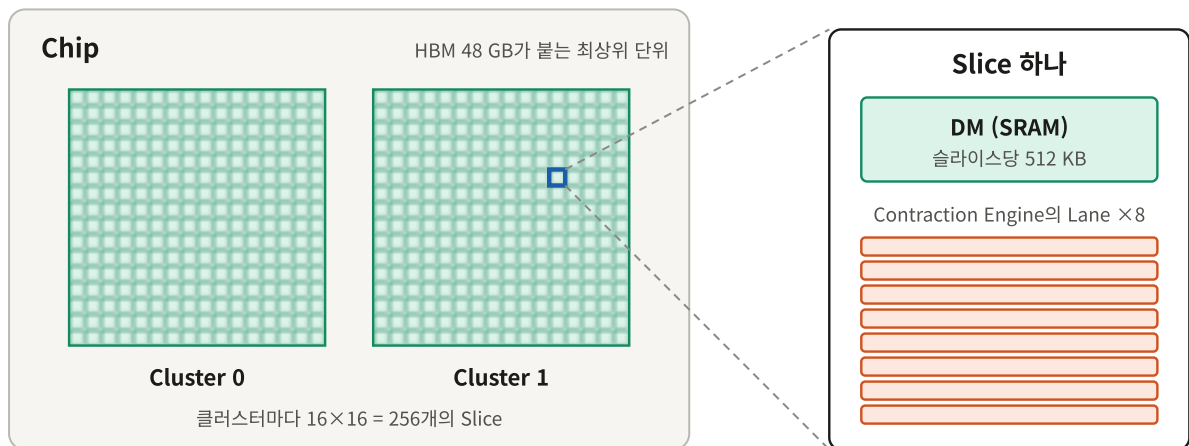


Tensor Contraction Processor 프로그래밍 입문

FuriosaAI의 *Programming Tensor Contraction Processors*(furiosa-opt Book) 전 장을
관련 분야를 전공하지 않은 컴퓨터공학 학부생도 따라올 수 있게 풀어 쓴 안내서



원문: developer.furiosa.ai/furiosa-opt/book/ (2026년 9월 11일에 확인한 온라인 판)

원문과 furiosa-opt 저장소는 Apache License 2.0으로 배포된다. 이 문서는 FuriosaAI가 만든 것이 아닌 비공식 해설이며,
API의 최종 기준은 원문이 밝히듯 `furiosa-opt-std` rustdoc이다.

이 안내서에 대하여

무엇을 다루나

FuriosaAI의 AI 가속기 RNGD는 **Tensor Contraction Processor(TCP)**라는 구조를 쓴다. 이 칩을 직접 프로그래밍하는 인터페이스가 **vISA**(Virtual Instruction Set Architecture)이고, vISA 커널을 Rust로 작성·검증·최적화하는 도구 모음이 **furiosa-opt**다. 원문 *Programming Tensor Contraction Processors*는 이 도구의 공식 설명서이며 다음 여덟 부분, 마흔다섯 쪽 남짓으로 이루어져 있다.

원문 부분	하위 페이지	이 안내서
Introduction	-	2장
Quick Start	Setup and Tooling, Tensor and Contraction, Kernel Design, Kernel Validation	2장 , 3장
Mapping Tensors	Mapping Expressions, Spatial and Temporal Dimensions, Tensor Semantics	4장
Moving Tensors	Sequencer, Fetch Engine, Commit Engine, Case Study: Tensor Unit I/O, DMA Engine, Memory Performance	5장 , 6장
Computing Tensors	Fetch Adapter, Switch Engine, Collect Engine, Register Files, Contraction Engine(Outer, Packet Reducer, Time Reducer, Lane Folder), Vector Engine(Intra-Slice Chain, Intra-Slice Reduce, Valid Count Generator, Inter-Slice Reducer), Cast Engine, Transpose Engine, Commit Adapter, Case Study: Chip/Cluster Reduction	7장 , 8장 , 9장 , 10장
End-to-End Cases	Case Study: Transformer	11장
Scheduling and Tuning	Schedule, Diagnosis, Tuning	12장
Tools	Kernel Optimizer, Language Server, Schedule Viewer	13장

원문의 모든 하위 페이지를 빠짐없이 다룬다. 다만 번역이 아니라 **해설**이다. 원문이 당연히 여기는 배경지식은 [1장](#)에 모았고, 긴 레퍼런스 절(예: 설정 비트맵 표 전체)은 규칙과 대표 예제 위주로 줄였으며, 원문에 없는 그림과 비유를 더했다. 수치, 타입 이름, 메서드 이름은 원문을 그대로 따른다.

누구를 위한 문서인가

C·Python·Java 같은 언어로 프로그래밍을 해 봤고, 컴퓨터 구조 수업에서 캐시·메모리 계층·파이프라인을 들어 봤으며, 행렬곱을 계산할 줄 아는 학부생을 가정한다. Rust, NPU, 양자화, 트랜스포머 모델은 몰라도 된다. 필요한 만큼은 본문에서 설명한다.

원문을 읽을 때 알아 둘 점

- 원문은 스스로 **알파 테스트 빌드용 실험 소프트웨어**의 문서라고 밝힌다. 실제 작업에 쓰기 전에는 Furiosa 엔지니어와 상의하라고 권한다.
- 원문은 사람뿐 아니라 **AI 에이전트에게 커널 작성을 맡기는 저자와**, vISA를 생성하는 **컴파일러 개발자**를 함께 독자로 삼는다. 그래서 "이 페이지가 무엇을 소유하는지" 같은 문장이 자주 나온다.
- 원문 안에서 예제의 주석과 코드, 또는 두 장의 수치가 서로 어긋나는 곳이 몇 군데 있다. 이 안내서는 그런 곳을 **원문 확인** 상자로 표시하고, 코드를 기준으로 계산한 값을 함께 적었다.

표기 규칙

표기	뜻
<code>B</code> , <code>KB</code> , <code>MB</code> , <code>GB</code>	바이트 단위. 원문은 2의 거듭제곱으로 쓴다(예: 512 KB = 524,288 B).
사이클(cycle)	하드웨어 클럭 한 틱. 원문의 모든 지연 시간과 스케줄 길이의 단위.
<code>B/cycle</code>	한 사이클에 옮길 수 있는 바이트 수. 원문이 말하는 대역폭(bandwidth) .
<code>m![A / 8, A % 8]</code>	매핑 표현식. 텐서의 축이 어떤 순서와 크기로 놓이는지 적는 Rust 매크로(4장).
<code>Time::SIZE</code>	매핑 타입 <code>Time</code> 의 원소 개수. Rust의 연관 상수 문법이다.
<code>§4.2</code> , 그림 3	이 안내서 안의 절과 그림. PDF에서 누르면 해당 위치로 이동한다.

본문의 상자는 여섯 종류다.

핵심

그 절에서 한 가지만 기억한다면 이것.

비유

익숙한 대상에 빗댄 설명. 정확한 정의는 본문을 따른다.

주의

틀리면 컴파일 에러, 잘못된 결과, 하드웨어 리셋으로 이어지는 규칙.

더 깊이

처음 읽을 때는 건너뛰어도 되는 세부 사항.

원문 확인

원문 내부의 불일치나 원문이 설명하지 않고 넘어가는 부분.

스스로 점검

절을 이해했는지 확인하는 질문.

그림의 색은 역할을 뜻한다. ■ DM(칩 안 SRAM), ■ TRF(Contraction Engine용 레지스터 파일), ■ VRF(Vector Engine용 레지스터 파일), ■ HBM과 DMA 전송, ■ 호스트. 계산 엔진은 흰 상자다. 색만으로 구분하지 않도록 모든 상자에 이름을 적었다.

읽는 순서

목적	추천 경로
개념만 빠르게	1장 → 2장 → 3장 → 4장 앞부분 → 8장 첫 절 → 12장 첫 절
커널을 직접 작성	1장부터 3장까지 순서대로 읽고, 4장·5장을 정독한 뒤, 7장~10장은 필요한 엔진만 레퍼런스로 찾아 읽는다. 부록 D의 메서드 체인 지도를 옆에 둔다.
성능 튜닝	6장의 메모리 규칙, 8장의 성능 절, 12장 전체, 부록 C 체크리스트

차례

1	시작하기 전에: 배경지식	7
1.1	왜 AI 전용 칩을 따로 만드는가	7
1.2	텐서, 이름 붙은 축, einsum	8
1.3	수 표현: i4부터 bf16까지	9
1.4	메모리 용어: SRAM, DRAM, 사이클, 대역폭	11
1.5	하드웨어 파이프라인과 스트림	12
1.6	원문 코드를 읽기 위한 최소한의 Rust	13
2	큰 그림: TCP, vISA, furiosa-opt	16
2.1	TCP, vISA, furiosa-opt	16
2.2	하드웨어 계층: Chip, Cluster, Slice, Lane	17
2.3	네 가지 메모리와 전송 경로	18
2.4	Tensor Unit 파이프라인	19
2.5	실행 컨텍스트: main, sub, tdma, pdma	20
2.6	커널 한 개의 일생	21
2.7	설치와 프로젝트 구조	21
2.8	원문의 학습 경로	22
3	첫 커널 다섯 개	24
3.1	다섯 패턴 한눈에	24
3.2	패턴 1: 상수 덧셈	24
3.3	패턴 2: 원소별 곱셈과 VRF	27
3.4	패턴 3: 내적과 Contraction Engine	28
3.5	패턴 4: GEMV, 출력 축을 슬라이스에	30
3.6	패턴 5: GEMM, 타일과 레인	31
3.7	커널 검증의 네 단계	33
3.8	다섯 패턴에서 배울 것	34
4	텐서 매핑	35
4.1	왜 매핑인가: 저장 순서와 지역성	35
4.2	Cell: 버퍼 위치 하나가 가리키는 것	36
4.3	매핑 생성자	37
4.4	동등한 매핑	40
4.5	공간 차원과 시간 차원	40
4.6	값 하나의 좌표 추적: NCHW	42
4.7	텐서 의미론: 담는다, 명세한다	43
4.8	축을 시간과 공간 사이로 옮기는 네 경로	43
5	텐서 이동 I: 시퀀서, Fetch, Commit	45
5.1	이동의 기본 규칙	45
5.2	시퀀서: 매핑 두 개를 중첩 루프로	46
5.3	Fetch Engine	49
5.4	Commit Engine	52
5.5	사례: Tensor Unit I/O	53
6	텐서 이동 II: DMA와 메모리 성능	55
6.1	DMA Engine: 파이프라인 없이 옮기기	55
6.2	DMA Engine의 구조	56
6.3	정렬과 패킷 크기 제약	57
6.4	DMA 성능을 정하는 세 가지	57
6.5	예제 여섯 개로 보는 DMA 성능	58
6.6	재분배 연산: 칩과 클러스터 사이로 값 보내기	59

6.7	Scatter와 Gather	60
6.8	PCIe DMA	61
6.9	DM의 물리 구조	61
6.10	뱅크 기아와 64회 규칙	63
6.11	HBM의 물리 구조	64
7	Tensor Unit 연산 I: 스트림을 계산할 준비	67
7.1	계산 경로 고르기	67
7.2	Fetch Adapter	67
7.3	Switch Engine: 슬라이스 사이의 링	69
7.4	Collect Engine: 32바이트 플릿으로 맞추기	72
7.5	레지스터 파일: TRF와 VRF	73
8	Tensor Unit 연산 II: Contraction Engine	78
8.1	네 단계 파이프라인	78
8.2	배치 행렬곱으로 보는 배치 전략	79
8.3	Outer: 브로드캐스트와 곱	80
8.4	Packet Reducer	82
8.5	Time Reducer	83
8.6	Lane Folder	84
8.7	설계 규칙 정리	85
9	Tensor Unit 연산 III: Vector Engine	87
9.1	구조: 두 부품과 두 가지 순서	87
9.2	Intra-Slice Chain: 고정된 12단계	88
9.3	피연산자: 상수, VRF, stash	90
9.4	ALU 규칙과 연산 클러스터	91
9.5	태그와 조건부 피연산자	93
9.6	Pair Mode: 두 그룹을 짝지어 계산하기	94
9.7	Intra-Slice Reduce	95
9.8	Valid Count Generator	96
9.9	Inter-Slice Reducer	99
10	Tensor Unit 연산 IV: 출력 단계와 칩·클러스터 리듀스	100
10.1	Cast Engine	100
10.2	Transpose Engine	101
10.3	Commit Adapter	103
10.4	사례: 칩과 클러스터를 넘는 리듀스	104
11	종합 사례: Transformer 디코더 한 스텝	109
11.1	End-to-End Cases의 성격	109
11.2	트랜스포머 디코더를 짧게	109
11.3	Qwen3-0.6B의 수치	110
11.4	커널 경계와 한 스텝의 흐름	111
11.5	결정 추적	112
11.6	검증과 점검	112
12	스케줄링과 튜닝	113
12.1	스케줄러가 하는 일	113
12.2	MNIST 예제로 타임라인 읽기	114
12.3	진단: 병목 가설 하나 세우기	116
12.4	튜닝: 통제된 실험	116
13	도구	119
13.1	Kernel Optimizer: cargo furiousa-opt	119

13.2	Language Server	120
13.3	Schedule Viewer	121
부록 A	용어집	123
A.1	하드웨어와 메모리	123
A.2	매핑과 스트림	124
A.3	엔진과 연산	125
A.4	스케줄링, 검증, 도구	126
부록 B	핵심 수치	128
B.1	하드웨어 계층과 용량	128
B.2	대역폭과 지연	128
B.3	규칙과 한도	128
B.4	사례의 수치	129
부록 C	설계 체크리스트	130
C.1	매핑과 크기	130
C.2	이동과 메모리	130
C.3	계산 엔진	130
C.4	출력과 리듀스	131
C.5	스케줄, 튜닝, 검증	131
부록 D	메서드 체인 지도	132
D.1	Tensor Unit 체인	132
D.2	Vector Engine 체인	133
D.3	DMA와 호스트 전송	134
부록 E	원문 대응표	135
E.1	장별 원문 페이지	135
E.2	이 안내서를 읽는 법	136

1장

시작하기 전에: 배경지식

원문은 독자가 Rust와 가속기 하드웨어에 익숙하다고 보고 곧장 API로 들어간다. 이 장은 그 사이의 빈칸을 메운다. AI 추론 칩이 왜 이렇게 생겼는지, 텐서를 "이름 붙은 축"으로 보는 법, 원문에 나오는 수 표현과 메모리 용어, 그리고 예제 코드를 읽는 데 필요한 최소한의 Rust를 차례로 설명한다. 이미 아는 절은 건너뛰어도 좋다.

이 장에서 배우는 것

- AI 추론이 행렬곱만큼이나 "데이터 옮기기" 문제인 이유
- 이름 붙은 축, einsum 표기, 그리고 모든 축약(contraction)을 이루는 세 단계
- $i4 \cdot i8 \cdot i32 \cdot f32 \cdot bf16 \cdot f8$ 같은 수 표현과 양자화의 zero-point
- SRAM과 DRAM, 사이클, 대역폭, 정렬, बैं크, 인터리빙 같은 메모리 용어
- 하드웨어 파이프라인의 지연과 처리량, 그리고 스트림의 Time-Packet
- 원문 예제를 읽기 위한 Rust 문법: 매크로, 제네릭과 터보피시, 빌림, 타입스테이트

1.1 왜 AI 전용 칩을 따로 만드는가

대형 언어 모델(LLM)이 토큰 하나를 만들 때 하는 일은 대부분 큰 행렬과 벡터의 곱이다. 11장에서 볼 Qwen3-0.6B는 디코더 레이어 하나에서 약 1,570만 개의 가중치를 입력과 곱하고, 이런 레이어를 28번 반복한다. 곱셈 한 번은 빠르다. 느린 것은 가중치를 계산 회로 앞까지 가져오는 일이다. 가중치는 크고, 크고 값싼 메모리(DRAM)는 계산 회로에서 멀리 있기 때문이다. 흔히 **메모리 벽(memory wall)**이라고 부르는 문제다.

원문이 설명하는 TCP는 이 문제를 세 가지 방식으로 다룬다.

1. **작고 많은 계산 단위.** 칩 하나에 *슬라이스(Slice)*라는 계산 단위가 512개(클러스터 2개 × 256개) 있고, 슬라이스마다 전용 SRAM이 붙어 있다. 데이터를 잘게 나눠 각 슬라이스 옆에 두면 계산 회로와 메모리 사이의 거리가 짧아진다.
2. **고정된 파이프라인에 흘리기.** 각 슬라이스에는 Fetch, Contraction, Vector, Commit 같은 전용 엔진이 일렬로 이어진 *Tensor Unit* 파이프라인이 있다. 데이터는 매 사이클 고정 크기 묶음으로 이 파이프라인을 흐른다.
3. **배치와 이동을 코드에 적는다.** 어떤 값을 어느 메모리의 어느 슬라이스에 두고 어떤 순서로 읽어 흘릴지를 프로그래머가 명시한다. PyTorch처럼 전부 숨기지도 않고, 바이트 주소를 직접 계산하게 하지도 않는다. 원문의 표현으로는 "텐서 단위로 생각하면서 메모리 할당과 Tensor Unit 스케줄링을 직접 관리"한다.

그래서 원문의 절반 이상은 계산 자체보다 **데이터를 어디에 두고 어떻게 옮기는가**를 다룬다. 이 점을 알고 읽으면 장 구성이 자연스럽게 이해된다.

비유: 공장으로 생각하기

원문의 용어를 공장에 빗대면 다음과 같다. 뒤에서 새 용어가 나오면 이 표로 돌아와도 좋다.

공장	원문 용어	설명
외부 공급처	Host	CPU 쪽 시스템 메모리. 모델 가중치가 처음 올라오는 곳
느린 트럭	PCIe DMA (<code>device.pdma</code>)	호스트와 HBM 사이 전송. 30 B/cycle로 가장 느리다
대형 창고	HBM	칩 패키지 위의 DRAM. 칩당 48 GB
지게차	Tensor DMA (<code>device.tdma</code>)	HBM과 DM 사이, DM끼리 데이터를 옮기는 전송 엔진
작업대	DM	슬라이스마다 512 KB씩 붙은 칩 안의 SRAM
작업자	Slice	클러스터마다 256명. 모두 같은 작업을 각자의 재료로 동시에 한다
조립 라인	Tensor Unit 파이프라인	Fetch → ... → Commit 순서로 고정된 엔진의 줄
작업자 옆 보조 선반	TRF, VRF	한쪽 피연산자(예: 가중치)를 올려 두는 작은 레지스터 파일

1.2 텐서, 이름 붙은 축, einsum

원문은 텐서를 "shape 안의 이름 붙은 인덱스 하나하나를 값에 대응시키는 것"으로 정의한다. 우리가 아는 다차원 배열과 다른 점은 **축에 이름이 있고 순서가 없다**는 것이다. 이미지 묶음의 shape를 $\{ N = 4, C = 3, H = 256, W = 512 \}$ 로 쓰면 N(장 수), C(채널), H(높이), W(너비)라는 네 축이 있다는 뜻일 뿐, 어느 축이 먼저 저장되는지는 말하지 않는다. 그래서 $\{ N = 4, C = 3 \}$ 과 $\{ C = 3, N = 4 \}$ 는 같은 텐서다. 인덱스는 모든 축에 값을 하나씩 준 것이다. 예를 들어 $\{ N: 0, C: 1 \}$.

텐서	차원	예	이름 붙은 shape
스칼라	0D	5.2	$\{ \}$
벡터	1D	[1, 2, 3]	$\{ I = 3 \}$
행렬	2D	2 × 4 격자	$\{ I = 2, J = 4 \}$
이미지 묶음	4D	RGB 이미지 네 장	$\{ N = 4, C = 3, H = 256, W = 512 \}$

순서는 텐서를 **저장할 때** 정한다. 행을 먼저 둘지 열을 먼저 둘지, 여러 슬라이스에 어떻게 나눌지를 정하는 것이 4장의 주제인 **매핑(mapping)**이다. 계산식은 축 이름으로만 쓰고 저장 순서는 매핑이 따로 책임진다. 이 분리가 원문 전체를 떠받치는 설계 원리다.

einsum 표기

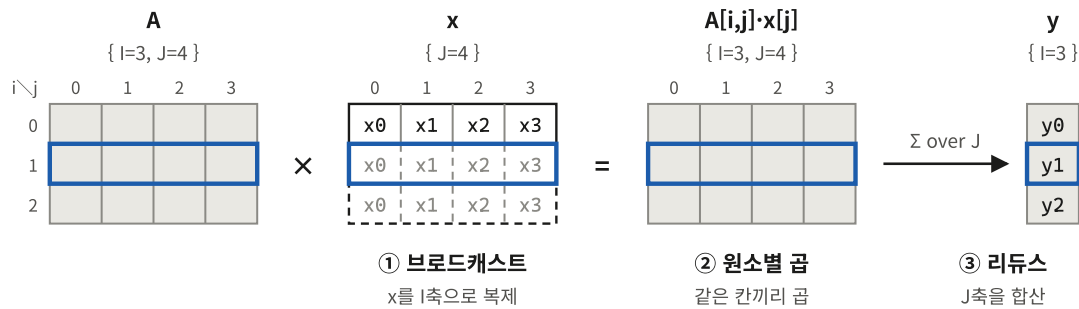
텐서를 곱하고 더하는 연산은 **einsum**(Einstein summation) 표기로 짧게 적을 수 있다. 입력의 축 이름을 쉼표로 늘어놓고 화살표 뒤에 출력 축을 쓴다. 규칙은 두 개다.

- 입력에 있지만 출력에 없는 축은 **합산해서 없앤다**.
- 한 입력에만 있는 축에 대해서는 다른 입력을 그 축을 따라 **복제한 것처럼** 취급한다. 이것이 **브로드캐스트(broadcast)**다. NumPy의 브로드캐스트와 같은 개념이다.

연산	einsum	브로드캐스트	곱	리듀스
내적	$I, I \rightarrow 1$	없음	$x_i \cdot y_i$	$\sum_i x_i \cdot y_i$
GEMV(행렬×벡터)	$IJ, J \rightarrow I$	x를 I축으로	$A_{ij} \cdot x_j$	$y_i = \sum_j A_{ij} \cdot x_j$
GEMM(행렬×행렬)	$IK, KJ \rightarrow IJ$	A를 J축으로, B를 I축으로	$A_{ik} \cdot B_{kj}$	$C_{ij} = \sum_k A_{ik} \cdot B_{kj}$

축약 = 브로드캐스트 → 곱 → 리듀스

원문은 두 입력을 원소별로 곱한 뒤 출력에 없는 공유 축을 없애는 연산을 **텐서 축약(tensor contraction)**이라 부르고, 모든 축약이 **브로드캐스트**, **원소별 곱**, **리듀스**의 세 단계로 이루어진다고 설명한다. 칩 이름의 "Contraction"이 바로 이것이다. **그림 1**은 GEMV를 세 단계로 나눠 보여 준다.



$$y[1] = A[1,0] \cdot x[0] + A[1,1] \cdot x[1] + A[1,2] \cdot x[2] + A[1,3] \cdot x[3]$$

그림 1. GEMV($IJ, J \rightarrow I$)를 세 단계로 나눈 모습. x에는 I축이 없으므로 점선처럼 I축을 따라 복제된 것으로 보고(①), A와 같은 칸끼리 곱한 뒤(②), 출력에 없는 J축을 따라 더한다(③). 파란 테두리는 출력 y[1] 하나를 만드는 데 쓰이는 칸이다.

여기서 **리듀스(reduce)**는 합, 최댓값, 최솟값처럼 한 축을 따라 값을 모아 그 축을 없애는 연산이다. **8장**에서 볼 Contraction Engine의 하드웨어 단계는 이 세 단계를 그대로 나눠 맡는다. Outer 단계가 브로드캐스트와 곱을, Packet Reducer와 Time Reducer가 리듀스를 담당한다.

핵심

모든 축약은 브로드캐스트, 곱, 리듀스로 나뉜다. 커널을 설계한다는 것은 결국 각 축을 하드웨어의 어느 차원(슬라이스, 사이클, 패킷, 레인)에 맡길지 정하는 일이다. 이 문장은 이 안내서 끝까지 반복해서 등장한다.

1.3 수 표현: i4부터 bf16까지

원문 예제에는 **i32**, **bf16**, **f8e4m3** 같은 타입 이름이 자주 나온다. 가속기가 작은 수 형식을 좋아하는 이유는 단순하다. 원소 하나의 바이트 수가 줄면 같은 대역폭으로 더 많은 원소를 옮기고, 같은 SRAM에 더 많은 원소를 넣을 수 있다. 줄어든 정밀도는 스케일 값 등으로 보완한다.

정수 타입

타입	비트	범위	원문에서 쓰이는 곳
i4	4	-8 ~ 7	양자화된 가중치. 한 바이트에 두 개가 들어간다
i5	5	-16 ~ 15	i4에서 zero-point를 뺀 중간값. 파이프라인 안에서만 존재한다

타입	비트	범위	원문에서 쓰이는 곳
<code>i8</code>	8	-128 ~ 127	양자화된 가중치와 활성화값
<code>i9</code>	9	-256 ~ 255	i8에서 zero-point를 뺀 중간값
<code>i16</code>	16	-32,768 ~ 32,767	Cast Engine의 출력 형식 중 하나
<code>i32</code>	32	약 ±21억	정수 누산기, Vector Engine의 정수 연산 폭

부동소수점 타입

IEEE 754 방식의 부동소수점은 부호, 지수, 가수 비트로 이루어진다. 지수 비트가 많을수록 표현 범위가 넓고, 가수 비트가 많을수록 값이 촘촘하다.

타입	부호 / 지수 / 가수	특징
<code>f32</code>	1 / 8 / 23	일반적인 float. 실수 누산기와 Vector Engine의 실수 연산 폭
<code>bf16</code>	1 / 8 / 7	bfloat16. f32와 표현 범위가 같고 정밀도만 낮다. 원문 예제에서 가장 많이 쓰는 가중치·활성값 형식
<code>f8e4m3</code>	1 / 4 / 3	8비트 부동소수점(FP8). 이름의 e4m3이 지수 4비트, 가수 3비트라는 뜻
<code>f8e5m2</code>	1 / 5 / 2	범위를 넓히고 정밀도를 줄인 FP8
<code>f4e2m1</code>	1 / 2 / 1	4비트 부동소수점. 크기가 0, 0.5, 1, 1.5, 2, 3, 4, 6 여덟 가지뿐이라 블록마다 스케일을 곱해 쓴다 (NVFP4·MXFP4 형식)

원문 확인

원문은 `bf16` 이 무엇인지 따로 정의하지 않는다. 예제에 나오는 `bf16::from_f32` 같은 API는 Rust `half` 크레이트의 `bfloat16` 타입과 같은 모양이어서, 이 안내서는 `bf16`을 `bfloat16`으로 읽는다.

양자화와 zero-point

양자화(quantization)는 실수를 작은 정수로 근사해 저장하는 방법이다. 흔한 형태는 $\text{실수} \approx \text{scale} \times (q - \text{zero_point})$ 이다. q 는 저장된 정수이고 `zero_point`는 실수 0에 대응하는 정수다. 계산할 때 q 에서 `zero_point`를 먼저 빼면 결과 범위가 한 비트 넓어진다. `i8` 두 값의 차는 $[-128, 127] - [-128, 127] = [-255, 255]$ 이므로 `i9`가 필요하다. 원문의 Fetch Adapter가 `i8`을 `i9`로, `i4`를 `i5`로 넓히는 이유가 이것이다(§ 7.2).

누산기와 확장(widening)

곱을 여러 개 더하면 원래 폭을 넘는다. `i8` 곱 하나는 최대 $(-128) \times (-128) = 16,384$ 이고, 이런 곱을 32개만 더해도 524,288이 되어 `i16`의 최댓값 32,767을 넘는다. 그래서 Contraction Engine은 곱하기 전에 `i4·i8`을 `i32`로, `f8·bf16`을 `f32`로 넓힌다. 원문은 이 넓힌 타입을 **누산기(accumulator) 타입**이라 부른다. 결과를 저장할 때는 Cast Engine이나 Commit Adapter가 다시 `bf16` 같은 작은 형식으로 줄인다. Vector Engine은 아예 32비트 타입(`i32`, `f32`)만 받는다.

wrapping, saturating, 고정소수점, 재해석

- wrapping 연산**(`AddFxp` 등): 오버플로가 나면 2의 보수 규칙대로 반대편 끝으로 돌아간다.
- saturating 연산**(`AddFxpSat`, `AddSat` 등): 최댓값이나 최솟값을 넘으면 그 값에 머문다.

- **고정소수점(fixed-point)**: 정수의 일부 비트를 소수점 아래로 해석하는 방식이다. 원문의 `vector_fxp_to_fp(int_width)` 는 i32를 f32로 바꿀 때 정수부 비트 수를 받으며, `int_width = 31` 이 보통의 i32 ↔ f32 변환이다.
- **캐스트와 재해석**: 캐스트는 값을 유지하며 형식을 바꾸고(`1.0 → 1`), 재해석(reinterpret)은 비트를 그대로 둔 채 다른 타입으로 읽는다(`1.0f32 → 0x3f80_0000`). 둘 다 원문에 나온다(§ 9.4).

1.4 메모리 용어: SRAM, DRAM, 사이클, 대역폭

구분	SRAM	DRAM
저장 방식	트랜지스터 여러 개로 된 래치	커패시터에 전하를 저장하고 주기적으로 재충전
속도	매우 빠르다	상대적으로 느리다
밀도와 가격	밀도가 낮고 비싸다	밀도가 높고 싸다
위치	계산 회로와 같은 칩 안	칩 밖(RNGD에서는 같은 패키지 위)
원문의 이름	DM, TRF, VRF (묶어서 SRAM 텐서)	HBM(High-Bandwidth Memory)

사이클

원문의 모든 시간은 **사이클**로 잰다. 사이클은 하드웨어 클럭의 한 틱이다. 스케줄 전체 길이도, DMA 명령의 시작 지연(약 500사이클)도, 가산기 트리의 지연(5~7사이클)도 이 단위로 적힌다. 원문은 부품에 따라 0.75 GHz, 1 GHz, 1.5 GHz 같은 서로 다른 클럭 기준을 적기도 하므로, 서로 다른 절의 사이클 수를 비교할 때는 기준 클럭을 확인하는 편이 좋다.

대역폭

통신 수업에서 말하는 주파수 대역폭과 달리, 원문의 **대역폭(bandwidth)**은 *단위 시간에 옮길 수 있는 데이터 양*이다. 칩 안에서는 B/cycle (사이클당 바이트), 칩 밖의 HBM은 TB/s 로 적는다. 여러 구간을 거치는 전송의 속도는 가장 느린 구간이 정한다. 원문의 대표 수치는 PCIe 30 B/cycle, DM 256 B/cycle(클러스터당), DMA 엔진 하나 256 B/cycle이다.

뱅크와 인터리빙

메모리는 한 덩어리가 아니다. 독립적으로 읽고 쓸 수 있는 **뱅크(bank)**, **채널(channel)**, **DMN** 같은 여러 통로로 나뉜다. 한 통로 안의 요청은 줄을 서서 하나씩 처리되고, 서로 다른 통로는 동시에 처리된다. RNGD의 메모리 주소는 순서대로 훑으면 요청이 여러 통로에 자연스럽게 흩어지도록 비트가 배치되어 있다. 요청을 여러 통로에 번갈아 배분하는 것을 **인터리빙(interleaving)**이라 한다. 반대로 같은 통로만 계속 두드리는 접근 패턴은 대역폭을 절반 이하로 떨어뜨린다. 구체적인 숫자는 6장에서 본다.

비유: 톨게이트

부스가 16개인 톨게이트에서 모든 차가 1번 부스로만 가면 처리량은 부스 하나만큼이다. 번호판 끝자리로 부스를 나눠 쓰면 16배가 된다. DM 주소의 하위 비트가 뱅크 번호가 되도록 설계한 것이 이 "번호판 끝자리" 규칙에 해당한다.

정렬과 Read-Modify-Write

하드웨어는 8바이트, 32바이트, 256바이트 같은 고정 단위로 읽고 쓴다. 쓰기가 단위의 일부만 덮으면, 나머지 바이트를 지키기 위해 단위 전체를 읽고(Read), 고친 다음(Modify), 다시 써야(Write) 한다. 이것이 **RMW**다. 원문에 따르면 DM에서는 쓰기 시간이 세 배로 늘고 해당 뱅크를 막으며, HBM에서는 약 50배 느려진다. 원문의 매핑에 크기를 $8 \cdot 32 \cdot 256$ 의 배수로 맞추는 **패딩**이 자주 등장하는 이유다.

우선순위와 기아

여러 엔진이 같은 बैं크를 원하면 정해진 우선순위로 순서를 받는다. 우선순위가 높은 엔진이 한 बैं크를 계속 쓰면 낮은 엔진은 한없이 기다린다. 운영체제 수업의 기아(starvation) 문제와 같다. RNGD에서는 우선순위가 가장 낮은 DMA 엔진이 4,096사이클 동안 응답을 받지 못하면 칩 내부 통신망(NoC)이 타임아웃을 선언하고 클러스터 전체가 리셋된다. 여기서 "같은 बैं크를 64번 이상 연속으로 접근하지 말라"는 규칙이 나온다([§ 6.10](#)).

1.5 하드웨어 파이프라인과 스트림

컴퓨터 구조 수업의 5단계 CPU 파이프라인처럼, 하드웨어 파이프라인은 일을 여러 단계로 쪼개 단계마다 매 사이클 조금씩 처리한다. 두 성능 지표를 구분해야 한다.

- **지연(latency)**: 입력이 들어가서 첫 결과가 나올 때까지 걸리는 사이클 수. 단계 수에 비례한다.
- **처리량(throughput)**: 파이프라인이 가득 찬 뒤 한 사이클에 나오는 결과의 수.



그림 2. 3단계 파이프라인의 시간표. 패킷은 한 사이클에 한 단계씩 대각선으로 내려간다. 파란 테두리의 P0은 사이클 2가 끝나야 출력되지만, 파이프라인이 찬 뒤로는 매 사이클 결과가 하나씩 나온다.

원문은 이 구분을 자주 쓴다. 예를 들어 32개 값을 더하는 가산기 트리는 깊이가 $\log_2 32 = 5$ 라서 첫 결과까지 5사이클이 걸리지만, 단계마다 레지스터가 있는 **완전 파이프라인**이라 그 뒤로는 매 사이클 결과 하나가 나온다. 원문의 "깊이는 첫 출력 지연을 늘리지만 정상 상태의 처리량은 줄이지 않는다"는 문장이 이 뜻이다.

스트림의 Time과 Packet

Tensor Unit 안의 데이터는 **스트림**으로 흐른다. 스트림은 매 사이클 고정 크기의 묶음, 즉 **패킷(packet)**을 하나씩 내보내는 데이터 흐름이다. 원문은 스트림을 두 매핑으로 설명한다.

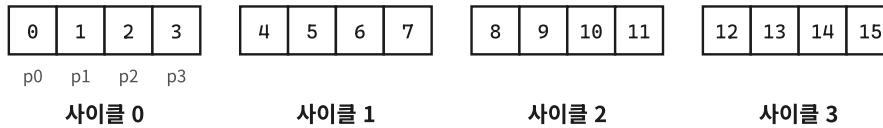
- **Time**: 몇 번째 묶음인가. 반복 순서이자 시간 축이다.
- **Packet**: 한 묶음 안에서 몇 번째 원소인가.

그리고 공간 차원인 Chip, Cluster, Slice마다 이런 스트림이 하나씩 동시에 흐른다([그림 3](#)).

DM에 저장된 원소 16개 (슬라이스 하나, $\text{Element} = m![A]$)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

$\text{fetch}::\langle \text{Time} = m![A / 4], \text{Packet} = m![A \% 4] \rangle$



Time: 사이클마다 다음 패킷 (값 = $A / 4$)

Packet: 패킷 안의 위치 p0~p3 (값 = $A \% 4$)



모든 슬라이스가 각자 데이터로 같은 스트림을 동시에 흘린다

그림 3. 스트림으로 읽기. 원소 16개를 $\text{Time} = m![A / 4]$, $\text{Packet} = m![A \% 4]$ 로 읽으면 사이클마다 4개씩 한 패킷으로 흐른다. 공간 차원(여기서는 Slice)마다 파이프라인이 하나씩 있어서, 256개 슬라이스가 각자의 16개 원소를 동시에 흘린다.

파이프라인 내부의 표준 묶음 크기는 32바이트이고, 원문은 이것을 **플릿(flit)**이라 부른다. Fetch Engine이 만든 패킷의 크기는 매핑에 따라 제각각이지만, Collect Engine이 이를 32바이트 플릿으로 맞추고 뒤쪽 엔진에 넘긴다([§ 7.4](#)).

핵심

스트림 텐서에는 차원이 다섯 개 있다. 공간 차원 **Chip·Cluster·Slice**는 "어느 작업자가 처리하나", 시간 차원 **Time**은 "몇 번째 사이클인가", **Packet**은 "한 사이클에 함께 가는 원소 중 몇 번째인가"를 뜻한다.

1.6 원문 코드를 읽기 위한 최소한의 Rust

원문의 모든 예제는 Rust이고, 원문은 "Rust 기본기가 있다고 가정한다"고만 적는다. 여기서는 C·C++이나 Java를 아는 학생이 예제를 읽는 데 꼭 필요한 문법만 짚는다. [3장](#)의 첫 예제 일부를 보자.

```
use furiousa_opt_std::prelude::*;

axes![A = 2048]; // ① 축 A를 크기 2048로 선언

pub type Cluster = m![1 # 2]; // ② 매핑을 "타입"으로 정의
pub type Slice = m![A / 8 # 256];

#[device(chip = 1)] // ③ 장치에서 실행되는 커널
pub fn constant_add_kernel(
    device: &mut Device, // ④ 장치를 가변 참조로 빌린다
    input: &HbmTensor<i32, Chip, m![A]>, // ⑤ 제네릭 타입: 원소 타입과 매핑들
) -> HbmTensor<i32, Chip, m![A]> {
    let dm = input.to_dm:::<Cluster, Slice, m![A % 8]>(&mut device.tdma); // ⑥ 터보피시
    let result = device
        .main
        .begin(dm.view()) // ⑦ 메서드 체인: 단계마다 타입이 바뀐다
        .fetch:::<m![1], m![A % 8]>()
        // ... (중간 단계 생략)
        .commit:::<m![A % 8]>();
    result.to_hbm(&mut device.tdma)
}
```

번호	문법	읽는 법
①	매크로 이름! [...]	느낌표가 붙은 것은 함수가 아니라 매크로다. <code>axes!</code> 는 축 이름과 크기를 선언하고, <code>m!</code> 은 매핑 타입을, <code>i!</code> 는 인덱스 값을 만든다.
②	<code>type</code> 이름 = ...;	타입 별칭. C의 <code>typedef</code> 와 같다. 매핑이 값이 아니라 타입이라는 점이 중요하다.
③	속성 #[...]	함수에 붙는 표시. <code>#[device]</code> 는 커널, <code>#[tokio::main]</code> 은 비동기 <code>main</code> , <code>#[unroll]</code> 은 루프 펼치기(§ 12.4).
④	<code>&mut T</code>	가변 참조(빌림). 빌린 동안에는 다른 코드가 그 값을 쓸 수 없다.
⑤	<code>Foo<A, B, C></code>	제네릭 타입. C++ 템플릿 인자처럼 읽으면 된다. <code>HbmTensor<i32, Chip, m![A]></code> 는 "원소가 i32이고 칩 차원 매핑이 Chip, 원소 매핑이 <code>m![A]</code> 인 HBM 텐서"다.
⑥	<code>f::<X, Y>(...)</code>	터보피시(turbofish). 제네릭 인자를 명시적으로 넘긴다. 이 API에서는 주로 목적지 매핑을 여기에 적는다.
⑦	<code>.a().b().c()</code>	메서드 체인. 각 메서드가 새 타입의 값을 돌려주므로 다음에 부를 수 있는 메서드가 달라진다.

매핑을 타입으로 넘기는 이유

보통의 라이브러리라면 `fetch(time_shape, packet_shape)` 처럼 값을 넘길 것이다. 원문의 API는 이것을 `fetch::<Time, Packet>( )` 처럼 **타입**으로 넘긴다. 타입은 컴파일할 때 모두 알려져 있으므로, 커널 컴파일러가 실행 전에 크기가 맞는지, 하드웨어 제약을 지키는지 검사하고 하드웨어 명령의 루프 설정까지 미리 계산할 수 있다. 원문의 API 코드에는 `verify_fetch`, `verify_collect` 같은 검사 함수가 들어 있고, 제약을 어기면 커널 컴파일 단계에서 실패한다.

타입스테이트: 순서를 타입으로 강제하기

Tensor Unit의 엔진 순서는 하드웨어에 고정되어 있다. 원문 API는 이 순서를 **타입스테이트(typestate)** 패턴으로 강제한다. `begin` 은 `BeginTensor` 를, `fetch` 는 `FetchTensor` 를, `collect` 는 `CollectTensor` 를 돌려주고, 각 타입에는 그다음에 올 수 있는 메서드만 정의되어 있다. 예를 들어 `commit` 은 `commit_trim` 뒤에서만 부를 수 있으므로 다음 코드는 컴파일되지 않는다.

```
device.main.begin(dm.view())
    .fetch::<m![1], m![A % 8]>()
    .collect::<m![1], m![A % 8]>()
    .commit();           // 컴파일 에러: commit은 commit_trim 뒤에만 있다
```

메서드가 `self` 를 소비하므로 지나간 단계로 되돌아갈 수도 없다. 하드웨어 파이프라인이 한 방향으로만 흐르는 것과 같다.

빌림과 수명: 컨텍스트 점유와 스트림의 수명

- `&mut device.main` 처럼 컨텍스트를 가변으로 빌리는 것은 "이 문장이 끝날 때까지 이 실행 컨텍스트를 독점한다"로 읽으면 된다. `device.main` 과 `device.sub` 은 서로 다른 필드여서 한 커널이 둘을 동시에 빌릴 수 있다.
- `begin(dm.view())` 는 DM 텐서의 뷰를 빌린다. 원문은 수명 매개변수 `'l'` 로 "스트림은 원본 뷰보다 오래 살 수 없다"는 규칙을 타입에 새긴다. `TuTensor<'l, ...>` 의 `'l'` 이 그것이다.
- `unsafe` 는 컴파일러가 증명할 수 없는 약속을 프로그래머가 대신 하는 곳이다. 원문에서는 `reshape` 처럼 "메모리 배치는 그대로 두고 매핑 이름만 바꾸는" 연산에 쓰인다.

async와 await

호스트와 HBM 사이의 전송(`to_hbm(&mut device.pdma)`, `to_host(...)`)과 커널 실행(`launch(...)`)은 비동기이며 `.await` 로 기다린다. 반면 커널 함수 안의 `to_dm`, `fetch`, `commit` 같은 호출은 그 자리에서 실행되는 것이 아니라 **선택한 컨텍스트의 큐에 장치 명령을 넣는다**. 실제 실행 순서와 명령끼리의 겹침은 스케줄러가 정한다(12장).

팁

원문 API 코드의 `impl<'l, const T: Tu, P: CanApplyFetch, D: Scalar, Chip: M, ...>` 같은 긴 제네릭 목록은 처음에는 건너뛰어도 된다. 메서드 이름과 터보피시 안의 매핑만 따라가도 흐름이 보인다. 긴 목록의 대부분은 "앞 단계에서 받은 차원을 그대로 넘긴다"는 뜻이다.

스스로 점검

1. `{ I = 2, J = 3 }` 행렬 A와 `{ J = 3 }` 벡터 x로 GEMV를 하면 무엇이 브로드캐스트되고, 어느 축을 따라 리듀스하나?
2. i8 곱을 64개 더할 때 i16 누산기로 충분한가?
3. 가산기 트리의 깊이가 6이면 첫 결과까지 몇 사이클이 걸리고, 그 뒤의 처리량은 얼마인가?
4. `x.fetch::<m![A], m![B]>(C)` 에서 `m![A]` 와 `m![B]` 는 각각 무엇을 정하나?

답: ① x가 I축으로 브로드캐스트되고 J축을 합산한다. ② $(-128)^2 \times 64 = 1,048,576$ 이므로 부족하다. ③ 6사이클, 이후 매 사이클 1개. ④ 출력 스트림의 Time(몇 번째 사이클인가)과 Packet(한 패킷 안의 원소 배치).

2장

큰 그림: TCP, vISA, furiosa-opt

원문: Introduction · Quick Start · Setup and Tooling · Computing Tensors 개요 · Schedule(실행 컨텍스트)

세부 엔진으로 들어가기 전에 전체 지도를 그린다. 칩의 하드웨어 계층, 네 종류의 메모리와 그 사이의 전송 경로, 슬라이스마다 있는 Tensor Unit 파이프라인, 그리고 연산을 동시에 흘리는 실행 컨텍스트. 이 장의 그림들은 이후 모든 장에서 "지금 어디를 보고 있나"를 확인하는 지도로 쓴다. 마지막으로 커널 하나가 작성되고 컴파일되어 실행되기까지의 흐름과 개발 환경을 정리한다.

이 장에서 배우는 것

- TCP·vISA·furiosa-opt가 각각 무엇이고 PyTorch 같은 프레임워크와 어떻게 다른지
- Chip → Cluster → Slice → Lane 계층과 각 계층의 개수
- Host·HBM·DM·TRF·VRF의 용량과 역할, 그 사이를 잇는 전송 경로
- Tensor Unit 파이프라인을 이루는 엔진 열 개의 순서와 역할
- main·sub·tdma·pdma 실행 컨텍스트와 "겹쳐 실행"의 의미
- 커널의 컴파일 단계(MIR → vISA → LIR → EDF)와 프로젝트 구조

2.1 TCP, vISA, furiosa-opt

원문 첫 문단의 요지는 다음과 같다. FuriosaAI의 **Tensor Contraction Processor(TCP)**는 추론(inference)을 겨냥한 대규모 병렬 AI 가속기다. PyTorch나 XLA 같은 고수준 프레임워크는 메모리 배치와 하드웨어 스케줄링을 감추고, 저수준 커널 API는 바이트 단위로 생각하기를 요구한다. TCP는 그 중간에서 프로그래머에게 직접적인 제어권을 주되 바이트 단위 추론은 요구하지 않는다.

수준	프로그래머가 생각하는 단위	메모리 배치와 스케줄
PyTorch, XLA	텐서 연산(<code>matmul</code> , <code>softmax</code>)	프레임워크가 숨기고 알아서 정한다
vISA (furiosa-opt)	텐서와 축, 매핑	프로그래머가 명시한다. 단, 주소 계산은 컴파일러가 한다
저수준 커널 API	바이트, 주소, 레지스터	프로그래머가 모두 계산한다

- **vISA**(Virtual Instruction Set Architecture): 이 제어권을 드러내는 프로그래밍 인터페이스다. 텐서 단위로 추론하면서 메모리 할당과 Tensor Unit 스케줄링을 직접 관리하게 한다.
- **furiosa-opt**(Furiosa IR Optimizer): vISA 커널을 Rust로 작성·검증·최적화하는 도구 모음이다. `rustc` 드라이버로 동작하며, Furiosa SDK 컴파일러를 보완한다. 사람이 vISA를 손으로 쓰거나 다른 컴파일러가 vISA를 생성할 때 세밀한 제어를 제공한다.
- **RNGD**: 이 구조를 구현한 FuriosaAI의 칩 이름이다. 원문의 표에 "Count (RNGD)"처럼 구체적인 개수의 기준으로 등장한다.

원문은 커널을 작성하는 사람이 **AI 에이전트를 지휘해** 커널을 쓰고 검증하고 최적화하는 상황을 주된 독자로 삼고, vISA를 생성하는 컴파일러 개발자를 두 번째 독자로 삼는다. 공개 릴리스의 API 기준은 `furiosa-opt-std` `rustdoc`이고, 원문은 "커널 설계 결정을 어떻게 내리고 검토하는지"를 설명한다고 스스로를 규정한다.

2.2 하드웨어 계층: Chip, Cluster, Slice, Lane

TCP에는 중첩된 네 단계의 하드웨어 단위가 있다. 모두 **공간 병렬성**의 단위다. 같은 단계의 단위들은 같은 매핑을 공유하며 각자의 데이터로 같은 일을 동시에 한다.

단계	개수 (RNGD)	역할
Chip	시스템에 따라 다름	HBM을 가진 최상위 단위
Cluster	칩당 2개	슬라이스 256개의 묶음
Slice	클러스터당 256개	Tensor Unit 하나. 전용 DM(512 KB)을 가진다
Lane	슬라이스당 8개	Contraction Engine의 곱셈-누산(MAC) 배열 한 행

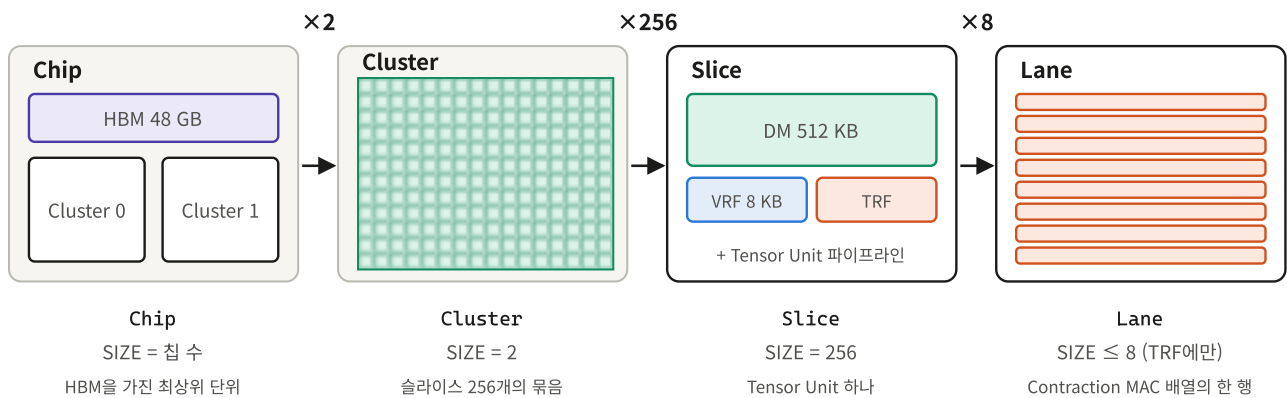


그림 4. 하드웨어 계층. 각 단계는 텐서 타입의 매개변수 하나에 대응하고, 크기가 하드웨어 개수와 정확히 맞아야 한다(Lane은 8 이하). 칩 하나에 슬라이스는 512개다.

텐서 타입은 원소 타입과 함께 **각 논리 축이 이 계층에 어떻게 분배되는지**를 담는다. 원문의 예를 하나 읽어 보자.

```
DmTensor<bf16, m![1], m![1 # 2], m![A / 8 # 256], m![A % 8]>
//      원소   Chip  Cluster  Slice      Element(슬라이스 안)
```

- **bf16**: 원소 타입.
- **m![1]**: Chip 차원에는 축이 없다. 칩 하나를 쓴다.
- **m![1 # 2]**: Cluster 차원도 비어 있고 크기 1을 2로 **패딩**했다. 클러스터 두 개 중 하나만 실제로 쓴다는 뜻이다.
- **m![A / 8 # 256]**: 축 A를 8개씩 묶은 **블록 번호**(A / 8)가 슬라이스 번호가 된다. A = 2048이면 블록이 정확히 256개이고, # 256 은 하드웨어 개수에 맞춘다는 표시다.
- **m![A % 8]**: 슬라이스 안에는 **블록 안의 위치**(A % 8), 즉 원소 8개가 들어간다.

그래서 A의 원소마다 "몇 번 슬라이스의 몇 번째 자리"라는 위치가 딱 하나로 정해진다. 매핑 연산자를 미리 요약하면 다음과 같다. 자세한 규칙은 4장에서 다룬다.

연산자	예	뜻
/	A / 8	스트라이드. 8개씩 묶은 블록의 번호
%	A % 8	블록 안의 위치(0~7). 새 안쪽 항을 만든다
#	1 # 2, A / 8 # 256	하드웨어 단위 수에 맞춰 패딩

연산자	예	뜻
=	<code>B / 2048 = 1 # 2</code>	기존 항이 덮는 범위를 줄인다. 타일 하나를 고를 때 쓴다

주의

런타임은 **칩 단위**로 동작한다(`#[device(chip = N)]`). 칩이나 클러스터의 일부만 쓰는 구성은 아직 지원하지 않으므로, 쓰지 않는 단위는 `m![1 # 2]` 처럼 패딩으로 채워 개수를 맞춘다. 원문은 이 제약이 이후 릴리스에서 완화될 수 있다고 적는다.

2.3 네 가지 메모리와 전송 경로

타입	위치	용량 (RNGD)	역할
<code>HostTensor</code>	호스트	-	CPU 쪽 값. 매핑 하나로 표현이 완전히 정해진다
<code>HbmTensor</code>	패키지 위 DRAM	48 GB, 1.5 TB/s	가중치와 활성값의 장기 저장소
<code>DmTensor</code>	칩 안 SRAM	전체 256 MB, 슬라이스당 512 KB	주 작업 메모리. Tensor Unit이 여기서 읽고 여기에 쓴다
<code>TrfTensor</code>	칩 안 SRAM	레인당 8 KB, 슬라이스당 레인 8개	Contraction Engine의 고정 피연산자 레지스터 파일
<code>VrfTensor</code>	칩 안 SRAM	슬라이스당 8 KB	Vector Engine의 고정 피연산자 레지스터 파일

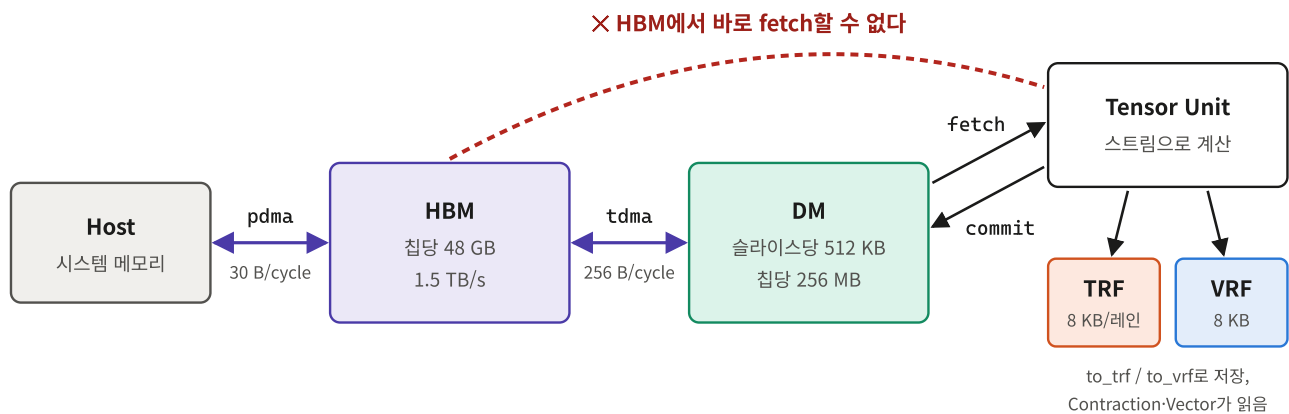


그림 5. 메모리 계층과 전송 경로. 보라색 화살표는 DMA 전송이다. Tensor Unit은 DM에서만 읽고(fetch) DM에만 쓴다(commit). 그래서 HBM의 데이터는 반드시 tdma로 DM에 옮긴 뒤 계산한다.

전송 경로는 원문의 이동 표로 정리된다.

지금 값	목적지	연산과 컨텍스트
<code>HostTensor</code>	<code>HbmTensor</code>	<code>to_hbm + device.pdma</code> (async)
<code>HbmTensor</code>	<code>HostTensor</code>	<code>to_host + device.pdma</code> (async)
<code>HbmTensor</code>	<code>DmTensor</code> / <code>HbmTensor</code>	<code>to_dm</code> / <code>to_hbm + device.tdma</code>
<code>DmTensor</code>	<code>HbmTensor</code> / <code>DmTensor</code>	<code>to_hbm</code> / <code>to_dm + device.tdma</code> , 또는 Tensor Unit의 fetch→commit 왕복
<code>DmTensorView</code>	Tensor Unit 스트림	<code>begin</code> 다음 <code>fetch</code> (main 또는 sub)

지금 값	목적지	연산과 컨텍스트
Tensor Unit 스트림	<code>DmTensor</code>	<code>commit</code> , 또는 기존 뷰에 쓰는 <code>commit_view</code>
Tensor Unit 스트림	<code>TrfTensor</code> / <code>VrfTensor</code>	<code>collect</code> 뒤 <code>to_trf</code> / <code>to_vrf</code>

핵심

모든 이동은 **수학적 텐서 이동**이다. 값은 그대로이고 목적지의 매핑(배치)만 바뀔 수 있다. `to_dm`에 넘기는 타입 매개변수가 곧 "어느 클러스터, 어느 슬라이스, 슬라이스 안 어떤 순서로 둘지"에 대한 지시이다.

2.4 Tensor Unit 파이프라인

Tensor Unit은 칩 안의 계산 파이프라인이다. DM에서 텐서를 읽어 엔진 열 개를 거쳐 변환한 뒤 DM에 다시 쓴다. 텐서는 사이클마다 패킷 하나씩 흐르는 스트림으로 이동하고, 엔진들은 스트림의 사이클당 배치(Packet)와 반복 순서(Time)를 바꿔 가며 처리한다. 슬라이스마다 이 파이프라인이 하나씩 있다.

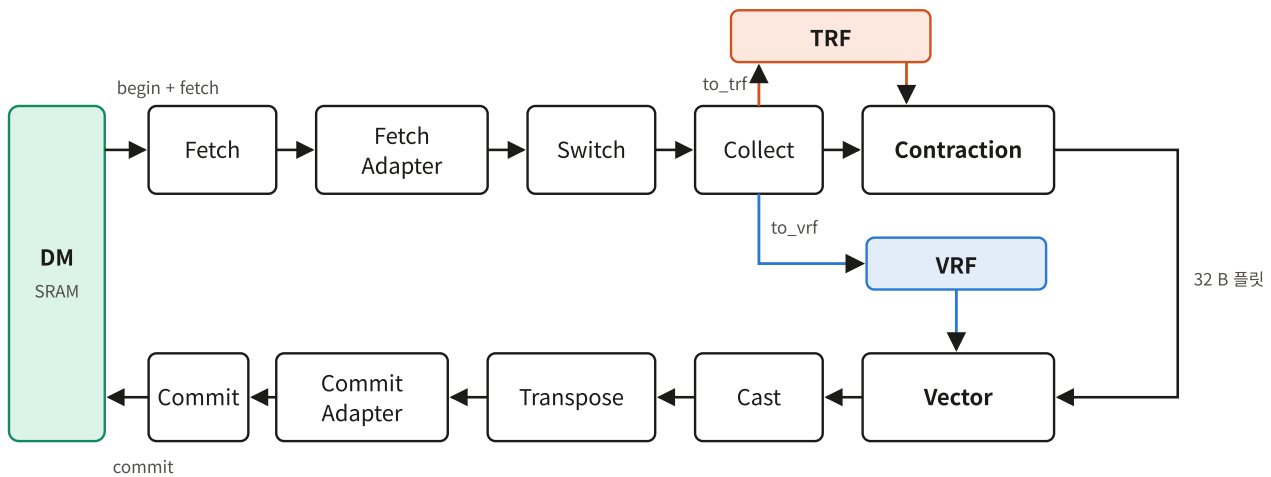


그림 6. Tensor Unit 파이프라인. 왼쪽에서 데이터를 읽고 정리한 뒤 오른쪽에서 곱해 아래쪽에서 계산 결과를 줄이고 정리해 DM으로 돌아간다. Collect 이후 모든 엔진은 32바이트 플릿 단위로 동작한다. 쓰지 않는 엔진은 건너된다.

엔진	하는 일	핵심 제약
Fetch	DM의 데이터를 파이프라인으로 읽어 들인다	패킷은 8바이트 정렬. Slice는 보존(축 올리기 예외)
Fetch Adapter	원소별 변환: 테이블 조회, 타입 변환, zero-point 빼기	선택 사항. 안 쓰면 그대로 통과
Switch	슬라이스 사이에 데이터를 옮긴다	링 네트워크. Slice 매핑이 바뀔 수 있다
Collect	패킷을 32바이트 플릿으로 맞춘다	출력은 정확히 플릿 하나
Contraction	einsum 축약: 행렬곱, 컨볼루션, 어텐션	한 피연산자는 TRF에 상주하고 다른 하나는 스트림
Vector	원소별 연산, 이항 연산, 리듀스	입력은 i32/f32만
Cast	정밀도를 낮춘다(f32→bf16 등)	출력은 정확히 플릿 하나
Transpose	플릿 안에서 원소를 재배열(Time과 Packet 교환)	플릿 내부에서만
Commit Adapter	기록 전 원소별 변환: 트리밍, f32→bf16(+ReLU)	<code>.commit()</code> 앞에 연결

엔진	하는 일	핵심 제약
Commit	결과를 DM에 쓴다	플릿 정렬 쓰기

스트림에는 차원이 다섯 개([Chip, Cluster, Slice, Time, Packet]) 있다. 엔진 대부분은 공간 차원을 그대로 두고 Time과 Packet만 바꾼다. 공간 차원을 바꾸는 예외는 셋이다. Switch Engine은 슬라이스 사이로 데이터를 옮기고, Fetch의 축 올리기(axis lifting)는 슬라이스마다 다른 주소에서 읽게 하며, Vector Engine의 inter-slice reducer는 클러스터 안 256개 슬라이스의 값을 합친다.

Contraction Engine과 Vector Engine은 피연산자 하나는 파이프라인 스트림에서, 다른 하나는 슬라이스 전용 레지스터 파일에서 받는다. TRF는 Contraction Engine에, VRF는 Vector Engine에 피연산자를 공급하며, 둘 다 해당 엔진이 돌기 전에 채워져 있어야 한다.

2.5 실행 컨텍스트: main, sub, tdma, pdma

`Device::new(Topology { chips, pes })`는 칩을 열고, 서로 독립인 실행 컨텍스트 네 개를 담은 Device를 돌려준다. 커널의 모든 연산은 이 중 하나를 골라 명령을 넣는다.

컨텍스트	담당
<code>device.main</code>	커널의 주 계산. Tensor Unit의 모든 엔진을 쓸 수 있다
<code>device.sub</code>	Tensor Unit 파이프라인의 부분집합. Contraction Engine 등 일부 기능이 없다. 주로 main이 계산하는 동안 TRF·VRF에 피연산자를 미리 올린다
<code>device.tdma</code>	Tensor DMA. HBM↔HBM, HBM↔DM, DM↔DM 이동. Tensor Unit 밖에 있다
<code>device.pdma</code>	PCIe DMA. 호스트↔HBM 이동. 비동기(.await)이며 주로 호스트 프로그램이 쓴다

규칙은 둘이다. 첫째, 한 컨텍스트 안의 연산은 순서대로 실행된다. 둘째, 다른 컨텍스트의 연산은 자원이나 메모리 의존성이 없으면 겹쳐 실행될 수 있다. 원문의 원소별 곱셈 예제로 보면 그림 7과 같다.

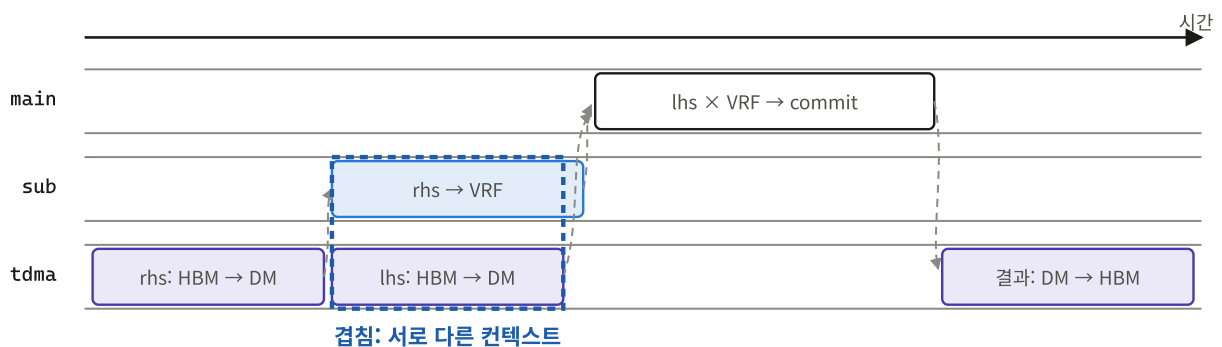


그림 7. 원소별 곱셈 커널을 컨텍스트별로 펼친 개념도. 점선 화살표는 "앞 연산의 결과를 기다린다"는 의존성이다. sub가 rhs를 VRF에 올리는 동안 tdma는 lhs를 옮길 수 있다. main은 lhs와 VRF가 모두 준비된 뒤에 시작한다. 실제 순서와 겹침은 스케줄러가 정하며 12장에서 다룬다.

비유

main은 조립 라인의 주 작업자, sub는 다음 공정에 쓸 부품을 미리 선반(TRF·VRF)에 올려 두는 보조 작업자, tdma는 창고와 작업대 사이를 오가는 지게차다. 셋은 서로 다른 사람이라 동시에 일할 수 있지만, 주 작업자는 필요한 부품이 선반에 올라오기 전에는 기다려야 한다.

2.6 커널 한 개의 일생

furiousa-opt 프로젝트의 코드는 두 부분으로 나뉜다. 장치에서 도는 **커널**(#[device] 함수)과, 커널을 부르고 결과를 검사하는 **호스트 프로그램**이다.

1. **호스트**: `Device::new(kernel.topology())` 로 장치를 연다.
2. **호스트**: 입력을 `HostTensor` 로 만들고 `to_hbm(&mut device.pdma).await` 로 HBM에 올린다.
3. **호스트**: `launch(kernel, (&mut device, &in_hbm)).await` 로 커널을 실행한다.
4. **커널**: `to_dm(&mut device.tdma)` 로 HBM의 입력을 DM에 배치한다.
5. **커널**: `device.main.begin(...).fetch(...).commit()` 으로 계산한다. 필요하다면 `device.sub` 가 피연산자를 TRF·VRF에 올린다.
6. **커널**: `to_hbm(&mut device.tdma)` 로 결과를 HBM에 돌려놓고 반환한다.
7. **호스트**: `to_host(&mut device.pdma).await` 로 결과를 받아 **호스트 오라클**(같은 계산을 CPU에서 한 기준값)과 비교한다.

같은 코드를 두 가지로 빌드할 수 있다. 평범한 `cargo` 로 빌드하면 **CPU 빌드**다. 커널이 호스트에서 실행되므로 NPU나 SDK 없이 값의 정확성을 확인할 수 있다. `cargo furiousa-opt` 로 빌드하면 **NPU 빌드**다. 커널을 미리 컴파일하고, Furiosa SDK의 커널 드라이버와 PE 런타임(`furiousa-driver-rngd`, `furiousa-smi` 등)을 거쳐 실제 칩에서 실행한다.

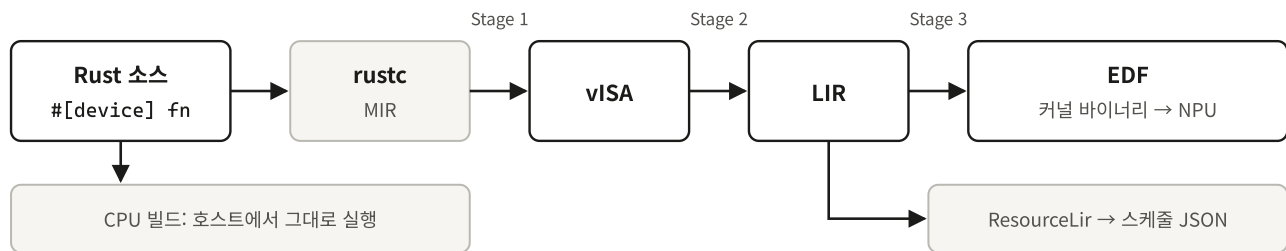


그림 8. 커널 컴파일 파이프라인. `cargo furiousa-opt compile` 은 Stage 1(MIR → vISA), Stage 2(vISA → LIR), Stage 3(LIR → EDF)를 거친다. 스케줄은 LIR을 자원 단위 표현(ResourceLir)으로 낮춘 뒤 계산되며, 단계별 산출물을 파일로 뽑는 옵션은 13장에서 정리한다.

2.7 설치와 프로젝트 구조

호스트 요구 사항

호스트는 **x86_64 Ubuntu 22.04 이상**이어야 한다. 배포 바이너리가 `GLIBC_2.34` 를 요구하는데 20.04에는 2.31이 들어 있기 때문이다. macOS나 arm64에서는 `linux/amd64` 컨테이너 안에서 모든 작업을 한다. 툴체인 설치 전에 다음 패키지가 필요하다. `rustc`가 호스트 바이너리를 시스템 `cc` 로 링크하고, `furiousa-opt-std` 의 빌드 스크립트가 `libclang.so` 를 불러오기 때문이다.

```

sudo apt install build-essential libclang-dev # 모든 빌드
sudo apt install gcc-aarch64-linux-gnu      # NPU 빌드(cargo furiousa-opt)에만

```

설치하고 첫 프로그램 실행

Furiosa optimizer는 `rustc` 드라이버이고, 고정된 nightly 툴체인과 ABI가 묶여 있다. 툴체인과 도구를 설치하고, 표준 스타터를 생성해 CPU에서 실행한 뒤 커널을 컴파일한다.

```
rustup toolchain install nightly-2026-05-01
cargo +nightly-2026-05-01 install cargo-binstall
cargo +nightly-2026-05-01 binstall cargo-furiosa-opt
cargo install cargo-generate
cargo generate furiosa-ai/furiosa-opt base-template
cd base-template
cargo run --release --bin gemm          # CPU에서 실행
cargo furiosa-opt compile              # 커널 컴파일(검증)
```

생성된 프로젝트의 `rust-toolchain.toml` 이 같은 `nightly` 채널을 고정하므로 그 디렉터리에서는 Cargo가 알아서 이 툴체인을 쓴다. NPU 빌드에는 Furiosa SDK와 실제 NPU가 추가로 필요하지만, CPU 빌드는 SDK 없이 호스트에서 돈다.

템플릿 구조

```
base-template/
├── Cargo.toml          # [package.metadata.furiosa-opt] → 커널 패키지 표시
├── rust-toolchain.toml # nightly-2026-05-01 고정
└── src/
    ├── lib.rs
    ├── kernel/
    │   ├── mod.rs      # 커널 모듈 재수출
    │   └── <name>_kernel.rs # #[device] 함수
    └── <name>.rs        # 호스트 프로그램 + 호스트 오라클 테스트
```

- `Cargo.toml` 의 `[package.metadata.furiosa-opt]` 가 이 패키지를 커널 컴파일 대상으로 표시한다.
- `src/kernel/` 에 `#[device]` 함수가 있고, `src/kernel/mod.rs` 가 이를 재수출한다.
- `src/*.rs` 는 `launch(kernel, ...)` 를 부르는 호스트 바이너리이며 오라클 테스트를 함께 담는다. 각각 `Cargo.toml` 에 `path = "src/<name>.rs"` 인 `[[bin]]` 항목이 있어야 한다.

주의

호스트 바이너리를 `src/bin/`, `examples/`, `tests/` 로 옮기지 않는다. 컴파일러 플러그인이 `src/` 를 뿌리로 하는 cargo 타깃만 옮기 때문이다.

커널을 하나 추가하는 순서는 다섯 단계다. ① `src/kernel/<name>_kernel.rs` 에 `#[device(...)] pub fn <name>_kernel(...)` 을 쓴다. ② `src/kernel/mod.rs` 에서 재수출한다. ③ `src/<name>.rs` 에 호스트 프로그램, 호스트 오라클, `launch(<name>_kernel, ...)` 호출을 쓴다. ④ `Cargo.toml` 에 `[[bin]]` 으로 등록한다. ⑤ Kernel Optimizer 명령으로 실행한다(§ 13.1). 원문은 에이전트가 이 작업을 할 때 Kernel Authoring Skill을 쓰라고 권한다.

개발 도구는 세 가지다. **Kernel Optimizer**(`cargo furiosa-opt`)는 빌드·테스트·컴파일을 담당하고, **Language Server**(`furiosa-rust-analyzer-proxy`)는 IDE에서 `Stride<Symbol<A>, 8>` 같은 긴 타입을 `m![A / 8]` 로 보여 주며, **Schedule Viewer**는 스케줄 JSON을 타임라인으로 그린다. 모두 13장에서 다룬다.

2.8 원문의 학습 경로

원문 Quick Start는 기본 패턴 다섯 개를 익힌 뒤 Mapping → Moving → Computing → End-to-End Cases → Scheduling → Tools 순서로 읽으라고 안내한다. 기본 패턴은 텐서가 **하드웨어 한 번의 처리(pass)**와 **슬라이스당 512 KB의 DM 용량** 안에 들어가는 경우를 다룬다. 그보다 크면 텐서를 타일로 나눠야 하는데, 타일을 시간에 따라 차례로 처리하면 **시간 분할(temporal partitioning)**, 병렬 하드웨어 단위에 나눠 주면 **공간 분할(spatial partitioning)**이다.

커널의 다음 제약	읽을 곳
한 번의 처리에 들어간다	기본 패턴이면 충분하다(3장)
배치 순서를 바꾸거나 패킷을 다뤄야 한다	Case Study: Tensor Unit I/O(§ 5.5)
리듀스 축이 칩이나 클러스터에 걸쳐 있다	Case Study: Chip/Cluster Reduction(§ 10.4)
여러 커널을 조합한 모델	Case Study: Transformer(11장)

스스로 점검

1. 칩 하나에 슬라이스는 몇 개이고, 슬라이스 하나의 DM은 몇 KB인가?
2. HBM에 있는 가중치로 행렬곱을 하려면 어떤 이동이 먼저 필요한가? 그 이동은 어느 컨텍스트가 하나?
3. `m![1 # 2]` 를 Cluster 차원에 쓰는 이유는?
4. sub 컨텍스트가 할 수 없는 대표적인 일은?

답: ① 512개, 512 KB. ② `to_dm` 으로 DM에 옮겨야 한다. `device.tdma`. ③ 클러스터 두 개 중 하나만 쓰면서 하드웨어 개수 2를 맞추려고. ④ Contraction Engine 실행.

3장

첫 커널 다섯 개

원문: Quick Start · Tensor and Contraction · Kernel Design(First Vector Kernel, Stationary Vector Operand, Contraction Patterns) · Kernel Validation

원문 Quick Start의 핵심은 실행 가능한 다섯 패턴이다. 패턴마다 하드웨어 결정이 하나씩 늘어난다. 상수 덧셈으로 파이프라인의 기본 모양을 보고, 원소별 곱셈에서 sub 컨텍스트와 VRF를, 내적에서 TRF와 Contraction Engine을, GEMV에서 출력 축을 슬라이스에 실는 법을, GEMM에서 타일과 레인을 배운다. 코드를 한 줄씩 따라가며 스트림의 Time과 Packet이 어떻게 바뀌는지 표로 추적한다.

이 장에서 배우는 것

- 커널 코드의 기본 체인: `to_dm` → `begin` → `fetch` → `collect` → (계산) → `commit_trim` → `commit` → `to_hbm`
- 고정 피연산자를 레지스터 파일(VRF·TRF)에 올리는 sub 컨텍스트의 역할
- Contraction Engine 체인 `contract_outer` → `contract_packet` → `contract_time` → `contract_lane`의 뜻
- 출력 축은 공간 차원(Slice·Lane)에, 리듀스 축은 Packet·Time에 두는 배치 원칙
- 커널 검증의 네 단계와 각 단계가 증명하지 않는 것

3.1 다섯 패턴 한눈에

패턴	계산	새로 등장하는 결정	원문이 드는 쓰임새
상수 덧셈	$out[i] = in[i] + 1$	파이프라인 기본 체인, 벡터를 슬라이스 256개에 분배, Vector Engine	잔차·bias 덧셈
원소별 곱셈	$lhs[i] \times rhs[i]$	sub 컨텍스트, 고정 피연산자를 담은 VRF	게이트 스케일링, GLU, 어텐션 스케일링
내적	$\sum_i x_i \cdot y_i$	TRF, Contraction Engine, 누산기 확장과 cast	어텐션 점수, 유사도
GEMV	$y_i = \sum_j A_{ij} \cdot x_j$	출력 축 I를 Slice에, 리듀스 축 J를 Time·Packet에	LLM 디코드
GEMM	$C_{ij} = \sum_k A_{ik} \cdot B_{kj}$	두 출력 축을 슬라이스 격자로 타일링, J의 일부를 Lane에	일반 행렬곱

다섯 예제 모두 `base-template`에 들어 있고, 장치 소스는 `src/kernel/<이름>_kernel.rs`, 호스트 프로그램과 오라클은 `src/<이름>.rs`에 있다. 모두 칩 1개, 클러스터 2개 중 1개를 쓴다.

3.2 패턴 1: 상수 덧셈

목표는 벡터의 모든 원소에 1을 더하는 것, 즉 $out[i] = in[i] + 1$ 이다. **데이터 이동**은 다음과 같다. 벡터를 HBM에서 DM으로 옮기고, 각 DM 슬라이스를 `Fetch` → `Collect` → `Vector Engine` → `Commit`으로 흘린 뒤, 결과를 DM에서 HBM으로 돌려놓는다.

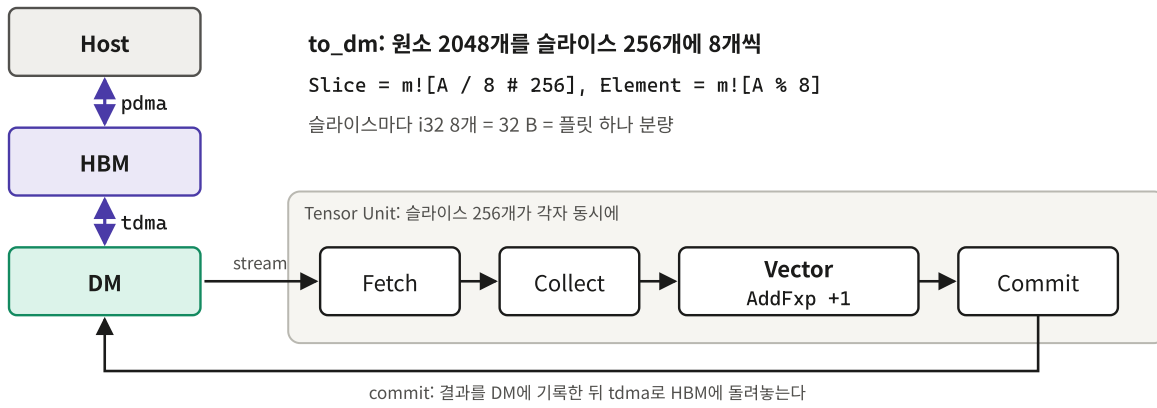


그림 9. 상수 덧셈의 데이터 이동. 호스트 프로그램이 입력을 HBM에 올리면 커널이 이를 DM의 슬라이스 256개에 나눠 놓고, 모든 슬라이스가 같은 파이프라인으로 자기 몫의 원소 8개에 1을 더한다.

장치 소스

```

use furiosa_opt_std::prelude::*;

axes![A = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![A / 8 # 256];

#[device(chip = 1)]
pub fn constant_add_kernel(device: &mut Device, input: &HbmTensor<i32, Chip, m![A]> -> HbmTensor<i32,
Chip, m![A]> {
    // HBM → DM: 원소 2048개를 슬라이스 256개에 8개씩 나눈다
    let dm = input.to_dm::<Cluster, Slice, m![A % 8]>(&mut device.tdma);

    let result = device
        .main
        .begin(dm.view())
        // Fetch: DM에서 원소 8개짜리 패킷을 파이프라인으로 흘린다
        .fetch::<m![1], m![A % 8]>()
        // Collect: 32바이트 플릿으로 정규화 (i32 8개 = 32 B)
        .collect::<m![1], m![A % 8]>()
        // Vector Engine 진입. TagMode::Zero = 조건 없이 모든 원소에 실행
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        // 모든 원소에 상수 1을 더한다
        .vector_fxp(FxpBinaryOp::AddFxp, 1)
        // Vector Engine을 나와 commit: 패킷을 쓰기 폭으로 트리밍한 뒤 DM에 기록
        .vector_final()
        .commit_trim::<m![A % 8]>()
        .commit::<m![A % 8]>();

    // DM → HBM
    result.to_hbm(&mut device.tdma)
}

```

한 줄씩 추적하기

#	호출	Time	Packet	무슨 일이 일어나나
1	<code>to_dm::<Cluster, Slice, m![A % 8]>(tdma)</code>	-	-	DMA가 HBM 벡터를 DM에 배치. 블록 번호 A / 8이 슬라이스 번호, A % 8이 슬라이스 안 위치
2	<code>main.begin(dm.view())</code>	<code>m![1]</code>	<code>m![A % 8]</code>	파이프라인 입구(<code>BeginTensor</code>). 아직 시간 반복은 없다
3	<code>fetch::<m![1], m![A % 8]>()</code>	<code>m![1]</code>	<code>m![A % 8]</code>	Fetch Engine이 슬라이스마다 i32 8개(32 B)를 한 번에 읽는다
4	<code>collect::<m![1], m![A % 8]>()</code>	<code>m![1]</code>	<code>m![A % 8]</code>	이미 정확히 32 B이므로 그대로 통과
5	<code>vector_init(), vector_intra_slice_tag(TagMode::Zero)</code>	//	//	Vector Engine 진입. 모든 원소의 태그를 0으로 두어 조건 없이 실행
6	<code>vector_fxp(FxpBinaryOp::AddFxp, 1)</code>	//	//	Fxp 클러스터에서 상수 1을 더한다 (wrapping 덧셈)
7	<code>vector_final()</code>	//	//	Vector Engine 종료
8	<code>commit_trim::<m![A % 8]>()</code>	//	<code>m![A % 8]</code>	플릿의 패딩을 잘라낸다. 여기서는 패딩이 없어 사실상 아무 일도 없다
9	<code>commit::<m![A % 8]>()</code>	-	-	Commit Engine이 <code>Element = m![A % 8]</code> 배치로 DM에 쓴다(<code>DmTensor</code>)
10	<code>to_hbm(tdma)</code>	-	-	DMA가 결과를 HBM으로 옮긴다

원문 설명대로 이 커널은 칩 1개, 클러스터 2개 중 1개, 그 클러스터의 슬라이스 256개를 모두 쓴다. 각 슬라이스는 i32 8개를 받고 `begin` → `fetch` → `collect` → `vector_init` → `vector_intra_slice_tag` → `vector_fxp` → `vector_final` → `commit` 체인을 한 번에 처리한다. § 5.3의 Fetch 공식으로 계산하면 슬라이스당 읽기는 1사이클이다.

호스트 프로그램과 오라클

호스트 프로그램은 입력을 만들어 HBM에 올리고, 커널을 실행하고, 결과를 호스트 쪽 수식과 비교한다. 원문 테스트의 핵심 부분이다.

```
#[tokio::test]
async fn matches_reference() {
    let mut device = Device::new(constant_add_kernel.topology()).unwrap();
    let mut rng = SmallRng::seed_from_u64(42);
    let input = HostTensor::<i32, m![A]>::rand(&mut rng);
    let in_hbm = input.to_hbm(&mut device.pdma).await.unwrap(); // 호스트 → HBM

    // 오라클: out[i] = in[i] + 1 (i32 wrapping 덧셈)
    let in_buf: Vec<i32> = input.into_vec();
    let expected: Vec<i32> = in_buf.iter().map(|&x| x.wrapping_add(1)).collect();

    let out_hbm = launch(constant_add_kernel, (&mut device, &in_hbm)).await.unwrap();
    let actual: Vec<i32> = out_hbm.to_host::<m![A]>(&mut device.pdma).await.unwrap().into_vec();
    for (i, (&e, &a)) in expected.iter().zip(&actual).enumerate() {
        assert_eq!(e, a, "constant_add mismatch at i={i}: expected {e}, actual {a}");
    }
}
```

핵심

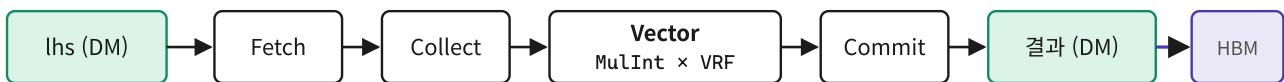
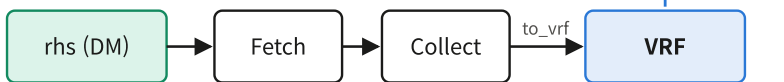
커널 코드에는 "원소마다 1을 더하라"는 반복문이 없다. 반복은 **매핑**이 만든다. `slice = m![A / 8 # 256]` 이 256개 슬라이스의 공간 병렬성을, `packet = m![A % 8]` 이 한 사이클 안의 8개 병렬성을 만든다.

더 깊이: %와 =의 차이

둘 다 축을 좁혀 보이지만 하는 일이 다르다. %는 **새 안쪽 항**을 만든다. `m![B / 2048, B % 2048]` 에서 앞 항은 2048개짜리 블록의 번호, 뒤 항은 블록 안 위치다. =는 항은 그대로 두고 **뷰가 읽는 범위**를 줄인다. `m![B / 2048 = 1 # 2]` 는 블록 두 개 중 하나를 고르며, # 2는 선택된 범위가 아니라 원래 축의 크기(2)를 유지한다는 표시다. 타일 커널은 둘을 함께 써서 `m![B / 2048 = 1 # 2, B % 2048]` 처럼 "블록 하나를 고르고 그 안의 모든 위치를 덮는다".

3.3 패턴 2: 원소별 곱셈과 VRF

목표는 $out[i] = lhs[i] \times rhs[i]$ 다. 두 피연산자가 모두 흘러야 할 것 같지만, 원문은 한쪽을 **고정 피연산자(stationary operand)**로 만든다. sub 컨텍스트가 rhs를 VRF에 미리 올려 두고, main 컨텍스트가 lhs를 흘리면서 Vector Engine이 매 사이클 VRF를 읽어 곱한다.

main 컨텍스트**sub 컨텍스트**

lhs와 rhs는 먼저 tdma로 HBM → DM에 옮긴다. 두 DM 영역은 서로 겹치면 안 된다.

그림 10. 원소별 곱셈. rhs는 VRF에 고정해 두고 lhs만 흘린다. Vector Engine은 슬라이스마다 자기 슬라이스의 VRF를 읽는다.

```

#[device(chip = 1)]
pub fn elementwise_mul_kernel(
    device: &mut Device,
    lhs: &HbmTensor<i32, Chip, m![A]>,
    rhs: &HbmTensor<i32, Chip, m![A]>,
) -> HbmTensor<i32, Chip, m![A]> {
    // 두 피연산자를 HBM → DM (DM 주소는 자동 배치)
    let lhs_dm = lhs.to_dm::<Cluster, Slice, m![A % 8]>(&mut device.tdma);
    let rhs_dm = rhs.to_dm::<Cluster, Slice, m![A % 8]>(&mut device.tdma);

    // sub 컨텍스트: rhs를 VRF에 올린다 (아래 main과 동시에 돌 수 있다)
    // VRF는 Vector Engine이 매 사이클 읽는 슬라이스별 피연산자다
    let rhs_vrf: VrfTensor<i32, Chip, Cluster, Slice, m![A % 8]> = device
        .sub
        .begin(rhs_dm.view())
        .fetch::<m![1], m![A % 8]>(C)
        .collect::<m![A % 8 / 8], m![A % 8 % 8]>(C)
        .to_vrf();

    // main 컨텍스트: lhs의 각 원소를 VRF의 대응 원소와 곱한다
    let result = device
        .main
  
```

```

    .begin(lhs_dm.view())
    .fetch::<m![1], m![A % 8]>()
    .collect::<m![1], m![A % 8]>()
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    // 슬라이스마다 lhs 원소 8개를 VRF의 rhs 원소 8개와 곱한다
    .vector_fxp(FxpBinaryOp::MulInt, &rhs_vrf)
    .vector_final()
    .commit_trim::<m![A % 8]>()
    .commit::<m![A % 8]>();

    result.to_hbm(&mut device.tdma)
}

```

새로 보이는 점을 정리하면 이렇다.

- **두 컨텍스트**: 모든 커널은 서로 다른 하드웨어 자원 위의 `device.main` 과 `device.sub` 를 가진다. `main` 은 주 계산을, `sub` 는 흔히 피연산자 선적재를 맡는다. `main` 은 `sub` 가 아직 만들지 않은 데이터가 필요하면 기다리며, 둘은 칩 안 SRAM 공간을 함께 쓴다. `Tu const 제네릭( { Tu::Main }, { Tu::Sub } )` 이 스트림이 어느 컨텍스트에 속하는지 타입에 기록한다.
- **DM 주소**: DM 주소를 생략하면 자동 배치된다. 알고리즘상 겹치지 않는 주소를 직접 지정해야 할 때만 `_at` API를 쓴다.
- **sub 쪽 collect**: `collect::<m![A % 8 / 8], m![A % 8 % 8]>()` 의 Time은 크기 $8/8 = 1$, Packet은 8이다. 결국 `collect::<m![1], m![A % 8]>()` 과 같은 배치를 스트라이드·모듈로로 쪼개 적은 것이다(§ 4.4).
- **VRF 저장**: `sub` 에서 부른 `to_vrf()` 는 인자가 필요 없다. VRF 텐서는 슬라이스마다 i32 8개(32 B)로, VRF 한도 8 KB에 한참 못 미친다.
- **VRF 피연산자**: `vector_fxp(FxpBinaryOp::MulInt, &rhs_vrf)` 처럼 VRF 텐서의 참조를 넘기면 그 값이 피연산자가 된다. 같은 메서드에 상수를 넘기면 상수 피연산자다(§ 9.3).

3.4 패턴 3: 내적과 Contraction Engine

목표는 $\sum_i x_i \cdot y_i$ 다. 한 피연산자는 파이프라인으로 흐르고, `sub` 컨텍스트가 다른 피연산자를 TRF에 올린다(Fetch → Collect → `to_trf()`). 곱하고 더하는 일은 Contraction Engine이 네 단계로 한다.

- `contract_outer`: **Stream Adapter**가 32 B 플릿 두 개를 64 B 패킷 하나로 묶고, **TRF Sequencer**가 TRF의 고정 피연산자를 읽는다. 두 피연산자가 레인별 곱셈기에 들어간다.
- `contract_packet`: 패킷 안의 곱들을 하드웨어 **리덕션 트리**로 공간적으로 합산한다.
- `contract_time::<m![1]>()`: 사이클에 걸쳐 누산해 슬라이스마다 스칼라 하나를 만든다.
- `contract_lane`: 레인 차원을 접는다. `Lane = m![1]` 이면 사실상 할 일이 없다.

이어서 `.cast()` 가 f32 누산기 값을 bf16으로 줄인다. 곱을 여러 개 누산하려면 폭이 넓어야 하므로 Contraction은 i4·i8을 i32로, fp8·bf16을 f32로 넓힌다(§ 1.3).

```

axes![A = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![1 # 256]; // 활성 슬라이스 1개. m![A / 8 # 256]이면 256개 전부에 분배된다
pub type Time = m![1];        // 시간 반복 없음
pub type Lane = m![1];        // 레인 병렬성 없음

```

```
#[device(chip = 1)]
pub fn dot_product_kernel(
    device: &mut Device,
    lhs: &HbmTensor<bf16, Chip, m![A]>,
    rhs: &HbmTensor<bf16, Chip, m![A]>,
) -> HbmTensor<bf16, Chip, m![1]> {
    // HBM → DM
    let lhs: DmTensor<bf16, Chip, Cluster, Slice, m![A]> = lhs.to_dm(&mut device.tdma);
    let rhs: DmTensor<bf16, Chip, Cluster, Slice, m![A]> = rhs.to_dm(&mut device.tdma);

    // sub 컨텍스트: rhs를 TRF에 올린다
    let rhs: TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![A]> = device
        .sub
        .begin(rhs.view())
        .fetch::<Time, m![A]>()
        .collect::<m![{ Time }, A / 16], m![A % 16]>()
        .to_trf();

    // main 컨텍스트: lhs를 Contraction Engine으로 흘리며 A축을 따라 리듀스
    let result: DmTensor<bf16, Chip, Cluster, Slice, m![1 # 8]> = device
        .main
        .begin(lhs.view())
        .fetch::<Time, m![A]>()
        .collect::<m![A / 16], m![A % 16]>()
        // 연속한 32 B 플릿 둘을 64 B 패킷으로 묶어 시간 단계를 반으로 (A/16 → A/32)
        .contract_outer::<m![A / 32], m![A % 32], _, _, _>(&rhs)
        .contract_packet::<m![1]>()
        .contract_time::<m![1]>()
        .contract_lane::<m![1], m![1 # 8]>(LaneMode::Interleaved)
        .cast::<bf16, m![1 # 16]>() // f32 누산기 출력을 bf16으로
        .commit_trim::<m![1 # 8]>()
        .commit();

    // DM → HBM
    result.to_hbm(&mut device.tdma)
}
```

main 컨텍스트 체인의 모양 변화 (슬라이스 하나 기준, bf16 = 2 B)

호출	Time 크기	Packet	설명
<code>fetch::<m![1], m![A]>()</code>	1	A (4,096 B)	패킷이 한 번에 읽는 최대 32 B보다 크므로 32 B씩 128번 읽는다
<code>collect::<m![A / 16], m![A % 16]>()</code>	128	16개 (32 B)	플릿 128개로 쪼갬
<code>contract_outer::<m![A / 32], m![A % 32]>(&rhs)</code>	64	32개 (64 B)	플릿 둘을 한 패킷으로 묶고 TRF의 같은 위치 값과 곱한다. 곱은 f32
<code>contract_packet::<m![1]>()</code>	64	1	패킷 안 곱 32개를 트리로 합산
<code>contract_time::<m![1]>()</code>	1	1	64사이클의 부분합을 누산기에 모은다
<code>contract_lane::<m![1], m![1 # 8]>(Interleaved)</code>	1	1 # 8	레인을 패킷 쪽으로 접는다. f32 8칸 중 1칸이 유효
<code>cast::<bf16, m![1 # 16]>()</code>	1	1 # 16	f32 → bf16. 다시 32 B 플릿 하나로 패딩
<code>commit_trim::<m![1 # 8]>(), commit()</code>	-	1 # 8	bf16 8칸(16 B)만 DM에 기록

sub 쪽 체인의 `m![{ Time }, A / 16]` 에 쓰인 중괄호는 매핑 안에서 타입 별칭을 부르는 **이스케이프 문법**이다(§ 4.3). `to_trf()` 가 만든 TRF 텐서는 레인 하나에 bf16 2048개(4 KB)를 담아 레인당 한도 8 KB 안에 든다. 이 예제는 `slice = m![1 # 256]` 이라 슬라이스 256개 중 하나만 쓴다. 교육용으로 단순화한 것이다.

호스트 오라클은 두 벡터를 f32로 곱해 더한 뒤 `bf16::from_f32` 로 반올림하고, 결과가 기대값의 2% 또는 0.5 이내인지 확인한다. 부동소수점 누산 순서와 bf16의 낮은 정밀도 때문에 정확한 일치 대신 허용 오차를 둔다.

더 깊이: 왜 64바이트 패킷으로 묶나

Contraction Engine의 곱셈기는 내부적으로 항상 64 B 패킷을 처리한다. 32 B 플릿을 그대로 넣으면(PackSize 1) 절반은 0이 곱해져 처리량이 반으로 준다. `contract_outer::<m![A / 32], m![A % 32]>` 처럼 OutPacket을 64 B로 고르면 플릿 두 개가 패킷 하나로 묶여 곱셈기를 모두 쓴다(§ 8.3).

3.5 패턴 4: GEMV, 출력 축을 슬라이스에

목표는 $y_i = \sum_j A_{ij} \cdot x_j$ 이고, 원문은 이 패턴이 LLM 디코드에 쓰인다고 적는다. 전략은 출력 축 I를 슬라이스에 매핑해 **슬라이스 하나가 행 하나를 계산하게** 하는 것이다. 행들은 서로 독립이지만 모든 행이 벡터 x 전체를 필요로 하므로, x는 모든 슬라이스에 브로드캐스트한다.



각 슬라이스: Time = J / 32 (64사이클), Packet = J % 32 (bf16 32개 = 64 B). x는 모든 슬라이스의 TRF에 같은 값으로 들어간다.

그림 11. GEMV의 배치. 출력 축 I가 슬라이스 차원을 차지하므로 256개 행이 동시에 계산된다. 리듀스 축 J는 한 슬라이스 안에서 시간 (64사이클)과 패킷(32개)으로 나뉜다.

```
axes![I = 256, J = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![I];           // 출력 차원을 슬라이스에 분배
pub type Time = m![J / 32];       // 리듀스 차원의 시간 반복
pub type Packet = m![J % 32];     // 리듀스 차원의 패킷 크기
pub type Lane = m![1];

#[device(chip = 1)]
pub fn gemv_kernel(
    device: &mut Device,
    matrix: &HbmTensor<bf16, Chip, m![I, J]>,
    vector: &HbmTensor<bf16, Chip, m![J]>,
) -> HbmTensor<bf16, Chip, m![I]> {
    // HBM → DM
    let matrix: DmTensor<bf16, Chip, Cluster, Slice, m![J]> = matrix.to_dm(&mut device.tdma);
```

```

let vector: DmTensor<bf16, Chip, Cluster, Slice, m![J]> = vector.to_dm(&mut device.tdma);

// 벡터를 TRF에 올린다
let vector_trf: TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![J]> = device
    .sub
    .begin(vector.view())
    .fetch::<m![1], m![J]>()
    .collect::<m![J / 16], m![J % 16]>() // 32 B 플릿으로 쪼갬다
    .to_trf();

// 행렬 × 벡터. 핵심 차이: I는 슬라이스로 보존되고 J는 리듀스된다
let result: DmTensor<bf16, Chip, Cluster, Slice, m![1 # 4]> = device
    .main
    .begin(matrix.view())
    .fetch::<m![J / 16], m![J % 16]>()
    .collect::<m![J / 16], m![J % 16]>()
    .contract_outer::<Time, Packet, _, _, _>(&vector_trf)
    .contract_packet::<m![1]>()
    .contract_time::<m![1]>()
    .contract_lane::<m![1], m![1 # 8]>(LaneMode::Interleaved)
    .cast::<bf16, m![1 # 16]>()
    .commit_trim::<m![1 # 4]>()
    .commit();

// 결과를 HBM으로
result.to_hbm(&mut device.tdma)
}

```

매핑을 해독해 보자.

- `Slice = m![I]`: $I = 256$ 이므로 패딩 없이 슬라이스 256개와 정확히 맞는다.
- 행렬의 DM 타입은 `DmTensor<bf16, Chip, Cluster, m![I], m![J]>` 다. 슬라이스 i 에는 행 i 의 원소 2048개(4 KB)가 들어간다.
- 벡터의 원래 타입 `HbmTensor<bf16, Chip, m![J]>`에는 I 축이 없다. 그런데 목적지 타입의 Slice는 `m![I]` 다. 이 불일치가 곧 **브로드캐스트**다. DMA가 x 를 256개 슬라이스에 똑같이 복사해 넣는다.
- `main` 체인의 흐름은 내적과 같다. 한 슬라이스에서 64사이클 동안 64 B 패킷을 처리하고, 패킷 안 32개를 트리로, 64사이클을 누산기로 더해 스칼라 하나를 만든다. 차이는 256개 슬라이스가 동시에 서로 다른 행을 처리한다는 점뿐이다.
- `commit_trim::<m![1 # 4]>()`: bf16 4칸(8 B)을 기록한다. 커밋은 유효 폭이 $8 \cdot 16 \cdot 24 \cdot 32$ B 중 하나여야 하므로 값 하나(2 B)가 아니라 8 B 단위로 쓴다. 마지막 `to_hbm`이 이를 `m![I]` 배치로 정리한다.

3.6 패턴 5: GEMM, 타일과 레인

목표는 $C_{ij} = \sum_k A_{ik} \cdot B_{kj}$ 다. A에는 J 축이, B에는 I 축이 없으므로 A는 J 축으로, B는 I 축으로 브로드캐스트된다. 원문은 두 출력 축을 모두 공간에 싣는다. `type Slice = m![I / 32, J / 32]`로 출력 행렬을 16×16 개 블록으로 나눠 블록 하나를 슬라이스 하나에 맡기고, J 의 가장 안쪽 조각(`J % 8`)은 **레인** 8개에 맡긴다.

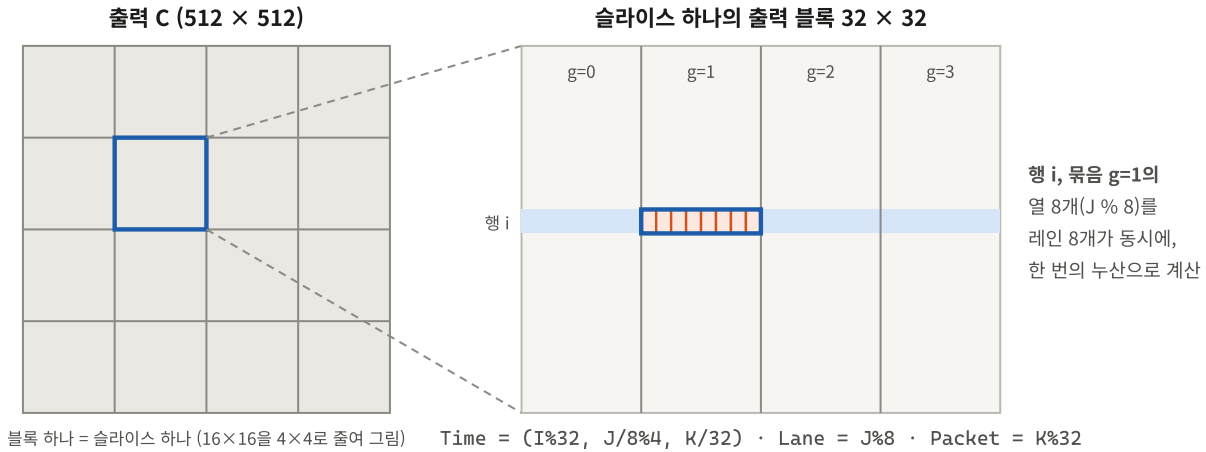


그림 12. GEMM의 배치. 출력 블록의 행($I \% 32$)과 열 묶음($J / 8 \% 4$)은 시간으로 반복하고, 묶음 안의 열 8개는 레인 8개가 동시에 계산한다. K축은 패킷(32개)과 시간(2단계)으로 나뉘어 리듀스된다.

원문 확인

원문 코드 주석은 "각 슬라이스가 16×16 출력 타일을 맡는다"고 적지만, 매핑을 계산하면 **슬라이스 격자가 16×16** ($I / 32$ 와 $J / 32$ 가 각각 16개)이고 슬라이스 하나가 맡는 출력 블록은 32×32 ($I \% 32, J \% 32$)다. 또 원문 설명은 Switch Engine이 B의 타일을 각 슬라이스로 보낸다고 하지만, 예제 코드에는 `.switch()` 호출이 없다. 코드에서는 `to_dm`의 목적지 매핑(`Slice = m![I / 32, J / 32]`, `Element = m![J \% 32, K]`)이 B의 J 타일을 해당 슬라이스들에 복사해 넣는다.

```
axes![I = 512, J = 512, K = 64];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
// 출력 차원 I와 J를 슬라이스에 분배
pub type Slice = m![I / 32, J / 32];
pub type Lane = m![J \% 8];

#[device(chip = 1)]
pub fn gemm_kernel(
    device: &mut Device,
    a: &HbmTensor<bf16, Chip, m![I, K]>,
    b: &HbmTensor<bf16, Chip, m![J, K]>,
) -> HbmTensor<bf16, Chip, m![I, J]> {
    // HBM → DM
    let a: DmTensor<bf16, Chip, Cluster, Slice, m![J \% 32, K]> = a.to_dm(&mut device.tdma);
    let b: DmTensor<bf16, Chip, Cluster, Slice, m![J \% 32, K]> = b.to_dm(&mut device.tdma);

    // 행렬 B를 TRF에 올린다. 슬라이스마다 K 전체와 자기 블록의 J 부분만 받는다
    let b_trf: TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![J / 8 \% 4, K]> = device
        .sub
        .begin(b.view())
        .fetch::<m![J \% 8, J / 8 \% 4], m![K]>()
        .collect::<m![J \% 8, J / 8 \% 4, K / 16], m![K \% 16]>()
        .to_trf();

    // A × B: K를 리듀스하고 I와 J는 보존한다
    let result: DmTensor<bf16, Chip, Cluster, Slice, m![I \% 32, J \% 32]> = device
        .main
        .begin(a.view())
        .fetch::<m![I \% 32, J / 8 \% 4], m![K]>()
        .collect::<m![I \% 32, J / 8 \% 4, K / 16], m![K \% 16]>()
```



```

.contract_outer::<m![I % 32, J / 8 % 4, K / 32], m![K % 32], _, _, _>(&b_trf)
.contract_packet::<m![1]>()
.contract_time::<m![I % 32, J / 8 % 4]>()
.contract_lane::<m![I % 32, J / 8 % 4], m![J % 8]>(LaneMode::Interleaved)
.cast::<bf16, m![J % 8 # 16]>()
.commit_trim::<m![J % 8]>()
.commit();

```

```

// 결과를 HBM으로
result.to_hbm(&mut device.tdma)
}

```

슬라이스 하나 기준 추적

단계	모양	설명
A의 DM 원소	$m![I \% 32, K] = 32 \times 64$	블록의 행 32개. J축이 없으므로 같은 I 블록의 슬라이스 16개가 같은 A를 가진다(브로드캐스트)
B의 DM 원소	$m![J \% 32, K] = 32 \times 64$	블록의 열 32개. 같은 J 블록의 슬라이스 16개가 같은 B를 가진다
B의 TRF	Lane $J \% 8$, Element $m![J / 8 \% 4, K]$	레인 I에는 열 번호가 $8g + I$ 인 B 행 4개가 들어간다. 레인당 bf16 256개 = 512 B
<code>fetch::<m![I % 32, J / 8 % 4], m![K % 32]>()</code>	Time 32×4 , Packet 64	A에는 J가 없으므로 Time의 $J / 8 \% 4$ 는 같은 행을 네 번 다시 읽는 시간 브로드캐스트 다
<code>collect::<..., K / 16>, m![K % 16]</code>	Time $32 \times 4 \times 4$, Packet 16	32 B 플릿으로 쪼갬
<code>contract_outer::<..., K / 32>, m![K % 32]</code>	Time $32 \times 4 \times 2$, Packet 32, Lane 8	64 B 패킷을 레인 8개로 브로드캐스트하고, 레인 I에서는 TRF의 열 $8g + I$ 과 곱한다
<code>contract_packet::<m![1]>()</code>	Packet 1	$K \% 32$ 를 트리로 합산
<code>contract_time::<m![I % 32, J / 8 % 4]>()</code>	Time 32×4	$K / 32$ (2단계)를 누산. 행 i와 묶음 g는 살아남는다
<code>contract_lane::<..., m![J % 8]>(Interleaved)</code>	Packet $J \% 8$	레인 8개의 결과가 한 패킷(f32 8개 = 32 B)이 된다
<code>cast</code> → <code>commit_trim</code> → <code>commit</code>	Element $m![I \% 32, J \% 32]$	Time (i, g)와 Packet I을 합치면 $J \% 32 = 8g + I$ 이 되어 32×32 블록이 완성된다

원문은 이 뒤에 두 가지 발전형을 언급한다. **Blocked GEMM**은 DM 용량을 넘는 행렬을 위해 K를 시간으로 나누고 I와 J를 칩에 걸쳐 나누는 타일링을 더하고, **Flash Attention**은 GEMM, Vector Engine의 softmax, main/sub 선적재를 트랜스포머 어텐션 헤드 하나에 조합한다. 두 페이지는 현재 원문 목차에 따로 공개되어 있지 않다.

3.7 커널 검증의 네 단계

원문은 검증을 네 단계로 구분하고, 각 단계가 무엇을 증명하지 **않는**지를 분명히 한다.

단계	방법	확인하는 것	확인하지 않는 것
1. 커널 컴파일	<code>cargo furiousa-opt compile</code>	모든 <code>#[device]</code> 본문이 번역·검증되는지. 매핑과 shape 오류를 실행 없이 잡는다	계산 결과가 맞는지

단계	방법	확인하는 것	확인하지 않는 것
2. CPU 실행	평범한 <code>cargo run</code> , <code>cargo test</code>	호스트에서 계산한 결과가 호스트 오라클과 같은지	하드웨어 동작, 성능
3. NPU 실행	<code>cargo furiousa-opt</code> + SDK + 실제 NPU	컴파일된 EDF가 실제 하드웨어에서 동작하는지	-
4. 스케줄 데이터	스케줄 JSON 덤프와 makespan 비교	정적 실행 계획, 연산 사이의 겹침과 대기	그 자체로는 처리량 측정이 아니다

주의

스케줄의 makespan(마지막 사이클)은 릴리스, shape, 데이터 타입, 매핑을 모두 고정했을 때만 비교한다. 비교할 때마다 소스 리비전, 백엔드, shape, 매핑, 오라클 결과, 스케줄 파일을 기록한다. 튜닝 절차는 12장에서 다룬다.

3.8 다섯 패턴에서 배울 것

- **체인의 모양은 늘 같다.** `to_dm` 으로 배치하고, `begin` → `fetch` → `collect` 로 스트림을 만들고, 계산 엔진을 거쳐 `commit_trim` → `commit` 으로 쓰고, `to_hbm` 으로 돌려놓는다. 커밋 전 `commit_trim` 은 생략할 수 없다.
- **고정 피연산자는 레지스터 파일에.** Vector Engine의 두 번째 피연산자는 VRF, Contraction Engine의 두 번째 피연산자는 TRF에 sub 컨텍스트가 미리 올린다.
- **출력 축은 공간에, 리듀스 축은 패킷과 시간에.** GEMV는 I를 Slice에, GEMM은 I·J를 Slice와 Lane에 두었다. 리듀스 축 J·K는 Packet(트리로 한 번에)과 Time(누산기로 여러 사이클)에 나뉘었다.
- **폭은 넓혔다가 줄인다.** 곱은 i32·f32 누산기에서 더하고, 저장 전에 cast로 bf16 같은 작은 형식으로 줄인다.
- **패딩으로 하드웨어 개수에 맞춘다.** `m![1 # 2]`, `m![A / 8 # 256]`, `m![1 # 8]` 처럼 개수와 32 B 플릿 크기를 맞추는 패딩이 곳곳에 있다.

스스로 점검

1. 상수 덧셈 커널에서 $A = 4096$ 으로 바꾸고 슬라이스 256개를 모두 쓰려면 `slice` 와 `element` 매핑을 어떻게 바꿔야 하나? 그러면 슬라이스당 몇 바이트인가?
2. 원소별 곱셈에서 rhs를 VRF 대신 상수로 바꾸려면 어느 줄을 어떻게 고치면 되나?
3. GEMV의 벡터 x를 256개 슬라이스에 복제한 것은 어느 호출인가?
4. GEMM에서 레인 8개가 모두 쓰이는 이유를 매핑으로 설명해 보라.

답: ① `slice = m![A / 16]`, `element = m![A % 16]`. i32 16개 = 64 B라 한 패킷이 플릿 두 개가 된다. ② sub 체인을 없애고 `vector_fxp(FxpBinaryOp::MulInt, 상수)` 로. ③ `vector.to_dm(&mut device.tdma)`. 원래 타입에 I축이 없는데 목적지 Slice가 `m![I]` 라서 복제된다. ④ TRF의 `lane = m![J % 8]` 이 크기 8이고, 출력 패킷도 `J % 8` 로 레인을 접는다.

4장

텐서 매핑

원문: Mapping Tensors · Mapping Expressions · Spatial and Temporal Dimensions · Tensor Semantics

원문은 매핑을 "논리 순서와 물리 배치의 유일한 기준(source of truth)"이라고 부른다. 텐서의 값을 버퍼의 어느 칸에, 어느 칩·클러스터·슬라이스에, 스트림의 몇 번째 사이클과 몇 번째 패킷 자리에 둘지를 모두 매핑이 정한다. 이 장은 매핑 표현식 `m![...]`의 생성자를 하나씩 익히고, 매핑이 여러 하드웨어 차원으로 쪼개지는 방식, 그리고 "텐서가 값을 담는다"는 말의 정확한 뜻을 다룬다.

이 장에서 배우는 것

- 같은 텐서를 다른 순서로 저장하면 무엇이 달라지는가(major/minor, 지역성)
- 매핑이 버퍼 위치를 `cell` (인덱스·패딩·범위 밖)로 바꾸는 방식
- 생성자: 심볼, 쌍(,), 항등(1), 패딩(#), 리사이즈(=), 스트라이드(/), 모듈로(%), 타일, 이스케이프({ })
- 디바이스 텐서가 매핑을 Chip·Cluster·Slice·Lane·Time·Packet으로 나누는 규칙과 크기 제약
- 값 하나의 물리 좌표가 HBM → DM → 스트림으로 가며 어떻게 바뀌는지
- "담는다(holds)", "명세한다(specifies)", "수학적 텐서 이동"의 정의

4.1 왜 매핑인가: 저장 순서와 지역성

텐서는 값을 갖지만 **고유한 저장 순서는 없다**. 순서는 매핑이 고른다. 하드웨어는 인접한 값들이 버퍼에서 연속한 자리에 있을 때 가장 효율적으로 접근한다. 매핑의 가장 바깥 차원은 **major**로 가장 느리게 바뀌고, 가장 안쪽 차원은 **minor**로 가장 빠르게 바뀐다. C 배열의 행 우선 순서에서 행이 major, 열이 minor인 것과 같다.

원문의 예로 H(6행) × W(8열) 텐서를 세 가지로 저장해 보자(그림 13).

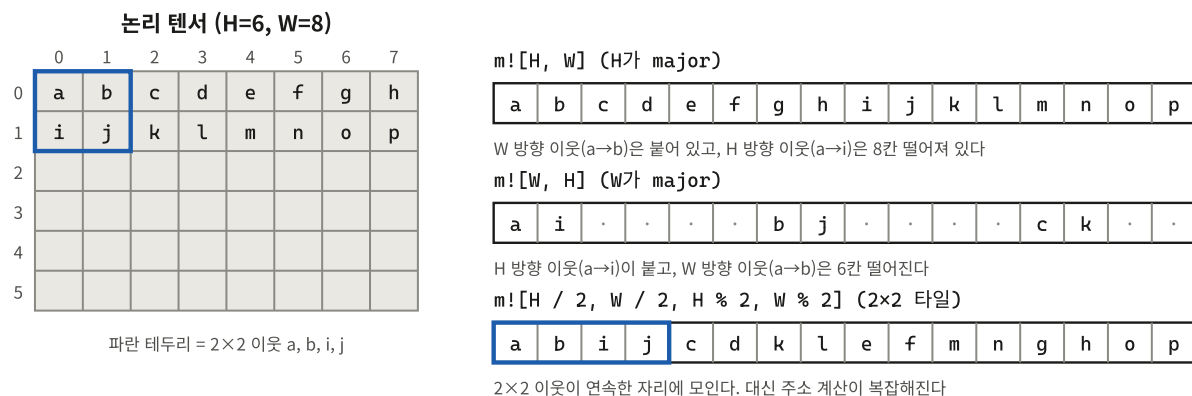


그림 13. 같은 값, 다른 물리 순서. 매핑의 바깥 항(major)은 천천히, 안쪽 항(minor)은 빠르게 바뀐다. 어느 순서가 좋은지는 접근 패턴에 달려 있다. 원문은 타일 배치가 지역성의 분포를 바꿀 뿐 그 자체로 더 빠른 접근을 보장하지는 않으며, 쪼갬 차원을 표현하는 매핑 상태가 추가 비용이라고 적는다.

매핑은 이후의 실행에도 제약을 건다. 할당된 뒤에 표현의 순서를 바꾸려면 보통 데이터를 복사하거나 전치해야 하므로, **할당 시점의 선택이 이후 연산에 영향을 준다**. 하드웨어의 기하 구조, 정렬, 스케줄링은 컴파일러가 낮은 수준으로 옮길 때 고려하는 구조적 제약으로 남는다. 매핑은 낱것의 주소 계산이 아니라 논리 순서를 표현하고, 주소 정렬 검사는 장치별로

따로 적용된다. 정렬이 어긋난 접근은 Read-Modify-Write를 부를 수 있으며, 원문은 영향받은 DM 접근에서 약 50배 느려진 사례가 있었다고 적는다.

핵심

매핑은 순서와 배치에 대한 완전한 기술적 설명이고, **물리 표현(physical representation)**은 그 매핑으로부터 만들어진 구체적인 버퍼, 장치 배치, 스트림 분해다. 같은 값은 논리 텐서를 바꾸지 않은 채 서로 다른 물리 표현 사이를 옮겨 다닐 수 있다.

원문은 이동이나 계산 연산을 고르기 전에 각 축을 어느 차원에 돌지부터 정하라고 권한다. 차원마다 정의되는 곳과 리듀스되는 곳이 정해져 있다.

차원	종류	정의되는 곳	리듀스하는 엔진
Chip	공간	HBM, DM, 스트림	DMA 재분배 + Vector Engine
Cluster	공간	DM, 스트림	DMA 재분배 + Vector Engine
Slice	공간	DM, 스트림	Vector Engine(inter-slice reducer)
Lane	공간	TRF	Contraction Engine
Time	시간	스트림	Contraction Engine(Time Reducer)
Packet	공간	스트림	Contraction Engine(Packet Reducer)

4.2 Cell: 버퍼 위치 하나가 가리키는 것

매핑 표현식은 Rust 타입이고, 모든 매핑은 `M` 트레이트를 구현한다. 핵심은 두 가지다. 버퍼 크기 `SIZE` 와, 버퍼 위치 `i` 를 `Cell` 로 바꾸는 `map(i)` 이다.

```
pub trait M: Debug + Clone {
    const SIZE: usize;           // 이 매핑의 버퍼 크기
    fn to_value() -> Mapping;    // 타입을 값으로 변환
    fn map(i: usize) -> Cell;    // 버퍼 위치 → Cell
}

pub enum Cell { Index(Index), Padding(PaddingKind), OutOfBounds }
```

`Cell` 은 세 종류다. 좌표를 해석하기 전에 먼저 종류를 확인한다.

- `Cell::Index`: 살아 있는 텐서 인덱스. 매핑의 항(term)을 그대로 보존하며, `Index::finalize()` 가 스칼라 좌표를 검증한다(모듈로 초과 같은 경우 에러를 낼 수 있다).
- `Cell::Padding(kind)`: 패딩 영역 안이지만 실제 값이 아닌 자리.
- `Cell::OutOfBounds`: 매핑의 범위 밖.

축 항이 하나도 없는 `Cell::Index(Index::new())` 도 유효한 살아 있는 결과이며, 패딩이나 범위 밖과는 다르다. 인덱스는 `i!` 매크로로 만든다. `i![A: 2, B: 3]` 은 `A = 2, B = 3` 인 인덱스이고, 적은 순서가 major에서 minor 순서다. 패딩이 섞이면 결합이 교환법칙을 따르지 않으므로 순서를 함부로 바꾸지 않는다.

```
axes![A = 4];
type E = m![A];
assert_eq!(E::map(0), i![A: 0]);           // 살아 있는 인덱스
assert_eq!(E::map(4), Cell::OutOfBounds); // 범위 밖
```

가장 단순한 구체 타입은 `HostTensor<D, E>` 다. 원소 타입이 `D` 인 호스트 버퍼이고, 표현이 매핑 `E` 하나로 완전히 정해진다. 버퍼 크기는 `E::SIZE` 이고, i 번째 원소는 `E::map(i)` 가 가리키는 인덱스의 값을 담는다. 예를 들어 `HostTensor<bf16, m![A, B]>` ($A = 8, B = 512$)는 bf16 4,096개를 담는다. 범위 밖 위치는 읽기라면 `PaddingKind::Top`, 쓰기라면 `PaddingKind::Bottom` 으로 마무리한다.

4.3 매핑 생성자

매핑 표현식은 작은 생성자를 조합해 만든다. 이 절 전체에서 원문처럼 `axes![A = 8, B = 512]` 를 가정한다.

심볼: `m![A]`

대문자 한 글자로 된 심볼은 크기를 축 선언에서 가져온다. `m![A]` 는 버퍼 위치 0~7을 $A = 0 \sim 7$ 에 그대로 대응시키고, 8 이상은 범위 밖이다. 원문 주: 모든 심볼 A 에 대해 `i![A: 0]` 은 빈 인덱스 `i![]` 와 같다.

쌍: `m![A, B]`

심표는 두 공간의 데카르트 곱을 선형 버퍼에 펼친다. 왼쪽이 major, 오른쪽이 minor이며 크기는 곱이다. `m![A, B]` 는 $\{A = 8, B = 512\}$ 텐서를 4,096칸에 저장한다. 버퍼 위치 i 는 $A = i / 512, B = i \% 512$ 로 분해된다.

버퍼 위치	인덱스
0	<code>i![A: 0, B: 0]</code>
511	<code>i![A: 0, B: 511]</code>
512	<code>i![A: 1, B: 0]</code>
519	<code>i![A: 1, B: 7]</code> ($519 = 512 \times 1 + 7$)

major 쪽 결과가 패딩이면 그 종류가 먼저 이긴다. `m![A, B, C, D]` 는 오른쪽 결합으로 `Pair<A, Pair<B, Pair<C, D>>>` 가 된다.

항등: `m![1]`

`m![1]` 은 크기 1인 버퍼로 위치 0을 빈 인덱스에 대응시킨다. 쌍의 항등원이어서 `m![1, A]` 와 `m![A, 1]` 은 `m![A]` 와 같다. 더 일반적으로 **브로드캐스트 매핑**은 크기 안의 모든 위치에 같은 빈 인덱스를 돌려준다. `m![256]` 같은 표기가 "256개 자리 모두 같은 값"을 뜻하는 이유다.

패딩: `#`

패딩은 쓰지 않는 버퍼 공간을 덧붙여 하드웨어 요구에 맞춘다. 원문의 예: DMA 엔진이 행을 64바이트 경계에서 시작하길 요구할 때, `axes![C = 13, D = 61]` 의 `m![C, D]` 는 61이 64로 나누어떨어지지 않아 행이 어긋난다. `m![C, D # 64]` 로 쓰면 행마다 3칸을 덧붙여 64바이트 경계에 맞춘다.

위치	m![C, D # 64]의 결과
0	i![C: 0, D: 0]
60	i![C: 0, D: 60]
61~63	Cell::Padding(PaddingKind::Top)
64	i![C: 1, D: 0]

패딩 자리의 **내용에 대한 약속**도 타입의 일부이며, 세 종류가 있다.

표기	종류	뜻
m![A # m], m![A #{*} m]	Top	접근할 수 있지만 값은 임의. 날것의 DM 텐서가 이 종류다. #의 기본값
m![A #{0} m]	Zero	접근할 수 있고 0임이 알려져 있다. 매핑은 이 성질을 기록할 뿐 직접 0을 채우지는 않는다
m![A #{!} m]	Bottom	접근할 수 없다. 읽거나 쓰면 정의되지 않은 동작. 컴파일러가 피해야 할 주소를 표현한다

리사이즈: =

리사이즈는 새 크기 너머의 인덱스를 잘라 매핑을 더 작은 논리 크기로 제한한다. 패딩이 버퍼를 늘린다면 리사이즈는 논리 뷰를 줄인다. axes![C = 2, D = 3]에서 m![C, D = 2]는 D의 앞 두 값만 쓰므로 크기 4다(위치 0~3이 (0,0), (0,1), (1,0), (1,1)).

스트라이드와 모듈로: /, %

스트라이드와 모듈로는 축 하나를 바깥(블록 번호)과 안쪽(블록 안 위치) 두 차원으로 쪼갬다. B = 512를 64개씩 8블록으로 나누면 m![B / 64, B % 64]이고, 이는 m![B]와 같다. 위치 130은 블록 2의 위치 2, 즉 B = 64 × 2 + 2 = 130이다. 스트라이드의 크기는 L / F(나누어떨어져야 함), 모듈로의 크기는 F다. 모듈로는 버퍼 크기를 유지한 채 같은 크기 블록으로 나누고, 리사이즈는 버퍼를 잘라낸다는 점이 다르다.

이 연산들은 겹쳐 쓸 수 있고, 겹치면 **비트 배치**를 바꿀 수 있다. 원문의 예 m![B / 64, B % 32, B / 32 % 2]는 버퍼 위치의 비트를 [8:6]_[5:1]_[0]으로, 텐서 인덱스 B의 비트를 [8:6]_[5]_[4:0]로 묶는다(그림 14 오른쪽). 원문은 이런 비트 재배치가 뱅크 인터리빙이나 캐시 효율을 위해 주소 비트를 재배열하는 하드웨어 표현에 자연스럽게 대응한다고 설명한다. 6장에서 DM과 HBM의 주소 비트가 실제로 이렇게 설계되어 있음을 본다.



그림 14. 스트라이드와 모듈로. 왼쪽은 축 하나를 블록 번호와 블록 안 위치로 쪼갠 모습이고, 오른쪽은 세 조각으로 쪼갠 매핑이 버퍼 위치와 텐서 인덱스 사이에서 비트 묶음의 자리를 바꾸는 모습이다.

타일: 복사 없는 부분 뷰

타일은 인덱스가 붙은 **뷰**로 구현된다. `tile`은 매핑 분할이 올바른지 검사한 뒤 데이터를 복사하지 않고 메타데이터 뷰만 만든다.

```
axes![A = 8, B = 512];
let tensor = HbmTensor::<bf16, m![1], m![A, B]>::new();
let view = tensor.view();
let tile01 = view.tile::<m![B], 2, m![A, B = 2 # 512]>(0); // B = 0..2
let tile23 = view.tile::<m![B], 2, m![A, B = 2 # 512]>(2); // B = 2..4
```

- 첫째 인자 `m![B]`: 타일 위치를 찾는 매핑(I).
- 둘째 인자 `2`: 타일에 들어가는 I 칸의 수(LEN).
- 셋째 인자 `m![A, B = 2 # 512]`: 뷰의 매핑. `B = 2 # 512`는 "물리적으로는 512칸 범위 안에서 살아 있는 B 자리가 2개"라는 뜻이다. 이 크기 정보가 없으면 원본 매핑과의 분할 검사를 통과하지 못한다.
- `(0)`, `(2)`: B축을 따라 켄 논리 시작 좌표. 타일 번호가 아니다.

읽기 뷰는 타일 바깥 칸을 `PaddingKind::Top`으로 채우고, 쓰기 가능한(`mutable`) 뷰는 바깥 칸이 `PaddingKind::Bottom`이어야 한다. 쓰기가 타일 밖으로 새지 못하게 하려는 것이다. HBM과 DM은 계층 전용 타일 래퍼를, TRF·VRF·DPE는 일반 뷰를 쓰며, `HostTensor`에는 뷰·타일 API가 없다.

블록 단위 타일도 가능하다. `view.tile::<m![B / 32], 2, m![A, B / 32 = 2 # 16, B % 32]>(i)`는 32개씩 16블록 가운데 연속한 두 블록을 고른다. `i = 0`이면 블록 `{0, 1}`, `i = 1`이면 `{1, 2}`이므로 이웃한 타일이 한 블록씩 **겹친다**. 블록 하나짜리 타일이라면 `B / 32 = 1`이 항등 `m![1]`로 간단해진다.

원문 확인

원문 본문은 일반 뷰 API를 `TensorView::tile<I, E2, LEN>(start)`로 적지만, 바로 아래 예제 코드는 `tile::<m![B], 2, m![A, B = 2 # 512]>`, 즉 I, LEN, E2 순서로 인자를 넘긴다. 이 안내서는 코드의 순서를 따랐다.

이스케이프: { }

복잡한 매핑은 타입 별칭으로 나눠 이름을 붙이고 종괄호로 불러 쓴다. `type L = m![A]; type R = m![B];`이면 `m![{ L }, { R }]`은 `m![A, B]`와 같다. 3장의 내적 예제에서 본 `m![{ Time }, A / 16]`이 이 문법이다.

생성자 요약

문법	Rust 타입	크기	뜻
<code>m![A]</code>	<code>Symbol</code>	<code>A::SIZE</code>	축 하나를 순서대로
<code>m![L, R]</code>	<code>Pair</code>	$L \times R$	L이 major, R이 minor인 곱
<code>m![1], m![n]</code>	<code>Identity, Broadcast</code>	$1, n$	모든 자리가 같은 빈 인덱스
<code>m![L # n]</code>	<code>Padding</code>	n	n칸으로 늘리고 남은 칸은 패딩(Top/Zero/Bottom)
<code>m![L = n]</code>	<code>Resize</code>	n	앞 n칸만 보는 논리 뷰
<code>m![L / n]</code>	<code>Stride</code>	L / n	n개씩 묶은 블록 번호
<code>m![L % n]</code>	<code>Modulo</code>	n	블록 안의 위치
<code>m![{ T }]</code>	(별칭)	<code>T::SIZE</code>	타입 별칭을 매핑 안에서 사용

4.4 동등한 매핑

서로 다른 생성자 조합이 같은 위치 동작을 낼 수 있다. 원문은 두 매핑 E1, E2가 **동등(equivalent)**하다는 것을 `E1::SIZE == E2::SIZE` 이고 모든 i에서 `E1::map(i) == E2::map(i)` 인 것으로 정의한다. 이는 Rust 값으로서의 구조적 동일성과 다르며, 반사·대칭·추이 관계다. 자주 쓰는 항등식은 다음과 같다.

- **쌍의 항등원:** `E == m![{ E }, 1] == m![1, { E }]`
- **스트라이드-모듈로 분해:** `E::SIZE`가 n 으로 나누어떨어지면 `E == m![{ E } / n, { E } % n]`
- **쌍의 사영:** `m![{ { A }, { B } } / B::SIZE] == m![A], m![{ { A }, { B } } % B::SIZE] == m![B]`
- **쌍의 결합법칙:** `m![{ { E1 }, { E2 }, { E3 } }] == m![{ { E1 }, { E2 } }, { E3 }] == m![{ E1 }, [{ E2 }, { E3 }]]`
- **역등 연산:** `E == m![{ E } / 1] == m![{ E } # E::SIZE] == m![{ E } = E::SIZE]`
- **1로 나눈 나머지:** `m![E % 1] == m![1]`

매핑은 표준 형태로 **정규화**할 수 있다. 원문은 정규화가 두 번 적용해도 같고(역등), 테스트 코퍼스의 위치·패딩 동작을 보존하며, 매핑 표현을 왕복해도 유지된다는 점만 테스트되었다고 밝힌다. 동등성을 증명하거나 유일한 표현을 보장한다는 주장은 하지 않는다.

4.5 공간 차원과 시간 차원

`HostTensor<D, E>` 는 매핑 하나로 표현이 끝나지만, 디바이스 텐서는 매핑을 전용 차원 여러 개로 나눈다. 공간 차원 `Chip`, `Cluster`, `Slice` 는 하드웨어 계층에 데이터를 분배하고, 스트림 텐서에서는 `Packet` 이 한 시간 반복 안의 병렬 전달 크기를 정한다. 시간 차원 `Time` 은 전달 반복의 순서를 정한다.

타입(원소 타입 D 뒤의 매개변수들)	뜻
<code>HostTensor<D, Element></code>	호스트 값. 매핑 하나
<code>HbmTensor<D, Chip, Element></code>	칩마다 원소 배치. 구체적인 HBM 주소를 가진다
<code>DmTensor<D, Chip, Cluster, Slice, Element></code>	슬라이스마다 원소 배치. 주소는 백엔드가 줄 수 있다

타입(원소 타입 D 뒤의 매개변수들)	뜻
<code>TrfTensor<D, Chip, Cluster, Slice, Lane, Element></code>	슬라이스마다 레인 8개에 분배
<code>VrfTensor<D, Chip, Cluster, Slice, Element></code>	슬라이스마다 원소 배치
<code>TuTensor<'l, P, D, Chip, Cluster, Slice, Time, Packet></code>	Tensor Unit을 흐르는 스트림. P는 파이프라인 위치

예를 들어 `HbmTensor<bf16, m![A], m![B]>`는 $8 \times 512 = 4,096$ 개 원소를 칩 8개에 512개씩 나누며, i 번째 칩의 j 번째 원소에는 `i![A: i, B: j]`의 값이 있다. 같은 단계의 모든 단위는 같은 매핑을 공유한다고 가정한다. 매핑 매개변수는 물리 표현을 정하고, 주소는 저장소가 따로 소유한다.

크기 제약

단위	개수	제약	패딩 예
<code>Chip</code>	시스템에 따라	<code>Chip::SIZE == NUM_CHIPS</code>	<code>m![1 # NUM_CHIPS]</code>
<code>Cluster</code>	칩당 2	<code>Cluster::SIZE == 2</code>	<code>m![1 # 2]</code>
<code>Slice</code>	클러스터당 256	<code>Slice::SIZE == 256</code>	<code>m![X / N # 256]</code>

하드웨어보다 적은 단위를 쓸 때는 어느 차원이든 `#`로 패딩한다. 원소 크기에도 제약이 있다. `Element::SIZE × size_of::<D>() ≤ 512 KB`가 단위당 SRAM 용량을 넘으면 안 된다.

타입	단위 용량	제약
<code>DmTensor</code>	슬라이스당 512 KB	<code>Element::SIZE × size_of::<D>() ≤ 512 KB</code>
<code>TrfTensor</code>	레인당 8 KB	<code>Lane::SIZE ≤ 8, Element::SIZE × size_of::<D>() ≤ 8 KB</code>
<code>VrfTensor</code>	슬라이스당 8 KB	<code>Element::SIZE × size_of::<D>() ≤ 8 KB</code>

주소 정렬 제약은 저장소 API마다 다르므로, 원문은 한 가지 규칙을 모든 텐서에 가정하지 말고 계층별 생성자와 백엔드 검사를 확인하라고 한다.

시간 차원과 파이프라인 위치

`TuTensor`는 SRAM 타입과 같은 `Chip·Cluster·Slice` 차원을 유지하고 `Time`과 `Packet`을 더한다. 공간 차원과 달리 **Time에는 하드웨어 크기 한도가 없다**. 처리할 데이터가 많아지면 늘어난다. `Packet`은 한 시간 반복에서 슬라이스마다 몇 개의 원소를 받는지를 정한다.

`TuTensor`의 `const` 매개변수 `P: Position`은 스트림이 파이프라인의 어디에 있는지를 기록하는 타입스테이트다. 값은 `B`, `egin`, `Fetch`, `Switch`, `Collect`, `Contraction`, `Reduce`, `Cast`, `Transpose`다. Vector Engine은 분가·ALU 같은 고유 상태를 추적해야 해서 별도 타입스테이트(`VectorBranchTensor` 등)를 쓰고, Commit 뒤의 데이터는 멈춰 있으므로 `TuTensor`가 아니라 `DmTensor`가 된다.

```
axes![N = 4, C = 64, H = 32, W = 32];
type T<'l> = TuTensor<
  'l,
  { Position::Fetch }, // Fetch Engine의 출력
  bf16,
  m![1], // Chip: 칩 하나
  m![1], // Cluster: 클러스터 하나
  m![C / 2], // Slice: 채널 64개를 슬라이스 32개에 분배
  m![N, H, W], // Time: 배치(N)와 공간(H, W)을 반복
  m![C % 2], // Packet: 사이클마다 채널 2개
>;
```

이 스트림은 $\{N = 4, C = 64, H = 32, W = 32\}$ 텐서를 슬라이스 32개로 흘린다. Time의 크기는 $4 \times 32 \times 32 = 4,096$ 이므로 시간 반복이 4,096번이고, 반복마다 슬라이스 하나가 채널 2개를 받는다. 슬라이스 32개가 병렬이므로 반복마다 채널 $32 \times 2 = 64$ 개가 처리된다. (원문의 이 예제는 설명용이어서 Slice 크기 32가 하드웨어 개수 256과 다르다.)

4.6 값 하나의 좌표 추적: NCHW

원문은 NCHW 텐서 하나를 세 가지 표현으로 옮기며 좌표가 어떻게 바뀌는지 추적한다. 처음에는 배치 N을 칩 4개에 나눈 HBM 표현이다.

```
axes![N = 4, C = 64, H = 32, W = 32];
type Hbm = HbmTensor<bf16, m![N], m![C, H, W]>; // 배치를 칩 4개에
type Dm = DmTensor<bf16, m![1], m![1], m![C / 2], m![N, H, W, C % 2]>; // 칩 1개, 채널 2개당 슬라이스 1개
// 스트림 T: Slice = m![C / 2], Time = m![N, H, W], Packet = m![C % 2]
```

추적할 값: Index (N=1, C=17, H=3, W=5) — 논리 인덱스는 끝까지 같다

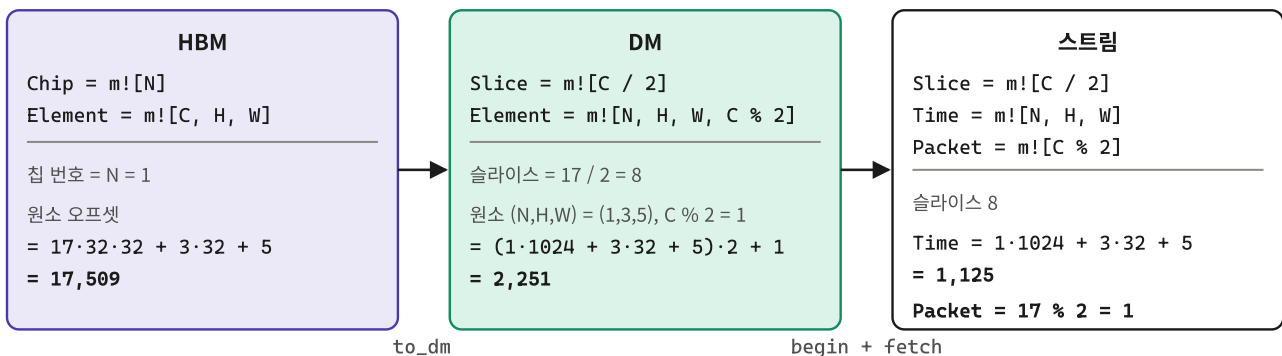


그림 15. NCHW 값 하나의 좌표 추적. 원문이 적은 HBM 오프셋(17,509), DM 슬라이스(8), 스트림 Time(1,125)·Packet(1)에 더해, DM 원소 오프셋을 같은 규칙으로 계산해 넣었다. DM 오프셋 2,251이 Time × 2 + Packet과 같다는 점에 주목하라. DM 배치와 스트림 순서가 일치하므로 Fetch가 주소를 연속으로 읽는다.

값의 물리 좌표는 단계마다 바뀌지만 **Index**는 끝까지 (1, 17, 3, 5)다. 원문은 이 추적을 "매핑을 고르고, 물리 표현을 각각 유도한 뒤, 추적한 인덱스의 값이 기대한 스트림 칸에 도착하는지 확인하라"는 절차로 제시한다.

4.7 텐서 의미론: 담는다, 명세한다

텐서는 HBM, DM, 파이프라인 스트림에 머물고 연산은 이를 변환한다. 원문의 Tensor Semantics 절은 "텐서 변수가 수학적 텐서를 **담는다(holds)**"와 "연산이 수학적 함수를 **명세한다(specifies)**"를 정의한다. 이 정의 덕분에 어떤 매핑과 메모리 계층을 쓰든 "출력이 올바른 수학적 텐서를 담으면 함수가 옳다"고 텐서 수준에서 추론할 수 있다.

- **HostTensor**: `HostTensor<D, E>` 는 모든 버퍼 위치 i 에 대해 $E::\text{map}(i) = \text{Cell}::\text{Index}(ti)$ 일 때 i 번째 원소가 T 의 ti 값을 저장하면 텐서 T 를 담는다.
- **HbmTensor**: `HbmTensor<D, Chip, Element>` 는 매핑을 둘로 나눈다. `Chip`이 칩 번호를 부분 인덱스 ti 로, `Element`가 칩 안 원소 번호를 부분 인덱스 tj 로 보내며, 두 매핑은 서로 다른 축 집합을 덮는다. i 번째 칩의 j 번째 원소가 T 의 $ti + tj$ 값을 저장하면 T 를 담는다.
- **나머지 타입**: 같은 규칙을 차원 수만 늘려 적용한다. 각 원소는 모든 매핑 매개변수가 돌려준 부분 인덱스의 합 위치의 값을 저장한다.

함수 명세는 입력이 담은 텐서로 출력이 무엇을 담는지를 선언하는 것이다. 예를 들어 `elementwise_add`가 $f(T1, T2) = T1 + T2$ 를 명세한다는 것은 "모든 $T1, T2$ 에 대해 lhs가 $T1$ 을, rhs가 $T2$ 를 담으면 반환값이 $T1 + T2$ 를 담는다"는 뜻이다. 특히 $f(T) = T$ 를 명세하는 연산을 **수학적 텐서 이동(mathematical tensor move)**이라 부른다. 표현은 달라도 출력이 입력과 같은 수학적 텐서를 담는다. `.to_dm()`, DMA 전송, Fetch, Commit이 모두 이것이다.

더 깊이: 선형 결합 매핑과 슬라이딩

원문은 여러 차원을 스트라이드와 함께 결합하는 선형 결합 표현 $\$(e1:n1, \dots, ed:nd)$ 도 정의한다. 크기는 $1 + \sum_k (\text{size}(e_k) - 1) \cdot n_k$ 이고, 공간 인덱스 si 와 텐서 인덱스 ti 가 각 항의 가중합으로 대응한다. 외적 합 $e1 * e2$ 는 $\$(e1 : \text{size}(e2), e2 : 1)$ 과 같으며, 축 순서를 바꿀 때 스트라이드를 손으로 고칠 필요가 없어 선호된다.

이 표현은 겹치는 블록을 읽는 **슬라이딩** 연산을 나타낸다. 원소 9개짜리 버퍼로 $\{N=5, F=3\}$ 텐서를 표현하되 각 행이 3원소 창이고 한 칸씩 미끄러진다면, (N, F) 의 원소는 버퍼 위치 $N + 2F$ 에 있다. 크기는 $1 + (5 - 1) \cdot 1 + (3 - 1) \cdot 2 = 9$ 다. 이때 버퍼 위치 4는 $\{N=4\}, \{N=2, F=1\}, \{F=2\}$ 세 인덱스에 동시에 대응하므로 매핑이 일대일이 아니다.

4.8 축을 시간과 공간 사이로 옮기는 네 경로

매핑 차원은 공간 차원(Chip, Cluster, Slice, Packet)이나 시간 루프(Time)에 명시적으로 할당된다. 이 할당은 파이프라인 전체가 아니라 **한 단계**에서만 유효하다. 원문은 축을 Time과 공간 차원 사이로 옮기는 네 경로를 정리한다.

경로	옮기는 방향	쓸 수 있을 때	비용
Switch Engine	Slice $\leftrightarrow$ Time, 양방향	설정 중 하나가 원하는 Slice 배치를 표현할 수 있을 때	$\text{ring_size} \times \text{Time}::\text{SIZE} \times \text{flits_per_packet}$ 사이클의 링 순회
DMA	Chip·Cluster·Slice로, 원하는 배치를 써서	항상	텐서 전체 전송
Inter-slice reducer	Time $\rightarrow$ 리듀스된 축이 비운 Slice 자리	어차피 Slice 축으로 리듀스할 때	리듀스 외 추가 비용 없음
축 올리기(fetch)	Time $\rightarrow$ Chip·Cluster·Slice	선택한 차원이 브로드캐스트를 담고 있을 때	원래 읽기 + DMA·덧셈·SFR 저장 (Slice의 경우)

Switch Engine은 어떤 슬라이스에 원래 없던 값을 가져다줄 수 있으므로 Slice/Time 경계를 넘는 일반 경로다. 다만 슬라이스 사이에서만 동작하므로 축을 Chip이나 Cluster로 옮기려면 원하는 배치를 쓰는 DMA가 필요하다. 축 올리기는 각

칩·클러스터·슬라이스가 이미 원본 영역을 똑같이 갖고 있을 때(브로드캐스트) 서로 다른 읽기 오프셋만으로 축을 Time에서 빼낸다. 데이터 전송이 없다. 자세한 동작은 § 5.3, § 7.3, § 9.9에서 다룬다.

스스로 점검

1. `axes![A = 6, B = 4]` 에서 `m![A, B]` 의 버퍼 위치 13은 어떤 인덱스인가? `m![B, A]` 에서는?
2. `m![D # 8]` ($D = 5$)과 `m![D #\{!\} 8]` 의 차이는?
3. `m![B / 16, B % 16]` 과 `m![B]` 가 동등한 이유를 원문의 항등식으로 설명하라.
4. 텐서가 슬라이스당 i32 3,000개라면 DM에는 들어가지만 VRF에는 왜 들어가지 않는가?

답: ① `m![A, B]` : $13 = 4 \cdot 3 + 1 \rightarrow (A=3, B=1)$. `m![B, A]` : $13 = 6 \cdot 2 + 1 \rightarrow (B=2, A=1)$. ② 둘 다 크기 8이지만 앞의 패딩 3칸은 임의 값이 있는 접근 가능 영역(Top), 뒤는 접근 금지 영역(Bottom)이다. ③ 스트라이드-모듈로 분해 항등식. ④ 12,000 B는 DM 한도 512 KB보다 작지만 VRF 한도 8 KB(8,192 B)보다 크다.

5장

텐서 이동 I: 시퀀서, Fetch, Commit

원문: Moving Tensors · Sequencer · Fetch Engine · Commit Engine · Case Study: Tensor Unit I/O

데이터를 옮기는 세 엔진 Fetch, Commit, DMA는 모두 **시퀀서**라는 공통 부품 위에 서 있다. 시퀀서는 버퍼 매핑과 스트림 매핑 두 개를 받아 하드웨어가 실행할 중첩 루프 설정으로 바꾼다. 이 장은 이동의 기본 규칙과 시퀀서의 동작을 먼저 설명한 뒤, DM에서 스트림을 만드는 Fetch Engine과 스트림을 DM에 쓰는 Commit Engine, 그리고 둘을 조합한 Tensor Unit I/O 사례를 다룬다. DMA Engine과 메모리 성능은 [6장](#)에서 이어진다.

이 장에서 배우는 것

- 이동 연산의 순서·수명 규칙과 엔진별 경계
- 시퀀서 설정 [size : stride] : packet_size 를 읽고, 접근 크기를 계산하는 법
- 전치·분할·슬라이싱·브로드캐스트·병합이 설정에 나타나는 모습과 하드웨어 한도
- Fetch의 read_size와 사이클 공식, 패킷 패딩의 득실, 인터리빙, 축 올리기
- Commit의 write_size, Element 매핑으로 전치하며 쓰기, 정렬 규칙

5.1 이동의 기본 규칙

어떤 값을 어떤 연산으로 옮기는지는 [§ 2.3](#)의 이동 표에 정리했다. 여기서는 그 표 뒤에 있는 규칙을 본다.

순서와 수명

- 연산마다 맞는 컨텍스트를 &mut 로 넘긴다. Device 는 main·sub Tensor Unit 컨텍스트와 tdma·pdma DMA 컨텍스트를 갖는다.
- begin 은 DmTensorView 를 빌리고, 스트림의 수명은 그 뷰를 넘을 수 없다. fetch 는 BeginTensor 를 소비해 스트림을 만들고, fetch_cast·collect 같은 어댑터는 스트림을 소비해 다음 단계를 돌려준다. commit 은 마지막 스트림을 소비하고, commit_view 는 넘겨받은 DmTensorViewMut 에 쓴다.
- 호스트 전송 to_hbm·to_host 는 async여서 .await 한다. 커널 안의 to_dm·to_hbm·fetch·commit은 선택한 컨텍스트에 장치 명령을 넣는다.
- 커널이나 백엔드 제출이 끝날 때까지 원본과 목적지 핸들을 살려 둔다.
- 목적지를 만드는 전송이나 commit이 끝나기 전에 그 목적지를 읽지 않는다. 같은 메모리를 가리키는 뷰에 순서 관계 없이 겹쳐 쓰지 않는다. 가변 뷰는 그 영역을 일부러 덮어쓸 때만 쓴다.
- DMA 호출은 타입 매개변수로 받은 매핑의 목적지를 새로 만들 뿐 원본을 바꾸지 않는다.

엔진 경계

엔진	하는 일	매핑이 정하는 것
Fetch	슬라이스마다 시퀀서 하나로 DM을 읽어 패킷 스트림을 만든다	출력 매핑이 스트림의 Time과 Packet을 고른다. 축 올리기로 Time의 축을 Chip·Cluster·Slice로 옮길 수도 있다
Commit	스트림을 DM에 쓴다	Element 매핑이 목적지 배치를 정하며, 쓰는 동안 스트림 축을 전치할 수 있다

엔진	하는 일	매핑이 정하는 것
DMA	읽기 시퀀서와 쓰기 시퀀서를 짝지어 메모리끼리 옮긴다	공개 경로는 HBM→HBM, HBM→DM, DM→HBM, DM→DM. 인덱스 텐서로 주소를 고르는 gather·scatter는 규칙이 따로 있다(§ 6.7)

튜닝 전에 지킬 제약

모든 이동은 수학적 텐서 이동이며, 컴파일러가 원본과 목적지 매핑에서 시퀀서 스트라이드를 유도한다. 원문이 나열하는 제약은 다음과 같다. DM의 Cluster는 1 또는 2, DM의 Slice는 $64 \cdot 128 \cdot 256$ 이어야 하고, DMA 패킷의 꼬리는 API가 검사하는 원소 크기 정렬을 만족해야 한다. Fetch와 Commit도 매핑별 시퀀서 규칙을 검사하며, 잘못된 매핑은 컴파일이나 명령 검증 단계에서 실패한다.

성능 면에서는 연속한 패킷 배치와, 트래픽을 클러스터·슬라이스·DM 뱅크·HBM 채널에 고루 흘리는 매핑을 고른다. 작거나 띄엄띄엄한 패킷은 시퀀서 접근 횟수를 늘리고, DM 뱅크를 서로 부딪치게 하는 패턴은 DMA를 굼길 수 있다(6장).

원문 확인

Moving Tensors 장은 DM Slice 크기로 $64 \cdot 128 \cdot 256$ 을 허용한다고 적지만, Spatial and Temporal Dimensions 장은 `Slice::SIZE` 가 하드웨어 개수 256과 같아야 한다고 적는다. 원문의 모든 예제는 # 256 패딩으로 256을 맞춘다.

5.2 시퀀서: 매핑 두 개를 중첩 루프로

시퀀서는 메모리 버퍼를 패킷 스트림으로 읽고, 패킷 스트림을 메모리에 다시 쓰는 부품이다. Fetch Engine과 Commit Engine은 각각 시퀀서 하나로 DM 주소를 훑고, DMA Engine은 읽기 시퀀서와 쓰기 시퀀서를 이어 중간 버퍼 없이 DM과 HBM 사이를 옮긴다. 프로그래머가 조절하는 것은 매핑과 패킷 크기다.

교육용 타입: MemTensor와 StreamTensor

원문은 엔진 타입을 모두 들여오는 대신 교육용 가짜 타입 둘로 원리를 보인다. `MemTensor<D, Buf>` 는 메모리 매핑 Buf 로 데이터를 담은 텐서이고(실제 API의 DM·HBM 텐서에 해당), `StreamTensor<'l, D, Time, Packet>` 은 흐르는 중인 텐서다. 수명 'l 이 스트림을 원본 버퍼에 묶는다.

```
impl<D: Scalar, Buf: M> MemTensor<D, Buf> {
    // (Time, Packet)은 Buf의 브로드캐스트여도 된다 (Fetch·Switch·DMA 읽기와 같다)
    pub fn read<'l, Time: M, Packet: M>(&'l self) -> StreamTensor<'l, D, Time, Packet>;
    // 브로드캐스트 금지: Buf의 칸마다 (Time, Packet)에 원본 위치가 정확히 하나 (Commit과 같다)
    pub fn write<'l, Time: M, Packet: M>(&mut self, stream: StreamTensor<'l, D, Time, Packet>);
}
```

한 버퍼를 읽는 Time·Packet 조합은 여러 가지다. 원문의 예 네 가지:

패턴	Buf	Time	Packet	결과
분할 읽기	<code>m![A, B] (8×512)</code>	<code>m![A, B / 32]</code>	<code>m![B % 32]</code>	32원소 패킷 128개
축 순서 바꾸기	<code>m![N, C, H, W]</code>	<code>m![W, H, C, N]</code>	<code>m![1]</code>	원소 하나씩, 축 순서를 뒤집어
타일 읽기	<code>m![A, B, C # 8]</code>	<code>m![A % 2, B % 4, A / 2, B / 4]</code>	<code>m![C # 32]</code>	A를 2×4, B를 4×128 블록으로

패턴	Buf	Time	Packet	결과
브로드캐스트 읽기	<code>m![A]</code>	<code>m![T, A]</code>	<code>m![P]</code>	같은 데이터를 T번 반복, 패킷을 같은 원소로 채움

브로드캐스트한 스트림을 같은 모양의 버퍼에 다시 쓰면 거부된다(원문 예제는 panic). 한 칸에 쓸 값이 여러 개가 되기 때문이다. 원문은 유효한 조합 가운데 Packet이 클수록 대역폭을 잘 쓴다고 적는다.

설정: `[size : stride] : packet_size`

컴파일러는 입력 매핑과 출력 매핑으로부터 시퀀서 하드웨어가 실행할 **중첩 루프 설정**을 만든다. 설정은 `[size_0 : stride_0, size_1 : stride_1, ...] : packet_size` 모양이고 첨자 0이 가장 바깥 루프다. size는 그 루프의 반복 횟수, stride는 반복마다 건너뛴 메모리 거리(원소 단위)다. **그림 16**은 작은 예로 이 루프가 주소를 어떻게 밟는지 보여 준다.

버퍼 Buf = `m![A, B]` ($A = 3, B = 4$): 주소 = $4 \cdot a + b$

0	1	2	3	4	5	6	7	8	9	10	11
a0 b0	a0 b1	a0 b2	a0 b3	a1 b0	a1 b1	a1 b2	a1 b3	a2 b0	a2 b1	a2 b2	a2 b3

Time = `m![B, A]`로 읽은 순서 (방문한 주소)

	+4	+4	-7	+4	+4	-7	+4	+4	-7	+4	+4
0	4	8	1	5	9	2	6	10	3	7	11
바깥 루프 b = 0			b = 1			b = 2			b = 3		

그림 16. 시퀀서가 주소를 밟는 모습. 설정은 `[B → 4 : 1, A → 3 : 4] : 1`이다. 안쪽 루프 A는 스트라이드 4로 세 칸을 밟고, 바깥 루프 B가 한 칸(+1) 나아갈 때 안쪽 루프가 두 번 건너뛴 거리(2×4)만큼 되감겨 주소가 -7 이동한다. 파란 테두리는 b = 0 반복이 방문하는 주소 0, 4, 8이다.

원문의 예는 `m![N, C, H, W]` 버퍼($N=4, C=3, H=8, W=8$)를 Time = `m![W, H, C, N]`, Packet = `m![1]`로 읽는 경우다. 설정은 `[8 : 1, 8 : 8, 3 : 64, 4 : 192] : 1`이고, 하드웨어는 다음 루프를 실행한다. 쓰기도 같은 설정으로 주소를 훑는다.

```
for w in 0..8 {
  for h in 0..8 {
    for c in 0..3 {
      for n in 0..4 {
        let addr = 1 * w + 8 * h + 64 * c + 192 * n; // 이 주소를 읽는다
      }
    }
  }
}
```

컴파일러는 스트림 매핑(Time 다음 Packet)의 항을 하나씩 처리하며 설정을 만든다. 항마다 엔트리의 size는 항의 크기이고, stride는 그 항이 *아직 남은 Buf*에서 차지하는 부피다. 처리한 축은 Buf에서 소비된 것으로 표시한다. `m![A, B, C # 32]` 버퍼($A = B = C = 8, i8$)를 Time = `m![B, A]`, Packet = `m![C # 16]`으로 읽는 원문의 예:

항	엔트리	스트라이드의 근거	처리 후 Buf
B	<code>8 : 32</code>	<code>m![C # 32]::SIZE</code>	<code>m![A, 1 # 8, C # 32]</code>
A	<code>8 : 256</code>	<code>m![1 # 8, C # 32]::SIZE</code>	<code>m![1 # 64, C # 32]</code>
C # 16	<code>16 : 1</code>	연속(Packet 차원)	<code>1 # 2048</code>

가장 안쪽 엔트리 `16 : 1` 이 연속이므로 하드웨어는 16원소 패킷을 한 번에 옮긴다.

한 번에 몇 개를 읽나: 접근 크기

한 번의 하드웨어 접근으로 옮기는 원소 수는 다음과 같다.

```
max_access_size = gcd(Packet::SIZE, contiguous_run)
```

`contiguous_run`은 가장 안쪽 엔트리부터 바깥으로 올라가며 물리적으로 연속인 엔트리들의 크기를 곱한 값이다. 이웃한 두 엔트리 $(n_{\text{outer}} : s_{\text{outer}}), (n_{\text{inner}} : s_{\text{inner}})$ 는 `s_outer == n_inner × s_inner` 일 때 연속이다. 바깥 루프가 한 칸 넘어가면 안쪽 루프가 옮은 영역의 바로 다음 주소로 간다는 뜻이다. 예를 들어 `[4 : 192, 3 : 64, 8 : 8, 8 : 1] : 8` 은 모든 엔트리가 연속이라 `contiguous_run = 8 × 8 × 3 × 4 = 768`이고, `max_access_size = gcd(8, 768) = 8`이다. 대부분의 경우 패킷 배치가 DM에서 완전히 연속이어서 `max_access_size`가 `Packet::SIZE`와 같다.

설정 패턴 모음

패턴	매핑에서 보이는 것	설정에서 보이는 것
전치	Buf와 다른 축 순서	스트라이드가 원래 배치의 부피를 따라 계산된다(위 표)
분할	<code>A % 2, A / 2</code> 처럼 축을 쪼갬	축 하나가 엔트리 여러 개가 된다. <code>m![A % 2, B % 4, A / 2, B / 4]</code> 는 <code>2 : 64, 4 : 8, 4 : 128, 2 : 32</code>
슬라이싱	<code>A % 4 = 3</code> 처럼 범위를 제한	엔트리의 size만 줄어든다(<code>3 : 64</code>)
브로드캐스트	Buf에 없는 축이 Time·Packet에 있다	스트라이드 0 엔트리. 같은 주소를 다시 방문한다
병합	엔트리가 8개를 넘을 때	연속한 이웃 $(n1 : s1), (n2 : s2)$ 를 <code>s1 == n2 × s2</code> 이면 $(n1 × n2 : s2)$ 하나로 합친다
비연속 패킷	패킷 안에 스트라이드 틈	<code>max_access_size</code> 가 <code>Packet::SIZE</code> 보다 작아져 패킷 하나에 여러 번 접근한다

병합의 원문 예는 `m![N, C, H, W]` ($N = C = H = 8, W = 32$) 버퍼를 `Time = m![W / 16, H % 2, H / 2, C / 2, C % 2, N / 2, N % 2, W / 8 % 2]`, `Packet = m![W % 8]` 로 읽는 경우다. 항을 처리하면 엔트리가 9개 나온다. `C / 2` (`4 : 512`)와 `C % 2` (`2 : 256`)은 `512 = 2 × 256`이라 `C` (`8 : 256`)로, `N / 2`와 `N % 2`는 `N` (`8 : 2048`)로, `W / 8 % 2` (`2 : 8`)와 `W % 8` (`8 : 1`)은 `Time/Packet` 경계를 넘어 `W % 16` (`16 : 1`)로 합쳐져 최종 6개가 된다. `H % 2` (`2 : 32`)와 `H / 2` (`4 : 64`)는 `32 ≠ 4 × 64`이므로 합칠 수 없다.

비연속 패킷의 원문 예 두 가지:

- `m![A, B # 16]` 버퍼($A = 4, B = 8$, 행마다 데이터 8칸 뒤 패딩 8칸)에 `Packet = m![A, B]` (32원소)를 쓰면 `A`의 스트라이드가 16이라 $8 × 1 ≠ 16$ 으로 연속이 아니다. `contiguous_run = 8, max_access_size = 8`이므로 패킷 하나를 네 번에 나눠 쓴다.
- `m![N, C, H, W]` ($N=4, C=3, H=4, W=8$)를 `Packet = m![W]` 로 읽으면 패킷마다 한 번(8원소)에 읽는다. 그런데 `Time = m![C]`, `Packet = m![N, H, W]` 로 `C`를 건너뛰면 `N`의 스트라이드 96이 $H × W = 32$ 와 달라 `contiguous_run = 32, max_access_size = gcd(128, 32) = 32`가 되고 패킷 하나에 네 번 접근한다.

하드웨어 한도

RNGD에서 다음 한도를 넘으면 컴파일 에러가 난다.

- 엔트리는 최대 8개. 넘으면 컴파일러가 병합을 시도한다.
- 엔트리당 size $\leq 65,536$.
- 패킷 크기는 1, 2, 4, 8, 16, 32바이트 중 하나.
- 가장 안쪽 엔트리 `n : s`는 연속 접근(`(s == 0 || s == 1) && n % packet_size == 0`)이거나 원소 하나짜리 패킷(`packet_size == 1`)이어야 한다.

병합이 실패하거나 한도를 넘으면 매핑을 다시 설계하거나 시퀀서 호출을 여러 개로 나눈다. 또 Buf와 스트림에 함께 나오는 축은 **호환되는 분해**를 써야 한다. Buf가 A(15)를 `m![A % 5, A / 5]`로, 스트림이 `m![A % 3, A / 3]`으로 쪼개면 원소 수가 같아도 $\gcd(5, 3) = 1$ 이라 어느 쪽도 다른 쪽을 세분하지 못하므로 거부된다.

5.3 Fetch Engine

Fetch Engine은 DM 텐서를 읽어 Tensor Unit용 패킷 스트림을 만든다. `BeginTensor`는 파이프라인 입구의 DM 텐서로, Time이 `m![1]`이고 Packet은 DM 안의 원소 배치다. `fetch::<OutTime, OutPacket>()`이 Fetch 시퀀서를 설정해 `FetchTensor`를 만들고, 이 스트림은 Fetch Adapter, Switch Engine, Collect Engine으로 이어진다.

```
axes![CH = 4, CL = 2, S = 256, A = 512, B = 32];

fn fetch_matrix_example<'l, const T: Tu>{
  input: BeginTensor<'l, T, i8, m![CH], m![CL], m![S], m![1], m![A, B]>,
} -> FetchTensor<'l, T, i8, m![CH], m![CL], m![S], m![A], m![B]> {
  input.fetch::<m![A], m![B]>() // 512사이클 동안 i8 32개(32 B)씩
}
```

`fetch`는 Chip·Cluster·Slice를 그대로 둔다. 슬라이스마다 Fetch 시퀀서가 자기 DM 구역을 독립적으로 읽기 때문이다. 위 예제에서는 칩 4개 × 클러스터 2개 × 슬라이스 256개가 모두 각자의 A×B 부분 텐서에 같은 시퀀서를 돌린다.

한 패킷을 여러 번 읽기

패킷 축이 DM에서 연속이 아닐 수 있고 하드웨어는 한 번에 최대 32바이트만 읽으므로, 패킷 하나를 만들려고 여러 번 읽을 수 있다. 한 번에 읽는 원소 수 **read_size**는 컴파일러가 원소 타입과 Fetch Adapter의 캐스트로부터 정한다.

- **main 컨텍스트**: `max_access_size`의 약수 가운데 `D[read_size]`가 1·2·4·8·16·32바이트가 되는 가장 큰 값.
- **sub 컨텍스트**: 8바이트로 고정.

$$\text{사이클 수} = \text{Time::SIZE} \times (\text{Packet::SIZE} / \text{read_size})$$

`Packet::SIZE`가 `read_size`보다 크면 여러 번 읽는다. 예를 들어 main에서 24바이트 패킷은 `read_size`가 8바이트가 되어 패킷마다 3번 읽는다. 원문은 같은 i4 텐서 `m![N, C, H, W]` ($N=4, C=3, H=4, W=16$)를 네 가지 OutPacket으로 읽는 예를 든다.

OutTime	OutPacket	패킷 크기	read_size	패킷당 읽기	사이클
<code>m![N, C, H]</code>	<code>m![W]</code>	16원소 (8 B)	16원소	1	48
<code>m![N, C, H / 2]</code>	<code>m![H % 2, W]</code>	32원소 (16 B)	32원소	1	24
<code>m![N, C]</code>	<code>m![H, W]</code>	64원소 (32 B)	64원소	1	12
<code>m![N]</code>	<code>m![C, H, W]</code>	192원소 (96 B)	64원소 (32 B)	3	12

핵심

Fetch의 사이클 수는 "시간 단계 $\times$ 패킷당 읽기 횟수"다. 패킷을 32바이트까지 키우면 시간 단계가 줄어드는 만큼 빨라지지만, 그보다 크게 키우면 읽기 횟수가 늘어 이득이 사라진다. 위 표의 마지막 두 줄이 같은 12사이클인 이유다.

인터리빙: 두 텐서를 한 번에

`begin_interleaved`는 **매핑이 같은** 두 텐서를 시퀀서 연산 하나로 합친다. 두 텐서가 같은 계산에 필요할 때 오버헤드를 줄이며, Time에 명시적 축(`I = 2`)을 두어 두 텐서의 교대를 표현한다. 첫 시간 반복은 lhs에서, 둘째는 rhs에서, 셋째는 다시 lhs에서 읽는 식이다. 한 번의 fetch에 최대 두 텐서까지 인터리빙할 수 있다.

```
axes![A = 16, B = 32, I = 2];
// 결과 스트림: Time = m![A, I], Packet = m![B]. A마다 lhs와 rhs를 번갈아 읽는다
device.main.begin_interleaved::<I, _, _, _, _, _>(lhs.view(), rhs.view()).fetch()
```

이렇게 번갈아 들어온 두 그룹은 Vector Engine의 Pair Mode가 짝지어 연산한다([§ 9.6](#)). [§ 10.4](#)의 `add_pair` 함수가 대표 예다.

Fetch 처리량을 정하는 세 가지

- **입력 대역폭:** `read_size`는 DM에서의 축 연속성과 패킷 크기에 묶인다. 이웃하지 않은 축은 `max_access_size`와 `read_size`를 줄이고, 연속 구간보다 작은 패킷도 `read_size`를 제한한다. 패킷을 더 큰 2의 거듭제곱 크기로 패딩하면 `read_size`가 커진다. 또 같은 बैं크를 64번 이상 연속 접근하면 우선순위가 낮은 Commit-DMA Engine을 굶겨 NoC 타임아웃을 부를 수 있다([§ 6.10](#)).
- **출력 대역폭:** 뒤의 Collect Engine이 패킷을 32바이트 플릿으로 바꾸므로 32바이트에 맞지 않는 패킷은 대역폭을 버린다. 20바이트 패킷은 플릿 하나에 12바이트 패딩($12/32 = 37.5\%$ 낭비), 40바이트 패킷은 플릿 두 개(64바이트)에 24바이트 패딩($24/64 = 37.5\%$ 낭비)을 남긴다.
- **공간 병렬성:** `fetch`를 여러 슬라이스에 분산할수록 처리량이 높다.

패킷 패딩의 원문 예는 f8e4m3 텐서 $A = 3, B = 5, C = 2$ (30바이트)다.

OutTime	OutPacket	Fetch 작업	사이클	Collect가 붙이는 패딩
<code>m![A, B]</code>	<code>m![C # 8]</code>	8 B 패킷 15개	15	패킷마다 24 B
<code>m![A]</code>	<code>m![[B, C] # 16]</code>	16 B 패킷 3개	3	패킷마다 16 B
<code>m![1]</code>	<code>m![[A, B, C] # 32]</code>	32 B 패킷 1개	1	없음

패딩은 실제 데이터 너머를 읽지만 패딩 값이 계산에 영향을 주지 않으므로 안전하다. 다만 32바이트 후보는 추가 물리 공간 때문에 DM 사용량이 늘 수 있고, 매핑이 달라지면 뒤 단계에도 영향이 간다. 원문은 읽기 횟수가 줄어드는 이득이 패딩과 하위 배치 비용보다 큰지는 **스케줄 비교로** 판단하라고 적는다.

축 올리기: 같은 데이터를 슬라이스마다 다른 곳부터 읽기

보통의 `fetch`는 모든 칩·클러스터·슬라이스가 같은 DM 오프셋에서 시작해 Time의 축을 순서대로 훑는다. **축 올리기 (axis lifting)**는 `fetch_chip_lift`, `fetch_cluster_lift`, `fetch_slice_lift`로 단위마다 다른 읽기 **시작 주소 (base)**를 줘서, 올린 축을 병렬로 읽고 Time에서 없앤다. 시작 주소만 바꾸는 것이므로 원본 영역이 그 차원을 따라 복제되어 있어야 한다. DM 배치에서는 이 복제가 브로드캐스트로 표현되고, 올리기가 그 브로드캐스트를 올린 축으로 바꾼다.

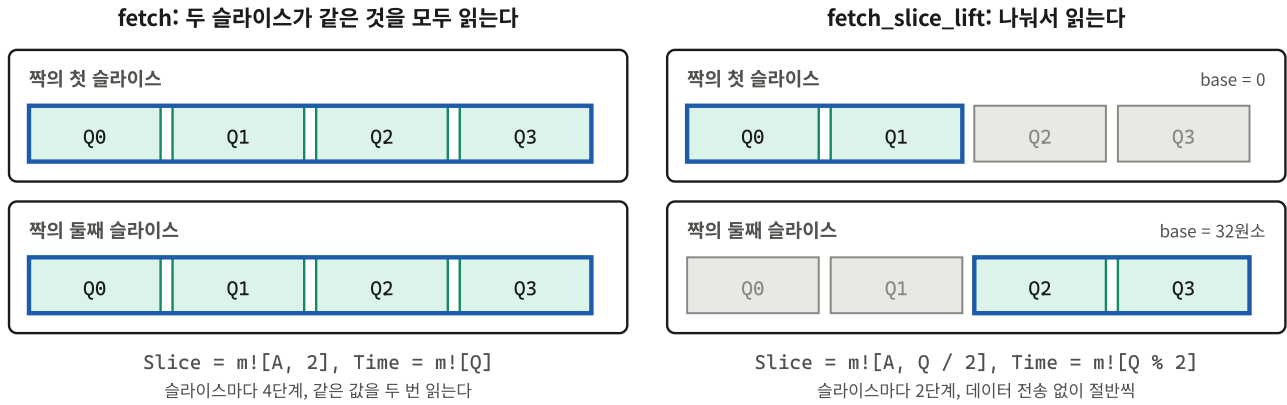


그림 17. 슬라이스 축 올리기. 두 슬라이스가 같은 데이터(Q 네 블록, 블록마다 bf16 16개)를 갖고 있을 때, 시작 주소만 다르게 주면 전송 없이 Q축을 슬라이스로 나눠 읽을 수 있다.

원문 예에서 $A = 128$ 이고 Slice 배치 $m![A, 2]$ 는 슬라이스 128쌍을 뜻한다. 한 쌍의 두 슬라이스는 같은 $m![Q, V]$ ($Q = 4, V = 16, \text{bf16}$) 데이터를 가진다. 올리기는 브로드캐스트 2를 $Q / 2$ 로 바꾸고 $Q \% 2$ 만 Time에 남긴다. base는 올린 축의 값에 원래 DM 스트라이드를 곱해 더한 것이다. 둘째 슬라이스는 $Q / 2 = 1$ 이므로 $\text{base} = 1 \times (Q \% 2 \text{의 크기 } 2) \times V(16) = 32\text{원소}(\text{bf16 } 64\text{바이트})$ 다.

```
axes![A = 128, Q = 4, V = 16];
fn fetch_halves_per_slice<'l, const T: Tu>(<
  input: BeginTensor<'l, T, bf16, m![1], m![1 # 2], m![A, 2], m![1], m![Q, V]>,
) -> FetchSliceLiftTensor<'l, T, bf16, m![1], m![1 # 2], m![A, Q / 2], m![Q % 2], m![V]> {
  input
    .fetch::<m![Q], m![V]>()
    .fetch_slice_lift::<m![A, Q / 2], m![Q % 2]>()
}
```

메서드	시작 주소를 얻는 방법
<code>fetch_chip_lift</code>	읽기 명령에 정적 base가 들어가므로 추가 설정 연산이 없다
<code>fetch_cluster_lift</code>	읽기 명령에 정적 base가 들어가므로 추가 설정 연산이 없다
<code>fetch_slice_lift</code>	base가 원본 텐서의 런타임 주소에 달려 있어 명령에 고정할 수 없다. 컴파일러가 상대 오프셋 상수를 DMA로 DRAM에서 SRAM으로 복사하고, fetch의 실행 컨텍스트에서 원본 주소를 더한 뒤, 슬라이스별 base SFR(특수 기능 레지스터)에 저장한다. fetch는 이 준비를 기다린다

sub 컨텍스트의 fetch라면 이 준비 저장이 fetch 자신이 쓸 SFR을 덮어쓰므로, 컴파일러가 base를 쓴 뒤 해당 매개변수를 복원한다. 두 컨텍스트 모두 세 가지 올리기를 지원한다. 요구 사항은 다음과 같다.

- 선택한 차원의 입력 크기와 출력 크기가 같아야 한다.
- 브로드캐스트만 바뀔 수 있다. 검증기는 매핑을 크기 2 묶음으로 비교하며, 바뀐 브로드캐스트 2는 크기 2인 축 성분이 되어야 한다. 기존 축이나 크기 3짜리 브로드캐스트는 바꿀 수 없다.
- 올린 축은 OutTime에서 빠져야 하고 OutPacket은 그대로다.
- Chip, Cluster, Slice 순서로, 차원마다 최대 한 번만 부른다.
- 계산된 모든 base는 8바이트의 배수여야 한다.
- `unsafe reshape` 로 브로드캐스트를 만들었다면 데이터가 실제로 복제되어 있어야 한다.

주의

`reshape` 는 원소 순서만 확인하고 복제 여부는 확인하지 않는다. 원문의 두 예가 차이를 보여 준다. DMA로 `[H, V]` 를 Rep축을 따라 실제로 복제한 뒤 `reshape`로 `m![A, 2]` 라고 부르면 올리가 유효하다. 반면 서로 다른 값을 담은 G축을 `reshape`로 브로드캐스트라고 우기면, 올리기 결과가 하드웨어와 CPU에서 달라질 수 있다.

데이터를 슬라이스 사이로 실제로 옮기거나 Slice에서 Time으로 옮겨야 한다면 Switch Engine의 `InterTranspose` 를 쓴다(§ 7.3). Chip과 Cluster에서 원본이 복제되어 있지 않다면 DMA가 대안이다.

5.4 Commit Engine

Commit Engine은 Fetch의 반대다. 어댑터들이 원소 타입과 배치를 확정한 스트림을 DM에 쓴다. 슬라이스마다 독립적으로 자기 DM 구역에 쓰는 Commit 시퀀서이며, 역시 수학적 텐서 이동이다.

```
// 스트림을 새 DmTensor로 기록
pub fn commit<Element: M>(self) -> DmTensor<D, Chip, Cluster, Slice, Element>;
// 이미 있는 DM 뷰(예: 타일)에 기록. Slice 배치가 보존되어야 한다
pub fn commit_view<DstSlice: M, Element: M>(self, dst: DmTensorViewMut<'l, D, Chip, Cluster, DstSlice, Element>);
```

`commit` 은 Chip·Cluster·Slice를 그대로 두고, 출력 `Element` 매핑이 Time과 Packet을 대신해 DM 안의 배치를 정한다. `Element`는 Commit 시퀀서와 Commit Adapter를 함께 설정하며, 입력 스트림과 다른 순서로 Time 축을 배치해 **쓰면서 전치**할 수도 있다. 원문 예는 Contraction 결과(`f32`, Time `M = 16`, Packet `N = 8`)를 `cast::<bf16, m![N # 16]>()` 로 `bf16` 8개를 담은 32바이트 플릿으로 줄이고, `commit_trim::<m![N]>()` 으로 유효한 8개만 남겨 슬라이스 256개에서 DM에 쓴다.

제약

- Chip·Cluster·Slice 크기가 하드웨어 구성과 맞아야 한다.
- 모든 시퀀서 스트라이드는 8바이트의 배수여야 한다.
- 쓰기 단위 `D[valid_size]` 는 `8·16·24·32`바이트 중 하나다. Commit Adapter의 트리밍이 이를 정한다(§ 10.3).

한 패킷을 여러 번 쓰기

패킷 축이 DM에서 연속이 아니면 여러 번 써야 한다. 컴파일러가 `write_size = gcd(valid_size, access_size)` 를 유도한다. `valid_size`는 Commit Adapter가, `access_size`는 시퀀서가 준다. sub 컨텍스트에서는 `D[write_size]` 가 8바이트로 고정이다.

$$\text{사이클 수} = \text{Time::SIZE} \times (\text{valid_size} / \text{write_size})$$

이 나눗셈은 항상 떨어진다. main에서는 `valid_size`와 `write_size`가 같아 패킷마다 1사이클이고, sub에서는 `write_size`가 8바이트, `valid_size`가 `8·16·24·32`바이트이므로 패킷마다 1~4사이클이다. 원문 예에서 `m![M, K]` 스트림 (`M = 4`, `K = 2`)을 `m![M, K, L]` 로 순서 그대로 쓰면 설정 `[4 : 64, 2 : 32, 32 : 1]` 이 모두 연속이다. `m![K, M, W]` 처럼 `M`과 `K`를 바꿔 쓰면 설정 `[4 : 8, 2 : 32, 8 : 1]` 의 연속이 깨지지만, `valid_size`와 `access_size`가 모두 8원소라 여전히 패킷마다 한 번에 쓴다.

Commit 처리량

- **순차 주소:** 슬라이스 안에서 연속한 DM 주소에 쓰면 बैं크를 병렬로 쓴다(DMN당 128 B/cycle, DMN 인터리빙이면 256 B/cycle). 같은 बैं크를 64번 이상 연속으로 쓰는 패턴은 बैं크 기아를 부른다.
- **공간 병렬성:** 쓰기를 모든 활성 슬라이스에 분산한다.
- **정렬된 쓰기:** 스트라이드 제약과 트리밍 제약 덕분에 쓰기 주소와 단위가 항상 8바이트 정렬이므로, बैं크의 일부만 쓰는 일은 일어나지 않는다.

5.5 사례: Tensor Unit I/O

원문의 Case Study: Tensor Unit I/O는 Fetch와 Commit으로 텐서를 DM에 거쳐 가게 하면서 모든 논리 인덱스를 보존하는 종합 예제다. 기존 예제 `fetch_commit_simple`은 `[A, B]` 배치의 i8 HBM 텐서를 받아 Tensor Unit에서 i32로 넓히고, `[B, A]` 배치의 HBM 텐서를 돌려준다.

```
axes![A = 4096, B = 8];
type Chip = m![1];
type Cluster = m![1 # 2];

#[device(chip = 1)]
pub fn fetch_commit_simple(
    device: &mut Device,
    input: &HbmTensor<i8, m![1], m![A, B]>,
) -> HbmTensor<i32, m![1], m![B, A]> {
    // Element의 가장 안쪽 축은 원본의 가장 안쪽 축(B, 스트라이드 1)과 같아야 한다.
    // 그래야 DMA 꼬리가 B 전체(i8 8개 = 8 B)를 담아 min_align = 8을 만족한다.
    let input_dm = input.to_dm::<Cluster, m![A / 16], m![A / 8 % 2, A % 8, B]>(&mut device.tdma);

    let fetch_and_commit_tensor: DmTensor<i32, Chip, Cluster, m![A / 16], m![A / 8 % 2, A % 8, B]> = device
        .main
        .begin(input_dm.view())
        .fetch::<m![A / 8 % 2], m![A % 8, B]>()
        .fetch_cast::<i32>()
        .collect::<m![A / 8 % 2, A % 8], m![B]>()
        .commit_trim::<m![B]>()
        .commit();

    fetch_and_commit_tensor.to_hbm(&mut device.tdma)
}
```

단계	선언한 선택	슬라이스 하나 기준
입력	A = 4096, B = 8, i8, HBM 매핑 <code>m![1], m![A, B]</code>	전체 32,768 B
HBM → DM	Cluster <code>m![1 # 2]</code> , Slice <code>m![A / 16]</code> (256개), Element <code>m![A / 8 % 2, A % 8, B]</code>	$2 \times 8 \times 8 = i8$ 128개
Fetch	OutTime <code>m![A / 8 % 2]</code> , OutPacket <code>m![A % 8, B]</code>	Time $2 \times$ 패킷 64원소. 캐스트 때문에 8원소씩 읽어 16사이클
캐스트, Collect	<code>fetch_cast::<i32>()</code> , Time <code>m![A / 8 % 2, A % 8]</code> , Packet <code>m![B]</code>	i32 8개(32 B) 플릿 16개
Commit → HBM	<code>commit_trim::<m![B]>()</code> 로 DM에 쓰고, <code>to_hbm</code> 이 DMA 재배치로 <code>m![B, A]</code> 를 만든다	-

캐스트가 있는 fetch는 캐스트 결과가 32바이트 플릿 하나에 들어가야 하므로 i8을 8개씩(캐스트 후 32 B) 읽는다(§ 7.2). 호스트 테스트는 입력을 0부터 32,767까지 i8로 바꾼 값으로 채우고, 모든 `output[b, a]` 가 넓은 `input[a, b]` 와 같은지 확인한다. 원문은 이 예제가 **스트리밍 도중의 전치를 보여 주는 것이 아니라고** 강조한다. `[A, B]` 에서 `[B, A]` 로의 재배치는 마지막 DMA 경계에서 일어난다. 실행 명령은 `cargo furiousa-opt test --test fetch_commit_tests` 다.

스스로 점검

1. `m![A, B]` ($A = 4, B = 8, i8$) 버퍼를 `Time = m![A]`, `Packet = m![B]` 로 읽을 때 설정과 `max_access_size`는?
2. main 컨텍스트에서 bf16 패킷 48원소(96 B)를 Time 10으로 읽으면 `read_size`와 사이클 수는?
3. 20바이트 패킷이 출력 대역폭을 몇 퍼센트 버리나?
4. 축 올리기가 데이터 전송 없이 가능한 조건은?

답: ① `[4 : 8, 8 : 1] : 8`, `contiguous_run 32`, `max_access_size 8`. ② 48의 약수 중 bf16으로 32 B 이하인 최대값 16원소, 패킷당 3번 → 30사이클. ③ 37.5%. ④ 원본 영역이 해당 차원으로 복제(브로드캐스트)되어 있을 때.

6장

텐서 이동 II: DMA와 메모리 성능

원문: DMA Engine(Interface, Architecture, Constraints, Optimizations, Detailed Examples, Redistribution Operations, Scatter and Gather, PCIe DMA) · Memory Performance

DMA Engine은 Tensor Unit 파이프라인을 거치지 않고 메모리 계층 사이로 텐서를 직접 옮긴다. 계산은 필요 없고 위치나 배치만 바꿀 때 쓴다. 원문의 이 부분은 가장 하드웨어에 가까운 설명이다. 같은 양의 데이터라도 주소를 어떤 순서로 훑느냐에 따라 전송 시간이 열 배 넘게 달라지고, 규칙 하나를 어기면 클러스터 전체가 리셋된다. 이 장은 DMA의 구조와 규칙, 칩·클러스터 사이의 재분배 연산, 그리고 그 뒤에 있는 DM과 HBM의 물리 구조를 설명한다.

이 장에서 배우는 것

- 읽기·쓰기 시퀀서 한 쌍으로 이루어진 DMA 전송과 칩당 DMA 엔진 8개
- 정렬·패킷 크기 제약, 명령당 약 500사이클의 시작 지연, 256바이트 단위 분할
- 원문 예제 여섯 개로 보는 좋은 접근 패턴과 병리적 접근 패턴
- chip_shuffle·cluster_swap·chip_slice 같은 재분배 연산과 gather·scatter
- DM의 뱅크·DMN 구조, 뱅크 기아와 64회 규칙
- HBM의 주소 비트 배치, 스택 비트, 정렬 위반과 뱅크 충돌의 비용

6.1 DMA Engine: 파이프라인 없이 옮기기

DMA 전송은 서로 맞물린 두 단계로 이루어진다. **읽기 시퀀서**가 원본 계층을 읽고, **쓰기 시퀀서**가 목적지 계층에 쓴다. 쓰는 동안 배치를 바꿀 수 있다. DMA 전송도 수학적 텐서 이동이어서 배치가 달라도 출력은 입력과 같은 텐서를 담는다. Tensor DMA는 DMN 사이, 클러스터 사이, 칩 사이 전송을 모두 다루며, 칩 번호는 시스템 전체가 합의한 전역 ID를 쓴다.

커널 작성자는 원본 텐서에서 `.to_dm()`, `.to_hbm()` 같은 메서드를 부르고 DMA 컨텍스트를 넘긴다. `device.tdma` 는 칩 안 전송(HBM↔HBM, HBM↔DM, DM↔DM), `device.pdma` 는 호스트↔HBM 전송이다. 컴파일러는 원본과 목적지 텐서 타입에서 두 시퀀서의 설정을 유도한다. HBM→DM 전송은 HBM의 Chip 차원을 유지하고 목적지의 Cluster·Slice·Element를 정한다. DM→DM 전송은 Chip·Cluster·Slice·Element를 모두 새로 정할 수 있고, HBM이 목적지면 Chip과 Element를 정한다.

```
impl<D: Scalar, Chip: M, Element: M> HbmTensor<D, Chip, Element> {
    pub fn to_dm<Cluster: M, Slice: M, Element2: M>(
        &self,
        _dma: &mut DmaContext<{ Dma::Tensor }>,
    ) -> DmTensor<D, Chip, Cluster, Slice, Element2> {
        assert_dma_layout:::<...>(DM_WRITE_ALIGN_BYTES);    // DM 쓰기 정렬 검사
        // ...
    }

    // 같은 HBM 주소에서 매핑 이름만 바꾼다. self를 소비한다
    pub unsafe fn reshape<Chip2: M, Element2: M>(self) -> HbmTensor<D, Chip2, Element2>;
}
```

`reshape`의 안전 조건은 "옛 매핑과 새 매핑이 원소를 **같은 물리 순서**로 놓아야 한다"는 것이다. 축을 합치거나 쪼개는 재목음은 순서를 보존하므로 유효하지만, 순열(전치)은 아니다. 크기가 같다고 순서가 같다는 보장은 없다. `self`를

소비하는 것은 옛 모양의 핸들이 같은 HBM 바이트를 다른 매핑으로 계속 가리키지 못하게 하려는 장치다.

원문 예제 두 개를 보자. 첫째는 HBM→HBM 전송 두 번으로 [A, B, C]를 [C, A, B]로 전치한다. 둘째는 HBM의 벡터를 DM에 분배한다.

```
// 1) [A, B, C] → [A, C, B] → [C, A, B]
let intermediate: HbmTensor<f32, m![1], m![A, C, B]> = input.to_hbm(&mut device.tdma);
let result: HbmTensor<f32, m![1], m![C, A, B]> = intermediate.to_hbm(&mut device.tdma);

// 2) A = 2048개를 클러스터 2개 중 1개, 슬라이스 256개에 8개씩
input.to_dm::<m![1 # 2], m![A / 8], m![A % 8]>(&mut device.tdma)
```

원문의 조언

SRAM끼리 옮길 때는 Tensor Unit의 Fetch→Commit 왕복이 DMA보다 효율적인 경우가 많다. DMA가 SRAM 슬라이스의 대역폭을 다 쓰지 못할 수 있기 때문이다. 다만 실제로는 HBM 대역폭이 병목인 경우가 많아 HBM↔DM 전송에서는 이 차이가 덜 중요하다.

6.2 DMA Engine의 구조

엔진 8개와 시퀀서 한 쌍

RNGD 칩 하나에는 **DMA Engine이 8개** 있다. DMN 두 개당 하나이며(칩당 DMN 16개), 서로 독립인 전송 8개를 병렬로 돌릴 수 있다. 예를 들어 0번 엔진이 HBM↔DM을 옮기는 동안 1번 엔진이 DM↔DM을 옮긴다. 엔진 하나는 읽기·쓰기 시퀀서를 보조를 맞춰 돌린다. 두 시퀀서는 루프 횟수가 같고 스트라이드와 시작 주소만 다르다. 같은 논리 원소가 원본과 목적지에서 다른 순서로 놓이기 때문이다. 컴파일러는 둘을 **루프 엔트리마다 스트라이드 한 쌍을 가진 시퀀서 하나로** 표현한다(그림 18).

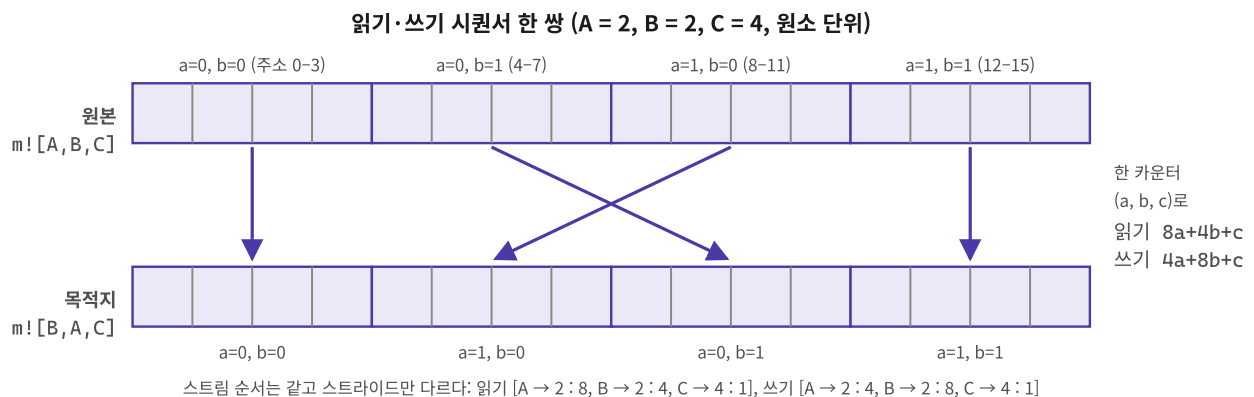


그림 18. DMA 전송 하나는 루프 카운터 하나로 읽기 주소와 쓰기 주소를 함께 계산한다. 원문 예의 256배 크기 버전(A = B = C = 256, m![A, B, C] → m![B, A, C])에서는 읽기 스트라이드가 (65,536, 256, 1), 쓰기 스트라이드가 (256, 65,536, 1)이 된다.

원문의 동적 동작 예는 base = (0, 256³)인 시퀀서 [A → 256 : (65,536, 256), B → 256 : (256, 65,536), C → 256 : (1, 1) : 256 이다. 카운터는 행 우선 순서로 늘어나고, $i = a \cdot 256^2 + b \cdot 256 + c$ 번째 반복의 읽기 주소는 i, 쓰기 주소는 $256 \cdot a + 256^2 \cdot b + c + \text{write_base}$ 다. 가장 안쪽 스트라이드 stride0이 256이므로 하드웨어는 반복마다 256바이트를 옮기고, (A, B, C) = (0, 0, 0..255)를 패킷 하나로 처리한다. 전체는 약 500사이클의 시작 지연과 256 × 256사이클의 데이터 전송이다.

컴파일러가 설정을 만드는 방법

- **읽기 시퀀서**: 원본 매핑 In을 스트림 모양에 사영해 원본 계층의 루프별 스트라이드를 만든다.
- **쓰기 시퀀서**: 목적지 매핑 Out을 스트림 모양에 사영해 목적지 계층의 스트라이드를 만든다.
- **통합 시퀀서**: 둘을 합쳐 엔트리마다 읽기·쓰기 스트라이드를 짝짓는다.
- **패킷 크기**: 연속한 읽기·쓰기 부피로부터 `stride0`을 추론한다. 읽기와 쓰기가 모두 256바이트씩 연속이면 최적의 stride0은 256이다.

여러 엔진에 나누기: 동질·이질 집계

컴파일러는 텐서 이동 하나를 칩·클러스터·슬라이스 차원으로 나눠 여러 DMA 엔진에 배정한다. 어떤 엔진이든 어떤 전송이든 처리할 수 있지만, 로컬 DMN 접근이 더 빠르므로 기본값은 원본 DM의 로컬 엔진이다. 원문은 커널 작성자가 엔진을 명시할 수도 있다고 적는다.

- **동질(homogeneous) 집계**: 텐서 모양이 DMN들에 고르게 나누어떨어질 때. 모든 엔진이 매개변수화된 디스크립터 템플릿 하나를 공유하고 시작 주소만 다르다.
- **이질(heterogeneous) 집계**: 나누어떨어지지 않을 때. DMN마다 자기 디스크립터(`HashMap<DmnIndex, DmaDescriptor>`)와 스트림 환경을 갖고, 경계의 DMN은 유효 영역 밖에 쓰지 않도록 일을 여러 DMA 명령으로 쪼갬다.

두 형태 모두 참여 엔진의 원본·목적지 매체가 같아야 하고, 전체 전송을 입력 매핑 하나와 출력 매핑 하나가 다스린다. 명령마다 시작 지점이 따로 붙으므로 DMN에 고르게 나누어떨어지는 모양을 고르는 것이 좋다.

6.3 정렬과 패킷 크기 제약

DMA의 정렬·패킷 규칙을 어기면 느려지는 데서 그치지 않고 결과가 틀리거나 하드웨어 예외가 난다. 컴파일러는 이를 하드웨어 불변식으로 강제한다.

계층	읽기 주소 정렬	쓰기 주소 정렬
HBM	1 B	1 B
DM (SRAM)	1 B	8 B

- HBM↔DM 전송은 위 표와 별개로 읽기 주소, 쓰기 주소, 패킷 크기 모두 8바이트 정렬이 필요하다.
- DM 규칙이 비대칭인 이유는 SRAM 하드웨어가 비대칭이기 때문이다. 읽기 포트는 바이트 선택 논리로 임의 바이트 범위를 뽑지만, 쓰기 포트는 뱅크 폭인 8바이트 단위로만 쓴다. 정렬이 어긋난 DM 쓰기는 Read-Modify-Write가 되어 **쓰기 시간이 세 배**가 되고 해당 뱅크의 다른 연산을 막는다.
- **패킷 최대 크기는 4,096바이트**다. 칩 안 버스(AXI) 트랜잭션이 256 beat × 16바이트 데이터 폭을 넘을 수 없어서다.

6.4 DMA 성능을 정하는 세 가지

메모리 대역폭

계층	최대 대역폭
HBM	칩당 1.5 TB/s (채널 32개 × 채널당 48 GB/s, 0.75 GHz 기준)
DM	클러스터당 256 B/cycle (DMN 인터리빙 시, DMN당 128 B/cycle)

계층	최대 대역폭
PCIe	읽기·쓰기 모두 30 B/cycle

DMA 엔진 하나는 최대 256 B/cycle을 옮긴다. HBM 대역폭은 HBM을 쓰는 모든 엔진이 나눠 쓰므로, HBM을 포화시키는 집계의 속도는 엔진별 합이 아니라 1.5 TB/s가 상한이다. 같은 클러스터 안의 DM→DM 전송은 읽기와 쓰기가 같은 DM 뱅크 접근을 두고 다투므로 **직렬화**되고, HBM↔DM처럼 계층을 건너는 전송은 읽기와 쓰기가 **파이프라인**된다.

채널과 DMN 인터리빙

최대 대역폭을 유지하려면 아래 메모리 구획들에 접근을 번갈아 흠어야 한다. HBM의 채널은 주소 비트 9~28로 선택되고, **주소 비트 8은 스택 비트**다. 채널 32개에 요청을 고루 보내려면 이 비트들이 모두 바뀌어야 한다. 스택 비트 하나만 고정되어도 요청이 채널 16개로만 가서 **대역폭이 절반**이 된다. 같은 HBM 뱅크를 반복해서 접근하면서 행 주소 비트(21번 이상)가 바뀌면 총돌마다 약 40사이클이 들어 대역폭이 한 자릿수 배로 떨어질 수 있다. DM에서는 두 DMN(각 128 B/cycle)을 번갈아 써야 하며, DMN 하나에만 접근하면 대역폭이 반으로 준다.

시작 지연과 패킷 분할

- **DMA 명령마다 약 500사이클의 고정 시작 지연**이 데이터 전송 전에 붙는다. 전송 여러 개를 명령 적게 합치면 이 비용이 분산되고, 이질 집계는 DMN별 명령마다 지연을 치른다.
- 한 명령 안에서 하드웨어는 패킷을 256바이트 단위로 쪼갬다. n바이트 패킷은 $\lceil n / 256 \rceil$ 개의 AXI 요청이 된다. 4,095바이트 패킷은 요청 16개이고, 4,096바이트 한도를 넘는 4,099바이트 패킷은 명령 여러 개로 나눠야 한다.
- 가장 안쪽 스트라이드 `stride0` 이 256바이트 정렬이면 사이클 수가 $\lceil \text{stride0} / 256 \rceil$ 이다. 정렬이 아니면 HBM 쓰기는 마지막 256바이트 조각에 RMW 벌칙을 추가로 치르고, HBM 읽기는 올림 오버헤드만 치르며, DM 연산은 이런 어긋남에 거의 영향을 받지 않는다.
- DMA는 DM 뱅크 접근 우선순위가 가장 낮다. Fetch나 Commit이 같은 뱅크에 64번 이상 연속 접근하면 DMA가 굶어 NoC 타임아웃이 난다(§ 6.10).

6.5 예제 여섯 개로 보는 DMA 성능

원문은 대표 전송 패턴마다 시퀀서 설정과 사이클 추정치를 준다. 예제 1~3은 계층 조합별로 잘 조율한 경우, 4~5는 열 배 이상 손해 보는 병리적 패턴, 6은 모양이 DMN에 나누어떨어지지 않을 때의 분할이다. 설정에 나오는 `slice_stride`는 슬라이스 하나의 DM 구획이 차지하는 가상 주소 폭(4 MB), `DMN_stride`는 같은 클러스터 안 두 DMN 사이의 주소 거리다.

#	전송	무엇이 속도를 정하나	대략 사이클
1	HBM → HBM 배치 변환 A=8, B=8, C=256, i8, [A, B, C] → [B, A, C]	바깥 스트라이드 256과 2,048이 HBM 주소 비트 8(스택)과 11(채널)을 토글해 요청이 여러 채널로 흠어진다. 256바이트 전송은 채널당 4사이클(0.75 GHz)이지만 채널 병렬로 거의 최대 대역폭을 유지한다	읽기 64 + 쓰기 64 요청(1 GHz) + 시작 500 ≈ 628. 단 엔진 4개가 HBM↔HBM을 나누면 엔진당 약 0.1875 TB/s라 사이클당 요청 1개도 못 채운다
2	HBM → DM 전 대역폭 A=B=C=256, i8	HBM 쪽 32,768 스트라이드가 채널을 인터리빙하고, DM 쪽 <code>slice_stride · DMN_stride</code> 가 연속한 256바이트 쓰기를 두 DMN에 번갈아 보낸다. 계층이 달라 읽기·쓰기가 파이프라인된다	$\max(\text{읽기 } 65,536, \text{ 쓰기 } 65,536) + 500 \approx 66,036$

#	전송	무엇이 속도를 정하나	대략 사이클
3	같은 클러스터 DM → DM	DMN·슬라이스 인터리빙으로 각 단계가 256 B/cycle을 쓰지만 읽기와 쓰기가 직렬화된다	65,536 + 65,536 = 131,072, 여기에 시작 500
4	HBM 뱅크 충돌 병리 A=64, B=2,048, C=1,024	A%32, A/32의 스트라이드가 HBM 뱅크 안의 행을 고르는 주소 비트 21과 26을 토글한다. 채널 안 연속 접근이 거의 매번 행을 닫고 새로 열어 요청마다 약 40사이클. $C / 256$ 으로 채널 32개에 흩어도 채널 안 행 충돌은 숨길 수 없다	요청 524,288개 ÷ 32채널 = 채널당 16,384 × 40 ≈ 655,360 + 500 ≈ 655,860 (잘 조율한 경우의 약 10배)
5	스택 비트 누락 병리 A=8, B=64, C=8, D=512	C의 스트라이드 512가 주소 비트 8(스택)을 한 번도 바꾸지 않아 DMN 8개가 채널 32개 중 16개에만 몰린다	요청 4,096개 ÷ 16채널 = 256 × 약 5.3 ≈ 1,357 + 500 ≈ 1,857. 스택 인터리빙을 되살리면 HBM 사이클이 절반
6	이질적 DMN 분할 칩 4개, A = 15	15 = 3·4 + 3이라 DMN 0~2는 4개씩, DMN 3은 3개만 말한다. 디스크립터 하나로는 네 번째 원소를 유효 영역 밖에 쓰므로 DMN 3의 일을 명령 두 개로 쪼갬다	DMN 0~2: 약 256 + 500 ≈ 756, DMN 3: 약 192 + 1,000 ≈ 1,192. 전체는 DMN 3에 묶여 약 1,192

핵심

같은 바이트를 옮기는데 예제 4는 잘 조율한 전송의 약 열 배가 걸린다. DMA 성능은 "몇 바이트를 옮기나"보다 "주소를 어떤 순서로 훑나"가 좌우한다. 원문은 가능하면 가장 안쪽 축 C를 256의 배수로 고르라고 권한다. $C = 256n + r (0 < r < 256)$ 이면 데이터 양은 조금 늘 뿐인데 접근이 더 많은 요청으로 쪼개져 사이클 수가 $n + 1$ 배로 늘어난다.

6.6 재분배 연산: 칩과 클러스터 사이로 값 보내기

SRAM 재분배는 DmTensor 나 DmTensorView 에서 **재분배 계획**을 쌓고 to_dm 또는 to_dm_view 로 실행한다. 마지막 호출 전의 모든 단계는 DMA 연산 하나로 합쳐진다. 칩이나 클러스터 사이로 부분 값을 보내고 각 목적지에 필요한 조각을 고르는 데 쓰며, 칩·클러스터 리듀스 사례(§ 10.4)의 기반이다.

API	뜻	모양 변화	실행
source.chip_shuffle(sources)	목적지 칩마다 원본 칩 하나를 고른다	없음	지연된 Tensor DMA
source.cluster_swap()	칩마다 클러스터 0과 1을 맞바꾼다	없음	지연된 Tensor DMA
source.chip_slice::<Axis, Output>(indices)	목적지 칩마다 Axis 좌표 하나를 고른다	Element → Output	지연된 Tensor DMA
source.cluster_slice::<Axis, Output>(indices)	목적지 클러스터마다 Axis 좌표 하나를 고른다	Element → Output	지연된 Tensor DMA
tensor.asymmetric_chip_slice::<Axis, Output>(…)	칩마다 로컬 Axis 좌표 하나를 고른다	Element에서 Axis 제거	즉시, sub 컨텍스트 병렬 복사
tensor.asymmetric_cluster_slice::<Axis, Output>(…)	클러스터마다 로컬 Axis 좌표 하나를 고른다	Element에서 Axis 제거	즉시, sub 컨텍스트 병렬 복사
source.hbm_chip_shuffle(dma, sources)	목적지 HBM 칩마다 원본 칩 하나를 고른다	없음	즉시 HBM DMA(tdma:pdma 모두 가능)

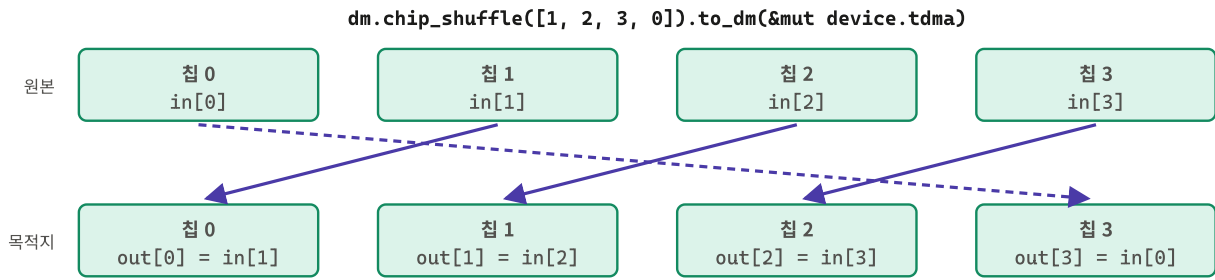


그림 19. `chip_shuffle(sources)`의 뜻은 `sources[목적지 칩] = 원본 칩`이다. 모양은 그대로이고 어느 칩이 값을 공급하는지만 바뀐다. 원본 칩이 바뀌므로 컴파일러가 칩 간 동기화(`ChipSync`)를 끼워 넣는다.

- **chip_slice:** `chip_slice::<m![B], m![C, D]>([0, 1, 2, 3])`는 목적지 칩 `t`가 `B`의 `t`번째 좌표만 갖게 하고 Element에서 `B`를 없앤다(`out[t, k, c, d] = in[t, k, t, c, d]`).
- **cluster_slice:** `cluster_slice::<m![B], m![C, D]>([1, 0])`는 클러스터 0이 `B = 1`을, 클러스터 1이 `B = 0`을 갖게 한다.
- **단계 조합:** 물리적 순서대로 선택적 chip shuffle → 선택적 cluster swap → chip slice 0개 이상 → cluster slice 0개 이상을 이어 부르고, 마지막 호출이 DMA 하나로 합친다. `to_dm`은 새 목적지 텐서를 만들고 `to_dm_view`는 타일 같은 기존 뷰에 쓴다. 축 순열은 목적지 매핑이 정하며 어느 마무리 호출을 쓰든 같다.
- **결과 매핑의 한계:** 결과 매핑은 목적지 배치만 기술하고, 어느 원본 배치나 슬라이스 좌표가 값을 공급했는지는 기록하지 않는다. 값의 뜻은 재분배 단계와 그 배열을 읽어서 판단해야 한다.

동기화의 자동 삽입

컴파일러는 완성된 계획에서 동기화를 유도한다. 원본 칩이 바뀌면 `ChipSync`를, 원본 클러스터가 바뀌면 `ClusterSync`를 넣고, 둘 다 바뀌면 둘 다 넣는다. 로컬 데이터만 자르는 계획에는 동기화가 없다. 원격 DMA 접근 전에는 목적지 준비 동기화를, 후에는 완료 동기화를 보내며, 자기 자신에게 가는 칩 경로는 칩 간 송수신을 만들지 않는다.

제약과 성능

- `cluster_swap()`은 클러스터가 정확히 두 개여야 한다. 원본 칩 목록은 순열이어야 하며, 살아 있는 목적지는 모두 살아 있는 원본 칩·클러스터를 읽어야 한다.
- slice 배열은 목적지 칩·클러스터마다 살아 있는 인덱스 하나를 가져야 한다. 합친 계획은 목적지 칩마다 원본 칩 하나, 목적지 클러스터마다 원본 클러스터 하나를 골라야 한다(가용 동기화 프로토콜과 맞추기 위해).
- `chip_slice::<m![B, C]>`는 `m![B, C]`를 선형 인덱스 하나를 가진 연속 매핑으로 취급한다. 떨어져 있는 축을 따로 자르려면 축마다 호출을 이어 붙인다.
- 성능 면에서는 DM 쓰기가 정렬되고 연속 구간이 긴 Element 매핑을 고른다. 로컬 축 하나만 자를 때는 sub 컨텍스트 slice를, shuffle과 합치거나 여러 축을 한 DMA로 자를 때는 뷰 체인을 쓴다.

6.7 Scatter와 Gather

Scatter와 gather는 고정 스트라이드가 아니라 **인덱스 텐서가 계산한 주소**로 원소를 옮긴다. `DmTensor::dma_scatter`는 DM 값을 인덱스가 고른 HBM 행에 쓰고, `HbmTensor::dma_gather_scaled`와 `dma_gather_unscaled`는 인덱스가 고른 HBM 행을 DM으로 읽는다. 두 gather는 인덱스가 어디 있고 값을 어떻게 읽는지만 다르다.

변형	인덱스 위치	인덱스 값의 뜻
<code>dma_gather_scaled</code> , <code>dma_scatter</code>	DRAM의 <code>HbmTensor</code>	축을 따른 바이트 오프셋 . 행 <code>r</code> 을 가리키려면 $r \times$ (행 하나의 바이트 수)를 넣는다. 예: 128원소 bf16 행이면 $r \times 256$
<code>dma_gather_unscaled</code>	칩 위의 <code>DmTensor</code> (DRAM에서 <code>to_dm</code> 으로 올린 것)	행 번호 그대로. paged attention의 블록 테이블처럼 칩에서 계산한 인덱스에 쓴다

```
axes![K = 512, D = 128, C = 612, G = 512, CL = 2];

// scatter: DM의 KxD 값을 HBM 캐시의 index가 가리키는 행에 쓴다
data_dm.dma_scatter::<m![K], _, _>(index, output);

// scaled gather: HBM index(바이트 오프셋)로 table의 행을 DM으로 모은다
table.dma_gather_scaled(index)          // -> DmTensor<bf16, ..., m![G / 2], m![G % 2, D]>

// unscaled gather: 칩 위 DmTensor index(행 번호)를 쓴다
let index_dm = index.to_dm(&mut device.tdma);
table.dma_gather_unscaled(&index_dm)
```

원문 확인

Moving Tensors 장은 "unscaled scatter는 현재 구현되지 않았다"고 적는다. 위 표의 scatter는 scaled 형태다.

6.8 PCIe DMA

PCIe DMA(`device.pdma`)는 호스트 시스템 메모리와 장치 HBM 사이로 텐서를 옮기는 별도의 물리 엔진이다. 호스트 ↔ HBM만 다루고, 칩 안 전송은 모두 Tensor DMA가 맡는다. `HostTensor`의 `.to_hbm()` (호스트 → 장치)과 `HbmTensor`의 `.to_host()` (장치 → 호스트)는 모두 async다. 호스트 메모리에는 칩·클러스터·슬라이스 분할이 없으므로 `HostTensor`는 Element 매핑만 갖고, 목적지 `HbmTensor`가 Chip 축을 더해 칩에 분배한다. `to_hbm`이 새 Chip과 Element를 받으므로 HBM 안의 배치는 호스트 배치와 달라도 된다.

주의

PCIe DMA 대역폭은 30 B/cycle로 칩 안 Tensor DMA(256 B/cycle)보다 한 자릿수 느리다. 데이터는 한 번 올리고 여러 칩 안 연산에서 재사용하도록 알고리즘을 짤다.

6.9 DM의 물리 구조

원문 Memory Performance 장은 메모리 성능이 커널 처리량을 제한하는 경우가 많다고 말한다. Fetch, Commit, DMA의 API 선택(패킷 크기, 접근 순서)은 곧바로 성능으로 이어진다. 칩당 최대 대역폭은 DM 2 TB/s, HBM 1.5 TB/s이지만, 이 값에 도달하려면 특정 접근 패턴이 필요하다. 먼저 규칙 표를 보고 각 규칙의 이유를 살핀다.

메모리	문제	규칙	위반 시 벌칙
DM	뱅크 기아	같은 뱅크 연속 접근 64회 미만	NoC 타임아웃 → 하드웨어 리셋
DM	DMN 인터리빙	클러스터의 DMN 두 개를 번갈아	대역폭 50% 손실
DM	슬라이스 인터리빙	DMN당 슬라이스 32개에 분산	명령 큐 경합

메모리	문제	규칙	위반 시 벌칙
HBM	정렬	256바이트 정렬 접근	정렬 안 된 읽기 2배, 쓰기 약 50배 (RMW)
HBM	뱅크 충돌	같은 뱅크 안에서 행 전환을 피한다	30~40배 저하
HBM	채널 인터리빙	채널 32개에 분산	병렬성 감소

DM은 칩당 256 MB이고 클러스터, DMN(Data Memory Network), 슬라이스, 뱅크로 계층을 이룬다.

단위	개수
클러스터	칩당 2
DMN	클러스터당 8
슬라이스	DMN당 32
뱅크	슬라이스당 16
행	뱅크당 4,096
바이트	행당 8

슬라이스 하나는 $16 \times 4,096 \times 8 = 524,288$ 바이트(512 KB)이고, 클러스터의 슬라이스 256개는 DMN 8개 $\times$ 32개다. 클러스터끼리는 Switch Engine으로도 데이터를 주고받을 수 있다.

슬라이스 안의 뱅크 16개

슬라이스의 512 KB는 데이터 폭 8바이트인 뱅크 16개로 나뉜다. 뱅크 하나에 대한 접근은 직렬이지만, 주소 공간이 연속한 128바이트를 뱅크 16개에 8바이트씩 흩어 놓아서 한 사이클에 128바이트를 병렬로 접근할 수 있다. 주소 비트 0~2는 바이트, 3~6은 뱅크, 7~18은 행이다(그림 20).

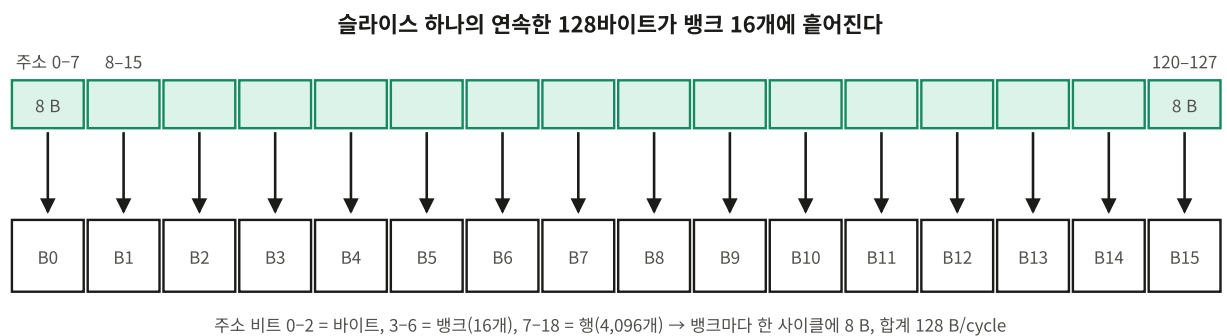


그림 20. DM 주소의 하위 비트가 뱅크를 정하므로 연속 주소를 순서대로 읽거나 쓰면 요청이 16개 뱅크로 자연히 퍼진다. 반대로 뱅크 번호 비트(3~6)가 같은 주소만 반복해 두드리는 패턴은 뱅크 하나의 속도에 묶인다.

DMN과 슬라이스 인터리빙

DMN 하나의 대역폭은 128 B/cycle뿐이다(DMN의 슬라이스 32개가 데이터 경로를 공유한다). 표준 전송 단위인 256바이트는 DMN 하나에서 2사이클이 걸리므로, 두 DMN에 걸쳐 접근을 파이프라인해야 끊임 없이 256 B/cycle을 낸다.

사이클	DMN 0	DMN 1
0	읽기 #0 (1/2)	(대기)

사이클	DMN 0	DMN 1
1	읽기 #0 (2/2)	읽기 #1 (1/2)
2	읽기 #2 (1/2)	읽기 #1 (2/2)
3	읽기 #2 (2/2)	읽기 #3 (1/2)
...	...	...

명령 큐가 이론상 약간의 몰아서 접근을 허용하지만, 원문은 DMA 스트림을 만들 때 항상 DMN을 인터리빙하라고 권한다. 슬라이스는 DMA·Fetch·Commit Engine이 함께 쓰므로 DMN당 32개 슬라이스에 요청을 흘리면 경합이 준다. Data Memory Router가 DMN 안의 슬라이스를 링으로 잇고(slice0_in → slice31_out, slice32_in → slice63_out), 슬라이스마다 대기 중인 DMA 요청을 담은 **2칸짜리 명령 큐**가 있다. 요청을 M개 슬라이스에 흘리면 슬라이스당 필요한 처리량이 1/M로 줄고, n사이클마다 DMN을 번갈아 쓰면 256 B/cycle 포화 대역폭에 도달한다.

6.10 뱅크 기아와 64회 규칙

뱅크 기아(bank starvation)는 DMA Engine이 우선순위 높은 엔진이 붙잡은 DM 뱅크를 끝없이 기다리는 상황이다. DM 뱅크는 공유 자원이고, DM 컨트롤러는 다음 순서로 요청에 우선순위를 준다.

1. main 컨텍스트 Fetch Engine
2. main 컨텍스트 Commit Engine
3. sub 컨텍스트 Fetch Engine
4. sub 컨텍스트 Commit Engine
5. DMA Engine

정상 동작에서 계산 엔진이 데이터에 먼저 접근해야 하므로 DMA가 가장 낮다. 문제는 높은 엔진이 같은 뱅크를 계속 쓸 때다. Tensor DMA는 NoC 허브를 거쳐 DRAM·DMN과 통신하고, 허브의 각 포트(DMA, DRAM, DMN)는 **4,096사이클 안에** 요청을 확인(acknowledge)해야 한다. 응답 없이 4,096사이클이 지나면 NoC 프로토콜이 트랜잭션을 죽은 것으로 보고 예외 상태로 들어간다. 교착이나 멈춘 트랜잭션을 잡기 위한 안전장치이지만, 하드웨어에는 우아한 복구 수단이 없어 **클러스터 도메인 전체를 리셋**하고 모든 계산 상태를 잃는다.

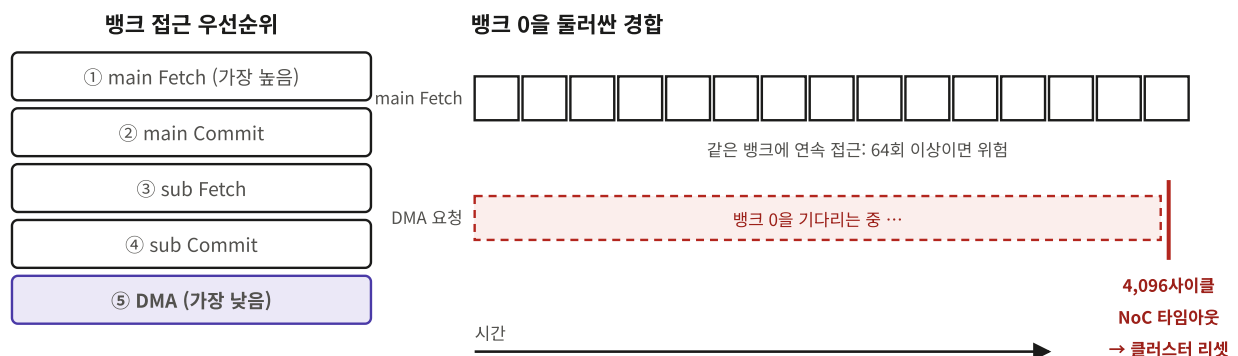


그림 21. 뱅크 기아. 우선순위가 가장 낮은 DMA는 높은 엔진이 뱅크를 놓아줄 때까지 기다리며, 4,096사이클 안에 응답을 받지 못하면 복구할 수 없는 리셋이 일어난다.

한계가 **64**인 이유는 최악의 경우에도 DMA 요청이 타임아웃 전에 끝나야 하기 때문이다. 원문의 식은 다음과 같다.

$$(TDMA_IO_BYTE / DMN_IO_BYTE) \times \text{Max_Consecutive_Access} \times DMN_SIZE < 4096$$

$$(256 / 128) \times \text{Max} \times 32 < 4096 \Rightarrow \text{Max} < 64$$

예를 들어 DMA Engine이 뱅크 0(과 다른 뱅크 15개)에 요청을 넣었는데 main 컨텍스트의 Fetch Engine이 뱅크 0을 계속 요청하면 DMA 요청이 멈추고, 이것이 4,096사이클을 넘으면 리셋이 난다. 그래서 DMA가 활성인 동안 Fetch와 Commit은 같은 뱅크를 64번 이상 붙잡으면 안 된다. 스케줄러가 이 규칙을 어떻게 강제하는지는 § 12.1에서 본다.

6.11 HBM의 물리 구조

HBM은 칩당 48 GB이고 합계 1.5 TB/s의 대역폭을 내지만, 그러려면 256바이트 정렬 접근과 32채널 인터리빙이 필요하다. 정렬 위반과 뱅크 충돌은 처리량을 30~50배까지 떨어뜨릴 수 있다.

단위	개수
스택	칩당 2
채널	스택당 16
슬라이스	채널당 3
뱅크 그룹	슬라이스당 4
뱅크	뱅크 그룹당 4
행	뱅크당 16K
바이트	행당 2K

최대 대역폭

채널 컨트롤러는 0.75 GHz에서 사이클당 64 B를 옮긴다. 원문 각주에 따르면 사이클마다 버스트 8번을 하므로 실효 주파수는 $0.75 \times 8 = 6$ GHz다. 그래서 채널당 48 GB/s($0.75 \text{ GHz} \times 64 \text{ B/cycle}$), 칩당 1.5 TB/s($48 \text{ GB/s} \times 32 \text{ 채널}$)가 나온다. 기본 전송 단위는 256바이트이며 채널당 4클럭 사이클이 걸린다. DMA 엔진 하나(약 256 GB/s)를 포화시키려면 여러 채널에 접근을 번갈아 해야 한다. 채널 컨트롤러마다 64칸짜리 명령 큐가 있어 접근을 섞어 벌칙을 줄이지만, 병리적인 패턴은 여전히 크게 느려진다.

칩 주소 공간의 비트 배치

HBM 주소는 순차 접근이 자연히 병렬이 되도록 비선형으로 비트를 배치한다(그림 22).

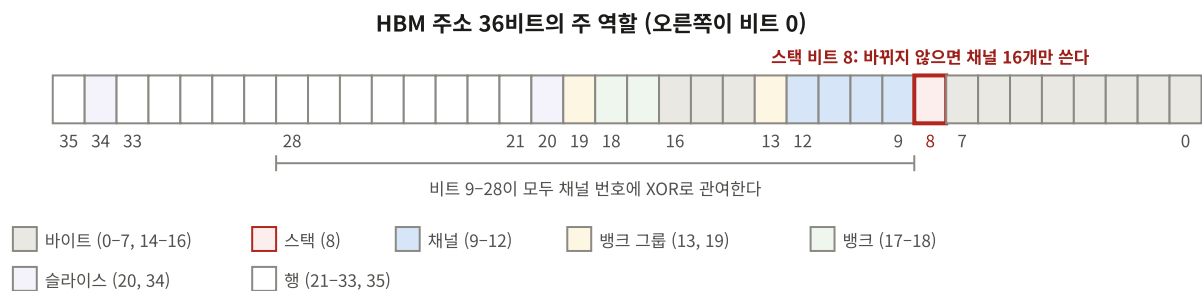


그림 22. HBM 주소 비트의 주 역할. 가장 낮은 8비트가 256바이트 전송 단위 안의 바이트이고, 그 바로 위의 스택 비트와 채널 비트가 연속한 256바이트 블록들을 서로 다른 스택·채널로 보낸다.

각 성분의 비트 수는 물리 기하와 맞는다. 예를 들어 바이트 성분은 비트 0~7과 14~16의 11비트로 행당 $2K(2^{11})$ 바이트를 표현한다. 예외가 세 가지 있다.

- **슬라이스**: 슬라이스가 세 개뿐인데도 두 비트(20, 34)를 쓴다.
- **연속 주소 공간**: 비트 34는 행 성분의 영향을 받아 비트 34와 35가 동시에 1이 되지 않는다. 그래서 48 GB 주소 공간이 연속이다.
- **채널 XOR 대응**: 채널 번호는 비트 9~12와 13~28의 XOR로 정해진다. 예를 들어 채널의 첫 비트는 비트 9, 13, 21, 25의 XOR다.

이 비트 순서 덕분에 순차 접근이 스택, 채널, 뱅크 그룹, 뱅크에 동시에 흩어진다. § 4.3에서 본 "매핑으로 비트 순서를 재배치한다"는 설명이 바로 이런 하드웨어를 겨냥한 것이다.

정렬이 어긋난 접근

256바이트 최소 접근 단위는 가장 낮은 8비트로 정해진다. 이 경계를 넘는 접근은 벌칙을 치른다.

- **정렬 안 된 읽기**: 256바이트 경계를 가로지르는 읽기는 NoC 전송이 두 번 필요해 대역폭이 사실상 절반이 된다.
- **정렬 안 되거나 일부만 쓰는 쓰기**: DMA 패킷은 내부적으로 256바이트 트랜잭션으로 쪼개진다. 2,800바이트 패킷이라면 256바이트 요청 10개와 240바이트 요청 1개가 되고, 마지막 "남은" 트랜잭션은 RMW가 된다. RMW는 256바이트 단위 전체를 읽고, 요청한 바이트를 고친 뒤, 전체를 다시 쓴다. 정렬된 쓰기보다 약 **50배** 느릴 수 있다.

뱅크 충돌

HBM 뱅크는 한 번에 행 하나만 열어 둔다. 같은 뱅크의 다른 행으로 가려면 현재 행을 닫고 새 행을 열어야 하며, 이때 40~50 ns(1.5 GHz 기준 60~75사이클)가 들어 이미 열린 행에 접근하는 것보다 30~40배 느리다. 모든 행은 닫힌 상태로 시작하므로 어떤 행이든 첫 접근은 여는 비용을 치른다.

채널 인터리빙이 뱅크 충돌을 완화한다. 비트 8~12(바로 다음 다섯 비트)가 독립적인 스택과 채널을 나타내므로, 이들을 낮은 주소에 둔 설계가 이웃 접근끼리의 간섭을 막아 연속 연산을 병렬화한다. 비연속 연산도 채널 성분이 비트 9~28에 걸쳐 있어 자연히 채널이 섞이는 경우가 많다. 그러나 **스택 성분은 비트 8 하나뿐**이므로, 두 스택을 번갈아 쓰도록 하는 것은 프로그래머가 명시적으로 챙겨야 한다.

컨트롤러도 행 전환 지연을 숨기려 한다. 채널 안에서 여러 뱅크로 명령을 섞어, 어떤 뱅크가 행을 바꾸는 동안 다른 뱅크로 유용한 전송을 하고, 명령 큐에서 **FR-FCFS**(First Ready-First Come First Served) 방식으로 이미 행이 열린 명령을 먼저 처리한다. 그래도 같은 뱅크 안에서 행을 계속 바꾸는 접근 패턴은 여전히 크게 느리므로, 원문은 컴파일러와 프로그래머가 코드를 만들 때 행 전환 비용을 추정하라고 권한다.

명령 사이 최소 간격 tCCD

tCCD (Column-to-Column Delay)는 같은 채널에서 연속한 읽기·쓰기 명령 사이의 최소 시간으로, 명령을 낼 수 있는 최대 속도를 정한다. 대부분은 뱅크 충돌이나 채널 인터리빙이 먼저 병목이 되지만 값은 다음과 같다.

연속한 명령의 관계	tCCD (1.5 GHz 사이클)	상대 성능	벌칙의 이유
같은 슬라이스, 다른 뱅크 그룹	2	1	뱅크 그룹의 이상적인 인터리빙
다른 슬라이스	3	2/3	데이터 경로 전환
같은 슬라이스, 같은 뱅크 그룹	4	1/2	네 뱅크가 I/O 버퍼를 공유

한 슬라이스 안에서 뱅크 그룹을 번갈아 쓰면 $t_{CCD} = 2$ 사이클(1.5 GHz)이라, 0.75 GHz 사이클마다 새 64 B 명령을 내 채널 속도를 다 쓴다. t_{CCD} 벌칙은 뱅크 충돌보다 덜 심각하다. 다른 슬라이스 간 인터리빙($t_{CCD} 3$)은 슬라이스 비트 20 바로 위가 행 비트 21이라 뱅크 충돌을 함께 일으키기 쉬워 t_{CCD} 가 부차적 효과가 되고, 같은 슬라이스·같은 뱅크 그룹 패턴($t_{CCD} 4$)은 채널 인터리빙에 가려지는 경우가 많기 때문이다.

스스로 점검

1. DMA 명령 3개로 나눈 전송과 1개로 합친 전송의 시작 지연 차이는 대략 몇 사이클인가?
2. DM 슬라이스 주소 0x40과 0x48은 같은 뱅크인가? (비트 3~6이 뱅크)
3. HBM에 2,800바이트 패킷을 쓰면 왜 느린가? 어떻게 고치나?
4. 64회 규칙의 식에서 32는 무엇을 뜻하나?

답: ① 약 1,000사이클(명령당 약 500). ② $0x40 = 64 \rightarrow$ 비트 3~6 = 8, $0x48 = 72 \rightarrow 9$. 다른 뱅크다. ③ 256바이트 단위 10개 + 240바이트 조각 1개로 쪼개져 마지막 조각이 RMW(약 50배)가 된다. 256의 배수로 맞춘다. ④ DMN 하나에 속한 슬라이스 수 (DMN_SIZE).

7장

Tensor Unit 연산 I: 스트림을 계산할 준비

원문: Computing Tensors · Fetch Adapter · Switch Engine · Collect Engine · Register Files

Computing Tensors 장은 올바른 Tensor Unit 스트림이 Vector Engine이나 Contraction Engine에 도착하기까지를 설명한다. 이 장은 그중 계산 엔진 **앞쪽**을 다룬다. Fetch 직후 원소를 바꾸는 Fetch Adapter, 슬라이스 사이로 데이터를 옮기는 Switch Engine, 패킷을 32바이트 플릿으로 맞추는 Collect Engine, 그리고 고정 피연산자를 담는 레지스터 파일 TRF와 VRF다.

이 장에서 배우는 것

- 필요한 계산에 따라 엔진 경로를 고르는 법과 main·sub 컨텍스트의 차이
- Fetch Adapter의 테이블 조회(FP4 해독), 타입 변환, zero-point 빼기
- Switch Engine의 링 구조, 정규 설정 다섯 가지, 비용 공식, 사용자 정의 설정의 제약
- Collect Engine이 패킷을 패딩하고 쪼개는 규칙
- TRF·VRF의 인터페이스, 저장 위치와 점유 컨텍스트, TRF의 물리 구조와 이중 버퍼링

7.1 계산 경로 고르기

원문은 커널이 필요로 하는 일에 따라 다음 경로를 고르라고 안내한다. 고정 피연산자는 레지스터 파일에 머물고 다른 피연산자가 엔진을 흐른다는 원칙은 모든 경로에 공통이다.

커널이 필요로 하는 것	고를 엔진	다음에 읽을 것
원소별 연산이나 리듀스	Vector Engine	Vector 스트림과 리듀스 규칙(9장)
행렬곱·컨볼루션 같은 축약	TRF + Contraction Engine	TRF 배치와 축약 매핑(8장)
슬라이스 사이 재분배나 리듀스	Switch Engine 또는 Inter-Slice Reducer	슬라이스 이동과 reducer 규칙(이 장, §9.9)
배치나 정밀도 변경	Fetch·Commit Adapter, Cast Engine, Transpose Engine	어댑터 단계와 출력 매핑 규칙(이 장, 10장)

스케줄러는 **실행 컨텍스트**를 독립적인 연산 흐름으로 다룬다. main은 Tensor Unit의 모든 엔진을 쓸 수 있다. sub는 Contraction Engine과 몇몇 기능이 빠진 부분집합이다(예: 테이블 조회와 Commit Adapter의 타입 변환은 main 전용). 나머지는 main과 같다. DMA 컨텍스트는 Tensor Unit 밖의 DMA Engine만 다룬다.

7.2 Fetch Adapter

Fetch Adapter는 Fetch Engine이 내보낸 패킷 스트림에 원소별 변환을 적용한다. 위치는 Fetch Engine과 Switch Engine 사이이며, Fetch Engine 자체는 이런 변환을 하지 않는다. 쓸 수 있는 단계는 셋이고 모두 선택 사항이다. 하드웨어 파이프라인 순서대로 `FetchTensor`에서 메서드를 부르면 되며, 아무것도 부르지 않고 Switch나 Collect로 바로 가도 된다.

1. **테이블 조회(Table Lookup)**: 하드웨어 조회 테이블로 값을 바꾼다.

2. **타입 변환(Type Casting)**: 원소 타입을 바꾼다.

3. **zero-point 빼기(Zero-Point Subtraction)**: 양자화 zero-point를 빼고 정수 스트림을 Contraction용 중간 타입으로 넓힌다($i4 \rightarrow i5$, $i8 \rightarrow i9$).

테이블 조회

fetch 단계에서 각 값을 미리 설정한 테이블의 인덱스로 보고 해당 항목을 출력한다. 일반 산술로 효율적으로 구현하기 어려운 연산에 쓴다. Sigmoid·GeLU 같은 비선형 활성화를 미리 계산한 테이블로 구현하거나, MXFP4·NVFP4 같은 특수 인코딩을 표준 형식으로 해독한다. 원문은 Sigmoid와 GeLU를 Vector Engine에서 직접 표현할 수도 있으므로 테이블 조회는 여러 선택지 중 하나라고 덧붙인다.

입력	출력	테이블
f4e2m1	f8e4m3, f8e5m2	짝(paired) 4비트 $\rightarrow$ 8비트 테이블
f8e4m3	bf16	짝 아닌 테이블
f8e5m2	bf16	짝 아닌 테이블

테이블은 런타임 인자가 아니라 **입력 스칼라 타입**(TableLookupCast 트레이트)으로 선택된다. 키 폭이 테이블을 고르고, 요청한 출력 타입이 항목의 인코딩을 고른다. 해독한 8비트 실수는 뒤따르는 `fetch_cast`로 f32까지 넓히고, NVFP4·MXFP4의 **블록별 스케일**은 뒤에서(Vector Engine이나 Contraction Engine에서) 곱한다. 스케일을 테이블에 넣으면 블록마다 다른 테이블이 필요해져 공유 정적 테이블의 의미가 사라지기 때문이다. f4e2m1 해독 테이블은 16항목짜리 블록 무관 해독이며 fetch 시퀀서 설정에 컴파일 시간 상수로 박혀 있어, 호출마다 테이블을 올리는 작업이 없다.

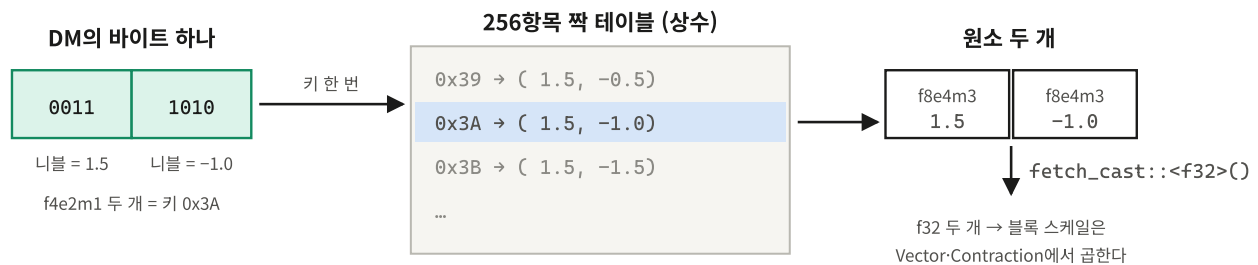


그림 23. 짝 테이블 조회. 하드웨어 시퀀서는 1~2바이트로 정렬된 키만 훑고 4비트 니블 하나를 키로 쓰지 않는다. 그래서 4비트 키는 짝 테이블을 쓴다. 한 바이트(니블 두 개)가 키이고, 한 번 조회로 f8e4m3 값 한 쌍을 얻는다. 원문은 이 사실만 적으며, 그림의 니블 순서와 값은 설명을 위한 예다.

```
axes![A = 8];
// e2m1(NVFP4 / MXFP4) 가중치 스트림을 하드웨어 테이블로 f8e4m3로 해독한 뒤 f32로 넓힌다
fn fetch_decode_e2m1<'l, const T: Tu>{
    input: BeginTensor<'l, T, f4e2m1, m![1], m![1], m![1], m![1], m![A]>,
) -> FetchCastTensor<'l, T, f32, m![1], m![1], m![1], m![1], m![A]> {
    input
        .fetch::<m![1], m![A]>()
        .fetch_table_lookup::<f8e4m3>()
        .fetch_cast::<f32>()
}
```

주의: 테이블 조치는 main 전용

상주 조회 테이블을 가리킬 수 있는 레지스터(`mode_indirect_table`)는 main Fetch Unit에만 있다. TRF 가중치 적재에 쓰이는 sub 컨텍스트 Fetch Unit에는 이 레지스터가 없다. 그래서 압축된 가중치를 Contraction에 쓰려면 main 컨텍스트 fetch에서 해독해 f8e4m3 스트림을 DM에 commit한 뒤, 그 DM 텐서를 단순 변환만 하는 StoTrf로 TRF에 올린다. f4e2m1 스칼라 타입은 니블 두 개를 담은 바이트로 모델링되며 BITS = 4로 표시해 fetch 매핑이 바이트가 아니라 원소 수를 세게 한다.

타입 변환

`fetch_cast::<OutD>()`는 원소 타입을 D에서 OutD로 바꾸고 Time-Packet 매핑은 그대로 둔다. 지연은 1~2사이클이다. RNGD가 지원하는 변환은 아래가 전부다. 넓히는 변환은 모두 연산 폭인 i32·f32로 가고, 좁히는 변환은 f32 → bf16 하나뿐이다.

입력	출력
i4, i8, i16	i32
f8e4m3, f8e5m2, bf16	f32
f32	bf16

8비트 실수를 bf16으로 바꾸는 캐스트는 없으므로 짝 아닌 테이블 조회로 해독하고, f32를 8비트 실수로 줄이려면 파이프라인 뒤쪽의 Cast Engine을 쓴다. 또 캐스트는 `read_size`에 제한을 더한다. **fetch 한 번의 캐스트 결과가 32바이트 플릿 하나에 들어가야 한다.**

- 가능: i4 → i32, `read_size` = 8원소(4 B) → $8 \times 4 = 32$ B. i8 → i32, `read_size` = 8원소(8 B) → 32 B.
- 불가: i4 → i32, `read_size` = 16원소(8 B) → 64 B. i8 → i32, `read_size` = 16원소 → 64 B.

§ 5.5의 예제가 i8을 8개씩 읽는 이유가 이것이다. zero-point를 담는 정수 확장(i4 → i5, i8 → i9)은 캐스트가 아니라 다음 단계의 일이며, `fetch_cast`는 i5·i9를 만들지 않는다.

zero-point 빼기

`fetch_zero_point_sub::<OutD>(zero_point)`는 원소마다 양자화 zero_point를 빼고 스트림을 Contraction Engine의 중간 타입으로 넓힌다. i4는 i5로, i8은 i9로 간다. i5·i9를 만드는 유일한 단계다. 한 비트가 더 필요한 이유는 § 1.3에서 본 대로 같은 폭 두 값의 차가 범위를 두 배로 늘리기 때문이다(i4: [-8, 7] - [-8, 7] = [-15, 15], i9: [-255, 255]). zero_point가 원본 타입의 범위 밖이면 panic한다. 범위 안의 zero_point라면 모든 잔차가 OutD 안에 들므로, 데이터와 무관한 이 검사 하나로 충분하다.

핵심: i5·i9는 Contraction 전용

i5·i9 스트림은 Switch Engine과 Collect Engine을 지나갈 수는 있지만, 그 뒤 합법적인 소비자는 `contract_outer` 하나뿐이다. 메모리에 commit할 수도, 레지스터 파일에 저장(`to_trf · to_vrf`)할 수도, 전치하거나 다른 엔진에 넣을 수도 없다. 이 제한은 런타임 검사가 아니라 컴파일 시간에 강제된다(i5·i9는 `MaterializableScalar`가 아니다).

7.3 Switch Engine: 슬라이스 사이의 링

다른 Tensor Unit 엔진은 모두 슬라이스별로 자기 DM 구획에서만 일한다. **Switch Engine만 슬라이스 사이로 데이터를 옮긴다.** 256개 슬라이스를 잇는 링 네트워크로, 한 슬라이스의 값을 그룹에 브로드캐스트하거나, 슬라이스끼리 값을

맞바꾸거나, 어느 슬라이스가 어느 값을 갖는지 순열로 바꾼다.

```
// FetchTensor에서 부른다. Chip·Cluster·Packet과 데이터 값은 보존, Slice와 Time만 바뀐다
pub fn switch<OutSlice: M, OutTime: M>(self, config: SwitchConfig)
    -> SwitchTensor<'l, T, D, Chip, Cluster, OutSlice, OutTime, Packet>;
```

작성자는 `OutSlice`, `OutTime`, 그리고 설정을 고르는 `SwitchConfig`를 정한다. `SwitchConfig`는 흔한 패턴을 위한 정규 변형 `Broadcast01`, `Broadcast1`, `Transpose`, `InterTranspose`, `TransposedBroadcast1`과 일반 패턴을 위한 `CustomBroadcast` 중 하나다. 이름의 숫자는 Slice에서 빠져나가는 슬라이스 하위 차원을 가리킨다(`Broadcast01`은 `slice0`과 `slice1`을, `Broadcast1`은 `slice1`만 브로드캐스트). 컴파일러가 `OutSlice·OutTime`이 설정의 차원 구조와 맞는지 검사하며, 모든 설정은 `InSlice::SIZE == OutSlice::SIZE`를 요구한다.

정규 설정

정규 설정은 칩의 256개 슬라이스를 `ring_size` 개씩 묶은 병렬 **부분 링(sub-ring)**으로 나눈다. `ring_size`는 보통 `slice1 × slice0`으로 설정의 매개변수에서 유도되며, 분할의 세밀함과 비용을 함께 정한다. 브로드캐스트 축(X, Y)을 새로 만드는 설정은 그 축이 입력 Slice나 입력 Time에 이미 있으면 안 된다.

설정	쓰임새	ring_size
Forwarding	슬라이스 간 교환 없이 그대로 통과. <code>SwitchConfig</code> 에 이 변형은 없고, 필요 없으면 <code>.switch()</code> 를 건너뛰고 <code>.collect()</code> 를 부른다	1
<code>Broadcast01</code>	Slice의 안쪽 두 하위 차원(<code>slice1</code> , <code>slice0</code>)을 부분 링 전체에 브로드캐스트	<code>slice1 × slice0</code>
<code>Broadcast1</code>	<code>slice1</code> 만 브로드캐스트하고 <code>slice0</code> 은 Slice에 유지	<code>slice1 × slice0</code>
<code>Transpose</code>	Slice 안에서 <code>slice1</code> 과 <code>slice0</code> 을 맞바꿈	<code>slice1 × slice0</code>
<code>InterTranspose</code>	Slice 하위 차원(<code>slice1</code>)과 Time 하위 차원(<code>time1</code>)을 맞바꿈	<code>slice1 × slice0</code>
<code>TransposedBroadcast1</code>	<code>slice0</code> 을 Time으로 브로드캐스트하면서 <code>slice1</code> 을 Slice 가장 안쪽으로 (<code>Transpose</code> 후 <code>Broadcast1</code>)	<code>slice1 × slice0</code>

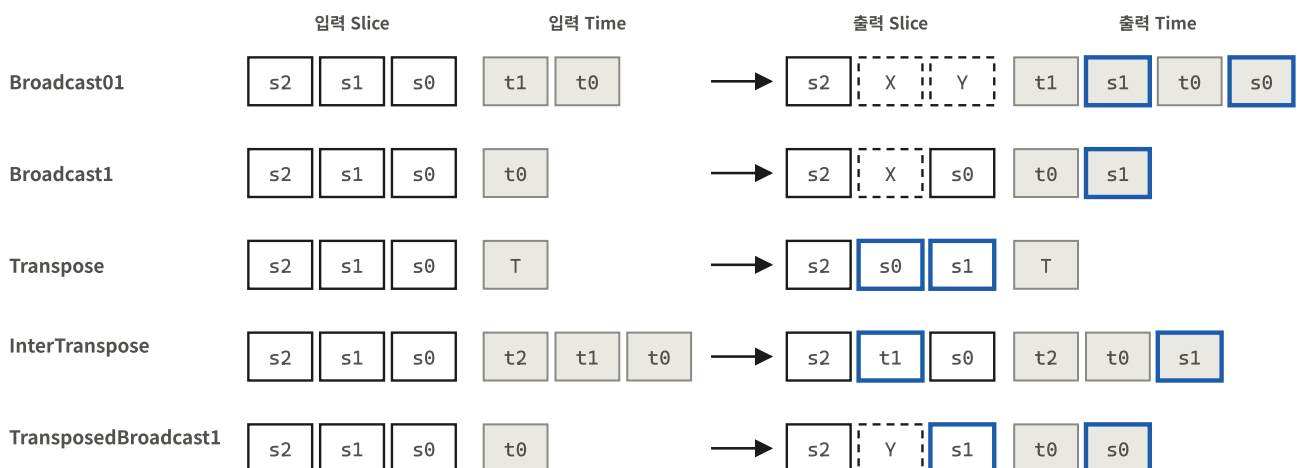


그림 24. 정규 설정의 차원 이동. 흰 칸은 Slice의 하위 차원, 회색 칸은 Time의 하위 차원(왼쪽이 바깥)이다. 파란 테두리는 자리를 옮긴 차원, 점선은 비워진 Slice 자리를 채우는 새 브로드캐스트 축이다. Slice에서 Time으로 간 차원은 항상 Time의 안쪽 자리로 간다.

원문의 `Broadcast01` 예로 매개변수가 매핑으로 바뀌는 과정을 보자. `axes![A = 256, B = 64, C = 63, D = 2, X = 2, Y = 2]`, 입력 Slice `m![A]`, Time `m![B]`, Packet `m![C # 64]` (f32)에 `SwitchConfig::Broadcast01 { slice1:`

2, slice0: 2, time0: 4 } 를 주면 컴파일러가 slice2 = 64, time1 = 16, ring_size = 4(부분 링 64개가 256개 슬라이스를 덮음)를 유도한다. 하위 차원은 slice2 = $A / 4$, slice1 = $A / 2 \% 2$, slice0 = $A \% 2$, time1 = $B / 4$, time0 = $B \% 4$ 가 되어 OutSlice = m![A / 4, X, Y], OutTime = m![B / 4, A / 2 \% 2, B \% 4, A \% 2] 다. 부분 링의 모든 슬라이스가 자기 패킷을 링에 돌리고 모든 슬라이스가 ring_size개의 패킷을 다 받으므로, 출력 슬라이스마다 그룹 전체의 데이터를 새 X, Y 축으로 인덱싱해 갖게 된다.

원문 예제 요약 (A = 256 슬라이스, 사이클 = ring_size × Time::SIZE × flits_per_packet)

설정	매개변수	OutSlice / OutTime	ring	사이클
Broadcast01	slice1 2, slice0 2, time0 4 (f32, Time 64, 패킷 256 B)	m![A / 4, X, Y] / m![B / 4, A / 2 \% 2, B \% 4, A \% 2]	4	$4 \times 64 \times 8 = 2,048$
Broadcast1	slice1 4, slice0 8 (i8, 64 B)	m![A / 32, X, A \% 8] / m![B, A / 8 \% 4]	32	$32 \times 64 \times 2 = 4,096$
Transpose	slice1 32, slice0 2 (i8, 64 B)	m![A / 64, A \% 2, A / 2 \% 32] / m![B]	64	$64 \times 64 \times 2 = 8,192$
InterTranspose	slice1 2, slice0 16, time0 2 (i8, Time 8, 32 B)	m![A / 32, B / 2 \% 2, A \% 16] / m![B / 4, B \% 2, A / 16 \% 2]	32	$32 \times 8 \times 1 = 256$
TransposedBroadcast1	slice1 8, slice0 8 (i8, Time 16, 32 B)	m![A / 64, Y, A / 8 \% 8] / m![B, A \% 8]	64	$64 \times 16 \times 1 = 1,024$

Transpose 는 입력과 출력 Time이 (정규화 후) 같아야 한다. InterTranspose 에는 크기 제약이 셋 있다. 입력 Slice가 256개 전체를 덮어야 하고(slice2 × slice1 × slice0 = 256), 맞바꾸는 두 차원의 크기가 같아야 하며(time1 = slice1), 입력 Time 크기가 slice1 × time0으로 나누어떨어져야 한다. 원문은 목적지 슬라이스가 모두 원본 영역의 사본을 이미 갖고 있다면 **축 올리기**(§ 5.3)가 부분 링 순회 없이 InterTranspose 를 대신할 수 있다고 적는다.

구조와 비용

Switch Engine은 칩의 256개 슬라이스를 슬라이스마다 라우터 하나씩인 **물리 링 하나**로 잇고, 이를 ring_size개씩의 병렬 부분 링 256 / ring_size개로 나눈다. 링크는 양방향이다. 정규 설정은 매개변수(slice1, slice0, time0)에서 ring_size를 유도하고, CustomBroadcast 는 작성자가 직접 정한다.

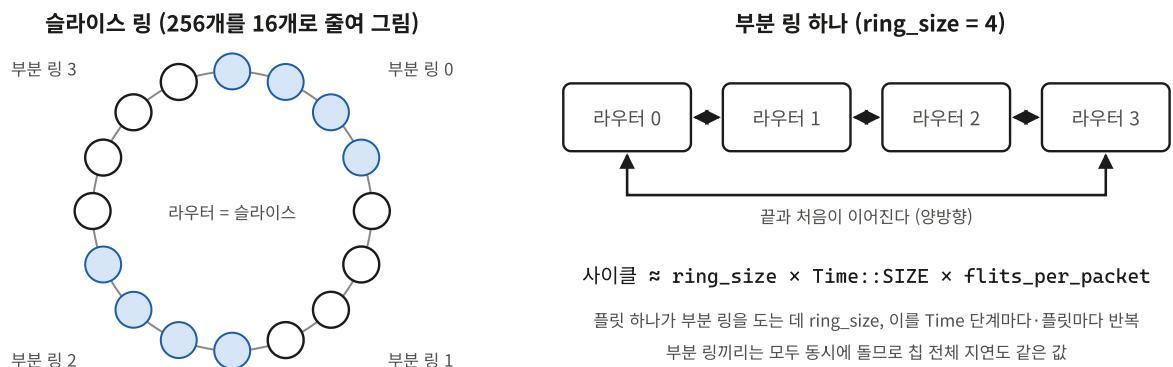


그림 25. Switch Engine의 링. ring_size가 크면 부분 링 하나가 더 많은 슬라이스에 닿지만 비용이 크고, 작으면 병렬 부분 링이 많아지고 비용이 작다. flits_per_packet은 $D[\text{Packet::SIZE}]$ 를 32바이트로 나눈 값이다.

사용자 정의 설정: CustomBroadcast

정규 설정으로 표현할 수 없는 이동, 예를 들어 임의의 차원 순열이나 일부 값만 뽑아내기에는 `SwitchConfig::CustomBroadcast { ring_size }` 를 쓴다. 정규 설정은 라우팅용 **snoop 비트맵**을 내장 생성기로 만들지만, 사용자 정의 설정은 컴파일러가 입력·출력 매핑과 `ring_size`로부터 비트맵을 직접 합성한다. 두 가지 패턴을 추가로 표현한다.

- **브로드캐스트를 곁들인 자유 전치**: 분할 차원의 임의 순열과 브로드캐스트.
- **일부 차원 추출**: 브로드캐스트하면서 차원 값의 일부만 Time으로 옮긴다. 정규 설정은 항상 차원 전체를 옮긴다.

원문 예	무엇을 하나	ring	사이클
임의 순열	안쪽 슬라이스 하위 차원 네 개($A / 4$, $A \% 4$, $B / 4$, $B \% 4$)의 순서를 [3, 2, 1, 0]으로 뒤집는다. 반복 구조가 없어 입력·출력 슬라이스가 링에서 임의로 멀 수 있으므로 최대 링이 필요하다	256	$256 \times 8 \times 8 = 16,384$
여러 차원 브로드캐스트	떨어져 있는 두 차원 $A \% 2$, $B \% 2$ 를 Slice에서 Time으로. Broadcast01은 두 차원이 붙어 있어야 해서 불가	32	$32 \times 8 \times 8 = 2,048$
일부 추출	$C \# 4$ (유효 3 + 패딩 1)에서 유효한 3개만 Time으로 옮기고 패딩 칸은 버린다	4	$4 \times 24 \times 1 = 96$

사용자 정의 비트맵을 쓰면 설정 데이터를 Switch Engine의 SFR(특수 기능 레지스터)에 스트리밍해야 하고, 이 쓰기가 끝날 때까지 **DMA Engine**과 **sub 컨텍스트**를 함께 점유한다. 고정된 사이클 정지가 아니라 스케줄링 병렬성이 줄어드는 형태로 비용이 나타난다. 원문이 나열하는 제약은 일곱 가지다.

1. **브로드캐스트 축은 새것이어야 한다**: 출력 Slice에 새로 넣는 축은 입력 Slice나 입력 Time에 없어야 한다.
2. **브로드캐스트 축은 정확히 한 번**: 같은 축을 출력 Slice 두 자리에 쓰면 비트맵의 뜻이 없다.
3. **브로드캐스트 축에는 패딩 금지**: 패딩 자리의 라우팅 목적지가 정의되지 않는다.
4. **순서 보존**: Slice에서 Time으로 가는 축들은 입력 Slice에서의 상대 순서를 지켜야 한다. 라우터마다 버퍼가 패킷 하나뿐이라, 받아마자 로컬로 내보낼지 전달할지 정해야 하고 여러 패킷을 모아 재정렬할 수 없기 때문이다.
5. **Time의 가장 안쪽 자리**: Slice에서 Time으로 가는 축은 출력 Time의 가장 안쪽 자리를 차지해야 한다. 다른 슬라이스에서 온 데이터는 각 패킷 안에서 마지막에 도착하므로 자연히 가장 안쪽이 되고, 다른 곳에 두려면 시간 시퀀스 전체를 버퍼링·재정렬해야 하는데 하드웨어가 못 한다. (`Broadcast01`은 `time0` 매개변수로 이를 우회하지만 사용자 정의 설정에는 그런 장치가 없다.)
6. **버리는 값은 패딩이어야 한다**: 일부 추출에서 남겨 두는 값이 유효 데이터면 입력을 조용히 읽으므로 검증기가 거부한다.
7. **ring_size**: 2의 거듭제곱이어야 하며, 컴파일러가 매핑에서 유도한 값과 다르면 거부한다.

7.4 Collect Engine: 32바이트 플릿으로 맞추기

Collect 뒤의 모든 엔진(Contraction, Vector, Cast, Transpose, Commit)은 **정확히 32바이트인 플릿**을 소비한다. Collect Engine은 임의 크기의 패킷을 두 단계로 플릿 하나로 정규화한다.

1. **패딩**: 입력 패킷을 다음 32바이트 경계까지 채운다. 이미 32바이트 정렬이면 건너뛴다.
2. **분할**: 플릿 경계에서 쪼갬다. 안쪽 32바이트가 새 Packet이 되고, 바깥의 플릿 개수는 새 Time에 흡수된다. 이미 32바이트면 건너뛴다.

`SwitchTensor` 와 `FetchTensor` 모두 같은 뜻의 `.collect::<Time2, Packet2>()` 를 가진다. `FetchTensor`에서 바로 부르면 슬라이스 분배가 필요 없을 때 `Switch Engine`을 건너뛴다. 결과 `CollectTensor` 는 다음 엔진으로 흐르거나 레지스터 파일에 저장된다.

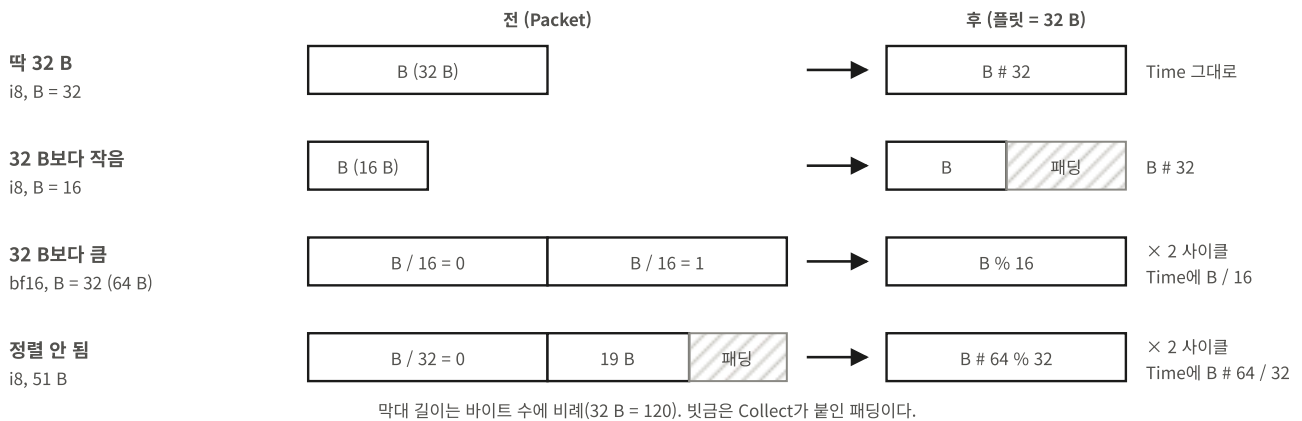


그림 26. Collect Engine의 네 경우. 패킷이 32바이트보다 크면 플릿 여러 개로 쪼개지고 그 개수가 Time의 새 안쪽 축(예: `B / 16`)이 된다. 원문의 마지막 예는 본문에서 51바이트로 설명하지만 예제 코드는 `B = 56` 으로 선언하고 있으며, 어느 쪽이든 64바이트로 패딩한 뒤 플릿 둘로 나뉜다.

정리하면 Collect의 출력 매핑은 늘 `Time2 = [Time, 플릿 번호]`, `Packet2 = [플릿 안 위치] # 32바이트` 모양이다. § 5.3에서 본 대로 32바이트에 맞지 않는 fetch 패킷은 여기서 패딩이 되어 대역폭을 버린다.

7.5 레지스터 파일: TRF와 VRF

Collect Engine의 스트림은 Contraction Engine과 Vector Engine으로 흐른다. 두 엔진은 입력을 하나 더 받는데, 슬라이스마다 하나씩 있는 **레지스터 파일**이다. TRF(Tensor Register File)는 Contraction Engine에, VRF(Vector Register File)는 Vector Engine에 공급하며, 소비 엔진이 돌기 전에 채워져 있어야 한다.

TRF 인터페이스

`TrfTensor<D, Chip, Cluster, Slice, Lane, Element>` 에서 `Chip·Cluster·Slice`는 원본에서 그대로 오고, `Lane` 은 공간 병렬성(활성 레인 1, 2, 4, 8개)을, `Element` 는 레인 안의 배치를 뜻한다. `CollectTensor` 의 `.to_trf::<Lane, Element>()` 가 TRF 텐서를 만들며, 레지스터 파일의 어느 영역에 돌지는 컴파일러가 정하므로 호출에 영역을 적지 않는다. `to_trf`는 스트림의 Time·Packet을 Lane·Element로 바꾼다.

$$\text{Lane} = \text{Time} / \text{FlitsPerLane} \quad \text{Element} = [\text{Time} \% \text{FlitsPerLane}, \text{Packet}]$$

`FlitsPerLane`은 컴파일러가 Lane과 Time에서 유도하며, 레인마다 연속한 플릿 `FlitsPerLane`개가 채워진다. 원문의 행렬곱 예에서는 `Lane` 이 출력 채널, `Element` 가 축약되는 축이다.

```
axes![V = 32, M = 32, N = 8, K = 32];
type Chip    = m![1];
type Cluster = m![V / 16];
type Slice   = m![V % 16 # 256];
type Lane    = m![N];

// 행렬곱 가중치를 TRF에 저장: Time = [N, K / 16]인 플릿이 레인 N개에 K개씩
fn store_bmatmul_trf<'l, const T: Tu>(<
  input: CollectTensor<'l, T, bf16, Chip, Cluster, Slice, m![N, K / 16], m![K % 16]>,
) -> TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![K]> {
  input.to_trf()
}
```

저장 위치 두 가지: Collect 직후, Vector Engine 뒤

저장은 `collect` 직후에만 하는 것이 아니다. main 컨텍스트 스트림이 Vector Engine을 먼저 거친 뒤 그 계산 결과를 저장할 수도 있다. 두 위치는 같은 일을 다르게 쓴 것이 아니라 **다른 데이터 경로**다.

- **collect 직후**: Fetch 네트워크가 플릿을 곧바로 레지스터 파일에 넘기고, 중간에 엔진이 돌지 않는다.
- **vector_final 뒤**: 패스가 Vector Engine 자신의 쓰기 포트에서 끝나므로, 레지스터에 들어가는 값은 패스가 계산한 값이다. Element는 리듀스나 widen이 바뀌 놓았을 수 있는 **패스 이후의 Time-Packet**을 펼친 것이다.

계산된 값이 VRF에 닿는 방법은 뒤쪽 경로뿐이다(DM 직접 저장과 collect 직후 저장은 모두 DM을 읽기 때문). Vector Engine 뒤에는 같은 명령 안에서 다른 엔진이 돌 수 없으므로, 저장을 `cast·transpose·commit`과 한 명령에 묶을 수는 없다.

```
axes![B = 64];
fn store_pass(device: &mut Device) -> VrfTensor<f32, m![1], m![1 # 2], m![1 # 256], m![B]> {
  let dm: DmTensor<f32, m![1], m![1 # 2], m![1 # 256], m![B]> = DmTensor::new();
  device.main
    .begin(dm.view())
    .fetch::<m![1], m![B]>()
    .collect::<m![B / 8], m![B % 8]>()
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_narrow_split::<m![B / 8, B % 8 / 4 % 2], m![B % 4]>()
    .vector_fp_unary(FpUnaryOp::Sqrt) // 원소마다 제곱근
    .vector_widen_concat::<m![B / 8], m![B % 8]>()
    .vector_final()
    .to_vrf(&mut device.sub) // main에서 저장: sub도 빌린다
}
```

저장이 차지하는 컨텍스트

main 컨텍스트에서 낸 저장은 어느 위치에서 하든 **쓰기를 sub 컨텍스트로 돌려서** 실행하므로 두 컨텍스트를 모두 점유하고, 그래서 `&mut device.sub`를 인자로 받는다. `device.main`과 `device.sub`는 서로 다른 필드이므로 `&mut Device`를 가진 커널이 둘을 동시에 빌릴 수 있다. 빌림은 그 문장 동안만 유지되고 명령이 끝나면 sub가 풀린다. **레지스터가** 얼마나 오래 점유되는지는 다른 문제로, 피연산자의 마지막 읽기까지이며 이는 컴파일러의 할당 문제다. 그래서 반환된 `VrfTensor`는 빌림을 들고 있지 않다. sub 컨텍스트에서 낸 저장은 이미 그 컨텍스트를 쥐고 있으므로 인자가 필요 없다.

StoTRF·StoVRF 짧은 명령

입력 접근이 완전히 연속이면(틈도 재정렬도 없음) TRF와 VRF는 **짧은 명령**(StoTRF, StoVRF)을 지원한다. DM의 데이터를 Fetch → Switch → Collect → `to_trf()` 전체 파이프라인을 거치지 않고 레지스터 파일로 바로 싣는 간결한 하드웨어 명령이다. 임의 배치를 지원하지 않는 대신 설정 오버헤드가 작다.

TRF의 읽기와 물리 구조

TRF는 뱅크로 나뉜 SRAM이며 구조는 **레인 8개 × 뱅크 2개 × 행 128개 × 320비트 = 슬라이스당 80 KB**다. 레인 8개가 병렬로 동작하며 접근마다 1, 2, 4, 8개가 활성이다. Contraction Engine 쪽 읽기는 한 번에 레인 8개 × 뱅크 1~2개 × 행 1개를 뺏는다. 뱅크 하나를 읽는 좁은(narrow) 읽기는 레인당 320비트, 두 뱅크를 모두 읽는 넓은(wide) 읽기는 레인당 640비트를 사이클마다 내며, 슬라이스 전체로는 320 또는 640 B/cycle이다. 320비트 행 하나에 들어가는 원소 수는 타입마다 다르다.

타입	저장 폭	행당 원소	비고
<code>i4 → i5</code>	5비트	64	하드웨어 동작이지만 아직 연결되지 않아 현재 i4 가중치는 i4로 남는다
<code>i8 → i9</code>	9비트(320비트 행에 맞게 올림)	32	zero-point 빼기의 한 비트 여유
<code>i8, f8</code>	8비트	32	행당 40바이트
<code>bf16</code>	16비트	16	행당 32바이트

저장할 때 넓어지는 것은 정수뿐이다(`i4 → i5`, `i8 → i9`). Fetch Adapter의 선택적 zero-point 빼기가 정수 가중치를 한 비트 넓히기 때문이다. `i8·f8`과 `bf16`은 제 폭을 유지하며, 320비트 행에는 그만큼 여유가 있어 같은 물리 행 폭이 모든 타입을 담는다. 활성 레인이 8개보다 적으면 활성 레인마다 더 많은 행을 쓰는 것처럼 동작한다. 레인 4개면 뱅크당 256행, 레인 1개면 1,024행이다.

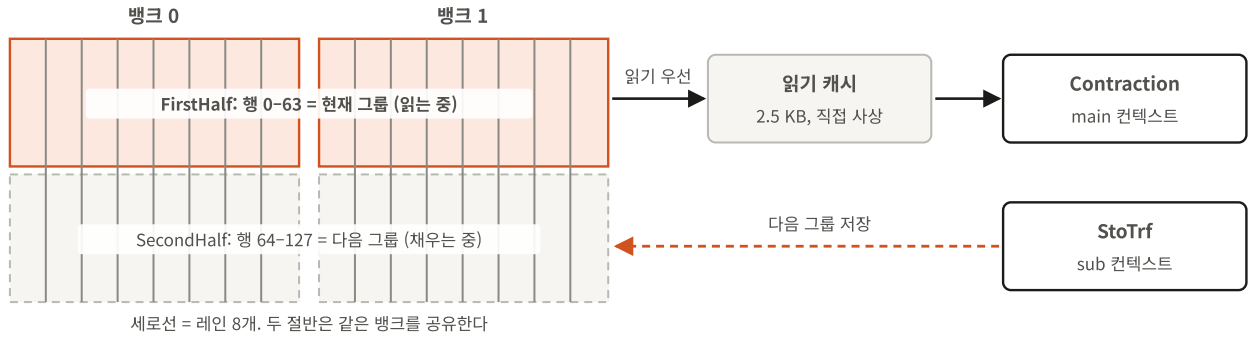
원문 확인

Quick Start의 메모리 표와 Spatial and Temporal Dimensions 장의 제약은 TRF를 "레인당 8 KB"로 적고(`Element::SIZE × size_of::<D>(C) ≤ 8KB`), Register Files 장은 물리 구조를 "슬라이스당 80 KB"로 적는다. 앞의 것은 API가 검사하는 텐서 크기 한도, 뒤의 것은 320비트 행으로 센 물리 용량으로 읽으면 된다.

이중 버퍼링

TRF는 뱅크마다 행을 반으로 나눠 **이중 버퍼링**을 할 수 있다. TRF Sequencer가 한쪽 절반에서 현재 가중치를 읽는 동안 저장이 다른 절반을 채우고, 반복마다 역할을 뒤집는다. 영역을 가리키는 주소 모드는 셋이며 스케줄러가 저장 시점에 고른다. Full 은 뱅크당 128행 전체, FirstHalf 는 0~63행, SecondHalf 는 64~127행이다. 절반 모드의 슬라이스당 용량은 40 KB다.

TRF (슬라이스 하나): 8레인 × 2뱅크 × 128행 × 320비트



다음 반복에는 두 절반의 역할을 뒤집는다. 캐시 적중과 뱅크 번갈기가 읽기와 저장의 경합을 줄인다.

그림 27. TRF 이중 버퍼링. main 컨텍스트의 축약이 현재 가중치 그룹을 읽는 동안 sub 컨텍스트가 다음 그룹을 나머지 절반에 저장해 두면, 반복 사이의 대기 시간이 준다. 튜닝 효과는 § 12.4에서 수치로 본다.

두 절반은 같은 뱅크를 공유하므로 서로 다른 행을 겨냥해도 뱅크 수준에서 경합한다. 한 사이클에 둘이 같은 뱅크를 원하면 **읽기가 우선**이다. 축약 파이프라인은 이번 사이클에 데이터가 필요하지만 저장은 기다릴 수 있기 때문이다. TRF는 이 경합을 두 가지로 줄인다.

- **읽기 캐시:** TRF 읽기는 같은 데이터가 여러 사이클에 걸쳐 브로드캐스트되어 재사용이 많다. 그래서 뱅크 앞에 직접 사상(direct-mapped) 캐시(레인 8 × 뱅크 2 × 행 4 × 320비트 = 2.5 KB)를 둔다. 적중하면 읽기가 뱅크를 건너뛰어 저장기 그 사이클에 뱅크를 쓸 수 있고, 실패하면 캐시가 뱅크에서 다시 채우며 그 사이클에 뱅크를 점유한다.
- **뱅크 번갈기:** 32바이트 이하의 좁은 읽기는 뱅크 하나만 쓰므로 32바이트 단위로 두 뱅크를 번갈아 읽을 수 있다. 그러면 연속한 사이클에서 읽기와 쓰기가 서로 다른 뱅크에 가서 캐시 실패 때도 경합을 피한다. 64바이트의 넓은 읽기는 매 사이클 두 뱅크를 다 쓰므로 캐시 실패마다 동시 저장을 막는다.

VRF

VRF는 Collect Engine이나 DM에서 직접 쓰이고 Vector Engine이 읽는다. 용량은 슬라이스당 8 KB다. `VrfTensor<D: V eScalar, Chip, Cluster, Slice, Element>`의 `Chip·Cluster·Slice`는 Vector Engine 연산이 피연산자를 스트림과 대조하는 기준이다. 그래서 스트림과 다른 분할로 운반된 피연산자는 다른 슬라이스의 데이터를 조용히 읽는 대신 컴파일 에러가 된다. 같은 슬라이스들을 다른 분할로 다시 표현하려면 `unsafe .reshape::<Chip2, Cluster2, Slice2, Element2>( )`를 쓴다. 예를 들어 익명 브로드캐스트(`m![256]`)로 복제된 피연산자를 이름 붙은 축으로 분할된 스트림에 먹일 때다. 데이터는 옮기지 않고 매핑 이름만 바꾸므로, 모든 물리 위치가 새 매핑이 주장하는 값을 이미 담고 있어야 한다.

`.to_vrf::<Element2>( )`는 스트림의 `Time·Packet`을 `Element2 = [Time, Packet]`으로 펼치며 `Element2`는 작성자가 고른다. 저장 메서드는 컨텍스트마다 시그니처가 하나씩이다. sub에서는 인자 없이, main에서는 `&mut device`. sub를 받는다.

```
axes![B = 64];
// sub 컨텍스트에서 i32 64개를 VRF에 저장: Time = B / 8, Packet = B % 8 → Element = B
fn store_vrf<'l>(<
  input: CollectTensor<'l, { Tu::Sub }, i32, m![1], m![1 # 2], m![1 # 256], m![B / 8], m![B % 8]>,
) -> VrfTensor<i32, m![1], m![1 # 2], m![1 # 256], m![B]> {
  input.to_vrf()
}
```

Vector Engine의 stash 기능도 VRF를 쓴다. stash를 쓰는 호출은 슬라이스당 1,024 B를 예약하므로 함께 읽는 VRF 피연산자는 7 KiB 이하여야 한다(§ 9.3).

스스로 점검

1. NVFP4 가중치를 행렬곱에 쓰려고 sub 컨텍스트에서 `fetch_table_lookup` 을 부르면 어떻게 되나? 올바른 순서는?
2. `i8` → `i32` 캐스트가 있는 `fetch`에서 `read_size`가 8원소로 제한되는 이유는?
3. `Broadcast1 { slice1: 4, slice0: 8 }` 에서 `ring_size`와, Time `64·i8` 64바이트 패킷의 사이클 수는?
4. `bf16` 패킷 24개(48 B)를 collect하면 Time과 Packet이 어떻게 되나?
5. TRF 이중 버퍼링에서 읽기와 저장이 같은 बैं크를 원하면 누가 먼저인가?

답: ① 테이블 조회는 main 전용이라 불가. main에서 해독해 DM에 commit한 뒤 StoTrf로 TRF에 올린다. ② 캐스트 결과가 플릿 하나(32 B)에 들어가야 해서. ③ ring 32, $32 \times 64 \times 2 = 4,096$. ④ 64 B로 패딩되어 플릿 두 개. Time에 플릿 번호 축이 붙고 Packet은 `bf16` 16칸(플릿 하나). ⑤ 읽기.

8장

Tensor Unit 연산 II: Contraction Engine

원문: Contraction Engine · Outer(Stream Adapter, TRF Sequencer, Multiplier) · Packet Reducer · Time Reducer · Lane Folder

칩 이름의 주인공이다. Contraction Engine은 행렬곱이나 컨볼루션 같은 이항 텐서 축약을 한다. § 1.2에서 본 브로드캐스트 → 곱 → 리듀스를 하드웨어 네 단계가 나뉘 말으며, 각 단계는 서로 겹치지 않는 차원(Lane, Packet, Time)을 담당한다. 이 장은 네 단계의 동작과 제약을 설명하고, 같은 배치 행렬곱을 세 가지로 배치했을 때 곱셈기 활용률이 얼마나 달라지는지 비교한다.

이 장에서 배우는 것

- Outer → Packet Reducer → Time Reducer → Lane Folder의 역할과 API 체인
- 리듀스 축을 Packet에 둘지 Time에 둘지에 따른 곱셈기 활용률 차이
- Stream Adapter의 묶기(PackSize)와 브로드캐스트, TRF Sequencer의 ReadSize
- 곱셈기가 받는 피연산자 타입 조합과 누산기 확장
- 가산기 트리 깊이, 누산 슬롯 용량, Interleaved-Sequential 레인 접기

8.1 네 단계 파이프라인

3장의 내적·GEMV·GEMM에서 본 대로 텐서 축약은 두 입력을 받아 공유 축을 따라 리듀스한다. 한 피연산자는 Collect Engine에서 흘러오고 다른 하나는 TRF에 있다. 축약은 main 컨텍스트에서 돌고, TRF 준비는 sub 컨텍스트에서 `.to_trf`()로 한다. 원문은 이 작업을 네 파이프라인 단계로 나눈다. 하나는 브로드캐스트와 곱을, 셋은 리듀스를 맡는다.

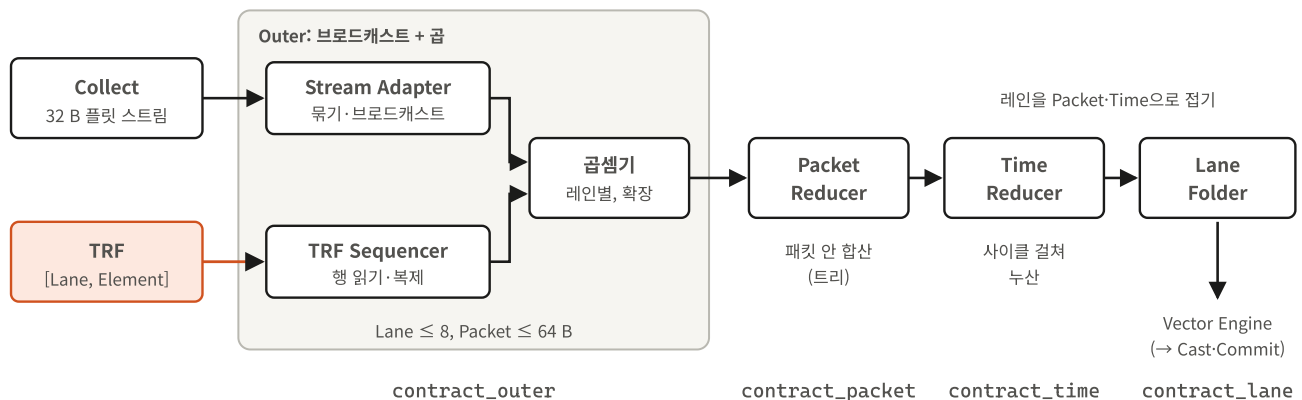


그림 28. Contraction Engine의 네 단계와 대응하는 API. 곱셈기 출력은 [Chip, Cluster, Slice, Lane, Time, Packet] 모양의 곱 텐서이고, 이후 세 단계가 Packet, Time, Lane 순서로 차원을 줄이거나 접는다.

- **Outer(브로드캐스트와 곱):** 두 피연산자를 [Chip, Cluster, Slice, Lane, Time, Packet]이라는 같은 모양으로 브로드캐스트하고 원소별로 곱해 곱 텐서 하나를 만든다. Chip·Cluster·Slice는 그대로 지나가고, Lane은 TRF와 뒤쪽 리듀서가 공유하는 공간 병렬성의 인덱스다. Stream Adapter가 스트림 피연산자를, TRF Sequencer가 TRF 피연산자를 브로드캐스트하고, 곱셈기가 누산기 타입($i4 \cdot i8 \rightarrow i32$, $f8 \cdot bf16 \rightarrow f32$)으로 넓혀 곱한다.
- **Packet Reducer(Packet 안의 리듀스):** Packet에 매핑된 축약 축을 레인마다 병렬 트리로 리듀스한다.
- **Time Reducer(Time에 걸친 리듀스):** 사이클마다의 결과를 공유 누산기에 모은다.

- **Lane Folder(Lane 접기)**: 버퍼를 출력 스트림으로 내보내며 모드에 따라 Lane을 OutPacket이나 OutTime에 흡수한다. 슬라이스나 칩에 걸친 리듀스는 뒤쪽 Vector Engine이 맡는다.

RNGD에서 Outer 단계는 **Lane ≤ 8 , Packet ≤ 64 B**로 상한을 둔다. 코드는 늘 네 호출이 이어진 모양이다.

```
input                                     // CollectTensor: Time, Packet
.contract_outer::<OutTime, OutPacket, _, _, _>(&trf) // Lane은 TRF에서 추론
.contract_packet::<ReducedPacket>()           // Packet 안의 축약 축을 합산
.contract_time::<SurvivingTime>()             // Time 안의 축약 축을 누산
.contract_lane::<FinalTime, FinalPacket>(LaneMode::Interleaved) // ContractTensor (i32/f32)
```

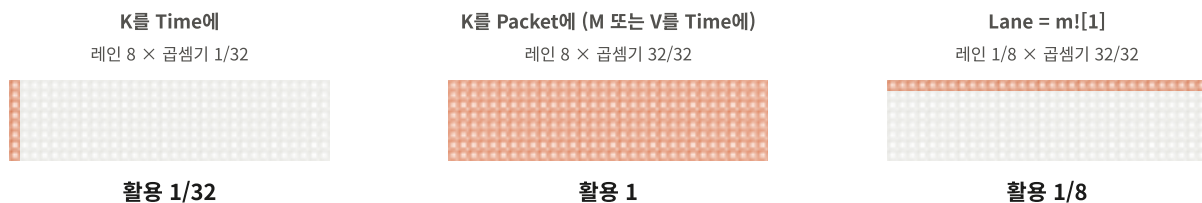
더 깊이: 호스트 백엔드의 지연 계산

원문의 API 코드를 보면 `contract_outer`는 넓은 두 피연산자와 매핑들을 `LazyContraction`에 담아 두기만 하고, `contract_packet`·`contract_time`은 검증과 타입 변경만 한다. 실제 합산은 `contract_lane`에서 한꺼번에 수행된다. CPU 백엔드에서 거대한 브로드캐스트 텐서를 만들지 않으려는 구현이며, 하드웨어의 네 단계 의미는 그대로다.

8.2 배치 행렬곱으로 보는 배치 전략

원문은 GEMM에 배치 축 V 를 더한 배치 행렬곱 $VMK, KN \rightarrow VMN$ 으로 설계 선택을 비교한다. V 개의 독립적인 $(M \times K)$ 입력과 공유 가중치 $(K \times N)$ 으로 $(M \times N)$ 곱을 만든다. 축 크기는 `axes![V = 32, M = 32, N = 8, K = 32]`이고 모든 변형에서 TRF는 `Lane = m![N]` (출력 채널 하나당 레인 하나), `Element = m![K]` 다. 세 변형은 어느 축을 **Time**에 두는가로 나뉜다.

변형	Cluster	Slice	스트림 Time	스트림 Packet	결과
K를 Time에	<code>m![M / 16]</code>	<code>m![M % 16, V % 16]</code>	<code>m![V / 16, K]</code>	<code>m![1 # 16]</code>	곱셈기 32개 중 1개만 일한다(bf16 기준 1/32). 교육용 퇴화 예
M을 Time에	<code>m![V / 16]</code>	<code>m![V % 16 # 256]</code>	<code>m![M, K / 16]</code>	<code>m![K % 16]</code>	K를 트리로 한 번에 합산. 레인 전체의 곱셈기를 쓴다
V를 Time에	<code>m![M / 16]</code>	<code>m![M % 16 # 256]</code>	<code>m![V, K / 16]</code>	<code>m![K % 16]</code>	배치가 지배적인 축일 때 (배치 추론)



격자 하나 = 슬라이스 하나의 곱셈기 배열(bf16, 64 B 패킷): 세로 레인 8줄 × 가로 레인당 곱셈기 32개. 칠한 칸이 실제로 일하는 곱셈기다. 사이클당 처리량은 칠한 칸의 비율에 비례한다.

그림 29. 배치에 따른 곱셈기(MAC) 활용. 왼쪽 두 격자는 원문의 배치 행렬곱 비교이고, 오른쪽은 원문의 "Lane이 8보다 작으면 쉬는 레인의 리덕션 트리가 높아 처리량이 Lane::SIZE / 8로 준다"는 설명을 그린 것이다. 3장의 내적과 GEMV 예제는 단순화를 위해 `Lane = m![1]`을 쓴다.

K를 Time에 둔 변형은 $V \times M = 1,024$ 가 칩당 공간 칸 512개에 들어가지 않아 V 의 바깥 조각($v / 16 = 2$)이 Time으로 가고, K 도 Time에 있다. Packet은 원소 하나를 `1 # 32`로 패딩한 것이라 리듀스가 Packet Reducer의 공간 트리가

아니라 사이클을 따라 순차적으로 일어나고, 사이클마다 곱셈기 32개 중 하나만 쓸모 있는 일을 한다. 원문은 이를 교육용 기준선으로만 보여 준다.

```
// K를 Time에 (되화 예)
input.contract_outer::<m![V / 16, K], m![1 # 16], _, _, _>(trf) // K 원소 하나를 N개 레인에 브로드캐스트
.contract_packet::<m![1]>(C) // 줄일 것이 없다
.contract_time::<m![V / 16]>(C) // K를 사이클마다 누산
.contract_lane::<m![V / 16], m![N]>(LaneMode::Interleaved)

// M을 Time에
input.contract_outer::<m![M], m![K], _, _, _>(trf) // K 원소 32개(64 B)를 N개 레인에, TRF는 M 사이클 동안
브로드캐스트
.contract_packet::<m![1]>(C) // K를 트리로 공간 합산
.contract_time::<m![M]>(C) // 줄일 것이 없다
.contract_lane::<m![M], m![N]>(LaneMode::Interleaved) // 슬라이스마다 (M × N) 행렬
```

핵심

병적인 경우를 피하려면 **축약 축 K는 Packet에** 두어 Packet Reducer의 트리로 병렬 리듀스하고, **살아남는 축 V·M·N은 Cluster·Slice·Lane에** 퍼뜨려 공간 병렬성을 최대로 쓴다. M-in-Time 전략은 슬라이스 수가 배치를 덮고, N이 레인에 들어가고, K가 패킷 하나에 들어갈 때 쓸 수 있다. K가 패킷보다 크면 K를 Packet(공간)과 Time(시간)으로 나눈다. 실제 커널은 크기에 따라 한 축을 여러 차원에 나누기도 한다.

8.3 Outer: 브로드캐스트와 곱

이름은 선형대수의 **외적(outer product)**에서 왔다. 길이 n 인 u 와 길이 m 인 v 의 외적 uv^T 는 $(uv^T)[i, j] = u[i] \times v[j]$ 인 $n \times m$ 행렬이고, u 를 열 축으로, v 를 행 축으로 브로드캐스트한 뒤 원소별로 곱한 것과 같다. Outer 단계의 세 하위 단계가 정확히 이 의미를 하드웨어로 구현하며 직렬로 돈다.

`contract_outer::<OutTime, OutPacket, Lane, TrfElement, TrfD>(&trf)` 에서 스트림 피연산자의 Time·Packet은 출력 OutTime·OutPacket으로 간다. OutPacket은 Time의 가장 안쪽 크기 1 또는 2 인자를 **묶기(Packing)**로 흡수하고, OutTime은 Time의 나머지 인자를 유지하되 **브로드캐스트** 인자를 가장 안쪽에 더한다. 브로드캐스트 인자는 TRF의 Lane·Element에서 오거나(스트림이 TRF 매핑을 따라 복제되는 경우), 입력에도 TRF에도 없이 출력에만 있는 축에서 온다(예: einsum $AB, BC \rightarrow ABCD$ 의 D). TRF 텐서 `[Chip, Cluster, Slice, Lane, Element]` 에서 Lane은 레인 1~8개에 데이터를 나누고, Element는 TRF Sequencer가 OutTime·OutPacket 모양으로 바꾼다.

Stream Adapter

Stream Adapter는 스트림의 Time·Packet을 계산 모양(Lane·OutTime·OutPacket)으로 바꾼다. 컴파일러는 사용자가 준 OutTime·OutPacket, TRF 피연산자, 출력 전용 브로드캐스트 축으로부터 자유 변수 셋(PackSize, LaneBroadcast, TimeBroadcast)을 유도한다.

```
Lane = LaneBroadcast
OutTime = [Time / PackSize, TimeBroadcast]
OutPacket = [Time % PackSize, Packet] # (64 / D::SIZE)
```

- **묶기:** Collect가 32 B 플릿을 내고 Outer는 $\text{PackSize} \times 32 \text{ B}$ (32 또는 64 B) 패킷을 낸다. $\text{PackSize} \in \{1, 2\}$ 는 OutPacket을 입력 Packet과 비교해 정해진다. OutPacket이 Time의 가장 안쪽 크기 2 인자를 흡수하면 2, 아니면 1이다. 즉 $\text{PackSize} = \text{OutPacket}::\text{SIZE} \times D::\text{SIZE} / 32$ 이므로 사용자는 OutPacket(32 B 또는 64 B)만 고르면

된다. 하드웨어는 내부적으로 항상 64 B 패킷을 처리하며, $\text{PackSize} = 1$ 이면 쓰지 않는 32 B 절반이 0으로 채워진다 (논리 타입에는 드러나지 않는다).

- **브로드캐스트**: 묶은 패킷을 **공간적으로** LaneBroadcast(TRF의 Lane, $1 \cdot 2 \cdot 4 \cdot 8$)만큼, **시간적으로** TimeBroadcast만큼 복제한다. TimeBroadcast에는 입력 Time에 없는 TRF Element 인자와 출력 전용 축이 들어가며, OutTime의 **가장 안쪽** 자리를 차지한다. 즉 바깥 반복으로 넘어가기 전에 같은 OutPacket을 그 인자 수만큼 다시 보낸다.

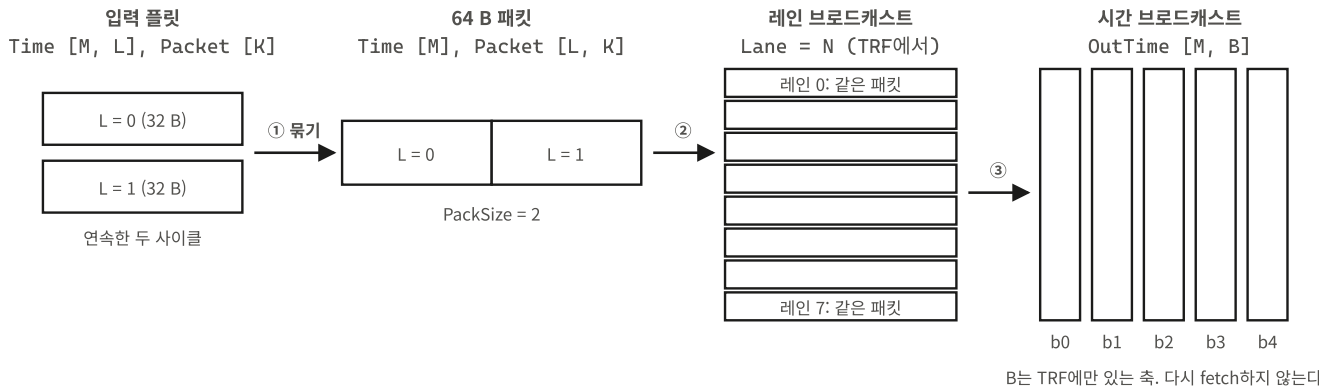


그림 30. 원문 예($M = 32, N = 8, K = 16, L = 2, B = 5, \text{bf16}$)의 Stream Adapter. 입력 Time의 가장 안쪽 인자 L(크기 2)을 Packet에 흡수해 64 B 패킷을 만들고(①), 그 패킷을 TRF의 레인 8개에 복제하며(②), TRF Element에만 있는 B축(크기 5)을 OutTime의 가장 안쪽에 더해 같은 묶음을 다섯 사이클 반복한다(③). 호출은 `contract_outer::<m![M, B], m![L, K], _, _, _>(trf)` 다.

제약: $\text{OutPacket}::\text{SIZE} \times \text{Storage}::\text{SIZE} \in \{32, 64\}$ 바이트다. Storage는 넓히기 전의 저장 타입이다(bf16이면 2 B이고, 결과 텐서가 가진 f32 누산기 폭이 아니다). $\text{PackSize} \in \{1, 2\}$, $\text{Lane}::\text{SIZE} \in \{1, 2, 4, 8\}$.

성능: PackSize가 곱셈기 활용률을 정한다. PackSize = 2는 64 B를 모두 채워 곱셈기를 다 쓰고, PackSize = 1은 32 B만 채워 0으로 채운 절반이 늘 0을 곱하므로 실효 처리량이 반으로 준다. PackSize = 2는 패킷 하나에 2사이클이 걸리지만 병목은 아니다. 앞 단계가 사이클마다 32 B 플릿 하나를 공급하고, Stream Adapter는 도착하는 대로 소비해 2사이클마다 패킷 하나를 내어 뒤 단계 소비와 맞춘다. 시간 브로드캐스트는 fetch 비용을 분산한다. 같은 스트림 패킷을 다시 가져오지 않고 재사용하므로 그 인자에 대한 대역폭 비용이 없다. 전체적으로 Stream Adapter는 fetch 대역폭(fetch 하나당 최대 32 B/cycle)에 묶이므로, fetch 패턴을 슬라이스에 걸쳐 섞어 활용률을 높인다.

TRF Sequencer

TRF Sequencer는 `TrfTensor`를 읽어 그 Element를 OutTime·OutPacket 모양으로 바꿔 Packet Reducer에 넘긴다.

```
OutTime = ([Element / ReadSize]를 차례로 읽기 + 브로드캐스트)
OutPacket = [PacketBroadcast, Element % ReadSize]
```

OutPacket은 사이클마다 TRF 읽기 한 번으로 채워진다. 레인당 두 बैं크에 걸친 640비트, 또는 한 बैं크만 읽으면 320비트다. 읽기는 Element의 가장 안쪽 연속 구간을 가져와 복제해서 64 B OutPacket을 채운다. 컴파일러는 $\text{Element} \% \text{ReadSize} == \text{OutPacket} \% \text{ReadSize}$ 이고 $\text{ReadSize} \times D::\text{SIZE} \leq 64$ 바이트인 가장 큰 ReadSize를 고른다. ReadSize가 넓으면 레인당 두 TRF बैं크를, 좁으면 한 बैं크를 쓴다. 사이클을 넘어서는 $\text{Element} / \text{ReadSize}$ (와 선택적 브로드캐스트)를 다른 시퀀서와 같은 중첩 루프 설정으로 차례로 읽는다. PacketBroadcast 인자는 한 사이클 안에서 같은 행을 복제해 추가 TRF 읽기 대역폭 없이 64 B를 채운다.

원문 예	TRF Element	ReadSize	PacketBroadcast	OutTime	OutPacket
한 번에 전부	$m![K]$ ($K = 32$, bf16)	32	1	$m![M]$ (M 브로드캐스트)	$m![K]$
일부씩	$m![0, K]$ ($O = 2$, K = 16)	16	L (2)	$m![0, M]$ (O를 차례로, M 브로드캐스트)	$m![L, K]$

제약: Chip·Cluster·Slice 크기가 하드웨어와 맞아야 하고, $\text{Element} \% \text{ReadSize}$ 가 64 B 전체를 덮으면 읽기가 레인당 두 बैं크에 걸치므로 시퀀서의 시작 주소와 모든 스트라이드가 64 B 정렬이어야 한다. **성능**은 사이클마다 레인당 OutPacket 하나이며, TRF 읽기 캐시와 बैं크 번갈기([§ 7.5](#))가 브로드캐스트 재사용과 좁은 읽기 동안 sub 컨텍스트의 동시 저장을 막지 않게 한다.

곱셈기와 피연산자 타입

곱셈기는 Stream Adapter와 TRF Sequencer가 맞춘 두 피연산자를 받아 각 원소를 축약 출력 타입으로 넓힌 뒤(누산기 오버플로 방지) 원소별로 곱한다. 출력은 [Chip, Cluster, Slice, Lane, Time, Packet] 모양이며, 사이클마다 모든 레인이 곱 패킷 하나를 병렬로 만든다.

스트림 (활성값)	가중치 (TRF)	누산기
i4 또는 i5	i4 또는 i5	i32
i8 또는 i9	i8 또는 i9	i32
f8e4m3	f8e4m3	f32
f8e5m2	f8e5m2	f32
bf16	bf16	f32

정수 피연산자는 원래 형태(i4·i8)이거나 zero-point를 뺀 중간 형태(i5·i9)일 수 있고, 같은 계열 안에서는 두 피연산자가 달라도 된다. 계열을 넘거나(i4와 i8), 종류를 넘거나(정수와 실수) 할 수는 없고, 실수 쌍은 정확히 같아야 한다. 이 표에 없는 원소 타입의 축약은 Vector Engine에서 한다. 원소별로 곱한 뒤 i32·f32로 계산하는 Intra-Slice Reduce로 리듀스한다([§ 9.7](#)).

8.4 Packet Reducer

Packet Reducer는 패킷 하나 안의 가장 안쪽 축약 축들을 레인마다 리덕션 트리로 합산한다. 각 레인은 Outer의 OutPacket에서 물려받은 32 B 또는 64 B 패킷(i4·i8·f8·bf16 원소)을 받는다. 수식으로는 $\text{output}[i] = \sum_j \text{input}[i, j]$ 이고, i는 살아남는 축, j는 Packet 안의 축약 축이다. 원문 예의 행렬곱은 레인 8개를 모두 쓰며, 깊이 5짜리 트리가 B축의 bf16 원소 32개를 합쳐 A의 위치마다 f32 하나를 만든다.

```
axes![A = 32, B = 32, C = 8];
// Packet Reducer: 깊이 5 트리가 B축의 bf16 32개를 f32 하나로 합산
// 출력(Interleaved): Time = [A], Packet = [C]
input.contract_outer::<m![A], m![B], _, _, _>(&trf)
    .contract_packet::<m![1]>()
    .contract_time::<m![A]>()
    .contract_lane::<m![A], m![C]>(LaneMode::Interleaved)
```

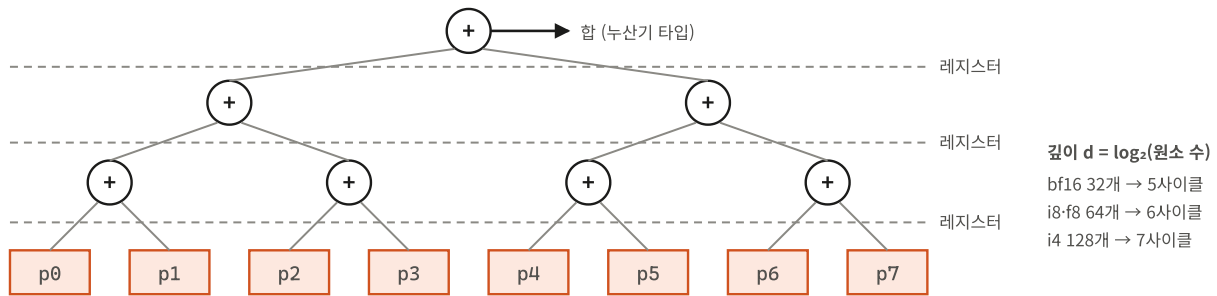


그림 31. 리덕션 트리(그림은 원소 8개로 줄였다). 단계마다 레지스터가 있는 완전 바이프라이너이라 깊이는 첫 출력 지연만 늘리고, 바이프라이너인 찬 뒤로는 매 사이클 패킷 하나가 들어가고 결과 하나가 나온다. 레인마다 이런 트리가 하나씩 있다.

$\text{ReducePacket} = \text{Packet} / 2^d$ ($0 \leq d \leq \log_2(\text{Packet}::\text{SIZE})$) $\text{OutPacket} = \text{ReducePacket}$ 에서 바깥 패딩 제거 (크기 ≤ 32)

트리의 앞은 입력 패킷의 원소이고, 한 단계 올라갈 때마다 쌍을 더해 원소 수가 반이 된다. 최대 깊이는 $\log_2(\text{Packet}::\text{SIZE})$ 로, i4는 7(Packet 128원소), i8·f8은 6(64), bf16은 5(32)다. 사용자가 OutPacket을 주면 컴파일러가 깊이 d를 유도하고, 트리는 가장 안쪽 2^d 개 원소를 소비해 ReducePacket을 만든다. 그 뒤 ReducePacket을 OutPacket으로 자르는데, 뒤쪽 Time Reducer의 레인별 누산기가 **열 32개**뿐이라 크기가 32로 제한된다. 예를 들어 i4는 128원소 패킷으로 들어오므로 $d = 0$ 이나 1이면 128 또는 64원소가 되고, 둘 다 가장 안쪽 32개만 남긴다. ReducePacket에 바깥 패딩이 있으면 살아 있는 열만 남긴다. [A # 16]은 OutPacket [A]로 줄어들며, OutPacket을 [A # 16]으로 적든 [A]로 적든 같은 결과다.

성능: 지연은 트리 깊이만큼(i4 7, i8·f8 6, bf16 5사이클)이다. 원소가 넓을수록 패킷에 들어가는 원소가 적어 깊이가 준다. 트리가 완전 바이프라이너이므로 정상 상태 처리량은 줄지 않는다. **Lane이 8보다 작으면 쉬는 레인의 트리가 높아 사이클당 처리량이 Lane::SIZE / 8로 준다.**

8.5 Time Reducer

Time Reducer는 Packet Reducer의 [Lane, Packet] 출력을 Time에 걸쳐 누산해 OutTime으로 줄인다. `contract_time::<OutTime>()`의 OutTime은 살아남는 Time 차원들이고 나머지는 합산된다. 원문 예 `reduce_b`는 2D 텐서를 B축으로 리듀스하고 A만 남긴다.

```
axes![A = 2048, B = 32];
// Slice = m![A / 8] (슬라이스 256개), Time = m![B / 16, A % 8], Packet = m![B % 16]
input.contract_outer::<m![B / 16, A % 8], m![B % 16], _, _, _>(trf)
  .contract_packet::<m![1]>()           // B % 16을 공간 합산 → 사이클마다 부분합
  .contract_time::<m![A % 8]>()         // 16플릿: B / 16 두 조각이 A % 8 슬롯 8개에 누산
  .contract_lane::<m![A % 8], m![1 # 8]>(LaneMode::Sequential)
```

OutTime은 Time의 부분집합이어야 하고 살아남는 차원의 상대 순서를 지켜야 한다(`verify_contract_time`). Time에 있지만 OutTime에 없는 차원은 합산되며, 그런 차원 중 가장 바깥 것이 플릿에 걸쳐 반복된다. **InnerTime**은 가장 바깥 리듀스 차원보다 안쪽에 있으면서 OutTime에 살아남는 차원들이다. 누산에는 InnerTime 값마다 [Lane, Packet] 슬롯 하나, 즉 InnerTime::SIZE개의 슬롯이 필요하고, 같은 InnerTime 값을 가진 플릿은 같은 슬롯에 더해진다. `reduce_b`에서는 `B / 16`이 가장 바깥 리듀스 차원(`Time::SIZE = 2 × 8 = 16플릿`)이고 InnerTime은 `m![A % 8]`이다.

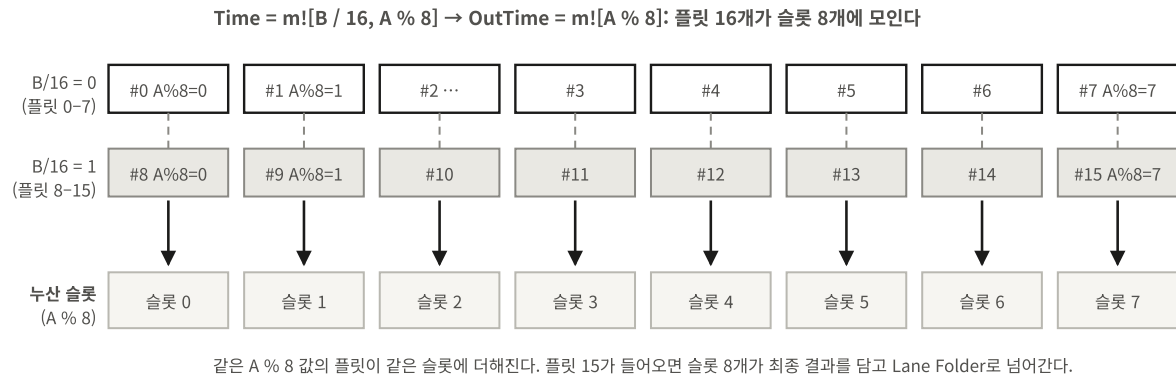


그림 32. Time Reducer의 누산 슬롯. 슬롯 수는 `InnerTime::SIZE`다. 리듀스 축을 Time의 가장 안쪽에 두면 정의상 `InnerTime`이 비어 슬롯 하나로 충분하다(§ 3.6의 $k / 32$ 가 그렇다).

제약: `InnerTime::SIZE`가 슬롯 용량을 넘으면 안 된다. 누산 버퍼는 셀 1,024개이고, 슬롯 하나는 `[Lane, Packet]` 덩어리인데 그 모양을 뒤쪽 Lane Folder의 모드가 정한다.

LaneMode	덩어리 모양	덩어리당 셀	슬롯 용량
Interleaved	<code>[Lane # 8, Packet]</code>	$8 \times \text{Packet}::\text{SIZE}$	$128 / \text{Packet}::\text{SIZE}$
Sequential	<code>[Lane, Packet # 32]</code>	$\text{Lane}::\text{SIZE} \times 32$	$32 / \text{Lane}::\text{SIZE}$

`reduce_b`는 Sequential에 `Lane::SIZE = 10`이라 슬롯 용량이 32이고 `InnerTime::SIZE = 8`이 넉넉히 들어간다. 넘친다면 Time을 다시 구성하거나(예: B를 더 쪼갬) 처리량을 조금 내주고 LaneMode를 바꿔 슬롯 여유를 얻는다. **성능:** 입력은 사이클당 패킷 하나이고, N개를 하나로 줄이면 출력 속도는 입력의 $1/N$ 이다. 크기 N인 Time 차원의 리듀스 지연은 약 N사이클이다.

8.6 Lane Folder

Lane Folder는 Contraction Engine의 마지막 단계다. Lane 차원의 값 8개를 `OutPacket(Interleaved)`이나 `OutTime(Sequential)`으로 옮겨 Lane을 없앤다. **값을 더하지 않고** 다른 축으로 접을 뿐이다. Time Reducer의 버퍼를 원소 8개 폭의 출력 버스로 한 사이클씩 비우며, LaneMode가 사이클마다 플릿에 무엇을 심는지 고른다.

모드	출력 매핑	사이클마다
Interleaved	<code>OutTime = [Time, Packet]</code> <code>OutPacket = [Lane # 8]</code>	레인 8개에 걸친 열 하나를 읽는다(레인당 값 1개, 플릿당 8개)
Sequential	<code>PadPacket = Packet # align_up(Packet::SIZE, 8)</code> <code>OutTime = [Time, Lane, PadPacket / 8]</code> <code>OutPacket = [PadPacket % 8]</code>	레인 하나의 Packet에서 열 8개를 읽고, 레인은 사이클을 따라 넘어간다

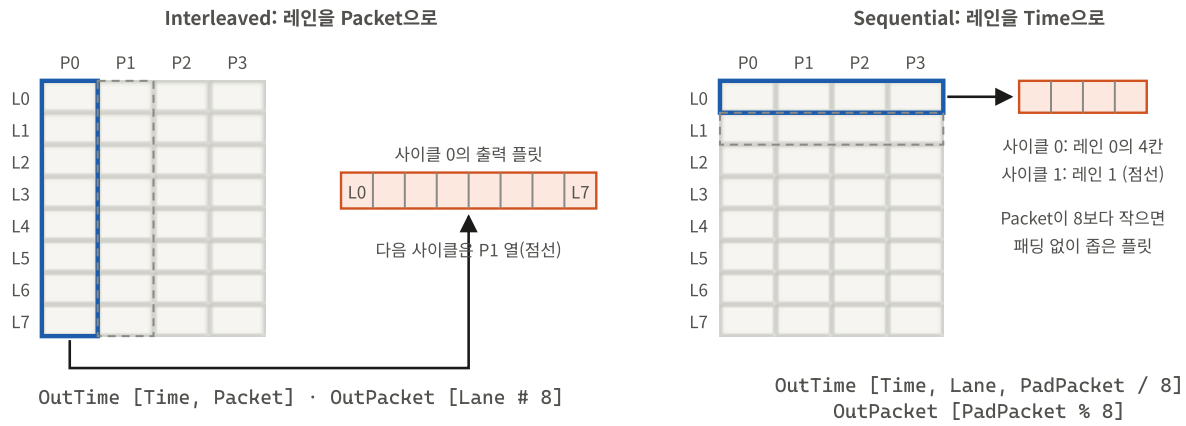


그림 33. Lane Folder의 두 모드(Packet 4칸 예). Interleaved는 사이클마다 모든 레인의 같은 열을 한 플릿으로 내므로 레인이 8개일 때 버스를 꽉 채운다. Sequential은 레인을 시간 축으로 펼쳐 레인 하나씩 내보낸다.

원문 예($N = 8, M = 4, P = 16$)에서 Time Reducer의 출력이 Lane N, Time M, Packet P라면, Interleaved는 `contract_lane::<m![M, P], m![N]>`으로 OutTime 64단계에 레인 값 8개씩을, Sequential은 `contract_lane::<m![M, N, P / 8], m![P % 8]>`으로 OutTime $4 \times 8 \times 2 = 64$ 단계에 한 레인의 원소 8개씩을 낸다. `Packet::SIZE`가 32보다 작으면 `[PadPacket / 8]::SIZE = [Packet::SIZE / 8]`이 패킷 하나의 사이클 수다(Packet 4면 1, 16이면 2).

제약: Lane Folder 자체의 제약은 없고, 여기서 고른 LaneMode가 앞 단계 Time Reducer의 슬롯 용량을 정한다. **성능:** Interleaved에서 Lane이 8보다 작으면 비활성 레인이 버스 자리를 비워 처리량이 `Lane::SIZE / 8`로 준다.

Sequential에서 `Packet::SIZE`가 8보다 작으면(예: Packet Reducer가 bf16 패킷의 절반을 접어 4가 된 경우) 사이클마다 버스 폭 8 대신 `Packet::SIZE`개만 실어 출력이 좁지만 패딩이나 낭비되는 버스 자리가 없다. 지연은 무시할 만하다. 레인별 출력의 모양만 바꿀 뿐 버퍼를 비우는 시간 외의 사이클을 더하지 않는다.

8.7 설계 규칙 정리

결정	원문이 원하는 선택	이유
축약 축의 위치	우선 Packet에, 넘치면 Packet과 Time에 나눠서	Packet은 트리로 한 사이클에, Time은 여러 사이클에 걸쳐 누산한다. K를 Time에만 두면 곱셈기 1/32(bf16)
OutPacket 크기	64 B (PackSize = 2)	32 B면 곱셈기 절반이 0을 곱한다
Lane	살아남는 축(출력 채널 등)으로 8까지 채운다	Lane < 8이면 트리와 출력 버스가 Lane::SIZE / 8만 일한다
살아남는 축	Cluster·Slice·Lane에 퍼뜨린다	공간 병렬성 최대화
Time의 리듀스 축	OutTime에 남는 축보다 안쪽에	InnerTime이 작을수록 슬롯이 적게 든다(가장 안쪽이면 1개)
LaneMode	레인이 8개면 Interleaved	슬롯 용량이 부족하면 Sequential로 여유를 얻는다
표에 없는 타입	Vector Engine에서 곱과 Intra-Slice Reduce	곱셈기가 받는 조합은 정해져 있다

스스로 점검

1. bf16 스트림을 `contract_outer::<..., m![K % 16]>` (32 B)로 부르면 PackSize와 곱셈기 활용률은?
2. i8 패킷 64원소를 한 번에 합산하는 트리의 깊이와 지연은?
3. $\text{Time} = m![K / 32, I]$ ($I = 64$)에서 $K / 32$ 를 리듀스하고 I 를 남기며 Interleaved·Packet 1이라면 슬롯이 충분한가?
4. $\text{Lane} = m![1]$ 인 GEMV를 레인 8개로 바꾸려면 어떤 축을 레인에 실을 수 있을까?

답: ① $\text{PackSize} = 16 \times 2 / 32 = 1$, 활용률 1/2. ② $\log_2 64 = 6$, 첫 결과까지 6사이클. ③ $\text{InnerTime} = I(64)$, 슬롯 용량 $128 / 1 = 128$ 이라 충분하다. ④ 출력 축 I 의 일부(예: $I \% 8$)를 TRF의 Lane에 두고 Slice를 $I / 8$ 로 줄이는 식이다. 이때 TRF에는 레인마다 x 가 복제된다.

9장

Tensor Unit 연산 III: Vector Engine

원문: Vector Engine · Intra-Slice Chain · Intra-Slice Reduce · Valid Count Generator · Inter-Slice Reducer

Vector Engine은 원소별 계산과 리듀스를 맡는다. GELU·SiLU 같은 활성화, softmax·layer norm 같은 정규화, 이항 연산, 슬라이스 안과 슬라이스 사이의 리듀스가 모두 여기서 일어난다. 원문에서 규칙이 가장 촘촘한 부분이어서, 이 장은 먼저 전체 구조와 단계 순서를 잡고, 피연산자·ALU·태그 규칙을 차례로 설명한 뒤, 패딩을 빼고 리듀스하는 Valid Count Generator와 슬라이스를 합치는 Inter-Slice Reducer로 마무리한다.

이 장에서 배우는 것

- `vector_init` 과 `vector_final` 사이의 두 부품과 실행 순서(IntraFirst, InterFirst)
- Intra-Slice Chain 12단계의 고정 순서, 8-way·4-way 구간과 Narrow·Widen
- 피연산자 세 종류(상수·VRF·stash)와 인자 모드
- "ALU는 호출 한 번에 한 번만" 규칙과 클러스터별 연산
- 태그로 원소마다 다른 피연산자를 고르는 조건부 연산, 두 그룹을 짝짓는 Pair Mode
- 슬라이스 안 리듀스, 패딩을 빼는 VCG, 슬라이스 사이 리듀스

9.1 구조: 두 부품과 두 가지 순서

Vector Engine은 **32비트 타입(i32, f32)**만 받는다. 앞에 Contraction Engine이 있으면 넓혀진 채로 들어오고(bf16 곱은 f32, i8 곱은 i32로 누산), Contraction을 거치지 않는다면 Fetch Adapter의 타입 변환으로 직접 넓혀야 한다(§ 7.2). Tensor Unit 호출 하나에서 Vector Engine 부분은 `vector_init()` 부터 `vector_final()` 까지의 메서드 체인이고, 그 안에 부품 둘이 있다.

- **Intra-Slice Chain**: 슬라이스마다 독립적으로 원소별 연산, 이항 연산, 슬라이스 안 리듀스를 한다. `vector_intra_slice_tag(TagMode)` 로 들어가거나, 입력이 두 그룹으로 나뉘는 축을 가졌다면 `vector_intra_slice_unzip()` 으로 들어간다(Pair Mode).
- **Inter-Slice Reducer**: 클러스터 안 256개 슬라이스에 걸쳐 리듀스한다. `vector_inter_slice_reduce()` 로 들어간다.

둘 중 하나만 써도, 둘 다 써도 된다. 둘 다 쓰면 순서는 **IntraFirst**(체인 → 리듀서)나 **InterFirst**(리듀서 → 체인)다. 체인은 `vector_init()` 직후나 리듀서 출력에서, 리듀서는 `vector_init()` 직후나 전환을 허용하는 체인 단계에서 시작한다.

원문 예	수식	흐름
ReLU	$\max(x, 0)$	<code>tag(Zero) → clip(Max, 0)</code>
ReLU 후 리듀스	$\sum_r \max(x_r, 0)$	<code>tag → clip(Max, 0) → inter_slice_reduce(AddSat)</code> (IntraFirst)
리듀스 후 bias	$(\sum_r x_r) + 100$	<code>inter_slice_reduce(AddSat) → tag → fxp(AddFxp, 100)</code> (InterFirst)
완전 합	$\sum_R x \times (R = 8,192)$	<code>tag → narrow_trim → intra_slice_reduce → widen_pad → inter_slice_reduce</code>

원문 예	수식	흐름
Pair Add	$g0 + g1$	unzip → clip_zip(AddFxp)

```
axes![B = 2, R = 8192];
// R을 Slice(R / 32 = 256), Time(R % 32 / 4 = 8), Packet(R % 4, 8칸으로 패딩)에 나눠 담았다
fn full_sum<'l, const T: Tu>{
    input: CollectTensor<'l, T, i32, m![1], m![B], m![R / 32], m![R % 32 / 4], m![R % 4 # 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![1 # 256], m![1], m![1 # 8]> {
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        .vector_narrow_trim::<m![R % 4]>() // Way8 → Way4 (뒤 4칸은 패딩)
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(IntraSliceReduceOpI32::AddSat) // Time·Packe
t의 R
        .vector_widen_pad::<m![1 # 8]>() // Way4 → Way8
        .vector_inter_slice_reduce::<m![1 # 256], m![1]>(InterSliceReduceOpI32::AddSat) // Slice의 R
        .vector_final()
}
```

축 하나가 Slice·Time·Packet에 걸쳐 있을 때 슬라이스 안 리듀서가 Time·Packet 부분을, 슬라이스 사이 리듀서가 Slice 부분을 없애는 전형적인 조합이다.

9.2 Intra-Slice Chain: 고정된 12단계

체인은 정해진 순서의 단계를 지난다. 필요한 단계만 부르고 나머지는 건너뛰는 건너뛰다. 원문의 `staged_pipeline` 은 $\max(\text{sigmoid}(x + 100), 0)$ 을 계산하며 Logic, FpDiv, Filter를 건너뛰는.

```
input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_fxp(FxpBinaryOp::AddFxp, 100) // x + 100
    .vector_fxp_to_fp(31) // i32 → f32 (int_width = 31)
    .vector_narrow_trim::<m![A % 2 # 4]>() // Way8 → Way4 (실수 경로 준비)
    .vector_fp_unary(FpUnaryOp::Sigmoid) // sigmoid(x + 100)
    .vector_widen_pad::<m![A % 2 # 8]>() // Way4 → Way8
    .vector_fp_to_fxp(31) // f32 → i32
    .vector_clip(ClipBinaryOpI32::Max, 0) // max(..., 0)
    .vector_final()
```

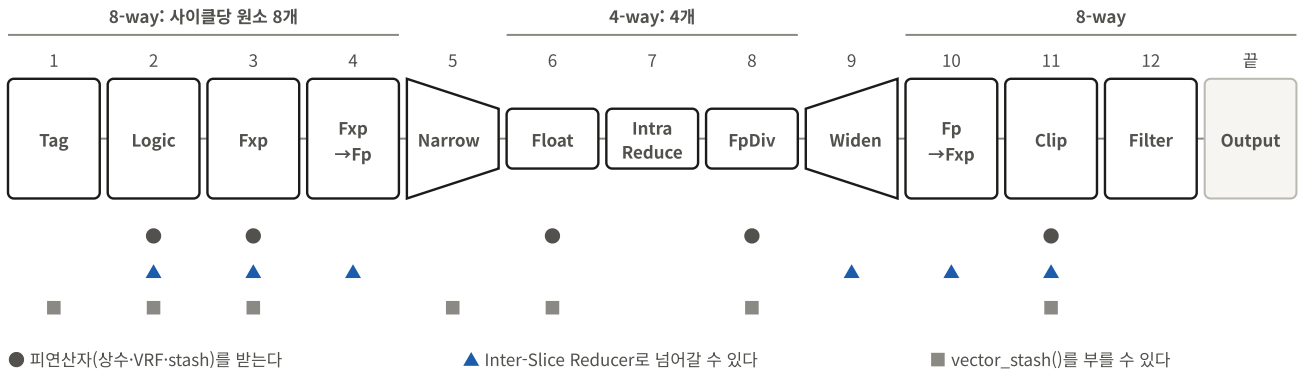



그림 34. Intra-Slice Chain의 단계와 폭. 실수 계열(Float, IntraSliceReduce, FpDiv)은 4-way로 돌므로 그 앞뒤에 Narrow와 Widen이 필요하다. 체인의 두 출구인 Output(`vector_final`)과 Inter-Slice Reducer로의 전환은 모두 8-way에서만 가능하다.

#	단계	메서드	Way	피연산자	Inter 전환
1	Entry(Tag)	<code>vector_intra_slice_tag()</code>	8	-	-
2	Logic	<code>vector_logic()</code>	8	있음	가능
3	Fxp	<code>vector_fxp()</code>	8	있음	가능
4	FxpToFp	<code>vector_fxp_to_fp()</code>	8	-	가능
5	Narrow	<code>vector_narrow_split()</code> / <code>vector_narrow_trim()</code>	8 → 4	-	-
6	Float	<code>vector_fp_unary/binary/ternary()</code>	4	있음	-
7	IntraSliceReduce	<code>vector_intra_slice_reduce()</code>	4	-	-
8	FpDiv	<code>vector_fp_div()</code>	4	있음	-
9	Widen	<code>vector_widen_concat()</code> / <code>vector_widen_pad()</code>	4 → 8	-	가능
10	FpToFxp	<code>vector_fp_to_fxp()</code>	8	-	가능
11	Clip	<code>vector_clip()</code>	8	있음	가능
12	Filter	<code>vector_filter()</code>	8	-	-

타입 시스템이 모든 단계 전환을 컴파일 시간에 강제하므로, 앞 체인이 호환되는 상태에 도달해야 다음 메서드를 부를 수 있다. `vector_reinterpret()`은 표에 없다. 어떤 단계도 차지하지 않아 두 단계 사이 어디에나 끼울 수 있다(‘**재해석: 비트는 그대로, 타입만 바꾸기**’).

단계는 8-way(사이클당 원소 8개)나 4-way(4개)로 돈다. 실수 클러스터는 처리량이 절반인 ALU를 나머지 체인과 맞추려고 4-way로 돈다. 그래서 실수 경로를 쓰는 체인은 8-way로 들어와 실수 단계 전에 Narrow를, 뒤에 Widen을 부른다.

메서드	쓸 때	효과 (모양)
<code>vector_narrow_split()</code>	두 절반 모두 실제 데이터	8-way 플릿 하나를 앞 4개·뒤 4개 패킷 둘로. [T], [P] → [T, P / 2], [P % 4]
<code>vector_narrow_trim()</code>	뒤 4개가 이미 패딩이거나 무관	앞 4개만 남김. [T], [P] → [T], [P = 4]
<code>vector_widen_concat()</code>	narrow_split을 되돌릴 때	4-way 패킷 둘을 8-way 플릿 하나로. [T, P / 2], [P % 4] → [T], [P]

메서드	쓸 때	효과 (모양)
<code>vector_widen_pad()</code>	<code>narrow_trim</code> 을 되돌릴 때	4-way 패킷을 무효 채움으로 8개까지 패딩. [T], [P] → [T], [P # 8]

원문은 좁히면 같은 논리 텐서에 두 배의 패킷이나 호출이 필요해 실수·리듀스 경로의 처리량이 절반이 된다고 적는다.

9.3 피연산자: 상수, VRF, stash

이항·삼항 연산에는 두 종류의 자리가 있다. 메서드 체인의 `self` 인 흐르는 텐서, 즉 **스트림**과, 하나 또는 두 개의 **피연산자**다. 피연산자는 세 곳에서 오며, 같은 메서드가 인자의 타입으로 출처를 구분한다. 삼항 연산(`FmaF`)은 피연산자 쌍(`operand0`, `operand1`)을 받는다.

출처	예	설명
상수	<code>100</code> , <code>2.5f32</code>	모든 원소에 브로드캐스트되는 스칼라
VRF 텐서	<code>&vrf_tensor</code>	Vector Engine에 들어가기 전에 <code>.to_vrf()</code> 로 올려 둔 값
stash	<code>Stash</code>	체인의 앞 단계에서 찍어 둔 스냅샷

```
.vector_fxp(FxpBinaryOp::AddFxp, 100)      // 상수 피연산자
.vector_fxp(FxpBinaryOp::MulInt, &vrf)     // VRF 텐서 피연산자
.vector_clip(ClipBinaryOpI32::Max, Stash)   // 앞서 찍어 둔 stash
```

VRF 피연산자의 세 규칙

- 분할이 같아야 한다:** 피연산자의 Chip·Cluster·Slice가 스트림의 것과 같아야 한다. 슬라이스 `s`의 연산은 슬라이스 `s`가 가진 값을 읽으므로, 다른 분할의 피연산자는 남의 슬라이스 데이터를 조용히 읽는 대신 컴파일 에러가 된다. 같은 슬라이스를 다른 이름으로 표현했을 뿐이라면 `VrfTensor::reshape` 로 스트림의 분할로 고쳐 쓴다. 예를 들어 익명 브로드캐스트 `m![256]` 로 복제한 피연산자를 이름 붙은 축으로 분할한 스트림에 먹일 때다.
- 접근 한 번은 인덱서 한 걸음:** 한 번의 접근(Packet, 8-way면 레인 8개, 4-way면 4개)이 인덱서의 한 걸음이며 읽기 방식은 두 가지뿐이다. *브로드캐스트*는 모든 레인에 한 주소의 값을, *연속*은 Element의 연속한 칸 Packet개를 준다. 따라서 패킷 축은 피연산자의 가장 안쪽 연속 축이거나 피연산자에 아예 없어야 한다. 패킷이 피연산자의 바깥 축을 건너, 한 접근 안에 브로드캐스트와 실제 레인이 섞이면 표현할 방법이 없어 거부된다.
- 축 불일치는 방향에 따라 다르다:** 피연산자에 있고 스트림에 없는 축은 그 칸들이 결코 읽히지 않으므로 `the stream leaves operand axes unmatched` 로 거부된다. 스트림에 있고 피연산자에 없는 축은 브로드캐스트다. 피연산자 하나가 스트림의 모든 행에 공급되는 방식이 이것이다.

stash: 잠깐 찍어 두고 한 번 쓰기

`vector_stash()` 는 흐르는 텐서의 스냅샷을 찍어 두고, 뒤의 이항·삼항 연산이 `stash` 피연산자로 읽게 한다. 원래 `x`가 중간 단계를 지나서도 살아 있어야 하는 `max(f(x), x)` 같은 잔차·스킵 연결이 대표적인 쓰임이다.

```

input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_stash()           // 원래 x 저장 (Way8)
    .vector_narrow_trim::<m![A % 2 # 4]>()
    .vector_fp_unary(FpUnaryOp::Sigmoid) // sigmoid(x) (Way4)
    .vector_widen_pad::<m![A % 2 # 8]>()
    .vector_clip(ClipBinaryOpF32::Max, Stash) // max(sigmoid(x), x)
    .vector_final()

```

- **VRF를 빌려 쓴다:** stash는 흐르는 텐서의 일부를 VRF에 저장했다가 다시 읽는다. 호출 하나에서 stash를 쓰면 컴파일러가 실제 양과 무관하게 **슬라이스당 VRF 1,024 B를 예약**한다. 같은 호출에서 VRF 피연산자도 읽는다면 두 용도가 8 KiB VRF를 나눠 쓰므로 피연산자는 슬라이스당 7 KiB 이하여야 한다.
- **부를 수 있는 단계:** Stashable 단계인 Branch(Tag), Logic, Fxp, Narrow, Fp, FpDiv, Clip 일곱 곳이다. VRF 쓰기 포트를 가진 체인 단계가 바로 이 일곱이며 Widen은 아니다. 그래서 4-way 구간에서 계산한 값을 stash하려면 먼저 Widen한 뒤 Clip 연산에서 stash한다.
- **한 번 쓰고 한 번 읽는다:** 두 번째 vector_stash() 도, 두 번째 Stash 읽기도 컴파일 에러다. 여러 번 읽을 값은 stash가 아니라 VRF(&vrf)에 둔다.
- **타입이 있다:** f32 stash는 f32 연산에만 먹일 수 있으므로 쓰기와 읽기 사이의 변환이나 재해석은 되돌려 놓아야 한다. 매핑은 흐르는 텐서를 따라가므로 Narrow 전에 찍은 stash를 Widen 뒤에 쓸 수 있다.
- **수명:** 스냅샷은 Tensor Unit 호출이 끝날 때까지 유효하다. 다만 Pair Mode에서는 쓸 수 없고, IntraFirst로 Inter-Slice Reducer에 넘어가면 사라진다.

인자 모드

인자 모드는 어느 자리에 스트림을, 어느 자리에 피연산자를 넣을지 고른다. 같은 연산으로 stream - operand도, operand - stream도 계산할 수 있다. 단항 연산은 모드 없이 늘 op(stream)이다.

BinaryArgMode	자리	계산
Mode00	stream / stream	op(stream, stream)
Mode01	stream / operand	op(stream, operand) (기본)
Mode10	operand / stream	op(operand, stream)
Mode11	operand / operand	op(operand, operand)

```

.vector_fxp_with_mode(FxpBinaryOp::SubFxp, BinaryArgMode::Mode10, 7) // 7 - x

```

삼항 연산의 TernaryArgMode 도 같은 방식이며 Mode012 (기본: stream, operand0, operand1), Mode002, Mode102, Mode112, Mode020, Mode021, Mode120 의 일곱 가지가 있다. 숫자 0은 스트림, 1은 operand0, 2는 operand1 자리를 뜻한다.

9.4 ALU 규칙과 연산 클러스터

원문은 단계 안의 연산 회로를 **ALU**(Arithmetic Logic Unit)라고 부른다. 규칙은 하나다. **한 단계 안에서 각 ALU는 Tensor Unit 호출 한 번에 최대 한 번만 돈다.** 여러 연산자가 단계별 ALU 풀을 나눠 쓰는 Logic, Fxp, Fp, Clip에서 특히

중요하다. 예를 들어 $\tanh(\sqrt{x})$ 는 $\tanh$ 와 $\sqrt{}$ 가 모두 `FpFpu` ALU를 쓰므로 호출 한 번에 넣을 수 없다.

```
// PANICS: "FxpAdd is already in use"
input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_fxp(FxpBinaryOp::AddFxp, 10) // FxpAdd 사용
    .vector_fxp(FxpBinaryOp::MulInt, 2)  // FxpMul 사용 ✓
    .vector_fxp(FxpBinaryOp::SubFxp, 5)  // FxpAdd 다시 ✗
    .vector_final()
```

클러스터	Way	ALU (각 1회)	연산 (괄호 안은 쓰는 ALU)
Logic	8	LogicAnd, LogicOr, LogicXor, LogicLshift, LogicRshift	i32: BitAnd, BitOr, BitXor, LeftShift, LogicRightShift·ArithRightShift(LogicRshift). f32: 비트 패턴에 BitAnd·BitOr·BitXor
Fxp	8	FxpAdd, FxpLshift, FxpMul, FxpRshift	AddFxp·AddFxpSat·SubFxp·SubFxpSat(FxpAdd), LeftShift·LeftShiftSat(FxpLshift), MulFxp·MulInt(FxpMul), LogicRightShift·ArithRightShift·ArithRightShiftRound(FxpRshift)
Float	4	FpFma, FpFpu, FpExp, FpMul0, FpMul1	단항: Exp·NegExp(FpExp), Sqrt·Tanh·Sigmoid·Erf·Log·Sin·Cos(FpFpu). 이항: AddF·SubF(FpFma), MulF(Mul0·Mul1·Fma 중 선택), DivF(FpFpu). 삼항: FmaF(FpFma)
IntraSliceReduce	4	전용 누산기 트리(선택 불가)	i32: AddSat, Max, Min. f32: Add, Max, Min
FpDiv	4	전용 나눗셈기(선택 불가)	DivF
Clip	8	ClipAdd, ClipMax, ClipMin	i32: Min·AbsMin(ClipMin), Max·AbsMax(ClipMax), AddFxp·AddFxpSat(ClipAdd). f32: Min, Max, AbsMin, AbsMax, Add

변환 단계 `vector_fxp_to_fp(int_width)`와 `vector_fp_to_fxp(int_width)`는 i32와 f32 사이를 값 보존으로 바꾼다 (int_width = 31이 표준). Filter는 태그 가드(보통 GroupId)로 만든 실행 마스크로 출력 플릿을 거르며 8-way, Standalone 컨텍스트 전용이다. Output(`vector_final`)은 Vector Engine을 나가 Cast·Transpose·Commit Engine으로 이어진다.

원문의 $\exp(\sqrt{((x+1) \times 2) \times 3})$ 예는 ALU 다섯 개를 모두 다르게 골라 호출 하나에 넣는다. $x+1$ 은 FpFma(AddF), $\times 2$ 는 FpMul0, $\times 3$ 은 FpMul1, $\sqrt{}$ 는 FpFpu, $\exp$ 는 FpExp다. 곱셈 ALU가 둘(Mul0, Mul1)이고 FMA ALU도 곱할 수 있어 가능한 조합이다.

재해석: 비트는 그대로, 타입만 바꾸기

`vector_reinterpret::<D2>()`는 스트림을 다른 32비트 스칼라로 다시 읽는다. 비트를 건드리지 않으므로 1.0f32는 1이 아니라 `0x3f80_0000`이 된다. 원소가 모두 32비트이고 각 클러스터가 실행하는 연산에서 피연산자 타입을 정하므로, 재해석은 명령을 내지 않고 ALU도 차지하지 않으며 현재 단계와 way를 유지한다. 그래서 체인 어디에나, 같은 클러스터의 두 연산 사이에도 몇 번이든 넣을 수 있다. 값을 보존하는 변환에는 FxpToFp·FpToFxp를 쓴다. 재해석은 뒤따르는 stash가 어떤 타입으로 저장될지도 정한다.

```
// f32의 부호 비트를 지우면 |x|: 실수 ALU 없이 LogicAnd 하나로
input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_reinterpret::<i32>()
    .vector_logic(LogicBinaryOpI32::BitAnd, 0x7fff_ffff) // 비트를 i32로 읽는다
    .vector_reinterpret::<f32>() // 부호 비트 지우기
    .vector_final() // 다시 f32로
```

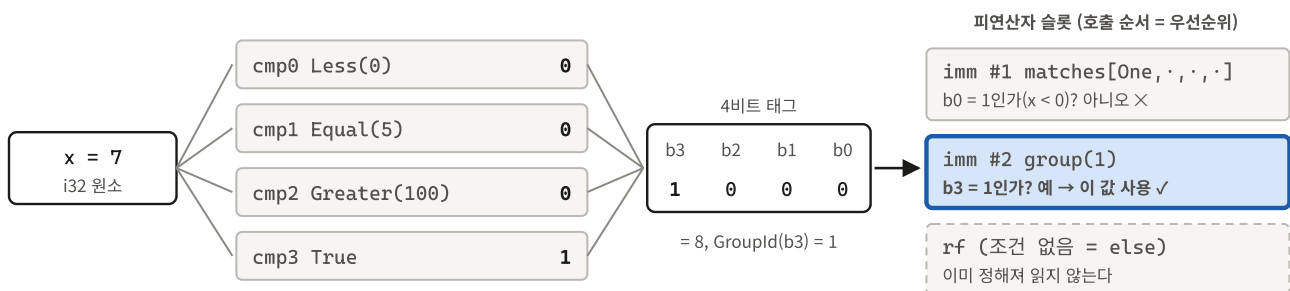
성능: Logic·Fxp·Clip 클러스터는 사이클당 원소 8개를 처리하고, Float 클러스터는 4개이며 앞뒤 Narrow·Widen 때문에 실질 처리량이 절반이 된다. 지연은 ALU 하나에 1사이클씩 더해진다. `exp(sqrt(x))`는 FpFpu와 FpExp를 써서 2사이클이다.

9.5 태그와 조건부 피연산자

체인의 첫 단계인 Tag는 플릿의 32비트 원소마다 4비트 **태그**(0~15)를 붙인다. 비트 3(MSB)은 **GroupId**로, Filter와 Pair Mode가 원소를 그룹 0·1로 나누는 데 쓴다. 비트 0~2는 비교 결과로 채우는 범용 플래그다. 이후의 이항·삼항 연산은 태그에 따라 원소마다 다른 피연산자를 줄 수 있다.

TagMode	비트를 채우는 방법	상태
Zero	네 비트 모두 0. 모든 원소의 태그가 0	동작
Comparison([cmp0, cmp1, cmp2, cmp3])	원소 x마다 $cmp_i(x)$ 가 참이면 비트 $i = 1$. <code>Cmp<D></code> 는 <code>Equal</code> , <code>Less</code> , <code>Greater</code> , <code>LessUnsigned</code> , <code>GreaterUnsigned</code> , <code>True</code> , <code>False</code> (Unsigned 쌍은 비트 패턴 그대로 비교)	동작
AxisToggle { axis }	GroupId = 축 인덱스 % 2, 비트 0~2는 0	호스트에서는 돌지만 커널 컴파일에서 거부. <code>vector_intra_slice_unzip()</code> 을 쓴다

`ValidCount` 와 `Vrf` 모드는 두 경로(호스트·장치)에서 모두 돌 때까지 enum에서 빠져 있다.



슬롯은 즉시값 레지스터 3개 + 레지스터 파일 포트(VRF 또는 stash) 1개. 원소는 가드를 만족하는 첫 슬롯을 쓴다.

그림 35. 태그 계산과 조건부 피연산자. 태그 비트는 LSB부터 비교 결과로 채워지고, 연산은 슬롯을 호출한 순서대로 검사해 원소마다 첫 번째로 맞는 슬롯의 값을 피연산자로 쓴다. 조건 없는 슬롯은 else 역할이다.

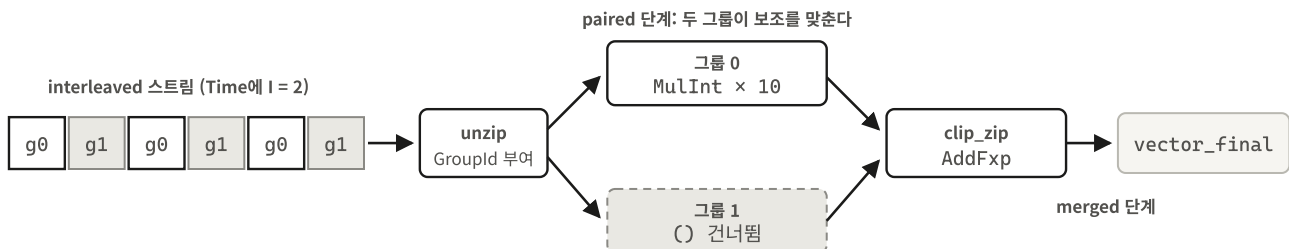
규칙은 한 문장이다. **조건이 없으면 피연산자를 그냥 쓰고, 조건이 있으면 그 값이 차지할 하드웨어 슬롯을 Branched 로 명시한다.** 연산 한 번(pass)에는 슬롯이 넷 있다. 즉시값(immediate) 레지스터 셋과, VRF 레지스터나 stash를 읽는 레지스터 파일 포트 하나다. 조건부 피연산자는 `Branched::imm` (즉시값)이나 `Branched::rf` (포트)로 어느 종류의 슬롯을 쓸지 밝히고, 이어서 올 수 있는 것만 제공하는 빌더를 돌려받는다. `imm` 은 세 번까지, `rf` 는 한 번만 마지막에 부를 수 있으며, 네 번째 `imm` 이나 `rf` 뒤의 `imm` 은 컴파일되지 않는다.

- 커널은 **어느** 즉시값 레지스터를 쓸지 고르지 않는다. imm은 다음 레지스터를 가져가고, 값이 실제로 어느 물리 레지스터에 들어갈지는 명령 생성이 정한다. 보존되는 것은 **호출 순서**이고, 이것이 매칭 우선순위다.
- 가드 TagGuard는 TagGuard::all(), 그룹 하나를 고르는 TagGuard::group(id), 또는 LSB부터 비트마다 BitReq (One, Zero, Ignore)를 적은 matches/not_matches다. 요구는 비트의 값에 대한 것이며, 어떤 태그 모드가 그 비트를 채웠는지와 무관하다.
- 두 가드는 아예 거부된다. 어떤 실행 id도 만족하지 않는 not_matches([Ignore, Ignore, Ignore, Ignore])와, 조건 없는 슬롯 뒤에 놓인 슬롯이다. 슬롯이 넷뿐인데 쓸모없는 슬롯이 자리를 차지하기 때문이다. matches([Ignore; 4])는 받아들여져 TagGuard::all()로 저장된다.
- 레이아웃을 만들어 돌려주는 헬퍼 함수는 보통 함수로 만들 수 있다. 다만 새 피연산자 타입을 만들 수 없고(피연산자가 stash에 무엇을 하는지 주장하는 트레잇이 닫혀 있다), 슬롯을 조건부로 채울 수도 없다(채운 슬롯 수가 빌더 타입의 일부라 if의 두 갈래가 일치하지 않는다). 슬롯 수 대신 값이나 가드를 바꾸거나 연산 전체를 분기한다.

원문은 이 규칙을 문장으로 다시 적는 대신 실제로 컴파일되는 예제를 가리킨다. furiousa-opt-examples의 ve_elementwise_branched (가드 붙은 즉시값 + 조건 없는 rf 읽기)와 ve_branch_bit_order (즉시값 셋 + 가드 붙은 stash 읽기)다.

9.6 Pair Mode: 두 그룹을 짝지어 계산하기

Pair Mode는 원소가 번갈아 끼워진 두 그룹으로 나뉘는 텐서에서 두 그룹 사이의 연산(짝별 덧셈, 한쪽만 스케일 등)을 한다. § 5.3의 begin_interleaved로 두 텐서를 번갈아 읽은 스트림이 전형적인 입력이다. vector_init() 직후, 두 갈래 그룹 축을 가진 스트림에서 vector_intra_slice_unzip()으로 들어간다. unzip은 그룹 축에서의 위치로 원소의 GroupId를 정한다. TagMode가 아니라 분기 유닛의 카운트 증강기를 구동하므로 처음 준 태그는 그대로 남으며, 그룹 분할의 지원 경로는 AxisToggle이 아니라 unzip이다. unzip으로 시작하면 뒤에서 Filter를 쓸 수 없다.



Pair Mode 내내 stash-filter-intra_slice_reduce를 쓸 수 없다. intra_slice_reduce는 zip 뒤에만 가능하다.

그림 36. Pair Mode의 흐름(원문의 "그룹 0만 10배 한 뒤 더하기" 예). unzip → paired 단계 → _zip → merged 단계 → vector_final 순서로 진행된다.

- **공통 단계** vector_fxp_to_fp, vector_narrow_split, vector_widen_concat, vector_fp_to_fxp는 두 그룹에 똑같이 적용된다. vector_narrow_trim과 vector_widen_pad는 쌍에 없으므로 split·concat 변형을 쓴다.
- **그룹별 연산**: 이항·삼항 연산(vector_fxp, vector_fp_binary, vector_fp_ternary, vector_clip 등)은 그룹마다 인자를 하나씩 받으며 ()로 한쪽을 건너뛰거나 양쪽에 다른 피연산자를 줄 수 있다. 단항 vector_fp_unary만 예외로 (op, group0_apply, group1_apply) 플래그를 받는다.
- **인자 모드의 재해석**: 그룹별 연산(vector_fxp_with_mode 등)의 모드는 각 그룹 안에서(0 = 그 그룹의 스트림, 1 = 그 그룹의 피연산자), _zip 연산의 모드는 두 그룹 사이에서(0 = 그룹 0 스트림, 1 = 그룹 1 스트림) 해석된다. 예를 들어 vector_fp_zip_with_mode(FpBinaryOp::SubF, BinaryArgMode::Mode10)은 group1 - group0이다.

```
// exp(group0) × group1 (실수 경로)
input
  .vector_init()
  .vector_intra_slice_unzip::<I, m![1]>()
  .vector_narrow_split::<m![1 # 2], m![A % 2 # 4]>() // 두 그룹 모두 Way8 → Way4
  .vector_fp_unary(FpUnaryOp::Exp, true, false) // 그룹 0: exp(x), 그룹 1: 건너뛴
  .vector_fp_zip(FpBinaryOp::MulF(FpMulAlu::Mul0)) // exp(group0) * group1
  .vector_widen_concat::<m![1], m![A % 2 # 8]>()
  .vector_final()
```

주의: Pair Mode의 제약

- `stash()`와 `filter()`는 `paired-merged` 단계 모두에서 쓸 수 없다.
- `vector_intra_slice_reduce()`도 `zip` 전후 모두 쓸 수 없다. 리듀서는 그룹 조건을 받지 않는다.
- `paired` 단계에서는 `Inter-Slice Reducer`로 넘어갈 수 없다. `zip` 뒤(`merged`)에는 다시 `Commit` 가능한 상태가 되어, 현재 단계가 허용하면 `vector_inter_slice_reduce()`를 부를 수 있다.
- `ALU` 사용은 두 그룹이 공유한다. 한 그룹에서 쓴 `ALU`는 두 그룹 모두에서 소모된 것으로 친다.

9.7 Intra-Slice Reduce

`IntraSliceReduce` 단계는 각 슬라이스의 `Time`과 `Packet`에 있는 차원을 리듀스한다. `Chip-Cluster-Slice`는 그대로이며, 클러스터의 256개 슬라이스에 걸친 리듀스는 `Inter-Slice Reducer`의 몫이다. 4-way로 돌고 `Pair Mode`에서는 쓸 수 없다. 호출 `vector_intra_slice_reduce::<REDUCE_LABEL, OutTime, OutPacket>(op)`의 매개변수는 다음과 같다.

- **REDUCE_LABEL**: 리듀스할 축. `Time`과 `Packet`에 있는 이 축의 인자를 모두 없앤다. `R`이 `Time`에 `R / 4`, `Packet`에 `R % 4`로 나뉘어 있어도 `REDUCE_LABEL = R` 하나로 둘 다 없앤다.
- **op**: `IntraSliceReduceOpI32`는 `AddSat·Max·Min`, `IntraSliceReduceOpF32`는 `Add·Max·Min`.
- **OutTime, OutPacket**: 입력의 `Time-Packet`에서 `REDUCE_LABEL` 인자를 모두 뺀 모양.

원문 예의 배치	Time	Packet	하는 일
<code>R</code> 이 <code>Time</code> 에만 ($R = 16$)	<code>m![R]</code>	<code>m![A % 2 # 8]</code>	시간 누산. 합은 <code>AddSat</code>
<code>R</code> 이 <code>Packet</code> 에만 ($R = 4$)	<code>m![A % 2]</code>	<code>m![R # 8]</code>	플릿 안 4-way 트리. 시간 누산 없음
둘 다 ($R = 16$)	<code>m![R / 4]</code>	<code>m![R % 4 # 8]</code>	$R % 4$ 는 트리, $R / 4$ 는 누산(Max)
<code>Slice</code> 까지 ($R = 13 \rightarrow R \# 32$)	<code>m![R # 32 / 4 % 2]</code>	<code>m![R # 32 % 4 # 8]</code>	<code>Slice</code> 부분($R \# 32 / 8$)은 결과에 남는다. 패딩은 <code>VCG</code> 가 뺀다(Min)

구조는 `Time` 경로와 `Packet` 경로가 따로다.

- **Time 경로**: `Contraction`의 `Time Reducer`와 같은 누산기 모델이되 **슬롯 용량이 8**이다(`InnerTime::SIZE ≤ 8`). 원문의 반례는 `Time = m![R, A % 3, B % 4]`에서 `InnerTime`이 $3 \times 4 = 12$ 여서 거부된다.
- **Packet 경로**: 플릿의 원소 4개는 두 길 중 하나를 탄다. `OutPacket = Packet`이면 리듀스 없이 네 값이 각자 `Time`을 따라 누산되고, `OutPacket = m![1 # 4]`이면 2단 트리 `op(op(a, b), op(c, d))`로 값 하나(+ 패딩 3칸)가 된다.

패딩을 어떻게 뺀까

리듀스 축을 하드웨어 차원에 맞추려고 패딩하면 그 자리에 임의 값이 들어 있어 리듀스에서 빼야 한다. 방법은 둘이다.

- **VCG(Valid Count Generator)**: 축 배치가 지원되면 이것을 쓴다. 컴파일러가 매핑으로부터 VCG를 자동 설정해 플릿마다 유효 원소 수를 붙이고, 패딩 원소가 자동으로 빠진다. 모든 배치를 지원하지는 않는다([§ 9.8](#)).
- **항등원 패딩**: 데이터가 체인에 닿기 전에 패딩 자리를 리듀스 연산의 항등원으로 채운다. AddSat·Add는 0·0.0, Max는 `i32::MIN·f32::NEG_INFINITY`, Min은 `i32::MAX·f32::INFINITY`. 단 리듀스 앞에 되돌릴 수 없는 변환이 없어야 한다. 예를 들어 $\exp(x) + \exp(y) + \dots$ 에는 $\exp(p) = 0$ 인 p 가 없어 쓸 수 없다. 원문은 이 용도로 쓸 공개 마스크 연산을 문서화하지 않으므로, 검증된 방법으로 항등값을 만들라고 적는다.

성능: 트리가 체인 안에서 파이프라인되므로 처리량은 사이클당 플릿 하나로 유지된다. 대신 리듀스 축의 시간 단계 수 n 만큼 첫 출력이 늦어지고, 여러 엔진이 이어진 파이프라인에서는 그동안 뒤 엔진이 첫 플릿을 기다린다.

9.8 Valid Count Generator

VCG는 "패딩된 리듀스 축에서 진짜 원소만 더하기"를 푼다. 사용자는 $R \# \text{PADDED_SIZE} / n \% m$ (스트라이드 n , 모듈로 m) 모양의 하위 표현식으로 쪼개 Slice·Time·Packet에 배치하고, 컴파일러가 VCG를 설정해 8원소 플릿마다 **valid_size**(0~8)를 붙인다. 플릿의 앞 valid_size개가 진짜이고 나머지는 패딩이다. 예를 들어 $R = 43$ 을 $R \# 48$ 로 패딩한 뒤 Slice에 $R \# 48 / 8$ (6칸), Time에 $R \# 48 / 2 \% 4$ (4칸), Packet에 $R \# 48 \% 2$ (2칸)로 나눌 수 있다.

```
struct VcgConfig {
    time_filters: [TimeFilterConfig; 3], // Time·Slice에 있는 R의 하위 표현식용
    packet_clipper: PacketClipperConfig, // Packet에 있는 R의 하위 표현식용
}
fn valid_size(&self, s: u64, t: u64) -> u32 { // s = 슬라이스 id, t = 시간 단계
    if self.time_filters.iter().all(|tf| tf.valid(s, t)) {
        self.packet_clipper.valid_size(t) // 모든 시간 필터가 유효라면 패킷 클리퍼가 개수를 정한다
    } else {
        0 // 하나라도 무효면 플릿 전체를 뺀다
    }
}
```

Time filter

시간 필터는 슬라이스 s 의 시간 단계 t 가 유효한 R 데이터를 담는지 판정한다. 필터 안의 시퀀서가 t 를 하위 표현식별 카운터로 분해하고, R 에 속한 카운터의 (값 $\times$ 스트라이드)를 더해 **idx**(Time 쪽 R 인덱스 기여)를 만든다. R 이 아닌 축(예: A)은 스트라이드 0이라 idx에 영향이 없다.

```
fn valid(&self, s: u64, t: u64) -> bool {
    let idx = self.sequencer.index(t);
    match ((s & self.slice_mask).cmp(&self.slice_thres), self.mode) {
        (Less, _) => true, // 경계보다 앞 슬라이스: 전부 유효
        (Greater, SliceMajor) => false, // 경계보다 뒤 슬라이스: 전부 무효
        - => idx < self.time_thres as u64, // 경계 슬라이스: idx로 판정
    }
}
```

배치	설정의 핵심	원문 예
R 이 Time 전부	<code>idx = t, slice_mask = 0, time_thres = R::SIZE</code>	$R = 12$, $R \# 16$: 앞 12단계 유효, 뒤 4단계 무효
R 이 Time에 흩어짐	R 카운터만 스트라이드를 갖고 다른 축은 0	$R = 10$, Time = $[R \# 12 / 4, A, R \# 12 \% 4]$: $t = 13 \rightarrow \text{idx } 5$ 유효, $t = 27 \rightarrow \text{idx } 11$ 무효

배치	설정의 핵심	원문 예
R이 Slice(major)와 Time(minor): <code>SliceMajor</code>	<code>slice_mask</code> 가 슬라이스의 R 기여를 뺀고, 경계 슬라이스만 <code>idx < time_thres</code> 로 판정	$R = 11, R \# 16$: 슬라이스 기여 $0 \cdot 4 \cdot 8 \cdot 12 \rightarrow$ 유효 단계 $4 \cdot 4 \cdot 3 \cdot 0$
R이 Time(major)와 Slice(minor): <code>TimeMajor</code>	시퀀서 스트라이드를 <code>slice_span</code> 으로 나누고 <code>time_thres = time_span - 1</code> 조건: $\text{PADDED_SIZE} - R::\text{SIZE} \leq \text{slice_span}$	$R = 13, R \# 16$: 슬라이스 기여 $0 \cdot 1 \cdot 2 \cdot 3 \rightarrow$ 유효 단계 $4 \cdot 3 \cdot 3 \cdot 3$

`SliceMajor`에서는 `Slice` 안의 하위 표현식이 스트라이드 내림차순(major 먼저)이어야 하고, 각각 크기와 스트라이드가 2의 거듭제곱이어야 `slice_mask`의 연속 비트를 차지한다. `Time` 안에서는 순서가 자유롭다. `TimeMajor`는 이 규칙을 물려받고 과패딩이 `slice_span` 이하라는 조건을 더하며, 그 덕분에 "전부 무효인 슬라이스"가 생기지 않는다. 원문은 두 모드 모두 모든 플릿에 대해 올바르게라는 증명을 실행한다. `SliceMajor`의 요지는 R 인덱스를 $r = (\text{슬라이스 기여}) + \text{idx}$ 로 쓰면, 슬라이스 기여가 경계보다 작을 때 r 은 늘 $R::\text{SIZE}$ 미만, 같을 때는 $\text{idx} < \text{time_thres}$ 일 때만 미만, 클 때는 늘 $R::\text{SIZE}$ 이상이라는 것이다. R이 `Time-Slice`에 없으면 `slice_mask = 0`, `slice_thres = 1`로 필터를 꺼서 늘 참이 되게 한다.

Packet clipper

```
fn valid_size(&self, t: u64) -> u32 {
    let idx = self.sequencer.index(t);           // Time 쪽 R 인덱스 기여
    (self.axis_size - idx).clamp(0, self.packet_span) // 남은 R 원소 중 이 플릿에 들어갈 수
}
```

패킷 클리퍼는 **t에만** 의존하고 슬라이스 s와는 무관하다. 그래서 모든 슬라이스가 같은 시간 단계에 같은 개수를 받으며, 이것이 표현할 수 있는 배치를 제한한다. Packet은 반드시 `m![R # PADDED_SIZE % packet_span # 8]` 모양이어야 한다. 예를 들어 $R = 10$ 을 $R \# 16$ 으로 패딩해 `Time R # 16 / 4`, `Packet R # 16 % 4 # 8`로 두면 `axis_size = 10`, `packet_span = 4`이고, 사이클마다 `valid_size`가 `clamp(10 - 0) = 4`, `clamp(10 - 4) = 4`, `clamp(10 - 8) = 2`, `clamp(10 - 12) = 0`이 된다. R이 Packet에 없으면 `axis_size = packet_span = 8`, 빈 시퀀서로 두어 늘 8을 준다.

한데 모으기

실제 리듀스는 `REDUCE_LABEL` 하나(R)에 패딩된 다른 축들이 더해진 경우가 많고, VCG는 패딩된 축마다 시간 필터 하나를, R의 Packet 부분에 패킷 클리퍼를 배정한다. 원문의 예는 원래 shape가 $[H, C, W] = [5, 5, 19]$ 다. H와 C는 8로 패딩해 `Slice(바깥 4칸, Ho-Co)`와 `Time(안쪽 2칸, Hi-Ci)`에 나누고, W는 24로 패딩해 `Time(w # 24 / 8, 3칸)`과 `Packet(w # 24 % 8, 8칸)`에 나눈다. 패킷 클리퍼(W)는 W 행마다 $[8, 8, 3]$ 을, C의 시간 필터는 $\text{Co} = 0 \cdot 1$ 유효, $\text{Co} = 2$ 경계($\text{Ci} = 0$ 만), $\text{Co} = 3$ 무효를, H의 시간 필터는 Ho에 대해 같은 판정을 낸다.

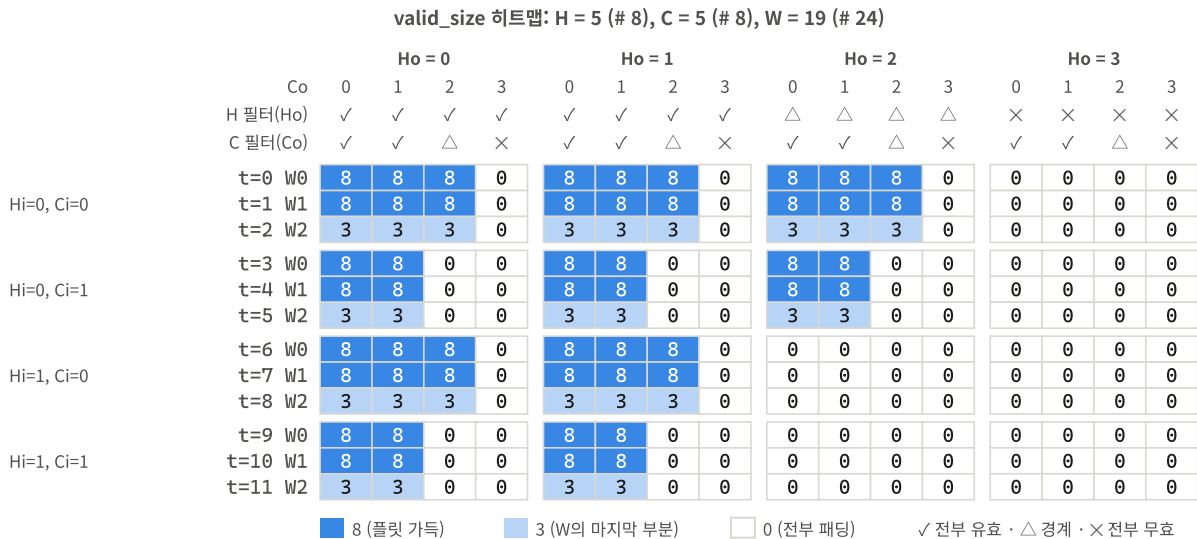


그림 37. VCG 세 부품의 합성. 슬라이스 16개(열, Ho와 Co로 묶음) × 플릿 12개(행, Hi·Ci·W 순서). Ho = 3이나 Co = 3인 열은 전부 0이고, 경계 열(Ho = 2 또는 Co = 2)은 해당 안쪽 카운터가 0인 행만 값을 가지며, 값이 있는 칸에서는 패킷 클리퍼의 [8, 8, 3] 톱니 무늬가 슬라이스와 무관하게 반복된다.

표현할 수 없는 배치와 용량

배치	왜 안 되나
Slice 안에서 순서가 뒤바뀜 (예: [X, R # 16 / 2 % 4, R # 16 / 8])	슬라이스별 R 인덱스 범위가 단조롭지 않아 slice_thres 하나로 가를 수 없다
Time-Slice-Time으로 끼워 넣음	슬라이스마다 필요한 유효 단계 수가 달라(예: 3/4, 2/4) 임계값 하나로 표현할 수 없다
TimeMajor 과패딩 (R = 14를 R # 20 으로)	20 - 14 = 6 > slice_span 4. time_span이 5가 되어 도달할 수 없는 패딩 인덱스 (R = 16)가 생긴다. CPU 엔진은 이를 panic으로 잡는다. 올바른 패딩은 R # 16
Packet에 R의 바깥 부분이 있거나 다른 축이 섞임	앞에서부터 세는 valid_size의 "연속 접두" 성질이 깨진다
R이 Slice와 Packet에 (R = 2045, Slice R # 2 048 / 8, Packet R # 2048 % 8)	t = 0에서 슬라이스 0~254는 8, 슬라이스 255는 5가 필요한데 valid_size(t)는 슬라이스마다 다른 값을 줄 수 없다. R::SIZE % packet_span = 0이면 Slice만의 경우로 줄어 지원된다

용량은 패킷 클리퍼 1개, 시간 필터 3개, 각각 시퀀서 엔트리 8개다. 그래서 호출 하나에서 유효성을 추적할 수 있는 패딩 축은 최대 4개이고, 패딩이 없는 축은 슬롯이 필요 없다. 리듀스 축(REDUCE_LABEL)은 하나뿐이므로 "여러 축"은 리듀스 축 하나에 패딩된 비리듀스 축들을 더한 것을 뜻한다.

VCG는 좁히기 전의 8-way 플릿마다 valid_size를 만든다. 뒤따르는 Narrow·Widen은 이를 다음과 같이 나눈다.

연산	입력 → 출력	유효 개수 변환
<code>vector_narrow_split</code>	8-way(v) → 4-way 둘	앞: min(v, 4), 뒤: max(v - 4, 0)
<code>vector_narrow_trim</code>	8-way(v) → 4-way 하나	min(v, 4). 매핑이 v ≤ 4를 정적으로 보장해야 데이터를 잃지 않는다
<code>vector_widen_concat</code>	4-way 둘 → 8-way	v _{low} + v _{high}
<code>vector_widen_pad</code>	4-way → 8-way	그대로

9.9 Inter-Slice Reducer

Inter-Slice Reducer는 클러스터 안 256개 슬라이스에 걸쳐 텐서를 리듀스한다. Chip·Cluster·Packet은 보존하고 Slice와 Time을 OutSlice·OutTime으로 다시 쓰며, 입력이 어느 모드였든 출력은 늘 Way8이다. `vector_init()` 직후 (InterFirst)나 호환되는 체인 단계에서(IntraFirst) 같은 이름의 메서드 `vector_inter_slice_reduce::<OutSlice, OutTime>(op)` 로 들어간다. 연산은 i32가 Add(wrapping)·AddSat·Max·Min, f32가 Add·Max·Min·Mul이다.

지원되는 모양은 네 규칙을 따른다.

1. **가장 안쪽부터 리듀스**: Slice에서 리듀스하는 부분은 가장 안쪽 인자들이어야 하고, 연속이며, 리듀스 비율 r 까지 스트라이드 1이어야 한다.
2. **비워진 자리 채우기**: 리듀스한 인자의 OutSlice 자리는 더미($1 \# n$), 새 차원으로의 브로드캐스트, Time에서의 승격 중 하나로 채운다.
3. **종류는 섞어도 된다**: 더미·브로드캐스트·승격 자리가 OutSlice에 어떤 순서로든 함께 올 수 있다.
4. **승격은 순서를 바꿀 수 있다**: Time $\rightarrow$ OutTime 부분은 살아남는 인자의 상대 순서를 지키지만, Time $\rightarrow$ OutSlice 승격은 순서를 지키지 않는다. 승격된 인자의 OutSlice 자리는 Time에서의 자리와 무관하다.

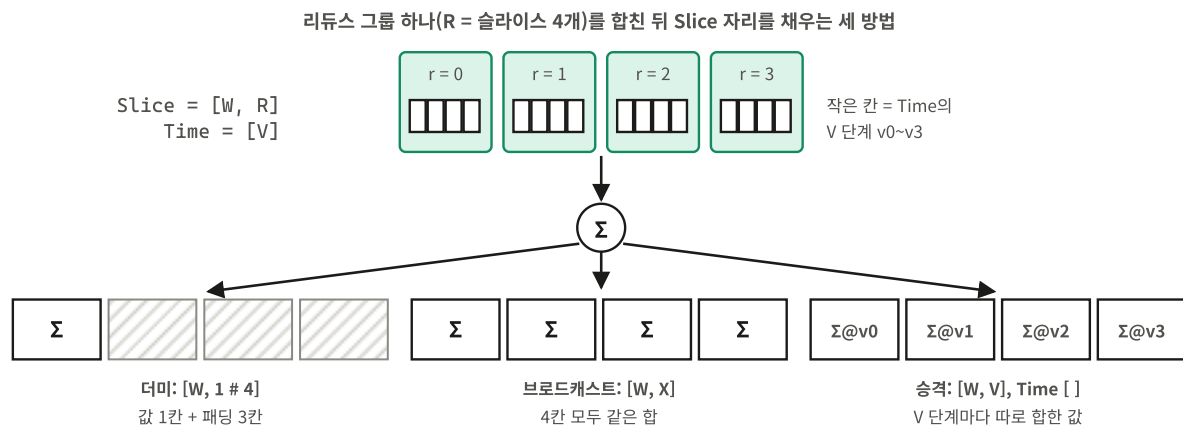


그림 38. Inter-Slice Reducer의 세 가지 자리 채우기. 원문 예는 더미 $AR \rightarrow A$ (Slice $[A / 8, R] \rightarrow [A / 8, 1 \# 4]$), 브로드캐스트 $PRW \rightarrow PWX$ ($[W, R] \rightarrow [W, X]$), 승격 $PRSUW \rightarrow PSUW$ (Slice $[W, R] \rightarrow [W, V]$, Time $[S, V, U] \rightarrow [S, U]$)다. 승격되는 V 가 Time의 가장 바깥에 있을 필요는 없다.

성능: 리듀스 그룹의 슬라이스 수 r 이 주된 조절 손잡이다. 지연은 $O(r)$ 사이클로, 대략 리듀스 그룹을 링으로 한 바퀴 도는 시간이다. 전체 시간은 입력을 스트리밍하는 시간에 이 링 꼬리가 더해진 값이다. 실제로는 앞 단계(부분합을 만드는 Contraction이나 리듀서 앞의 체인 작업)가 지배적이라 리듀서가 병목이 되는 일은 드물다. r 이 크거나(꼬리가 길다) 텐서가 작아 고정 꼬리를 여러 패킷에 분산하지 못할 때 드러난다.

스스로 점검

1. `tanh(sqrt(x))`를 Tensor Unit 호출 한 번에 넣을 수 없는 이유는?
2. stash를 쓰는 호출에서 VRF 피연산자가 슬라이스당 8 KiB라면 어떻게 되나?
3. `Comparison([Less(0), Equal(5), Greater(100), True])`에서 $x = -3$ 의 태그는?
4. $R = 20$ 을 $R \# 32$ 로 패딩해 Time 전체에 두면 시간 필터의 `time_thres`는?
5. Inter-Slice Reducer 뒤 비워진 Slice 자리를 채우는 세 방법은?

답: ① 둘 다 FpFpu ALU를 쓴다. 호출을 둘로 나눈다. ② stash가 1,024 B를 예약하므로 7 KiB까지만 가능해 들어가지 않는다. ③ $b0(x < 0) = 1, b1 = 0, b2 = 0, b3 = 1 \rightarrow 0b1001 = 9$. ④ $R::SIZE = 20$. ⑤ 더미, 새 축으로 브로드캐스트, Time에서 승격.

10장

Tensor Unit 연산 IV: 출력 단계와 칩·클러스터 리듀스

원문: Cast Engine · Transpose Engine · Commit Adapter · Case Study: Chip and Cluster Reduction

계산이 끝난 스트림은 DM에 쓰이기 전에 몇 가지 정리를 거친다. Cast Engine은 정밀도를 줄이고, Transpose Engine은 플릿 안에서 Time과 Packet을 맞바꾸며, Commit Adapter는 패딩을 잘라내고 필요하면 f32를 bf16으로 줄인다. 장의 뒷부분은 원문의 사례 연구인 칩·클러스터 리듀스를 다룬다. 칩이나 클러스터마다 흩어진 부분 값을 DMA 재분배와 Vector Engine 덧셈으로 합치는 방법이다.

이 장에서 배우는 것

- Cast Engine이 지원하는 변환과 "병목이 아닌" 이유
- Transpose Engine의 다섯 단계(Unpack, Gather, Transpose, Trim, Align)와 버퍼링 모드별 지연 공식
- 모든 commit 앞의 트리밍, 그리고 Cast Engine 대신 `commit_cast` 를 쓰는 이유
- ReduceScatter·AllGather·AllReduce가 무엇이고 칩·클러스터에서 어떻게 구현되는지

10.1 Cast Engine

Cast Engine은 f32·i32 파이프라인 결과를 bf16 같은 낮은 정밀도로 줄여, Commit Engine이 DM에 쓸 때의 저장 비용을 낮춘다. `CollectTensor`, `ContractTensor`, `VectorFinalTensor` 가 모두 같은 뜻의 `.cast::<OutD, OutPacket>()` 를 가진다. 하드웨어 제약상 입력은 Vector Engine의 스칼라 타입(`veScalar`, 즉 i32·f32)만 받는다. 작성자는 목표 타입 OutD와 출력 원소 배치 OutPacket을 고르고 나머지는 컴파일러가 유도한다.

Cast Engine은 값을 바꾸므로 수학적 텐서 이동은 아니지만 텐서의 shape는 보존하고 원소 타입만 바꾼다. 모든 차원은 그대로이고, 출력이 여전히 32바이트 플릿 하나에 들어가도록 Packet 배치만 다시 패딩한다.

원문 예	입력 Packet	출력 Packet
i32 8개(32 B) → i8	<code>m![A]</code> (A = 8)	<code>m![A # 32]</code> (i8 8개 + 패딩 24칸 = 32 B)
유효 4개인 i32 → i8	<code>m![A # 8]</code> (A = 4, 32 B)	<code>m![A # 32]</code> (같은 4개를 32칸으로 패딩)

입력 타입	지원하는 출력 타입
i32	i4, i8, i16
f32	f8e5m2, f8e4m3, bf16

성능: Cast Engine은 결코 파이프라인의 병목이 되지 않는다. 플릿에 유효 데이터가 얼마나 들었든 사이클마다 플릿 하나를 처리하고, 뒤의 Commit Engine이 덜 찬 플릿들을 모아 뽁뽁한 DM 쓰기로 만들기 때문에 DM 대역폭도 낭비되지 않는다.

10.2 Transpose Engine

Transpose Engine은 스트림의 **Time**과 **Packet** 차원을 맞바꾼다. Chip·Cluster·Slice는 그대로다. `CollectTensor`와 `VectorFinalTensor`가 `.transpose::<OutTime, OutPacket>()`를 가지며, 후자는 Vector Engine 출력을 곧바로 받는다. i5·i9 중간 타입은 전치할 수 없다. 원문의 기준 예는 16폭 행이 입력 패킷 두 개(D = 2)에서 모이는 8×16 i8 행렬이다.

```
axes![P = 256, B = 2, C = 8, D = 2, E = 8];
// 입력: Time [B, C, D], Packet [E # 32] → 출력: Time [B, D, E], Packet [C # 32]
fn basic_transpose<'l, const T: Tu>(  
    input: CollectTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B, C, D], m![E # 32]>,  
) -> TransposeTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B, D, E], m![C # 32]> {  
    input.transpose()  
}
```

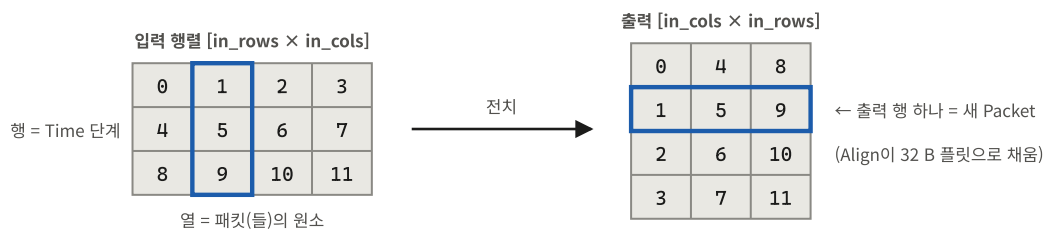
매개변수

valid_size는 Transpose Engine이 32바이트 입력 버스에서 사이클마다 읽는 유효 원소 수다. 입력 플릿은 늘 비트 폭 $\times$ **valid_size** 형태로 들어오고, 나머지 바이트는 Unpack 단계가 패딩으로 버린다. 8비트와 16비트 원소는 **이중 패킹 (doubled packing)**으로 플릿마다 기본 8개 대신 16개를 읽을 수 있고, 4비트와 32비트는 이중 패킹을 지원하지 않는다. Contraction Engine은 32비트 $\times$ 8 형태만 내보내고, Vector Engine과 Fetch Engine은 아래 표의 어떤 조합이든 내보낸다.

원소 크기	단일 valid_size	이중 valid_size	최대 in_rows	가능한 in_cols
4비트	16	지원 안 함	16	16, 32
8비트	8	16	8	8, 16, 32
16비트	8	16	4	8, 16, 32
32비트	8	지원 안 함	2	8, 16, 32

in_cols (입력 행 하나의 폭), **in_rows** (입력 행 수), **out_rows** (출력 행 수)는 작성자가 고른 OutTime·OutPacket으로 정해진다. 기준 예(i8, valid_size = 8)에서는 **in_cols** = 16(패킷 D = 2개 $\times$ 8), **in_rows** = 8(C), **out_rows** = 16(D $\times$ E, **in_cols**를 짝 채움)이다.

다섯 단계



Trim은 유효 원소가 valid_size보다 적어 생긴 패딩 행을 버리므로 $\text{out_rows} \leq \text{in_cols}$ 다.

그림 39. Transpose Engine의 동작. 입력의 열(Packet 방향)이 출력의 행이 되고, 입력의 행(Time 방향)이 출력의 원소 위치가 된다. 파란 테두리는 입력 열 1과 그것이 된 출력 행 1이다.

1. **Unpack**: 32바이트 입력 패킷마다 `valid_size`개의 유효 원소를 남기고 나머지를 버린다. 8·16비트 패킷에 기본 8개 너머의 유효 원소가 있으면 컴파일러가 이중 패킹을 고른다. 예를 들어 i8 패킷 `[B=16 # 32]`는 한 사이클에 16개를 모두 내고, i4는 이중 형태를 요청할 수 없다(유일한 `valid_size`가 16). 기준 예: `[C, D, E # 32] → [C, D, E]`.
2. **Gather**: 입력 행 하나(`in_cols = packets_per_col × valid_size`)는 연속한 `packets_per_col`개 패킷, 즉 가장 안쪽 시간 단계들로 조립된다. 그 위의 `in_rows`개 시간 단계가 쌓여 `[in_rows × in_cols]` 입력 행렬이 된다. Unpack과 Gather는 엔진이 입력을 읽는 동안 함께 일어나며 별도의 버퍼 단계가 아니다.
3. **Transpose**: `[in_rows × in_cols] → [in_cols × in_rows]`. 기준 예: `[C, D, E] → [D, E, C]`.
4. **Trim**: 일부 입력 패킷의 유효 원소가 `valid_size`보다 적으면 전치된 행렬에 패딩 행이 생기고, Trim이 이를 버려 `[out_rows × in_rows] (out_rows ≤ in_cols)`를 만든다. 기준 예는 꼭 차 있어 버릴 행이 없다.
5. **Align**: 전치된 행의 폭은 `in_rows`개 원소지만 DM 패킷은 32바이트여야 하므로, 각 행을 32바이트 플릿으로 패딩해 `[out_rows × (in_rows # F)]`를 만든다. F는 `D[F]`가 32바이트가 되는 값이다. 기준 예: `[D, E, C # 32]`.

버퍼링 모드와 지연

한 번의 버스트는 `n = OutTime::SIZE / out_rows` 번의 전치 반복이다. 반복마다 입력 플릿 `in_flits = in_rows × (in_cols / valid_size)` 개를 받고 출력 플릿 `out_rows`개를 낸다. 엔진 안에는 열 16개짜리 버퍼가 두 개 있고, 컴파일러는 `in_cols ≤ 16`이면 이중 버퍼링, 아니면 단일 버퍼링을 고른다. 이중 버퍼링은 입력과 출력을 겹쳐 전체 사이클을 줄이고, 단일 버퍼링은 두 단계를 직렬로 한다.

$$\begin{aligned} \text{단일 버퍼링: } & n \times (\text{in_flits} + \text{out_rows}) \\ \text{이중 버퍼링: } & \text{in_flits} + (n - 1) \times \max(\text{in_flits}, \text{out_rows}) + \text{out_rows} \end{aligned}$$

이중 버퍼링 식은 세 구간으로 나뉜다. 첫 버퍼가 차는 **입력 전용** 구간(`in_flits`), 한 버퍼가 입력을 받는 동안 다른 버퍼가 출력을 내는 **겹침** 구간(`(n - 1) × max(...)`, 느린 쪽이 반복을 정한다), 마지막 버퍼를 비우는 **출력 전용** 구간(`out_rows`)이다.

원문 예	in_cols	in_rows	out_rows	in_flits	n	모드	사이클
기준 예 (i8)	16	8	16	16	2	이중	$16 + 1 \times 16 + 16 = 48$
작은 행렬 (i8, B = 2)	8	4	2	4	1	이중	$4 + 0 + 2 = 6$
큰 열 (i8, D = 4)	32	8	32	32	2	단일	$2 \times (32 + 32) = 128$
bf16	8	4	8	4	8	이중	$4 + 7 \times 8 + 8 = 68$
i4	16	16	16	16	4	이중	$16 + 3 \times 16 + 16 = 80$
f32	8	2	8	2	4	이중	$2 + 3 \times 8 + 8 = 34$

작은 행렬 예에서는 B = 2개 원소가 8칸으로 패딩되어 `in_cols = 8`이 되고, Trim이 패딩 행 6개를 버려 `out_rows = 2`가 된다. 큰 열 예는 `in_cols`를 16보다 크게 만들어 단일 버퍼링을 강제하므로 입력과 출력이 겹치지 못해 사이클이 늘다. bf16은 원소가 넓어 최대 `in_rows`가 4로 줄고 32바이트 출력 플릿에 16개가 들어가며, i4는 `valid_size` 16으로 사이클당 원소 수가 두 배이고 최대 `in_rows`가 16($16 \times 4\text{비트} = 8\text{바이트}$)이며 플릿에 64개가 들어간다. f32는 Contraction Engine이 내보내는 형식(32비트 × 8)으로 최대 `in_rows`가 2다.

10.3 Commit Adapter

Commit Adapter는 Commit Engine이 DM에 쓰기 전 패킷 스트림에 원소별 변환을 적용한다. Tensor Unit 출력 쪽의 Fetch Adapter에 해당한다. 단계마다 `.commit_xxx(...)` 메서드가 있고, 체인은 늘 실제 DM 쓰기인 `.commit(...)` 으로 끝난다.

연산	main	sub
트리밍	✓	✓
타입 변환 (ReLU 결합 선택)	✓	×
Generate 모드 (내부 <code>memset</code> 경로)	×	✓

트리밍: 모든 commit의 첫 단계

파이프라인의 패킷은 항상 32바이트 플릿이지만 플릿에 유효 원소가 용량보다 적게 들어 있을 수 있고, 나머지는 패딩이다. 플릿을 그대로 쓰면 유효 영역 너머의 DM 바이트를 패딩 값으로 덮어쓴다. 트리밍은 각 플릿의 앞 `valid_size`개만 DM에 써서 이를 막는다. `valid_size`는 컴파일러가 출력 매핑에서 유도하며 사용자가 직접 정하지 않고, `D[valid_size]`는 $8 \cdot 16 \cdot 24 \cdot 32$ 바이트 중 하나여야 한다(32면 트리밍 없음). 지연은 거의 0이다.

트리밍은 필수 첫 단계다. `.commit()` · `.commit_view()` 는 `.commit_trim(...)` 뒤에서만 부를 수 있으므로 모든 commit이 트리밍을 거친다. `valid_size`가 이미 32바이트라서 할 일이 없더라도 체인의 닛 역할을 한다.

원문 예	입력 Packet	<code>commit_trim</code> 의 OutPacket
i8 패딩 제거	<code>m![W # 32] (W = 8)</code>	<code>m![W] : 32칸 중 유효 8개</code>
f32 일부만 유효	<code>m![W] (W = 8)</code>	<code>m![W = 4] : 8개 중 앞 4개(리사이즈)</code>
bf16 일부만 유효	<code>m![N] (N = 16)</code>	<code>m![N = 8] : 16개 중 앞 8개</code>
i4 트리밍 없음	<code>m![J] (J = 64)</code>	<code>m![J] : OutPacket = Packet</code>

타입 변환: `commit_cast`와 `commit_cast_relu`

Commit 경로의 타입 변환은 f32를 bf16으로 줄이며, 선택적으로 ReLU를 같은 패스에 결합한다(`commit_cast::<bf16>()`, `commit_cast_relu::<bf16>()`). 대부분의 타입 변환은 Cast Engine이 하는데 왜 이 경로가 따로 있을까? 원문의 설명은 이렇다.

핵심: Vector Engine을 비워 두려고

Cast Engine은 Vector Engine 위에 얹혀 있어서 변환하는 동안 Vector Engine을 점유한다. main 컨텍스트의 축약 결과를 Cast Engine으로 f32 → bf16 변환하면 그동안 Vector Engine이 바빠 sub 컨텍스트의 Vector Engine 작업이 병렬로 돌 수 없다. 변환을 Commit Adapter로 돌리면 Vector Engine이 비어 sub 컨텍스트가 쓸 수 있다. sub 컨텍스트 자체는 이 타입 변환을 지원하지 않는다.

두 메서드가 모드 인자 하나가 아니라 별도 메서드인 이유는 서로 다른 하드웨어 변환(`CommitF32ToBf16`, `CommitF32ToBf16Relu`)이기 때문이다. ReLU는 런타임에 고를 수 있는 독립 단계가 없고, 좁히는 캐스트와 결합된 형태로만 존재한다.

예를 들어 `commit_cast_relu`는 `[-5.0, -0.1, 0.0, 3.7]`을 `[0.0, 0.0, 0.0, 3.7]`로 바꾼다. 실제 main commit은 `.commit_trim()` 을 먼저 부르고 `.commit(...)` 으로 끝낸다.

Generate 모드는 `memset` 이 즉시 채우기에 쓰는 sub 컨텍스트의 내부 경로로, 앞선 Tensor Unit 스트림을 소비하지 않고 상수를 직접 쓴다. `TuTensor` 에서 체인으로 부르는 단계가 아니다.

10.4 사례: 칩과 클러스터를 넘는 리듀스

한 칩(또는 클러스터)이 가진 데이터를 다른 칩과 합쳐야 할 때가 있다. 텐서를 여러 칩에 나눠 계산하는 분산 추론에서 각 칩이 부분 결과를 가지고 있고, 이를 모두 더해 모든 칩이 전체 결과를 갖게 해야 하는 경우다. 분산 시스템에서는 이런 연산을 **집합 통신(collective)**이라 부른다. 원문의 사례 연구는 이를 DMA 재분배(§ 6.6)와 Vector Engine 덧셈으로 구현한다.

세 가지 집합 연산의 모양

[차원], [로컬] 표기는 차원(칩이나 클러스터)의 모양과, 각 차원 인덱스에 로컬로 저장된 모양을 구분한다. R은 리듀스할 차원, S는 분배하거나 모을 로컬 축, P...와 Q...는 S 앞뒤의 로컬 모양이다. ReduceScatter와 AllGather에서는 R과 S의 크기가 N으로 같다. `Broadcast(N)` 은 N개 인덱스가 모두 같은 값을 가진다는 뜻이다.

연산	모양 변환	뜻
ReduceScatter	$[R], [P..., S, Q...] \rightarrow [S], [P..., Q...]$	R을 리듀스하고 S를 분배한다. 차원 인덱스 s는 S의 좌표 s만 갖는다
AllGather	$[S], [P..., Q...] \rightarrow [\text{Broadcast}(N)], [P..., S, Q...]$	분배를 되돌려 모든 차원 인덱스가 S의 모든 좌표를 갖는다
AllReduce	$[R], [P...] \rightarrow [\text{Broadcast}(N)], [P...]$	로컬 모양은 유지하고 R을 리듀스한 결과를 모두에게 복제한다

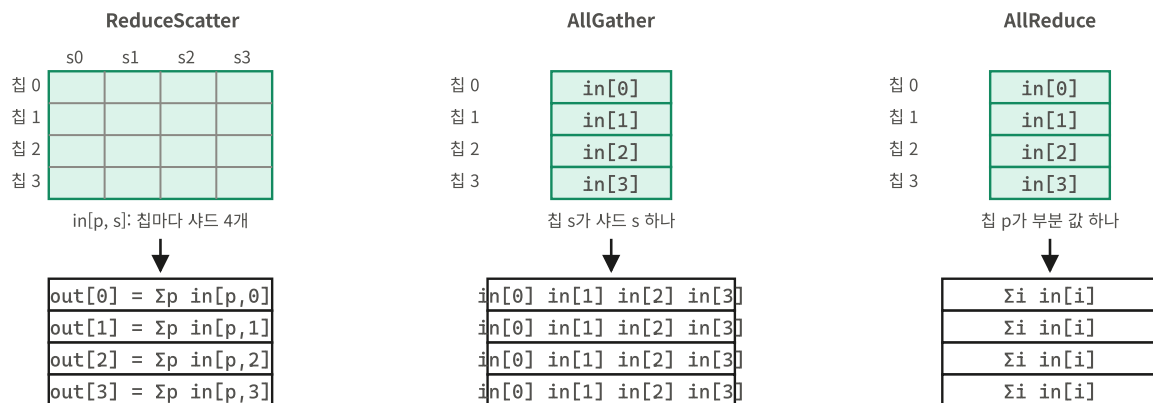


그림 40. 세 집합 연산의 입력(위)과 결과(아래). ReduceScatter 뒤에 AllGather를 하면 모든 칩이 합 전체를 가지므로 AllReduce와 같은 모양 변환이 된다. 원문의 칩 예제는 칩 4개, 로컬 페이지드 $[c, d] = i32$ 64개(256바이트)를 쓴다.

직접적인 순환 리듀스, ReduceScatter 뒤 AllGather, 버터플라이 교환은 모두 같은 AllReduce 모양 변환을 한다. 이 변환은 칩 차원과 클러스터 차원 모두에 적용된다.

주의: 부동소수점 리듀스 순서

`Broadcast(N)` 은 논리적 배치의 보장이다. 독립적으로 계산한 부동소수점 복제본은 서로 다를 수 있다. 순환 리듀스는 칩마다 $0 + 1 + 2 + 3$, $1 + 2 + 3 + 0$ 처럼 더하는 순서가 달라 반올림 결과가 달라질 수 있기 때문이다. ReduceScatter 뒤 AllGather는 샤드를 한 번씩만 계산해 모으므로 복제본이 똑같다. 원문 예제는 반올림 순서 문제가 없는 `i32`를 쓴다.

재분배와 동기화

예제들은 지연된 SRAM 재분배 계획을 쓴다. `to_dm` 앞의 `shuffle`과 `slice` 단계는 DMA 하나로 합쳐진다. 반면 `asymmetric_chip_slice`·`asymmetric_cluster_slice`를 직접 부르면 차원을 따라 데이터를 옮기지 않고 sub 컨텍스트에서 처리한다. 컴파일러는 `to_dm`이 계획을 구체화할 때 동기화를 유도한다. 원본 칩이 바뀌면 `ChipSync`, 원본 클러스터가 바뀌면 `ClusterSync`를 넣고, 로컬 `DmTensor` 자르기에는 넣지 않는다. 원격 DMA 접근 전에 목적지 준비 동기화를, 후에 완료 동기화를 보내며, 자기 자신으로 가는 칩 경로는 칩 간 송수신을 만들지 않는다.

예제	재분배 호출	자동으로 넣는 동기화
칩 ReduceScatter	<code>chip_shuffle().chip_slice().to_dm()</code> 체인 4개	<code>ChipSync</code> 3개 (라운드 0은 자기 자신)
칩 AllGather	차원을 원소로 옮기는 <code>to_dm()</code> 1개	<code>ChipSync</code> 1개
칩 AllGather after ReduceScatter	둘을 합침	<code>ChipSync</code> 4개
칩 AllReduce	<code>chip_shuffle().to_dm()</code> 3개(로컬 값은 이미 있음)	<code>ChipSync</code> 3개
칩 Butterfly AllReduce	<code>chip_shuffle().to_dm()</code> 2개	<code>ChipSync</code> 2개
클러스터 ReduceScatter	로컬 slice 2개 + <code>cluster_swap().to_dm()</code> 1개	<code>ClusterSync</code> 1개
클러스터 AllGather	차원을 원소로 옮기는 <code>to_dm()</code> 1개	<code>ClusterSync</code> 1개
클러스터 AllGather after ReduceScatter	둘을 합침	<code>ClusterSync</code> 2개
클러스터 AllReduce	<code>cluster_swap().to_dm()</code> 1개	<code>ClusterSync</code> 1개

이 개수는 예제의 명시적 재분배 호출만 센 것이다. HBM 전송과 로컬 Vector Engine 작업에도 의존성은 있지만 칩·클러스터 재분배 동기화를 더하지는 않는다.

칩 예제

예제는 `axes!` [A = 4, B = 4, C = 8, D = 8, I = 2, X = 2]를 쓴다. A가 칩 차원, [C, D]가 256바이트 로컬 페이로드다. 모든 리듀스가 공유하는 덧셈 함수는 §5.3의 인터리빙과 §9.6의 Pair Mode를 조합한 것이다.

```

/// DM 텐서 [C, D] 두 개를 Vector Engine으로 더한다
pub fn add_pair(
    device: &mut Device,
    lhs: &DmTensor<i32, m![A], m![1 # 2], m![1 # 256], m![C, D]>,
    rhs: &DmTensor<i32, m![A], m![1 # 2], m![1 # 256], m![C, D]>,
    output: &mut DmTensor<i32, m![A], m![1 # 2], m![1 # 256], m![C, D]>,
) {
    device
        .main
        .begin_interleaved::<I, _, _, _, _, _>(lhs.view(), rhs.view()) // lhs, rhs를 번갈아
        .fetch::<m![C, I], m![D]>()
        .collect::<m![C, I], m![D]>()
        .vector_init()
        .vector_intra_slice_unzip::<I, m![C]>() // 두 그룹으로
        .vector_clip_zip(ClipBinaryOpI32::AddFxp) // lhs + rhs
        .vector_final()
        .commit_trim::<m![D]>()
        .commit_view(output.view_mut());
}

```

- **ReduceScatter**: 입력은 A를 칩에 두고 [B, C, D]를 로컬로 가진다. `chip_shuffle().chip_slice().to_dm()` 체인 네 개가 각각 B 좌표 하나를 골라 목적지 칩으로 보내고, Vector Engine 체인 세 개가 도착한 [C, D] 값 네 개를 더한다. 목적지 칩 s는 $out[s] = \sum_p in[p, s]$ 를 갖는다. 라운드 0은 칩 s가 자기 $in[s, s]$ 를, 라운드 1~3은 칩 s+1, s+2, s+3(mod 4)의 $in[:, s]$ 를 받는다.
- **AllGather**: 칩 s가 샤드 $in[s]$ 하나를 가질 때, `to_dm()` 하나가 칩 차원을 로컬 원소 모양으로 옮기고 그 모양을 모든 칩에 브로드캐스트한다.
- **AllGather after ReduceScatter**: ReduceScatter 결과를 곧바로 AllGather에 넣는다. 목적지 칩 b가 좌표 b를 가지므로 `unsafe { shard.reshape::<m![B], ...>() }`로 축 이름을 A에서 B로 바꿔도 물리 순서가 보존된다.

```

/// 칩 순환 회전 네 개를 더해 모든 칩이 전체 합을 받는다
#[device(chip = 4)]
pub fn all_reduce(device: &mut Device, hbm: &HbmTensor<i32, m![A], m![C, D]>) -> HbmTensor<i32, m![A], m![C, D]> {
    let round0: DmTensor<i32, m![A], m![1 # 2], m![1 # 256], m![C, D]> = hbm.to_dm(&mut device.tdma);
    let round1 = round0.chip_shuffle([1, 2, 3, 0]).to_dm::<m![C, D]>(&mut device.tdma);
    let round2 = round0.chip_shuffle([2, 3, 0, 1]).to_dm::<m![C, D]>(&mut device.tdma);
    let round3 = round0.chip_shuffle([3, 0, 1, 2]).to_dm::<m![C, D]>(&mut device.tdma);

    let mut sum01 = DmTensor::new();
    let mut sum012 = DmTensor::new();
    let mut sum = DmTensor::new();
    add_pair(device, &round0, &round1, &mut sum01);
    add_pair(device, &round2, &sum01, &mut sum012);
    add_pair(device, &round3, &sum012, &mut sum);
    sum.to_hbm(&mut device.tdma)
}

/// XOR 짝 버터플라이 두 라운드로 리듀스한다
#[device(chip = 4)]
pub fn butterfly_all_reduce(device: &mut Device, hbm: &HbmTensor<i32, m![A], m![C, D]>) -> HbmTensor<i32, m![A], m![C, D]> {
    let partials: DmTensor<i32, m![A], m![1 # 2], m![1 # 256], m![C, D]> = hbm.to_dm(&mut device.tdma);
    let swapped = partials.chip_shuffle([1, 0, 3, 2]).to_dm::<m![C, D]>(&mut device.tdma); // 0↔1, 2
    ↪3
    let mut first = DmTensor::new();
    add_pair(device, &swapped, &partials, &mut first);

    let swapped = first.chip_shuffle([2, 3, 0, 1]).to_dm::<m![C, D]>(&mut device.tdma); // 0↔2, 1
    ↪3
    let mut sum = DmTensor::new();
    add_pair(device, &swapped, &first, &mut sum);
    sum.to_hbm(&mut device.tdma)
}

```

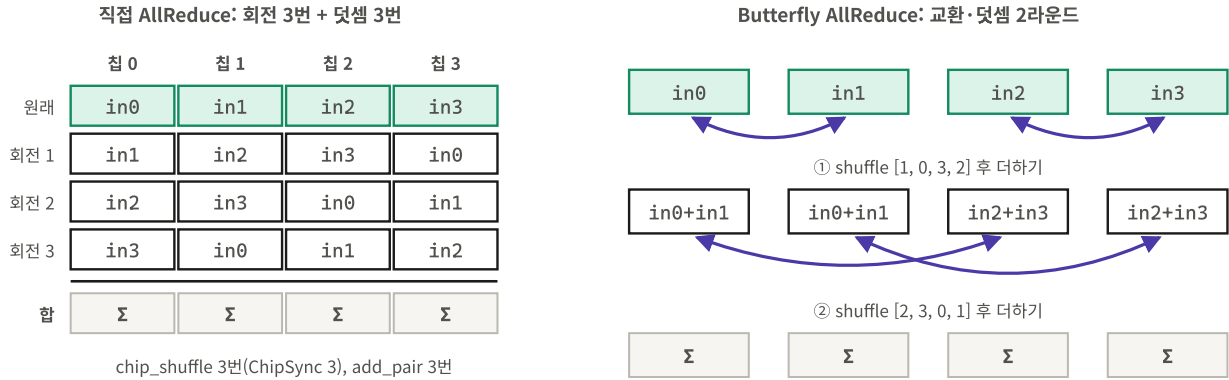


그림 41. 두 AllReduce 구현의 비교. 직접 방식은 원본의 회전 사본 세 개를 만들어 차례로 더한다. 버터플라이 방식은 칩 번호의 비트 하나씩 (XOR 짝)을 기준으로 교환하고 더하는 라운드를 두 번 반복해, 재분배와 덧셈 횟수를 줄인다(칩 4개 = 2비트).

클러스터 예제

클러스터 예제는 원소 두 개짜리 A·B 축(`axes! [A = 2, B = 2, C = 8, D = 8, I = 2]`)과 같은 256바이트 `[C, D]` 페이로드를 쓰며, 칩 하나(`#[device(chip = 1)]`)의 두 클러스터 사이에서 동작한다. 덧셈 함수는 클러스터 차원용으로 바뀐 `add_pair` 다.

- **ReduceScatter:** 첫 번째 로컬 `cluster_slice` 는 B 좌표 `[0, 1]`을(클러스터마다 하나), 두 번째는 `[1, 0]`을 고르고 `cluster_swap().to_dm()` 으로 반대 클러스터로 보낸다. Vector Engine이 로컬 값과 넘어온 값을 더한다. 클러스터 0은 `in[0, 0] + in[1, 0]`, 클러스터 1은 `in[1, 1] + in[0, 1]`을 갖는다.
- **AllGather:** `to_dm()` 하나가 클러스터 차원을 로컬 원소 모양으로 옮기고 두 클러스터에 브로드캐스트한다. 두 클러스터 모두 `[in[0], in[1]]`을 갖는다.
- **AllReduce:** `cluster_swap()` 한 번으로 두 부분 값을 맞바꾸고 덧셈 한 번으로 두 클러스터 모두 전체 합을 갖는다.

```
/// 로컬 값과 맞바꾼 클러스터 값을 더해 두 클러스터 모두 전체 합을 받는다
#[device(chip = 1)]
pub fn all_reduce(device: &mut Device, hbm: &HbmTensor<i32, m![1], m![A, C, D]>) -> HbmTensor<i32, m![1], m![A, C, D]> {
    let partials: DmTensor<i32, m![1], m![A], m![1 # 256], m![C, D]> = hbm.to_dm(&mut device.tdma);
    let remote = partials.cluster_swap().to_dm::<m![C, D]>(&mut device.tdma);
    let mut sum: DmTensor<i32, m![1], m![A], m![1 # 256], m![C, D]> = DmTensor::new();
    add_pair(device, &partials, &remote, &mut sum);
    sum.to_hbm(&mut device.tdma)
}
```

DMA 배치 요구 사항: 재분배는 § 6.6의 정렬, 살아 있는 인덱스, 256바이트 연속 구간 요구를 따른다. 원문 예제는 `m![B, C, D]` 를 써서 안쪽 `m![C, D]` 구간이 정확히 i32 256바이트가 되게 했다(§ 6.4에서 본 256바이트 전송 단위와 맞는다).

스스로 점검

1. main 컨텍스트의 축약 결과를 bf16으로 줄일 때 `cast` 대신 `commit_cast` 를 쓰면 무엇을 얻나?
2. `commit_trim::<m![w = 3]>()` (f32)가 거부될 이유는?
3. Transpose에서 `in_cols = 32`이면 어떤 버퍼링 모드가 되고, `n = 3`, `in_flits = 16`, `out_rows = 32`일 때 사이클은?
4. 칩 8개에서 버터플라이 AllReduce는 몇 라운드가 필요할까?

답: ① 변환 동안 Vector Engine이 비어 sub 컨텍스트의 Vector Engine 작업이 병렬로 돌 수 있다. ② f32 3개 = 12바이트라 유효 폭 $8 \cdot 16 \cdot 24 \cdot 32$ 바이트 규칙에 맞지 않는다. ③ 단일 버퍼링, $3 \times (16 + 32) = 144$. ④ 칩 번호가 3비트이므로 3라운드(원문의 두 라운드 예를 일반화한 추론).

11장

종합 사례: Transformer 디코더 한 스텝

원문: End-to-End Cases · Case Study: Transformer

원문 End-to-End Cases는 지금까지의 매핑·이동·계산·스케줄링 규칙이 실제 모델에서 어떻게 맞물리는지 보여 준다. 사례는 하나, Qwen3-0.6B 디코더가 토큰 하나를 처리하는 한 스텝이다. 원문은 구현 코드를 실지 않고 소스와 테스트 이름을 가리키므로, 이 장도 모델의 구조와 커널 경계, 결정 과정에 집중한다. 트랜스포머를 모르는 독자를 위해 모델 설명을 먼저 둔다.

이 장에서 배우는 것

- 디코더 전용 트랜스포머가 토큰 하나를 처리하는 과정
- Qwen3-0.6B의 크기와 파라미터 수를 직접 계산하는 법
- 원문 예제의 커널 경계 여섯 개와 각 경계가 맡는 일
- 설계 결정을 내리는 순서와 검증 방법

11.1 End-to-End Cases의 성격

원문은 이 부분의 페이지를 두 등급으로 표시한다. **Verified and tested**는 현재 소스와 테스트가 있어 바로 읽고 따라 해도 되는 패턴이고, **Design guide**는 설계 선택을 설명하는 자료로 구현 전에 API를 확인해야 한다. 원문의 권고는 "필요한 API 동작을 Quick Start 패턴으로 표현할 수 있으면 거기서 시작하라"다. 현재 제공되는 사례는 Qwen3 디코더 스텝 하나이며, 모델의 멘탈 모델, 기준 커널 지도, 디코드 전용 경계, 이식 가능한 오라클, 현재 예제의 스케줄 데이터를 다룬다.

11.2 트랜스포머 디코더를 짧게

대형 언어 모델은 문장을 **토큰**(단어 조각) 번호의 열로 보고, 지금까지의 토큰을 보고 다음 토큰을 예측한다. Qwen3-0.6B 같은 **디코더 전용** 모델에서 토큰 하나는 다음 과정을 거친다.

1. **임베딩**: 토큰 번호로 표를 조회해 크기 1,024의 벡터(은닉 벡터)를 얻는다.
2. **디코더 레이어 28개**를 차례로 지난다. 레이어마다:
 - **RMSNorm**: 벡터를 원소 제곱 평균의 제곱근으로 나눠 크기를 맞추고, 학습된 가중치를 곱한다.
 - **Q/K/V 투영**: 행렬곱 세 번으로 질의(Query), 키(Key), 값(Value) 벡터를 만든다.
 - **QK-Norm과 RoPE**: Q와 K를 헤드마다 RMSNorm하고, 위치에 따라 정해지는 $\cos \cdot \sin$ 값으로 회전시켜 위치 정보를 넣는다(RoPE).
 - **어텐션**: 현재 토큰의 Q와 이전 모든 토큰의 K를 내적해 점수를 만들고, softmax로 가중치로 바꾼 뒤 V들의 가중합을 구한다. 이전 토큰의 K·V는 다시 계산하지 않고 **KV 캐시**에 저장해 둔 것을 쓴다.
 - **출력 투영과 잔차**: 어텐션 결과를 행렬곱으로 은닉 크기로 되돌리고, 레이어 입력을 더한다(잔차 연결).
 - **RMSNorm → SwiGLU MLP → 잔차**: gate·up·down 세 행렬곱과 SiLU 게이팅으로 이루어진 MLP를 지나고 다시 입력을 더한다.
3. **최종 RMSNorm과 LM head**: 마지막 은닉 벡터를 어휘 크기의 행렬과 곱해 토큰마다 점수(logits)를 만든다.

프롬프트 전체를 한꺼번에 처리하는 단계를 **prefill**, 이후 토큰을 하나씩 만드는 단계를 **decode**라 한다. 원문 예제는 **디코드 전용, 토큰 하나**이며 prefill, 샘플링(점수에서 다음 토큰 고르기), 장치 쪽 토큰 조회를 구현하지 않는다.

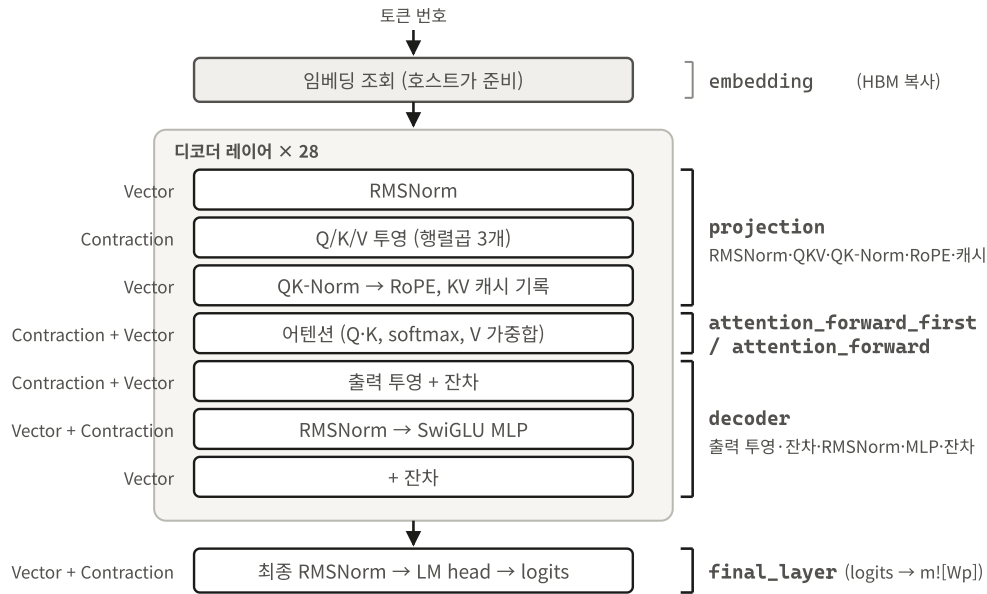


그림 42. Qwen3-0.6B 디코더 한 스텝과 원문 예제의 커널 경계(오른쪽). 왼쪽 엔진 표기는 원문 결정 추적의 요약(투영과 LM head의 축약은 Contraction Engine, RMSNorm과 원소별 단계는 Vector Engine)을 각 단계에 적용한 이 안내서의 정리다.

11.3 Qwen3-0.6B의 수치

원문이 적는 모델 설정은 은닉 크기 1,024, 질의 헤드 16개, 키·값 헤드 8개, 헤드 차원 128, MLP 중간 크기 3,072다. **그룹 질의 어텐션(GQA)**으로 키·값 헤드 하나를 질의 헤드 둘이 공유하고, RoPE의 θ 는 1,000,000이며, 모델 설계상 임베딩과 출력 가중치가 묶여(tied) 있다.

구성 요소	크기 또는 개수
디코더 레이어	28
어휘	151,936 토큰
임베딩과 묶인 LM head	$151,936 \times 1,024$
레이어당 어텐션 투영	6,291,456 파라미터
레이어당 SwiGLU MLP	9,437,184 파라미터
모델 전체	약 5억 9,600만 파라미터

더 깊이: 파라미터 수를 직접 계산해 보기

- Q 투영: $1,024 \rightarrow 16 \times 128 = 2,048$ 이므로 $1,024 \times 2,048 = 2,097,152$
- K, V 투영: $1,024 \rightarrow 8 \times 128 = 1,024$ 이므로 각각 1,048,576
- 출력 투영: $2,048 \rightarrow 1,024$ 이므로 2,097,152. 어텐션 합계 6,291,456 ✓
- SwiGLU: $\text{gate} \cdot \text{up}(1,024 \rightarrow 3,072)$ 과 $\text{down}(3,072 \rightarrow 1,024)$ 이 각각 3,145,728이므로 합계 9,437,184 ✓
- $28 \times (6,291,456 + 9,437,184) = 440,401,920$, 여기에 임베딩 $151,936 \times 1,024 = 155,582,464$ 를 더하면 595,984,384. 정규화 가중치 같은 작은 항을 빼면 약 5억 9,600만이다.

하드웨어 감각을 위해 하나 더 계산하면, 어휘 행렬 하나를 bf16으로 저장하면 $155,582,464 \times 2\text{바이트} \approx 297\text{ MB}$ 로 칩 전체 DM(256 MB)보다 크다. 이런 가중치는 HBM에 두고 필요한 부분만 DM으로 옮겨 계산해야 한다는 뜻이다([§ 2.3](#)). 이 상자의 계산은 원문의 수치를 확인하려고 이 안내서가 덧붙인 것이다.

11.4 커널 경계와 한 스텝의 흐름

원문은 로컬 경로 `furiosa-opt-examples/src/transformer/mod.rs` 가 다음 장치 진입점을 노출한다고 적고, 이 launch 지도를 사례 연구의 표준 실행 표면으로 삼는다. 축 크기와 텐서 타입은 `axes.rs` 에 정의되어 있다.

커널	하는 일
<code>embedding</code>	호스트가 준비한 HBM 복사. <code>HbmTensor<bf16, ..., m! [H]></code> 를 받아 뷰를 출력 HBM 텐서로 <code>to_hbm_view</code> 복사한다. 원문은 이것이 토큰 번호로 장치에서 조회하는 연산이 아니며 그렇게 설명해서는 안 된다고 못 박는다. 소스는 <code>ops::embedding</code> , 스모크 테스트는 <code>test_embedding</code>
<code>projection</code>	은닉 벡터 하나, 투영-RMSNorm 가중치, 위치에 따른 cos·sin 벡터, 가변 K/V 캐시를 받는다. 입력 RMSNorm, Q/K/V 투영, Q/K RMSNorm, RoPE를 하고, Q를 HBM에 쓰고 캐시를 갱신한다
<code>attention_forward_first</code>	첫 어텐션 청크. 온라인 softmax 어텐션 청크 하나를 계산한다
<code>attention_forward</code>	이후 청크. 앞 청크가 남긴 온라인 리듀스 상태를 이어받아 계속한다
<code>decoder</code>	어텐션 출력과 리듀스 상태를 받아 출력 투영, 잔차, 어텐션 후 RMSNorm, MLP, 잔차를 하고 잔차 스트림을 HBM에 쓴다
<code>final_layer</code>	잔차 스트림에 최종 RMSNorm을 하고, 어휘 가중치 뷰를 패딩된 출력 표현으로 바꿔 logits를 <code>m! [Wp]</code> 에 쓴다. 별도의 <code>lm_head_weight</code> 를 받으며 임베딩과 묶었다고 가정하지 않는다



그림 43. 한 토큰의 커널 호출 순서. 원문의 경계 테스트 `test_decoder_step_wires_kernel_boundaries` 는 0 가중치로 projection, 어텐션 청크 하나, decoder를 이어 붙여 경계가 제대로 연결되는지 확인한다.

배경: 온라인 softmax

원문은 어텐션을 "온라인 softmax 어텐션 청크"라고만 적는다. 일반적으로 온라인(스트리밍) softmax는 모든 점수를 한꺼번에 메모리에 올리지 않고, 청크를 하나씩 처리하면서 지금까지의 정규화 상수와 가중합 같은 누적 상태만 갱신해 최종 결과를 얻는 방식이다. KV 캐시가 길어져도 한 번에 처리할 양을 청크 크기로 묶어 둘 수 있다. 이 안내서의 해석으로는 이 점이 슬라이스당 DM이 512 KB로 제한된 하드웨어와 잘 맞고, 원문이 청크 사이에 이어받는다고 적은 "온라인 리듀스 상태"가 이 누적 상태에 해당한다. 구체적인 구현은 원문이 가리키는 `mod.rs`에서 확인해야 한다.

11.5 결정 추적

원문은 사례 연구가 책 전체와 같은 순서로 결정을 내린다고 설명한다. 이 순서를 이 안내서의 장과 짝지으면 다음과 같다.

1. **워크로드 의미**: 토큰 하나와 자라나는 KV 캐시가 데이터 이동을 정한다.
2. **강한 제약**: 모든 커널에는 타입이 붙은 공개 경계가 있다. 어휘 출력은 패딩된 `wp`를 쓰고, 모델의 실제 어휘는 `w`다.
3. **커널 경계**: Rust 모듈이 `embedding`, `projection`, `attention_forward_first`, `attention_forward`, `decoder`, `final_layer`를 각각 장치 진입점으로 노출한다.
4. **매핑과 이동**: 호스트가 준비한 텐서를 HBM과 DM으로 옮기고 엔진마다 필요한 표현을 쓴다(4장, 5장, 6장).
5. **엔진 선택**: 투영과 LM head의 축약은 Contraction Engine, RMSNorm과 원소별 단계는 Vector Engine, 전송은 DMA와 Fetch/Commit 경로(8장, 9장).
6. **검증**: 호스트 오라클 테스트가 값을, 덤프한 스케줄이 정적 실행 계획을 기록한다(12장).

원문은 이 사례가 결정들이 어떻게 조합되는지를 보여 줄 뿐, 어떤 엔진 API도 새로 정의하지 않는다고 강조한다. vISA와 API 계약은 레퍼런스 장에 남아 있다.

11.6 검증과 점검

```
cargo furiosa-opt test --test transformer_tests
```

테스트는 시드를 고정한 작은 입력으로 각 경계를 호스트 수식과 비교하며, 모델 파일이나 샘플링 루프가 필요 없다.

- `test_embedding`: 임베딩 경계의 스모크 테스트.
- `test_decoder_step_wires_kernel_boundaries`: 0 가중치로 `projection` → 어텐션 청크 하나 → `decoder`를 통과시켜 경계 연결을 확인한다. 투영, 청크 단위 어텐션, `decoder` 산술은 스칼라 오라클 테스트가 따로 다룬다.
- `test_final_layer_matches_scalar_reference`: 유효한 `w` logits를 호스트의 RMSNorm과 내적 오라클과 비교하는 결정적 테스트. 패딩은 텐서가 요구하는 모양과 저장 방식의 일부이며, 패딩 칸을 어떻게 해석할지는 호출자의 몫이다.

선택한 장치 진입점을 `--dump-schedule`로 컴파일하면 스케줄 데이터가 나오고, Schedule Viewer로 열어 12장의 비교 절차를 따른다.

원문이 원하는 작업 규율

오라클을 먼저 세운다. 기준 스케줄을 기록한다. 매핑, 이동, 엔진, 모양 가운데 **하나만** 바꾼다. 같은 산출물로 비교한다. 오라클 결과나 소스 순서에서 처리량을 추론하지 않는다.

12장

스케줄링과 튜닝

원문: Scheduling and Tuning · Schedule · Diagnosis · Tuning

올바른 커널을 측정된 실행 계획으로 바꾸는 단계다. 원문은 이 과정을 하나의 루프로 제시한다. **스케줄**에서 컨텍스트·의존성·연산 순서를 이해하고, **진단**에서 덤프한 스케줄을 보고 병목 가설 하나를 세우고, **튜닝**에서 바꿀 것 하나를 바꿔 같은 지표로 비교한 뒤 유지하거나 되돌린다. 스케줄은 컨텍스트 겹침, 메모리 의존성, 자원 충돌에 대한 "검증 가능한 가설"로 취급된다. 컴파일, CPU, NPU, 스케줄 검증이 각각 무엇을 뜻하는지는 § 3.7에서 다뤘다.

이 장에서 배우는 것

- 스케줄러가 연산을 겹치거나 기다리게 하는 규칙: 컨텍스트, 자원, 메모리 해저드
- 뱅크 경합의 64회 규칙이 스케줄에 미치는 영향
- 스케줄 타임라인을 읽는 법과 makespan
- 통제된 튜닝 실험의 절차, `#[unroll]`, 이중 버퍼링 실험 결과

12.1 스케줄러가 하는 일

스케줄러는 컨텍스트, 소스 순서, 메모리 위치를 써서 연산을 배치하고, 순차 실행과 같은 결과를 내는 데 필요한 의존성과 자원 제약을 지킨다. 공개 실행 컨텍스트는 셋이다.

- **Main**(`device.main`): 주 Tensor Unit 파이프라인을 구동한다.
- **Sub**(`device.sub`): 레지스터 파일 선적재 같은 연산에 쓰이는 Tensor Unit 파이프라인의 부분집합을 구동한다.
- **DMA**(`device.tdma`): HBM, DM 등 메모리 계층 사이의 텐서 이동을 구동한다.

한 컨텍스트 안의 연산은 순서대로 실행된다. 스케줄링 모델은 컨텍스트 점유 정보를 쓴다. 연산 A가 어떤 컨텍스트를 점유하면, 같은 컨텍스트를 쓰는 다음 연산 B는 A가 끝날 때까지 기다린다. 서로 다른 컨텍스트의 연산은 스케줄링 자원을 공유하지 않고 메모리 의존성을 어기지 않으면 겹친다. 예를 들어 DMA의 적재는 소비자가 그 값을 필요로 할 때까지 main 컨텍스트의 계산과 겹칠 수 있다. 어떤 연산이 어떤 컨텍스트를 점유하는지 알면 병렬 실행을 예측할 수 있다.

뱅크 경합과 64회 규칙

§ 6.10에서 본 뱅크 기아 규칙은 스케줄에도 반영된다.

- Tensor Unit 연산이 64회 제한을 어길 것 같으면 컴파일러는 그 연산을 **DMA를 점유하는 것처럼** 스케줄해 동시 DMA 연산을 막는다. TCP 구조 본연의 main·sub·DMA 병렬성(데이터 준비와 계산을 동시에)을 희생하지만 치명적인 하드웨어 리셋을 피한다. 원문은 이를 강한 제약으로 보고 같은 뱅크 연속 접근이 64회 이상인 패턴을 아예 쓰지 말라고 한다.
- 한도는 **누적**이다. 명령들이 번갈아 접근하더라도 모든 엔진이 같은 뱅크에 한 접근을 합쳐 64 미만이어야 한다. $\text{main } 30 + \text{sub } 20 + \text{DMA } 1 = 51$ 은 안전하지만, $\text{main } 30 + \text{sub } 35 + \text{DMA } 1 = 66$ 은 기아를 일으킨다.
- 컴파일러는 명령 하나하나를 64회 미만으로 유지하지만, 여러 명령이 동시에 돌 때 합계가 64에 이르는 것까지 막지는 못한다.

- 실제로 sub 컨텍스트는 같은 बैं크를 연속으로 접근하는 일이 드물다. StoTrf·StoVrf는 보통 순차 주소를 쓰고 타일링이 같은 बैं크 접근을 막는다. 한도를 넘을 sub 연산도 DMA와 동시에 스케줄하지 않는다.

main과 sub의 경합

main 컨텍스트도 sub 컨텍스트를 굵길 수 있지만 덜 심각하다. sub의 기아는 NoC 타임아웃이나 리셋을 일으키지 않고 처리 시간만 늘린다. 충돌 확률도 낮다. DMA Engine은 한 번에 बैं크 16개를 점유하지만 sub의 fetch·commit 엔진은 하나만 점유하기 때문이다. 같은 컨텍스트 안의 fetch와 commit 사이에서는 파이프라인 역압(back-pressure) 덕분에 기아가 생기지 않는다. main의 실행 명령이 특정 बैं크를 계속 쓰는 동안 sub의 저장 명령이 예약되면 sub가 늦어지며, 최악의 경우 전체 시간은 main 시간 + sub 시간, 이상적으로 둘이 다른 बैं크를 쓰면 $\max(\text{main 시간}, \text{sub 시간})$ 이다.

의존성: 순서를 바꿀 수 없을 때

스케줄러는 겹침을 드러내려고 독립적인 연산의 순서를 바꿀 수 있다. 순서를 바꾸면 값이 달라질 수 있을 때만 의존성이 재배치를 막는다. 그래서 원문은 **소스에 적은 순서로 성능 향상을 추론하지 말고** 방출된 스케줄을 보고 makespan을 비교하라고 한다. Tensor Unit 연산은 HBM 텐서를 직접 소비할 수 없어 데이터가 메모리 계층을 거쳐 올라와야 하고, 메모리 위치를 여러 컨텍스트가 공유하므로 스케줄러는 다음 해저드를 지킨다.

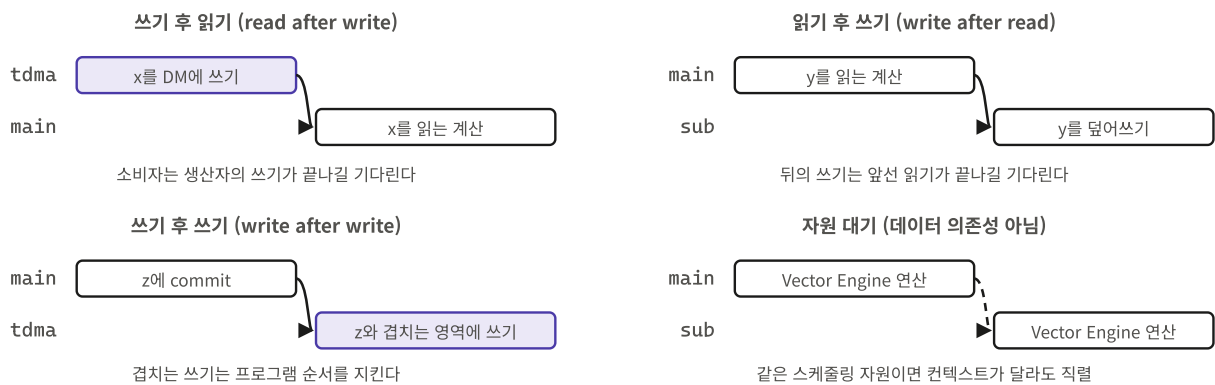


그림 44. 스케줄러가 연산을 기다리게 하는 네 경우. 앞의 세 가지는 여러 컨텍스트가 메모리 위치를 공유해서 생기는 해저드이고, 네 번째 (오른쪽 아래)는 자원 대기다. 칸마다 적은 컨텍스트와 연산은 이해를 돕기 위해 이 안내서가 고른 예다. 원문은 진단할 때 자원 대기과 데이터 의존성을 구분해야 올바른 튜닝 수단을 고를 수 있다고 강조한다.

12.2 MNIST 예제로 타임라인 읽기

원문은 2층 MLP로 스케줄의 겹침과 대기를 보여 준다. 입력 $X = 800$ 에서 은닉 $H = 256(\text{FC1} + \text{ReLU})$ 을 거쳐 출력 $C = 16(\text{FC2})$ 으로 가며, 완전연결층마다 행렬-벡터 곱에 bias를 더하고 첫 층은 같은 패스에서 ReLU를 한다. 장치 함수는 두 층을 소스 순서대로 부르고, 각 층은 계산 경로가 요구하는 메모리 계층에 피연산자를 준비한다.

```

axes![X = 800, H = 256, C = 16];
type Chip = m![1];
type Cluster = m![1 # 2];

#[device(chip = 1)]
pub fn forward(
    device: &mut Device,
    input: &HbmTensor<bf16, Chip, m![X]>,
    fc1_weight: &HbmTensor<bf16, Chip, m![H, X]>,
    fc1_bias: &HbmTensor<bf16, Chip, m![H]>,
    fc2_weight: &HbmTensor<bf16, Chip, m![C, H]>,
    fc2_bias: &HbmTensor<bf16, Chip, m![C]>,
) -> HbmTensor<bf16, Chip, m![C]> {
    let hidden = fc1_relu(device, input, fc1_weight, fc1_bias);
    fc2(device, hidden, fc2_weight, fc2_bias)
}

```

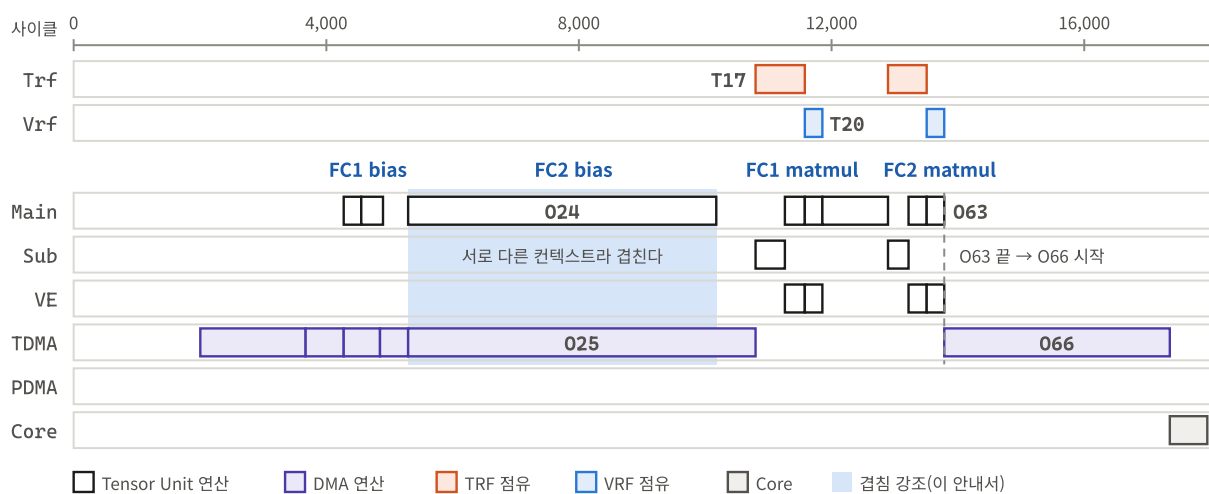


그림 45. 원문 Schedule 절의 MNIST 스케줄(전체 타임라인과 확대 그림 다섯 장)을 인쇄물에서도 읽히도록 다시 그렸다. 블록 위치는 원문 그림에서 꽤 근사값이고, 주소 공간을 보여 주는 Spm·Sram 행은 뺐다. 파란 라벨(FC1 bias 등)과 O24·O25·O63·O66·T17·T20 표시는 원문 확대 그림에 있는 것이다. 원문은 행의 뜻을 따로 설명하지 않으므로 아래 목록은 그림에서 읽은 내용이다.

원문은 이 스케줄을 확대한 그림 다섯 장으로 겹침과 대기를 보여 준다. 각 그림에서 읽을 수 있는 것은 다음과 같다.

- **컨텍스트:** Main·Sub·VE·TDMA 행마다 연산이 겹치지 않고 차례로 놓인다. VE 행의 블록은 같은 시각의 Main 연산과 나란히 있어, Vector Engine이라는 스케줄링 자원의 점유를 따로 보여 주는 것으로 읽힌다.
- **입력을 가져오는 동안:** 원문 그림은 TDMA 연산 O25를 중심으로 main의 O24와 Sram 행의 텐서 T14를 함께 강조한다. O24와 O25는 약 5,300~10,800사이클에 서로 다른 컨텍스트에서 겹쳐 돈다.
- **재배치된 연산:** 원문 그림은 FC1 bias와 FC2 bias 준비가 FC1 matmul보다 앞에 놓인 것을 표시한다. 소스는 `fc1_relu` → `fc2` 순서로 부르지만, 긴 입력 fetch가 진행되는 동안 bias 준비가 행렬-벡터 연산보다 먼저 올 수 있다는 것이 원문의 설명이다. 소스 순서와 실행 순서가 다를 수 있다는 예다.
- **메모리 계층 오르기:** 원문 그림은 입력 데이터가 올라가는 경로로 Sram 행의 T14, Trf 행의 T17, Vrf 행의 T20을 강조한다.
- **DMA가 main을 기다린다:** 원문 그림은 마지막 TDMA 연산 O66과, main과 VE를 함께 점유하고 O66 시작 직전에 끝나는 O63을 강조한다.

12.3 진단: 병목 가설 하나 세우기

진단은 스케줄 화면을 **검증 가능한 병목 가설 하나**로 바꾸는 일이다. 데이터 의존성과 공유 자원 대기를 구분해야 하며, 화면을 보는 것만으로 속도 향상이 증명되지는 않는다.

먼저 § 13.3의 방법으로 JSON을 만들어 연다. 뷰어는 스케줄된 타임라인, 연산의 컨텍스트, 수명, 연결된 텐서, 연산 설명을 보여 준다. 노드를 골라 입력과 출력을 추적하고, 의심 구간으로 사이클 범위를 좁힌다. 타임라인은 다음 순서로 읽는다. 마지막으로 스케줄된 사이클이 **makespan**이다.

1. 마지막 사이클에 닿는 구간을 찾는다.
2. 그 구간의 앞선 원인이 메모리 의존성인지, 공유 자원인지, 스케줄되지 않은 빈틈인지 확인한다.
3. 관련 컨텍스트 행을 비교해 다른 연산이 겹칠 수 있었는지 본다.
4. 그 구간을 바꿀 후보 변경 하나와 이유를 적는다.

스케줄은 컴파일러의 정적 동작이다. 값 오라클을 대신하지 않고 장치 처리량을 증명하지도 않는다. 비교가 유효하려면 후보마다 소스 리비전, 릴리스 프로파일, 장치 함수 필터, shape, 원소 타입, 매핑, 스케줄 파일을 기록하고, 후보 사이에는 고른 커널 선택 하나만 바꾼다. 이 중 무엇이랄도 바뀌면 튜닝 비교가 아니라 새 기준선으로 다룬다.

12.4 튜닝: 통제된 실험

튜닝은 정확성을 지키고, 기준선을 기록하고, 지원되는 수단 하나를 바꾸고, 같은 스케줄 지표로 비교한 뒤 유지할지 정하는 통제된 실험이다. 원문의 절차는 여섯 단계다.

1. 타입 검사 경로를 돌려 매핑과 shape 제약을 검증한다.
2. 경계 사례를 포함한 호스트 오라클로 CPU 테스트를 돌린다.
3. 기준선을 컴파일하고 스케줄 makespan을 기록한다.
4. 릴리스, shape, 데이터 타입, 매핑을 고정한 채 수단 하나를 바꾼다.
5. 같은 검사를 돌리고 후보의 스케줄을 덤프한다.
6. 정확성이 여전히 통과하고 측정 지표가 좋아졌을 때만 유지하고, 아니면 기준선으로 되돌린다.

이 장의 주 정적 지표는 스케줄 makespan이다. 재현 가능한 스케줄 비교나 따로 문서화한 장치 증거 없이 처리량이나 사이클 개선을 보고하지 않는다. 수단은 진단이 겨냥한 의존성이나 자원을 보고 고른다.

진단이 가리키는 것	바꿀 수단
현재 자원이 병목	실행 엔진 경로를 바꾼다(이 안내서의 예: Vector Engine을 함께 점유하는 Cast Engine 대신 <code>commit_cast</code> , § 10.3)
피할 수 있는 직렬 작업이나 맞지 않는 리듀스	타일이나 분할 모양을 바꾼다
이동이나 주소 의존성이 제한 구간	매핑, 패딩, 전송 경계를 바꾼다

루프 펼치기: #[unroll]

#[unroll]은 작은 정적 루프의 모든 반복을 스케줄러에 드러낸다.

```
#[feature(proc_macro_hygiene, register_tool, stmt_expr_attributes)]
#[register_tool(furiosa_opt)]
axes![Group = 4];

#[device]
fn example(_: &mut Device) {
    #[unroll]
    for group in 0..Group::SIZE {    // 반복 횟수는 컴파일 시간 상수여야 한다
        consume(group);
    }
}
```

- 커널 크레이트에 `#[feature(proc_macro_hygiene, stmt_expr_attributes)]` 가 필요하다.
- 반복 횟수는 컴파일 시간 상수여야 한다. 루프는 장치 진입점에 있어도, 진입점이 부르는 헬퍼에 있어도 된다.
- 중첩 루프는 자동으로 펼쳐지지 않는다. 모든 단계를 펼쳐려면 루프마다 `#[unroll]` 을 붙인다.
- 컴파일러는 펼친 뒤 남은 루프가 있는지 검사한다. **레지스터 파일 재사용은 커널 어디에도 루프가 남지 않았을 때만 켜진다.** 일부만 펼쳐도 그 반복은 스케줄에 드러나지만, 남은 바깥·중첩 루프가 있으면 루프가 없어야 가능한 최적화는 풀리지 않는다.

이중 버퍼링 실험

이중 버퍼링([§ 7.5](#))은 SubContext에서 다음 가중치 그룹을 TRF에 싣는 일과 MainContext에서 현재 그룹을 축약하는 일을 겹치게 할 수 있다. 다만 스케줄러는 **두 연산이 같은 스케줄링 단위 안에 보일 때만** 이 겹침을 고려한다. 원문은 가중치 그룹 20개, 같은 스케줄 창으로 세 가지 루프 형태를 비교한다.

```
// 일반 루프: 반복마다 그룹 하나를 TRF에 싣고(sub) 축약한다(main)
for g in 0..Group::SIZE {
    let weight_group = weight.view().tile::<m![Group], 1, m![1 # 20, Out, Red]>(g);
    let weight_trf: TrfTensor<bf16, Chip, Cluster, Slice, m![Out], m![Red]> = device
        .sub.begin(weight_group)
        .fetch::<m![Out, Red / 16], m![Red % 16]>()
        .collect::<m![Out, Red / 16], m![Red % 16]>()
        .to_trf();                                // StoTrf(g)
    device
        .main.begin(activation.view())
        .fetch::<m![Red / 16], m![Red % 16]>()
        .collect::<m![Red / 16], m![Red % 16]>()
        .contract_outer::<m![Red / 32], m![Red % 32], _, _, _>(&weight_trf)
        .contract_packet::<m![1]>()
        .contract_time::<m![1]>()
        .contract_lane::<m![1], m![Out]>(LaneMode::Interleaved)
        .cast::<bf16, m![Out # 16]>()
        .commit_trim::<m![Out]>()
        .commit_view(output.view_mut().tile::<m![Group], 1, m![1 # 20, Out]>(g)); // Contraction(g)
}
// 수동 2단 파이프라인: 한 반복에 first, second 두 그룹을 StoTrf한 뒤 두 번 축약
// 전체 펼치기: 위 루프 앞에 #[unroll]을 붙인다
```

루프 형태	스케줄 단위에 보이는 그룹	반복 사이 겹침	makespan
일반(rolled) 루프	1	없음	19,106사이클
수동 2단 소프트웨어 파이프라인	2	있음	15,477사이클
전체 펼치기(<code>#[unroll]</code>)	20	있음	15,086사이클

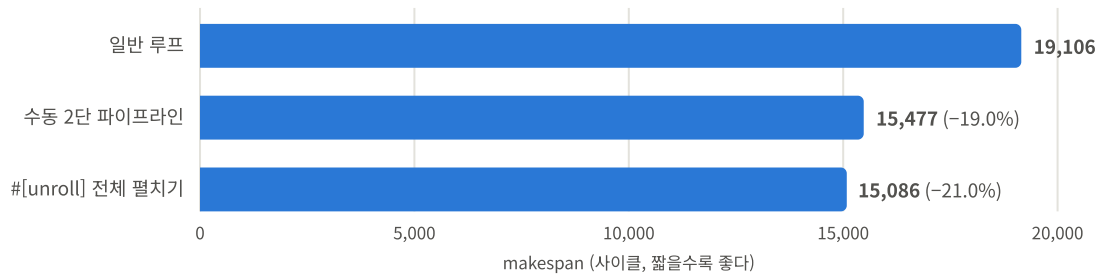


그림 46. 원문의 이중 버퍼링 실험 결과(가중치 그룹 20개). 괄호 안은 일반 루프 대비 감소율로, 원문 수치로 이 안내서가 계산했다. 같은 커널 연산이라도 스케줄러에 무엇을 한꺼번에 보여 주느냐에 따라 makespan이 약 5분의 1 줄었다.

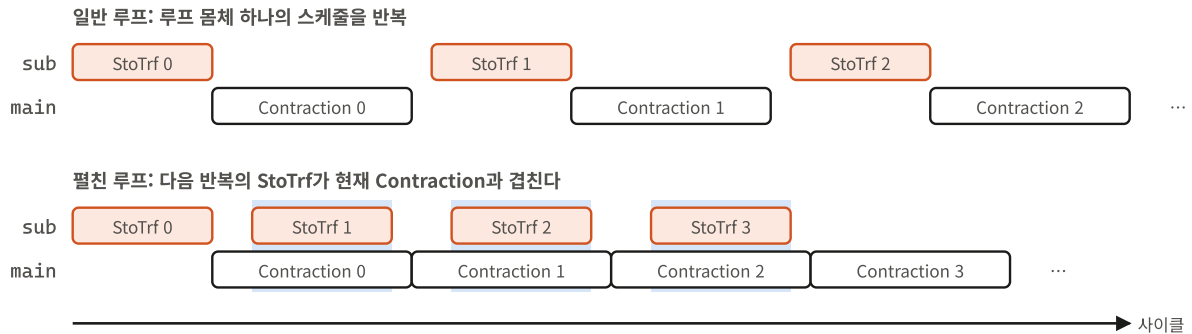


그림 47. 겹침이 생기는 원리를 줄여 그린 개념도. 원문은 세 경우의 스케줄을 같은 사이클 4,000~8,000 창으로 캡처했는데, 이 창에 일반 루프는 Contraction 0~4의 다섯 개가, 펼친 루프는 Contraction 0~7의 여덟 개가 들어 있다. 일반 루프는 스케줄러가 루프 몸체 하나의 스케줄을 만들어 반복마다 되풀이하므로 Contraction(i)과 StoTrf(i + 1)이 겹칠 수 없다. 반복을 펼치면 두 연산이 같은 스케줄 단위에 보여, TRF의 두 절반을 번갈아 쓰는 이중 버퍼링으로 겹친다(파란 띠).

수동 2단 파이프라인은 루프 몸체 하나에 두 그룹을 넣어 Contraction(first)과 StoTrf(second)를 함께 보여 주고, #[unroll]은 손으로 짤지 않고 20개 그룹을 모두 드러낸다. TRF는 한 절반에 그룹 하나씩 둘 수 있지만 두 절반이 बैं크를 공유한다는 점(§ 7.5)도 기억해야 한다. 원문은 결과로 생긴 겹침을 Schedule Viewer로 확인하라고 권한다.

스스로 점검

1. main 40회, sub 20회, DMA 1회가 같은 बैं크에 누적 접근하면 안전한가?
2. 두 커널 후보를 비교하는데 한쪽만 매핑과 원소 타입을 함께 바꿨다. 이 비교를 튜닝 결과로 보고해도 되나?
3. 가중치 그룹 루프 안에 작은 루프가 하나 더 있고 바깥 루프에만 #[unroll]을 붙였다. 레지스터 파일 재사용이 켜지나?
4. 일반 루프에서 StoTrf(i + 1)이 Contraction(i)과 겹치지 못하는 이유는?

답: ① 61로 64 미만이라 한도 안이다(여유는 적다). ② 아니다. 바꾼 것이 둘이므로 새 기준선으로 다룬다. ③ 아니다. 남은 루프가 하나라도 있으면 켜지지 않는다. ④ 스케줄러가 루프 몸체 하나의 스케줄을 만들어 반복마다 되풀이하므로 다른 반복의 연산이 같은 스케줄 단위에 보이지 않는다.

13장

도구

원문: Tools · Kernel Optimizer · Language Server · Schedule Viewer

furiosa-opt의 개발 도구는 셋이다. 빌드·테스트·컴파일을 맡는 Kernel Optimizer(`cargo furiosa-opt`), IDE에서 매핑 타입을 읽기 쉽게 보여 주는 Language Server, 스케줄 JSON을 타임라인으로 그리는 Schedule Viewer. 이 장은 원문의 명령 레퍼런스를 정리한다.

13.1 Kernel Optimizer: cargo furiosa-opt

`cargo-furiosa-opt`는 Furiosa NPU 컴파일러 툴체인을 위한 얇은 cargo 래퍼다. `cargo furiosa-opt`는 NPU 빌드이고, 평범한 `cargo build / test / run`은 CPU 빌드로, 커널이 호스트에서 돌고 NPU 하드웨어나 SDK가 필요 없다. cargo 인자는 모두 그대로 전달되며, 래퍼는 빌드에 필요한 커널을 알아서 컴파일한다. 먼저 쓰는 명령은 다음과 같다.

```
cargo run --release --bin gemm           # CPU에서 실행
cargo furiosa-opt compile                # 매핑 검사(모든 커널 컴파일)
cargo test --release --bin gemm          # 오라클과 값 비교
cargo furiosa-opt compile gemm_kernel --dump-schedule schedule.json # 스케줄 데이터 내보내기
```

사용법은 두 형태다.

```
cargo furiosa-opt <command> [args]      # cargo 통과: build, test, run 등을 NPU용으로
cargo furiosa-opt compile [FILTER]... [options] # 커널 직접 컴파일
```

평범한 cargo와 비교해 래퍼가 더하는 것은 둘이다. **NPU를 대상으로 한다**(커널이 실제 하드웨어로 디스패치되며 NPU와 Furiosa SDK가 필요하다). 그리고 **필요한 커널을 자동으로 빌드한다**. cargo 빌드 전에 필요한 `#[device]` 함수를 컴파일해 결과 바이너리가 실행 시간에 불러올 수 있게 한다.

자동 커널 빌드

사전 컴파일 단계는 필요할 때만 돈다. cargo 하위 명령이 코드를 빌드하거나 실행하는 것(`build`, `check`, `test`, `run`, `bench`, `doc`)이고, `-h / --help` 질의가 아닐 때다. 둘 때는 빌드에 실제로 필요한 커널만 컴파일한다.

- cargo의 단위 그래프를 읽어 `-p / --package`와 워크스페이스 선택을 존중하며 커널 패키지를 찾는다. `Cargo.toml`에 `[package.metadata.furiosa-opt]`가 있으면 커널 패키지다.
- 명령이 테스트·예제·바이너리 같은 특정 실행 타겟으로 풀리면, 컴파일러가 타겟마다 도달 가능한 `#[device]` 함수만 컴파일한다. 그렇지 않으면 선택한 패키지의 모든 커널을 컴파일한다.
- 커널 단위로 캐시되어 바뀌지 않은 커널은 다시 컴파일하지 않는다. 산출물은 `FURIOSA_OPT_OUT_DIR`(기본값 `<워크스페이스 target>/furiosa-opt/kernel`)에 쓰인다.

직접 컴파일과 덤프 옵션

```
# 모든 커널 패키지의 모든 #[device] 함수 컴파일
cargo furiosa-opt compile

# 패키지 하나에서 필터에 맞는 함수만
cargo furiosa-opt compile transpose_simple -p my_kernels

# 함수 하나를 컴파일하고 Schedule Viewer용 스케줄을 덤프
cargo furiosa-opt compile transpose::transpose_simple --dump-schedule schedule.json
```

커널 컴파일은 세 단계로 이루어진다(그림 8). Stage 1은 MIR → vISA, Stage 2는 vISA → LIR, Stage 3은 LIR → EDF(커널 바이너리)다. 덤프 옵션은 각각 한 단계의 산출물을 특정 소비자용으로 내보내며, 모두 **커널 하나를 컴파일할 때만** 쓰는 옵션이다.

옵션	단계	산출물
<code>--dump-visa <file></code>	Stage 1	vISA 텍스트. 컴파일러·백엔드 점검용
<code>--dump-dfg <file></code>	Stage 1	vISA 데이터 흐름 그래프(컴파일러 IR 바이너리). 그래프 점검용
<code>--dump-ir <file></code>	Stage 2	LIR(Low-Level IR)을 bincode 바이너리로. 컴파일러 도구용
<code>--dump-graph <file></code>	Stage 2	LIR 그래프 JSON. IR 그래프 뷰어용
<code>--dump-summary <dir></code>	Stage 3	LIR → EDF 컴파일 요약물 디렉터리에. 진단 도구용
<code>--dump-schedule <file></code>	Stage 3	LIR을 ResourceLir 로 낮춘 뒤의 스케줄 JSON. Schedule Viewer용

- **[FILTER]...**: 컴파일할 `#[device]` 함수 집합. 필터는 함수의 전체 경로(`abc::def::foo`)에 부분 문자열로 대조하며, 하나라도 맞으면 컴파일한다. 생략하면 전부 컴파일한다.
- **-p, --package <name>**: 이름 붙은 커널 패키지로 제한한다. 여러 번 쓸 수 있고, 생략하면 모든 커널 패키지를 컴파일한다.
- **--message-format <format>**: 컴파일러에 넘기는 진단 형식(예: `json`). 도구가 커널 컴파일 실패 진단을 기계적으로 읽을 수 있게 한다.

13.2 Language Server

`furiosa-rust-analyzer-proxy` 는 `rust-analyzer` 를 감싸는 프록시다. 일반적인 Rust 언어 서버 트래픽은 `rust-analyzer` 에 그대로 전달하고, 편집기에 보이는 결과만 고쳐 매핑 타입을 `m![...]` 표기로 보여 준다. 예를 들어 `Stride<Symbol<A>, 8>` 같은 긴 타입이 `m![A / 8]` 로 보인다. 편집기 표시만 바꿀 뿐 타입 검사는 바꾸지 않는다.

설치와 설정

1. `rust-analyzer` 를 설치해 PATH 에 둔다. 프록시가 이 원래 `rust-analyzer` 프로세스를 띄워 표준 기능을 제공한다.
2. GitHub 릴리스에서 프록시 바이너리를 받아 실행 권한을 준다.
3. IDE가 기본 언어 서버 대신 이 바이너리를 쓰게 설정한다.


```
curl -L -o furiosa-rust-analyzer-proxy \
  https://github.com/furiosa-ai/furiosa-opt/releases/latest/download/furiosa-rust-analyzer-proxy-x86_64-unknown-linux-gnu
chmod +x furiosa-rust-analyzer-proxy
```

```
// VSCode settings.json
{
  "rust-analyzer.server.path": "/path/to/furiosa-rust-analyzer-proxy",
  "rust-analyzer.inlayHints.maxLength": null, // '_'로 잘리는 일을 줄이도록 권장
  "rust-analyzer.server.extraEnv": {
    "FURIOSA_RUST_ANALYZER_PROXY_UPSTREAM": "/path/to/rust-analyzer" // 선택: 기본은 PATH의 rust-analyze
  }
}
```

기능과 주의점

프록시는 표준 Rust IDE 기능을 rust-analyzer에 맡기고, 편집기에 보이는 결과의 매핑 표현식만 바꾼다. 대상 기능은 호출 계층(call hierarchy), 코드 액션(제목과 텍스트 편집), 코드 완성(레이블·상세·편집), 진단(rust-analyzer와 rustc의 메시지), 호버(추론된 타입·시그니처·문서), 인레이 힌트, 시그니처 도움말(LSP 클라이언트가 돌려주는 오프셋 기반 매개변수 레이블 포함)이다. 인레이 힌트를 가장 정확히 바꾸려면 `rust-analyzer.inlayHints.maxLength`를 `null`(무제한)로 둔다.

주의

사용자 정의 타입의 이름이 내부 매핑 구성 요소와 같으면 매핑 표현식으로 잘못 보일 수 있다. 예를 들어 직접 만든 `Symbol` <T> 구조체가 IDE에서 `m! [T]`로 표시될 수 있다. 순전히 화면 표시 문제이며 다른 LSP 동작에는 영향이 없다.

13.3 Schedule Viewer

Schedule Viewer는 `cargo furiosa-opt`가 만든 스케줄 JSON을 불러와 대화형 실행 타임라인으로 보여 준다. 어떤 연산이 병렬로 도는지, 연산마다 어떤 컨텍스트나 자원을 점유하는지, 어떤 연산이 파이프라인 진행을 막는지 살펴볼 수 있다. 정적 실행 계획을 보여 줄 뿐 값이나 처리량을 검증하지는 않는다.

설치, 실행, JSON 만들기

```
cargo install furiosa-schedule-viewer
furiosa-schedule-viewer # 127.0.0.1:9254에 서버를 띄우고 브라우저를 연다
furiosa-schedule-viewer --host 127.0.0.1 --port 9254 # 주소 바꾸기

cargo furiosa-opt compile <device-function> --dump-schedule <path-to-json-file>
```

`--dump-schedule`은 단일 커널 컴파일에서만 쓸 수 있고, 함수 이름을 위치 필터로 준다. 이름이 모호하면 `kernel::gemm` `_kernel::gemm_kernel`처럼 전체 Rust 경로를 쓴다. 명령은 평소의 커널 산출물도 그대로 만든다.

사용법

- **불러오기:** 드롭 영역을 누르거나 스케줄 JSON 파일을 페이지로 끌어 놓는다.

- **텐서와 연산 살피기:** 노드를 누르면 왼쪽 사이드바에 이름, 수명, 컨텍스트, 연결된 노드가 나온다. 노드에 올리거나 선택하면 관련 노드가 강조된다. 연산을 고르면 입력·출력 텐서가, 텐서를 고르면 연결된 연산이 강조된다.
- **하드웨어 명령 기준:** 텐서와 연산은 실제 하드웨어 명령에 기반하므로 vISA에서 정의한 것과 다를 수 있다. 가능하면 vISA 텐서 이름과 shape, 연산 설명을 함께 보여 준다.
- **확대:** 사이클 범위만 바꾸려면 위쪽 타임라인을 드래그하거나 오른쪽 위의 **Cycle Range** 입력을 쓴다. 메모리 주소 범위까지 좁히려면 **Enable Brush**를 누르고 스케줄 화면을 드래그한다(사이클 범위와 주소 범위를 함께 설정). 전체 보기로 돌아가려면 타임라인 배경을 누르거나 **Reset**을 누른다.

스케줄을 검토하는 기준

고정된 커널 워크로드를 비교할 때는 스케줄의 makespan, 즉 마지막으로 스케줄된 사이클을 쓴다. 후보 사이에 릴리스, shape, 데이터 타입, 매핑을 바꾸지 않고, 차이는 타임라인의 겹침, 의존성, 자원 점유로 설명한다(§ 12.3).

부록 A

용어집

이 안내서에 나오는 용어를 네 묶음으로 정리했다. 괄호 안은 자세히 다룬 곳이다. 원문 표기가 영어인 용어는 영어를 그대로 두었다.

A.1 하드웨어와 메모리

TCP (Tensor Contraction Processor)

텐서 축약을 기본 연산으로 삼는 FuriosaAI의 NPU 구조(§ 2.1).

vISA

furiosa-opt 컴파일러가 Rust 커널을 하드웨어 명령어로 낮추기 전에 거치는 가상 명령어 집합 표현. 컴파일 1단계(MIR → vISA)의 결과다(§ 2.1, § 13.1).

Chip · Cluster · Slice · Lane

하드웨어 계층. 칩 하나에 클러스터 2개, 클러스터 하나에 슬라이스 256개가 있고, 슬라이스마다 Tensor Unit 파이프라인과 DM 구획이 하나씩 있다. Lane은 슬라이스 안에서 TRF와 Contraction Engine이 쓰는 병렬 단위로 8개다(§ 2.2).

HBM

칩 패키지 위의 고대역폭 DRAM. 칩당 48 GB, 최대 1.5 TB/s. 가중치와 활성값을 오래 두는 곳이며 Tensor Unit이 직접 읽지 못한다(§ 2.3, § 6.11).

DM (Data Memory)

칩 위 SRAM 작업 메모리. 칩당 256 MB, 슬라이스당 512 KB. Fetch가 읽고 Commit이 쓰는 곳이다(§ 2.3, § 6.9).

TRF (Tensor Register File)

Contraction Engine의 고정 피연산자를 담는 슬라이스별 SRAM. API 한도는 레인당 8 KB이고, 물리적으로 두 절반을 따로 쓸 수 있어 이중 버퍼링이 가능하다(§ 7.5).

VRF (Vector Register File)

Vector Engine의 피연산자를 담는 슬라이스별 SRAM. 슬라이스당 8 KB(§ 7.5, § 9.3).

SPM

원문이 공개 텐서 타입이 없다고만 적는 메모리 계층. 스케줄 뷰어에는 Spm 행으로 나타난다(§ 12.2).

뱅크 (bank)

DM 슬라이스를 이루는 병렬 접근 단위. 16개이고 각각 8바이트 폭이라 슬라이스당 128 B/cycle을 낸다. 같은 뱅크에 대한 접근은 직렬이다(§ 6.9).

DMN

DM 접근 경로의 구획. DMN당 128 B/cycle이며, 두 DMN을 번갈아 접근해야 클러스터당 256 B/cycle이 나온다(§ 6.4).

HBM 채널과 스택 비트

HBM의 병렬 접근 단위는 칩당 채널 32개(채널당 48 GB/s)다. 채널은 주소 비트 9~28로 고르고 주소 비트 8이 스택 비트다. 스택 비트가 고정되면 채널 16개만 쓰여 대역폭이 반으로 준다(§ 6.4, § 6.11).

사이클 (cycle)

하드웨어 클럭 한 번. 원문은 지연과 대역폭을 모두 사이클 단위로 적는다(§ 1.4).

NoC 타임아웃

칩 안 네트워크(Network on Chip) 요청이 제때 끝나지 않는 오류. DMA가 뱅크를 얻지 못해 굼으면 생기며 하드웨어 리셋으로 이어진다(§ 6.10).

A.2 매핑과 스트림

매핑 (mapping), `m![...]`

텐서 인덱스를 선형 버퍼 위치에 대응시키는 Rust 타입. 저장 순서, 패딩, 분할을 정한다(4장).

`axes![...]`

축 이름과 크기를 선언하는 매크로. 예: `axes![A = 8, B = 512]` (§ 4.3).

Cell

버퍼 위치 `i`에 대해 `map(i)` 가 돌려주는 값. 그 자리가 어떤 텐서 인덱스를 담는지(또는 패딩인지) 알려 준다 (§ 4.2).

major · minor

매핑의 가장 바깥 차원(가장 느리게 바뀜)과 가장 안쪽 차원(가장 빠르게 바뀜). C 배열의 행과 열에 해당한다 (§ 4.1).

패딩 # · 리사이즈 =

패딩은 쓰지 않는 버퍼 공간을 덧붙여 크기를 하드웨어 요구에 맞추고, 리사이즈는 논리 크기를 줄여 앞쪽 인덱스만 쓴다 (§ 4.3).

스트라이드 / · 모듈로 %

축 하나를 블록 번호와 블록 안 위치로 쪼갬다. `m![B / 64, B % 64]` 는 `m![B]` 와 같다 (§ 4.3).

타일 (tile)

데이터를 복사하지 않고 텐서의 한 부분을 가리키는 인덱스 뷰 (§ 4.3).

동등한 매핑

크기가 같고 모든 위치에서 같은 인덱스를 돌려주는 두 매핑. 구조가 달라도 서로 바꿔 쓸 수 있다 (§ 4.4).

공간 차원 · 시간 차원

Chip·Cluster·Slice(와 스트림의 Packet)는 데이터를 하드웨어에 펼치는 공간 차원이고, Time은 전달 반복의 순서를 정하는 시간 차원이다. Time에는 하드웨어 크기 한도가 없다 (§ 4.5).

Packet

한 시간 반복에서 슬라이스마다 전달되는 원소 묶음, 또는 그 배치를 정하는 매핑 (§ 4.5).

스트림, TuTensor

Tensor Unit 안을 흐르는 텐서. 차원은 [Chip, Cluster, Slice, Time, Packet] 이고, 파이프라인 위치가 타입에 새겨진다 (§ 4.5, 부록 D).

플릿 (flit)

Collect 이후 엔진이 사이클마다 처리하는 정확히 32바이트짜리 단위 (§ 7.4).

시퀀서 (sequencer)

두 매핑에서 유도한 스트라이드로 주소를 중첩 루프처럼 훑는 하드웨어 카운터. Fetch, Commit, DMA가 쓴다 (§ 5.2).

축 올리기 (axis lifting)

`fetch_chip_lift` · `fetch_cluster_lift` · `fetch_slice_lift` 로 칩·클러스터·슬라이스마다 다른 읽기 시작 주소를 줘서 Time의 축을 공간 차원으로 옮기는 것 (§ 5.3).

뷰 (view)

`DmTensorView` · `DmTensorViewMut` 처럼 텐서를 복사 없이 빌려 쓰는 핸들. `begin` 은 뷰를 빌리고 `commit_view` 는 가변 뷰에 쓴다(§ 5.1).

A.3 엔진과 연산

텐서 축약, einsum

여러 텐서를 곱하고 일부 축을 합쳐 없애는 연산. 행렬곱, 내적, 컨볼루션이 모두 축약이다(§ 1.2).

bf16

16비트 부동소수점 형식. 가중치와 활성화 저장에 주로 쓴다(§ 1.3).

Tensor Unit

DM에서 읽어 엔진 열 개를 거친 뒤 DM에 다시 쓰는 칩 안 계산 파이프라인. 슬라이스마다 하나씩 있다(§ 2.4).

Fetch Engine · Commit Engine

DM을 읽어 패킷 스트림을 만드는 입구와, 스트림을 DM에 쓰는 출구(§ 5.3, § 5.4).

Fetch Adapter

fetch 직후의 원소별 변환. 테이블 조회, 타입 변환, zero-point 빼기 셋이며 모두 선택 사항이다(§ 7.2).

Switch Engine

256개 슬라이스를 잇는 링으로 슬라이스 사이에 데이터를 옮기는 유일한 Tensor Unit 엔진(§ 7.3).

Collect Engine

임의 크기 패킷을 패딩하고 쪼개 32바이트 플릿으로 맞춘다(§ 7.4).

Contraction Engine

스트림과 TRF 피연산자의 축약을 계산한다. Outer(브로드캐스트와 곱) → Packet Reducer(사이클 안 합산) → Time Reducer(사이클에 걸친 누산) → Lane Folder(레인 결과 합치기)의 네 단계다(8장).

LaneMode

Lane Folder가 레인 결과를 내보내는 방식. Interleaved와 Sequential이 있다(§ 8.6).

Vector Engine

i32·f32 스트림에 원소별 연산, 이항·삼항 연산, 리듀스를 하는 엔진. Intra-Slice Chain과 Inter-Slice Reducer로 이루어진다(9장).

Intra-Slice Chain

슬라이스마다 독립적으로 도는 고정된 12단계 연산 체인. 단계는 정해진 순서로만 부를 수 있다(§ 9.2).

Way (8-way · 4-way)

Vector Engine 단계가 사이클마다 다루는 원소 수. 실수 계열 단계는 4-way라서 앞뒤에 Narrow와 Widen이 필요하다(§ 9.2).

ALU

Vector Engine 단계 안의 연산 유닛. Tensor Unit 호출 하나에서 같은 ALU는 한 번만 쓸 수 있다(§ 9.4).

stash

체인 중간 값을 저장해 뒤 단계의 피연산자로 다시 쓰는 기능. VRF 공간 1,024 B를 예약한다(§ 9.3).

TagMode · Branched

체인 첫 단계에서 원소마다 4비트 태그를 붙이는 방식과, 태그 조건에 따라 다른 피연산자를 고르는 조건부 피연산자(§ 9.5).

Pair Mode

번갈아 끼워진 두 그룹을 짝지어 계산하는 방식. `begin_interleaved` 로 읽고 `vector_intra_slice_unzip` 으로 들어간다 (§ 9.6).

Valid Count Generator (VCG)

리듀스할 때 패딩이 섞인 플릿마다 유효 원소 수(`valid_size`)를 만들어 패딩을 걸러 내는 부품 (§ 9.8).

Inter-Slice Reducer

클러스터 안 256개 슬라이스에 걸쳐 값을 리듀스한다. 체인과 함께 쓰면 순서는 IntraFirst 또는 InterFirst다 (§ 9.9).

Cast Engine · Transpose Engine

Cast는 i32·f32를 낮은 정밀도로 줄이고, Transpose는 플릿 안에서 Time과 Packet을 맞바꾼다 (§ 10.1, § 10.2).

Commit Adapter

commit 직전의 원소별 변환. 패딩 트리밍과 f32 → bf16 변환(ReLU 결합 가능)을 한다 (§ 10.3).

DMA Engine (Tensor DMA · PCIe DMA)

Tensor Unit 밖에서 메모리끼리 텐서를 옮기는 엔진. Tensor DMA(`tdma`)는 칩 안 HBM·DM 사이를 엔진당 최대 256 B/cycle로, PCIe DMA(`pdma`)는 호스트와 HBM 사이를 30 B/cycle로 옮긴다(6장).

재분배 계획

`chip_shuffle · cluster_swap · chip_slice · cluster_slice` 를 쌓아 DMA 하나로 실행하는, 칩·클러스터 사이의 값 보내기 (§ 6.6).

ReduceScatter · AllGather · AllReduce

분산 계산의 집합 연산. 리듀스한 결과를 조각내 나누기, 조각을 모아 모두에게 복제하기, 리듀스한 결과를 모두에게 복제하기다 (§ 10.4).

이중 버퍼링 (double buffering)

TRF의 두 절반을 번갈아 써서, 한 절반의 가중치로 추약하는 동안 다른 절반에 다음 가중치를 싣는 것 (§ 7.5, § 12.4).

A.4 스케줄링, 검증, 도구

실행 컨텍스트 (main · sub · tdma · pdma)

커널 연산이 명령을 넣는 독립 흐름. 한 컨텍스트 안은 순서대로, 서로 다른 컨텍스트는 자원과 메모리 의존성이 없으면 겹쳐 돈다 (§ 2.5, § 12.1).

StoTrf · StoVrf

레지스터 파일에 값을 싣는 연산. 스케줄에 이 이름으로 나타나며 보통 sub 컨텍스트에서 돈다 (§ 12.1).

makespan

마지막으로 스케줄된 사이클. 원문이 튜닝 후보를 비교할 때 쓰는 주 정적 지표다 (§ 12.3).

해저드 (hazard)

공유 메모리 때문에 순서를 지켜야 하는 관계. 쓰기 후 읽기, 읽기 후 쓰기, 쓰기 후 쓰기의 세 가지다 (§ 12.1).

뱅크 기아 (bank starvation)

같은 DM 뱅크에 연속 접근이 누적 64회 이상 몰려, 우선순위가 가장 낮은 DMA가 뱅크를 얻지 못하는 현상 (§ 6.10).

#[unroll]

작은 정적 루프의 모든 반복을 스케줄러에 드러내는 속성. 커널에 루프가 하나도 남지 않아야 레지스터 파일 재사용이 커진다 (§ 12.4).

#[device]

장치에서 실행할 커널 진입점을 표시하는 속성. `#[device(chip = N)]` 으로 칩 수를 정한다([§ 2.6](#)).

호스트 오라클

같은 수식을 CPU에서 계산해 커널 결과와 비교하는 기준 구현. 값의 정확성은 오라클로, 실행 계획은 스케줄로 확인한다([§ 3.7](#)).

MIR · LIR · ResourceLir · EDF

컴파일 단계의 표현. Rust의 MIR이 vISA로, vISA가 LIR로, LIR이 커널 바이너리 EDF로 낮아진다. 스케줄 덤프는 LIR을 ResourceLir로 낮춘 뒤에 나온다([§ 13.1](#)).

cargo furiosa-opt

NPU 빌드와 커널 직접 컴파일을 맡는 cargo 래퍼(Kernel Optimizer)([§ 13.1](#)).

Schedule Viewer · Language Server

스케줄 JSON을 타임라인으로 보여 주는 도구와, IDE에서 매핑 타입을 `m![...]` 로 보여 주는 rust-analyzer 프록시([§ 13.3](#), [§ 13.2](#)).

부록 B

핵심 수치

설계할 때 자주 찾게 되는 수치를 원문에서 모았다. 오른쪽 칸은 이 안내서에서 설명한 곳이다.

B.1 하드웨어 계층과 용량

항목	값	참고
Cluster	칩당 2개 (<code>cluster::SIZE == 2</code>)	§ 2.2
Slice	클러스터당 256개 (<code>slice::SIZE == 256</code>)	§ 2.2
Lane	슬라이스당 8개 (TRF와 Contraction Engine)	§ 7.5
DMA Engine	칩당 8개	§ 6.2
HBM	칩당 48 GB	§ 6.11
DM	칩당 256 MB, 슬라이스당 512 KB, 뱅크 16개 × 8 B 폭	§ 6.9
TRF	API 한도 레인당 8 KB(<code>lane::SIZE <= 8</code>). 물리 구조는 8 레인 × 2 뱅크 × 128 행 × 320 비트 = 슬라이스당 80 KB	§ 7.5
VRF	슬라이스당 8 KB. stash를 쓰면 그중 1,024 B가 예약된다	§ 9.3
DMA 패킷	최대 4,096 B. 하드웨어가 256 B 단위로 쪼갬다	§ 6.3

B.2 대역폭과 지연

경로	값	참고
HBM	칩당 1.5 TB/s (채널 32개 × 채널당 48 GB/s). 전송 단위 256 B, 채널당 4사이클	§ 6.4
DM	DMN당 128 B/cycle, 두 DMN을 번갈아 쓰면 클러스터당 256 B/cycle	§ 6.4
Tensor DMA	엔진당 최대 256 B/cycle	§ 6.4
PCIe DMA	읽기·쓰기 모두 30 B/cycle	§ 6.8
Fetch	fetch 하나당 최대 32 B/cycle(원문 Outer 성능 절)	§ 8.3
DMA 시작 지연	명령마다 약 500사이클	§ 6.4
정렬 어긋난 DM 쓰기	Read-Modify-Write가 되어 쓰기 시간 3배	§ 6.3
HBM 행 주소 충돌	충돌마다 약 40사이클	§ 6.4

B.3 규칙과 한도

규칙	값	참고
뱅크 연속 접근	모든 엔진의 누적 접근이 64회 미만. DMA Engine은 한 번에 뱅크 16개를 점유한다	§ 6.10

규칙	값	참고
주소 정렬	DM 쓰기 8 B, DM 읽기·HBM 읽기·HBM 쓰기 1 B. HBM↔DM은 읽기·쓰기 주소와 패킷 크기 모두 8 B	§ 6.3
HBM 주소 비트	비트 8 = 스택 비트, 비트 9~28 = 채널 선택	§ 6.4
플릿	Collect 이후 모든 엔진이 정확히 32 B 단위로 처리	§ 7.4
Contraction	OutPacket 64 B(PackSize 2) 권장, Lane은 8까지 채움	§ 8.7
Vector Engine	입력은 i32·f32만. Intra-Slice Chain 12단계, 실수 단계는 4-way	§ 9.2
Cast Engine	i32 → i4·i8·i16, f32 → f8e5m2·f8e4m3·bf16	§ 10.1
Transpose Engine	최대 in_rows: 4비트 16, 8비트 8, 16비트 4, 32비트 2. i5·i9는 전치 불가	§ 10.2
Switch Engine	<code>InSlice::SIZE == OutSlice::SIZE</code>	§ 7.3

B.4 사례의 수치

사례	값	참고
Qwen3-0.6B	레이어 28, 은닉 1,024, 질의 헤드 16, 키·값 헤드 8, 헤드 차원 128, MLP 중간 3,072, 어휘 151,936, RoPE θ 1,000,000, 약 5억 9,600만 파라미터	§ 11.3
MNIST MLP	$X = 800 \rightarrow H = 256 \text{ (FC1 + ReLU)} \rightarrow C = 16 \text{ (FC2)}$	§ 12.2
이중 버퍼링 (가중치 그룹 20개)	일반 루프 19,106 · 수동 2단 15,477 · 전체 펼치기 15,086사이클	§ 12.4

부록 C

설계 체크리스트

커널을 짜고 튜닝할 때 순서대로 훑어볼 항목이다. 각 항목은 원문의 규칙에서 왔고, "놓치면" 칸은 원문이 적은 결과를 줄였다.

C.1 매핑과 크기

확인할 것	놓치면	참고
☐ Chip·Cluster·Slice 크기가 하드웨어 개수와 같은가(Cluster 2, Slice 256, 남는 자리는 # 패딩)	컴파일이나 명령 검증에서 실패한다	§ 4.5
☐ Element 크기 × 원소 바이트가 DM 512 KB, TRF 레인당 8 KB(Lane ≤ 8), VRF 8 KB 안인가	컴파일이나 명령 검증에서 실패한다	§ 4.5
☐ 축약할 축을 Packet에 먼저 두고, 넘치는 부분만 Time에 두었는가	곱셈기 대부분이 논다(K를 Time에만 두면 bf16에서 1/32만 일함)	§ 8.7
☐ 살아남는 축을 Cluster·Slice·Lane에 퍼뜨렸는가	공간 병렬성을 잃는다	§ 8.7
☐ fetch 패킷이 32 B 플릿에 맞는가	Collect가 패딩을 붙여 대역폭을 버린다	§ 7.4

C.2 이동과 메모리

확인할 것	놓치면	참고
☐ DM 쓰기와 HBM↔DM 전송의 주소·패킷이 8 B 정렬인가	DM 쓰기가 Read-Modify-Write가 되어 3배 느려지고 그 बैं크의 다른 연산을 막는다	§ 6.3
☐ 같은 DM बैं크에 대한 모든 엔진의 연속 접근이 누적 64회 미만인가	DMA가 굼어 NoC 타임아웃과 리셋이 난다. 컴파일러가 DMA 병렬성을 희생해 막기도 한다	§ 6.10
☐ HBM 접근이 채널 32개(스택 비트 포함)에, DM 접근이 두 DMN에 고루 흩어지는가	대역폭이 절반 이하로 떨어진다	§ 6.4
☐ 작은 DMA 명령을 적은 수로 합쳤는가	명령마다 약 500사이클의 시작 지연이 쌓인다	§ 6.4
☐ DMA 패킷이 4,096 B 이하이고 연속 구간이 긴가	명령을 나눠야 하고 접근 횟수가 는다	§ 6.3
☐ 호스트 데이터를 한 번 올려 여러 연산에서 재사용하는가	PCIe DMA(30 B/cycle)가 칩 안 DMA보다 한 자릿수 느려 병목이 된다	§ 6.8

C.3 계산 엔진

확인할 것	놓치면	참고
☐ TRF·VRF를 소비 엔진보다 먼저 채우는가(보통 sub 컨텍스트)	소비 엔진이 레지스터 파일을 기다린다	§ 7.5

확인할 것	놓치면	참고
☐ Vector Engine 입력이 i32·f32인가	넓히는 단계(<code>fetch_cast</code> 또는 <code>Contraction</code>)를 따로 뒤야 한다	§ 9.1
☐ Lane을 8까지, <code>Contraction OutPacket</code> 을 64 B로 채웠는가	트리가 <code>Lane::SIZE / 8</code> 만 일하거나 곱셈기 절반이 0을 곱한다	§ 8.7
☐ Tensor Unit 호출 하나에서 같은 ALU를 두 번 쓰지 않는가	한 호출에 담을 수 없어 호출을 나눠야 한다	§ 9.4
☐ 체인 단계를 정해진 순서로 부르고, 실수 단계 앞뒤에 <code>Narrow-Widen</code> 을 두었는가	타입 검사에서 막힌다	§ 9.2

C.4 출력과 리듀스

확인할 것	놓치면	참고
☐ f32 → bf16 변환만 필요할 때 Cast Engine 대신 <code>commit_cast (_relu)</code> 를 쓰는가	Cast Engine이 sub 컨텍스트의 Vector Engine 작업과 자원을 다룬다	§ 10.3
☐ 부동소수점 리듀스 결과가 복제본끼리 똑같아야 하는가	순환 리듀스는 칩마다 더하는 순서가 달라 복제본이 달라질 수 있다. <code>ReduceScatter</code> 뒤 <code>AllGather</code> 는 한 번 계산해 복제한다	§ 10.4
☐ 재분배 계획의 원본 칩 목록이 순열이고, 목적지마다 원본이 하나인가	원문이 정한 계획 제약을 어긴다	§ 6.6
☐ Transpose 입력이 i5·i9 중간 타입이 아닌가	전치할 수 없다	§ 10.2

C.5 스케줄, 튜닝, 검증

확인할 것	놓치면	참고
☐ 호스트 오라클 CPU 테스트가 경계 사례를 포함하는가	값 오류는 스케줄로 찾을 수 없다	§ 3.7
☐ 기준선을 컴파일해 <code>makespan</code> 을 기록했는가	비교할 기준이 없다	§ 12.4
☐ 후보 사이에 수단 하나만 바꿨는가(릴리스·shape·원소 타입·매핑 고정)	튜닝 비교가 아니라 새 기준선이 된다	§ 12.3
☐ 대기가 데이터 의존성인지 자원 대기인지 구분했는가	영똥한 튜닝 수단을 고른다	§ 12.3
☐ 판단 근거가 소스 순서가 아니라 덤프한 스케줄인가	스케줄러가 독립 연산의 순서를 바꾸므로 추론이 틀린다	§ 12.1
☐ 겹쳐야 할 연산이 같은 스케줄 단위에 보이는가(수동 파이프라인이나 <code>#[unroll]</code>)	반복 사이의 겹침이 생기지 않는다	§ 12.4
☐ 커널 어디에도 루프가 남지 않았는가	레지스터 파일 재사용이 켜지지 않는다	§ 12.4

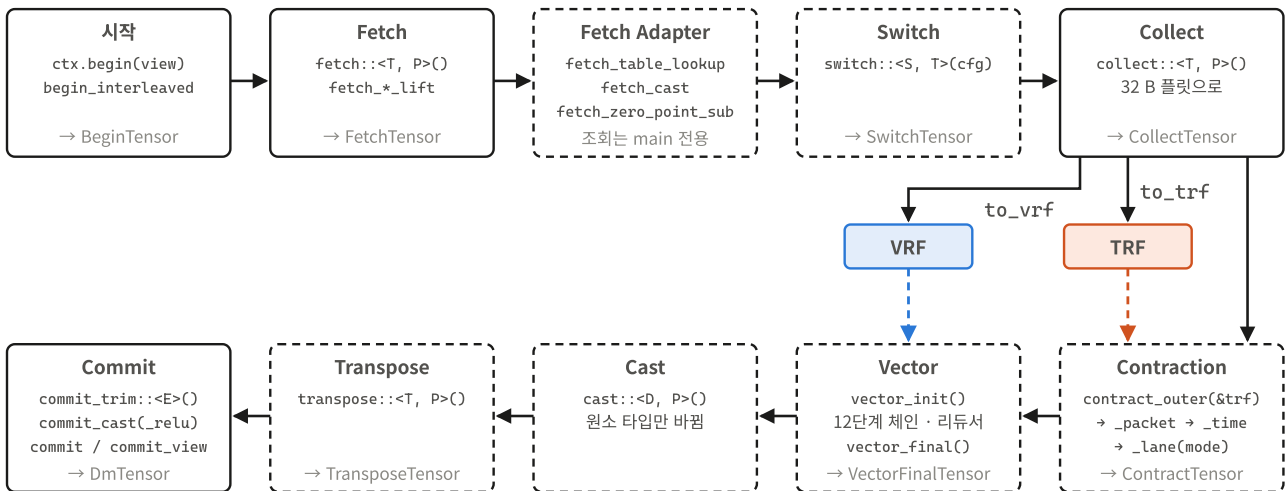
부록 D

메서드 체인 지도

커널 코드는 타입이 바뀌어 가는 메서드 체인이다. 각 호출은 이전 단계의 텐서를 소비하고 다음 파이프라인 위치의 타입을 돌려주므로, 순서에 맞지 않는 호출은 컴파일 오류가 된다. 이 부록은 원문의 설명과 예제 코드에 나온 연결을 한곳에 모았다. 정확한 제네릭 인자와 제약은 각 장에서 확인한다.

D.1 Tensor Unit 체인

윗줄: 읽고 정리하기 (왼쪽 → 오른쪽)



아랫줄: 계산하고 쓰기 (오른쪽 → 왼쪽)

그림 48. Tensor Unit 메서드 체인의 뼈대. 실선 상자는 늘 거치는 단계, 점선 상자는 필요할 때만 부르는 단계이며 엔진 순서는 그림 6와 같다. Collect 결과는 레지스터 파일에 저장할 수도 있고(주로 sub 컨텍스트), TRF는 Contraction에, VRF는 Vector에 피연산자를 준다. Contraction Engine과 Commit Adapter의 타입 변환은 main 컨텍스트 전용이다.

지금 가진 타입	부를 수 있는 메서드	결과	참고
DmTensorView	ctx.begin(view) begin_interleaved (Pair Mode 입력)	BeginTensor	§ 5.3
BeginTensor	fetch::<Time, Packet>()	FetchTensor	§ 5.3
FetchTensor	fetch_chip_lift fetch_cluster_lift fetch_slice_lift	Time 축을 공간 차원으로 옮긴 스트림	§ 5.3
FetchTensor	선택, 이 순서로: fetch_table_lookup::<D>() (main 전용) → fetch_cast::<D>() → fetch_zero_point_sub	변환된 fetch 스트림	§ 7.2
FetchTensor	switch::<Slice, Time>(SwitchConfig)	SwitchTensor	§ 7.3
FetchTensor SwitchTensor	collect::<Time, Packet>()	CollectTensor	§ 7.4

지금 가진 타입	부를 수 있는 메서드	결과	참고
CollectTensor	to_trf::<Lane, Element>() to_vrf()	TrfTensor VrfTensor	§ 7.5
CollectTensor	contract_outer(&trf) → contract_packet → contract_time → contract_lane(LaneMode) (main 전용)	ContractTensor	8장
CollectTensor ContractTensor	vector_init() → 체인이나 리듀서 → vector_final()	VectorFinalTensor	9장
CollectTensor ContractTensor VectorFinalTensor	cast::<D, Packet>()	원소 타입이 바뀐 스트림	§ 10.1
CollectTensor VectorFinalTensor	transpose::<Time, Packet>()	TransposeTensor	§ 10.2
commit 직전 스트림	commit_trim::<Element>() commit_cast · commit_cast_relu (main 전용)	기록할 스트림	§ 10.3
기록할 스트림	commit(...) commit_view(view_mut)	DmTensor	§ 5.4

원문 코드 블록에서 이어진 호출을 세어 보면 가장 흔한 연결은 `fetch` → `collect` (20회), `commit_trim` → `commit` (14회), `collect` → `contract_outer` · `to_trf` (각 7회), `contract_lane` → `cast` (7회), `cast` → `commit_trim` (8회) 이다. 이 횟수는 이 안내서가 원문 예제를 세어 얻은 것이다.

D.2 Vector Engine 체인

`vector_init()` 다음에는 체인 입구(`vector_intra_slice_tag` 또는 Pair Mode의 `vector_intra_slice_unzip`)나 `vector_inter_slice_reduce`가 온다. 체인 단계는 아래 순서로만 부를 수 있고 필요 없는 단계는 건너뛴다. `vector_reinterpret()`는 단계를 차지하지 않아 어디에나 끼울 수 있다(§ 9.2).

#	단계	메서드	Way	리듀서로 전환
1	Tag	<code>vector_intra_slice_tag(TagMode)</code>	8	-
2	Logic	<code>vector_logic</code>	8	가능
3	Fxp	<code>vector_fxp</code>	8	가능
4	FxpToFp	<code>vector_fxp_to_fp</code>	8	가능
5	Narrow	<code>vector_narrow_split</code> / <code>vector_narrow_trim</code>	8 → 4	-
6	Float	<code>vector_fp_unary</code> / <code>_binary</code> / <code>_ternary</code>	4	-
7	IntraSliceReduce	<code>vector_intra_slice_reduce</code>	4	-
8	FpDiv	<code>vector_fp_div</code>	4	-
9	Widen	<code>vector_widen_concat</code> / <code>vector_widen_pad</code>	4 → 8	가능
10	FpToFxp	<code>vector_fp_to_fxp</code>	8	가능
11	Clip	<code>vector_clip</code>	8	가능

#	단계	메서드	Way	리듀서로 전환
12	Filter	<code>vector_filter</code>	8	-

Pair Mode에서는 그룹별 변형(`vector_clip_zip`·`vector_fp_zip_with_mode`·`vector_fxp_with_mode` 등)을 쓴다 (§ 9.6). 중간 값을 다시 쓰려면 `vector_stash` 를 둔다 (§ 9.3).

D.3 DMA와 호스트 전송

출발	메서드	결과	컨텍스트	참고
<code>HostTensor</code>	<code>to_hbm(...)</code> (async)	<code>HbmTensor</code>	pdma	§ 6.8
<code>HbmTensor</code>	<code>to_host(...)</code> (async)	<code>HostTensor</code>	pdma	§ 6.8
<code>HbmTensor</code>	<code>to_dm(&mut device.tdma)</code>	<code>DmTensor</code>	tdma	§ 6.1
<code>DmTensor</code> , 뷰	<code>to_hbm(...)</code> <code>view().to_hbm_view(...)</code>	<code>HbmTensor</code>	tdma	§ 6.1
<code>DmTensor</code> , 뷰	(선택) <code>chip_shuffle</code> → <code>cluster_swap</code> → <code>chip_slice</code> * → <code>cluster_slice</code> * → <code>to_dm</code> 또는 <code>to_dm_view</code>	<code>DmTensor</code> (DMA 하나로 합쳐 실행)	tdma	§ 6.6
<code>DmTensor</code>	<code>asymmetric_chip_slice</code> <code>asymmetric_cluster_slice</code>	로컬 축을 자른 텐서	sub (즉시)	§ 6.6
<code>HbmTensor</code>	<code>hbm_chip_shuffle(dma, sources)</code>	<code>HbmTensor</code>	tdma 또는 pdma	§ 6.6
<code>HbmTensor</code>	<code>dma_gather_scaled(index)</code> <code>dma_gather_unscaled(&index_dm)</code>	<code>DmTensor</code>	tdma	§ 6.7
<code>DmTensor</code>	<code>dma_scatter(index, output)</code>	HBM 행에 기록	tdma	§ 6.7

*는 0번 이상 이어 부를 수 있다는 뜻이다.

부록 E

원문 대응표

원문은 FuriosaAI가 공개한 **furiosa-opt Book**(Programming Tensor Contraction Processors)이다. 아래 경로는 모두 `https://developer.furiosa.ai/furiosa-opt/book/` 뒤에 붙는다. 이 안내서는 2026년 9월 11일에 받은 원문 인쇄본(`print.html`)을 바탕으로 했다.

E.1 장별 원문 페이지

이 안내서	원문 페이지 경로
1장 배경지식	대응하는 원문 페이지 없음. 이 안내서가 덧붙인 배경 설명이다
2장 큰 그림	introduction.html quick-start.html quick-start/setup-and-tooling.html computing-tensors/index.html scheduling/schedule.html
3장 첫 커널 다섯 개	quick-start/tensor-and-contraction.html quick-start/kernel-design.html quick-start/kernel-validation.html
4장 텐서 매핑	mapping-tensors/index.html mapping-tensors/mapping-expressions.html mapping-tensors/spatial-temporal-dimensions.html mapping-tensors/tensor-semantic.html
5장 텐서 이동 I	moving-tensors/index.html moving-tensors/sequencer.html moving-tensors/fetch-engine.html moving-tensors/commit-engine.html moving-tensors/tensor-unit-io.html
6장 텐서 이동 II	moving-tensors/dma-engine.html moving-tensors/memory-performance.html
7장 Tensor Unit 연산 I	computing-tensors/index.html computing-tensors/fetch-adapter.html computing-tensors/switch-engine.html computing-tensors/collect-engine.html computing-tensors/register-files.html
8장 Tensor Unit 연산 II	computing-tensors/contraction-engine/index.html computing-tensors/contraction-engine/outer.html computing-tensors/contraction-engine/packet-reducer.html computing-tensors/contraction-engine/time-reducer.html computing-tensors/contraction-engine/lane-folder.html

이 안내서	원문 페이지 경로
9장 Tensor Unit 연산 III	computing-tensors/vector-engine/index.html computing-tensors/vector-engine/intra-slice-chain.html computing-tensors/vector-engine/intra-slice-reduce.html computing-tensors/vector-engine/vcg.html computing-tensors/vector-engine/inter-slice-reducer.html
10장 Tensor Unit 연산 IV	computing-tensors/cast-engine.html computing-tensors/transpose-engine.html computing-tensors/commit-adapter.html kernel-examples/chip-cluster-reduce.html
11장 Transformer 사례	kernel-examples/index.html kernel-examples/transformer.html
12장 스케줄링과 튜닝	scheduling/index.html scheduling/schedule.html scheduling/diagnosis.html scheduling/tuning.html
13장 도구	tools/index.html tools/kernel-optimizer.html tools/language-server.html tools/schedule-viewer.html
전체	print.html (한 페이지 인쇄본)

E.2 이 안내서를 읽는 법

- 이 안내서는 번역이 아니라 해설이다. 원문의 코드, API 이름, 수치는 원문 그대로 옮겼다.
- 원문에 없는 배경 설명, 비유, 점검 문제는 따로 상자로 구분했다. 그림과 계산 가운데 원문에 없는 것은 캡션이나 본문에서 이 안내서가 덧붙였다고 밝혔다.
- 원문의 두 곳이 서로 어긋나는 부분은 "원문 확인" 상자에 적었다. 구현할 때는 최신 원문과 API로 다시 확인해야 한다.
- 원문과 furiosa-opt 저장소는 Apache License 2.0으로 배포된다.